

DSA 2026

Black-and-white print PDF with rendered concept diagrams

One main problem per page - paraphrased self-contained prompts - Python and commented Go kernels

Black-and-white scheme

All panels use grayscale fills, dark borders, line patterns, and text labels. Meaning does not depend on color.

Rendered diagrams

The old text-only concept diagram panel has been replaced with vector shapes: arrays, pointers, trees, graphs, heaps, DP grids, tries, and more.

Page contract

229 main problem pages plus this cover. Each problem is isolated on one page with PDF bookmarks.

Each problem page includes

Prompt, example input/output, rendered diagram, brute approach, optimized approach, proof/invariant, complexity reasoning, variations, mistakes, Python, and commented Go.

Print note

Use grayscale or black-and-white printing. Borders and labels remain visible even when scaled with fit-to-page.

Source note

Problem names and study-plan scope are based on public lists. Statements are paraphrased for a self-contained guide.

30-minute interview-page workflow

- Study loop for each page:
- 1) Restate the prompt in your own words.
 - 2) Read the rendered diagram and name the state/invariant.
 - 3) Give brute force and optimized approach.
 - 4) Dry-run the sample using the diagram.
 - 5) Code Go from the commented kernel and test edge cases.
 - 6) Solve one variation without looking.

Tip: The kernels are interview templates. Adapt the exact function signature and helper predicates to the platform prompt.

MAIN PROBLEM 001 of 229: 1768. Merge Strings Alternately

Pattern: Array / String | Sources: LC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Merge characters by alternating from word1 and word2, appending any leftover characters after one word ends.

Example input/output:
Input: word1="abc", word2="pq"

Output: "apbqc"

Why: Take a/p, b/q, then leftover c.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

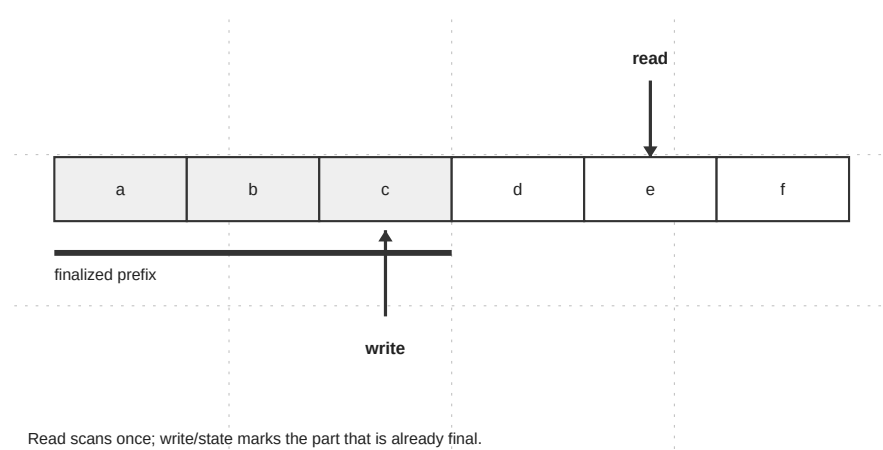
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1768. Merge Strings Alternately
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

- V1: Require in-place edits instead of building a new array/string.
- V2: Add Unicode/case-normalization rules before comparing characters.
- V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def merge_strings_alternately(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 002 of 229: 1071. Greatest Common Divisor of Strings

Pattern: Array / String | Sources: LC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Find the longest non-empty string that can be repeated some number of times to build both input strings; return empty if none exists.

Example input/output:
Input: str1="ABABAB", str2="ABAB"

Output: "AB"

Why: "AB" repeated 3 times and 2 times forms both strings.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

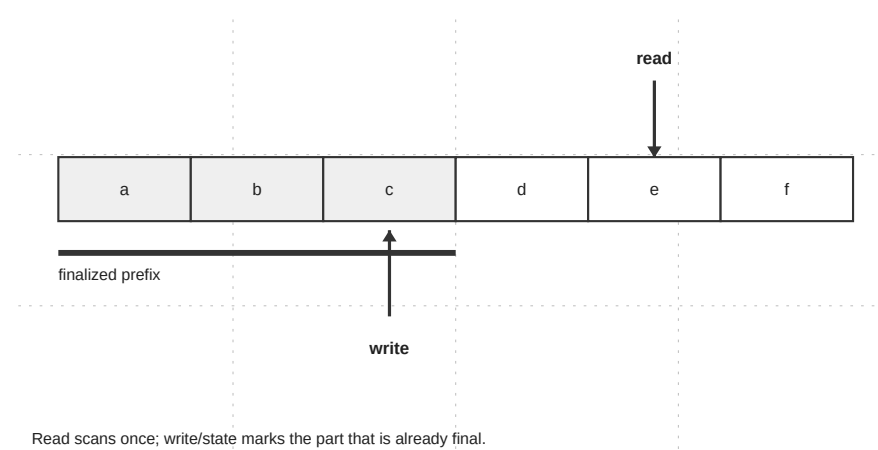
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1071. Greatest Common Divisor of Strings
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def greatest_common_divisor_of_strings(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 003 of 229: 1431. Kids With the Greatest Number of Candies

Pattern: Array / String | Sources: LC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
For each child, decide whether giving them all extraCandies would make their candy count at least the current maximum.

Example input/output:
Input: candies=[2,8,4], extraCandies=3

Output: [false,true,false]

Why: Only the child with 8 can still tie or exceed the maximum after adding 3 to each candidate.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

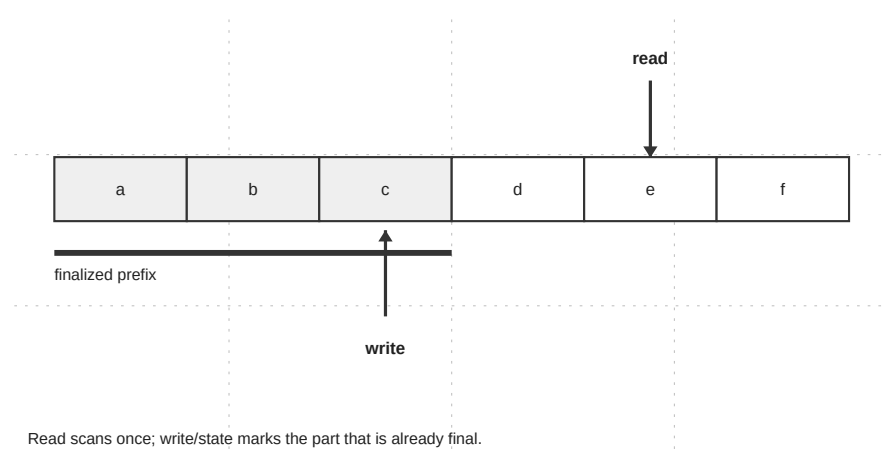
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1431. Kids With the Greatest Number of Candies
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def kids_with_the_greatest_number_of_can(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 004 of 229: 605. Can Place Flowers

Pattern: Array / String | Sources: LC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Given a flowerbed of 0/1 slots, decide whether n new flowers can be planted without adjacent flowers.

Example input/output:
Input: flowerbed=[1,0,0,0,1], n=1

Output: true

Why: Plant in the middle zero slot.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

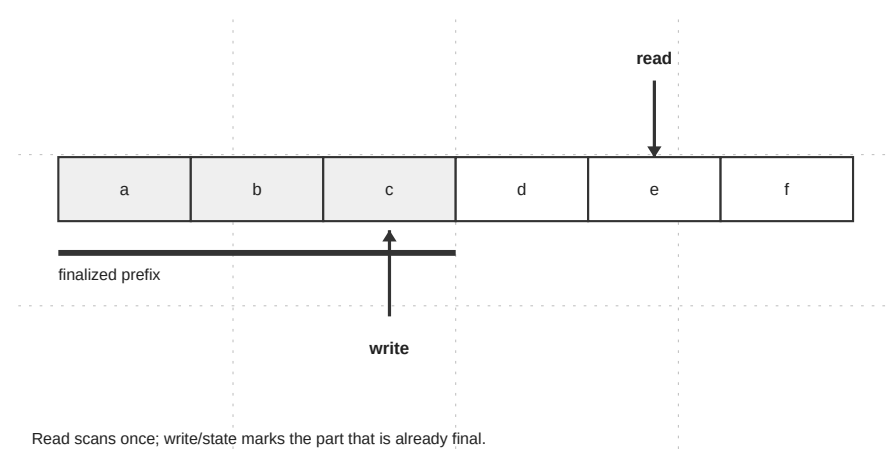
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 605. Can Place Flowers
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:
V1: Require in-place edits instead of building a new array/string.
V2: Add Unicode/case-normalization rules before comparing characters.
V3: Stream inputs so only a small buffer may be kept.

Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def can_place_flowers(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 005 of 229: 345. Reverse Vowels of a String

Pattern: Array / String | Sources: LC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Reverse only the vowels in a string while leaving all consonants and positions of non-vowels unchanged.

Example input/output:
Input: s="leetcode"

Output: "leotcede"

Why: The vowels e,e,o,e are reversed to e,o,e,e.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

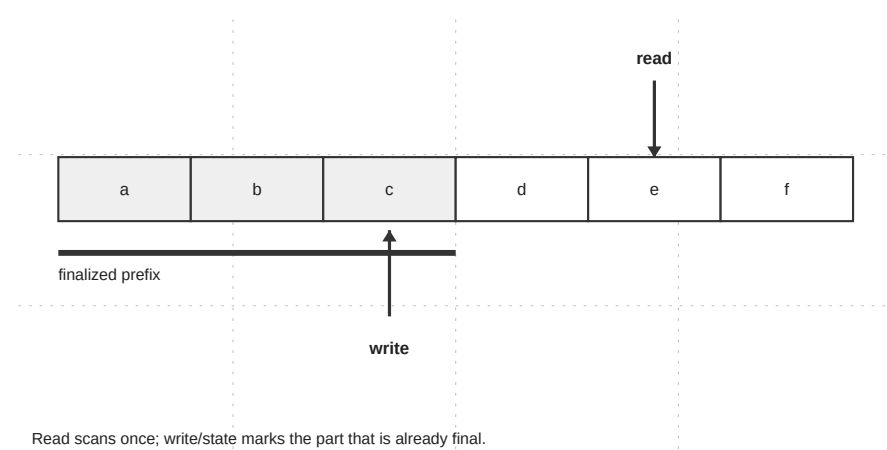
Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Annotated Go kernel - comments explain the reasoning

```
// 345. Reverse Vowels of a String
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

V1: Require in-place edits instead of building a new array/string.

V2: Add Unicode/case-normalization rules before comparing characters.

V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def reverse_vowels_of_a_string(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 006 of 229: 151. Reverse Words in a String

Pattern: Array / String | Sources: LC75, Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return the words of a string in reverse order, trimming outer spaces and reducing multiple spaces between words to one.

Example input/output:
Input: s=" blue sky "

Output: "sky blue"

Why: Words are [blue, sky], then reversed.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

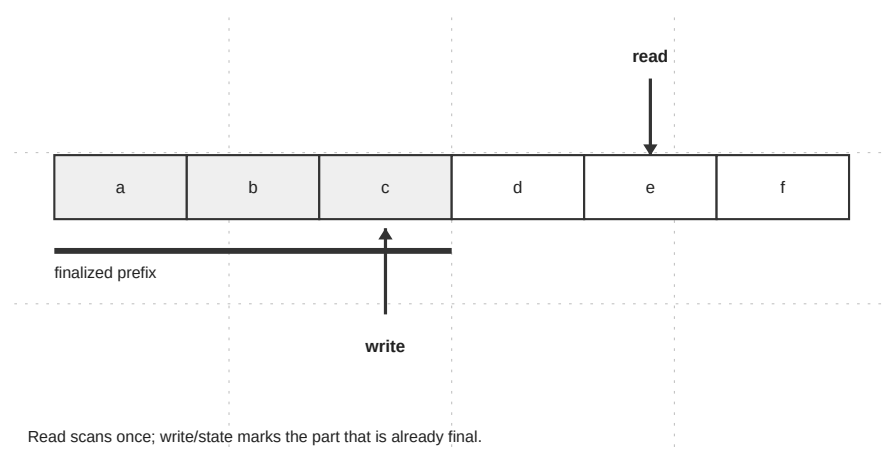
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 151. Reverse Words in a String
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def reverse_words_in_a_string(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 007 of 229: 238. Product of Array Except Self

Pattern: Prefix / Scan | Sources: LC75, Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
For each index, return the product of all array elements except `nums[i]`, without using division.

Example input/output:
Input: `nums=[2,3,4]`

Output: `[12,8,6]`

Why: Each output multiplies the other two numbers.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recompute every left/right sum or product per index. This repeats work and often becomes $O(n^2)$.

Optimized approach:
Carry a running prefix/suffix value. Update answer at the exact point where left and right states are both known.

Alternative:
Two passes are simplest; one pass works when the equation can be rearranged.

Implementation checkpoints:

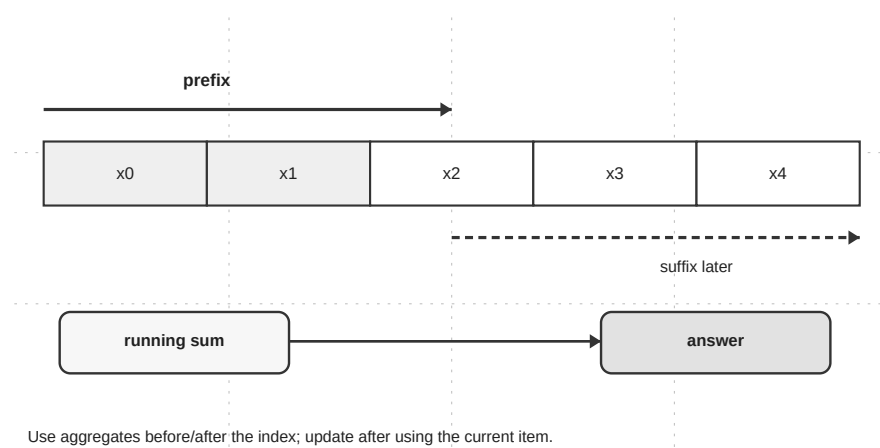
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 238. Product of Array Except Self
func solvePrefixScan(nums []int) int {
    total := 0
    for _, x := range nums { total += x }
    prefix, ans := 0, defaultAnswer()
    for i, x := range nums {
        suffix := total - prefix - x
        if condition(prefix, x, suffix) {
            ans = updateAnswer(ans, i)
        }
        prefix += x // next index sees this x on its left
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before index `i`, prefix equals the aggregate of all elements strictly left of `i`.

Complexity target:
Time $O(n)$; Space $O(1)$ to $O(n)$

Why:
Each element is added/multiplied a constant number of times, so time is $O(n)$; space is $O(1)$ or $O(n)$ for output arrays.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: `left <= right?` `i < n?`
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using updated prefix too early; not handling index 0.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
zeros; negative numbers; overflow in Go int; empty arrays; inclusive/exclusive index boundary.

Variations and follow-ups

Three drills:

- V1: Answer many range queries after one preprocessing pass.
- V2: Handle zeros/negative numbers without division.
- V3: Return both value and index where the best prefix occurs.

Follow-ups:

- Can you answer many range queries after preprocessing?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def product_of_array_except_self(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before index `i`, prefix equals the aggregate of all elements strictly left of `i`.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 008 of 229: 334. Increasing Triplet Subsequence

Pattern: Math / Greedy | Sources: LC75 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Return true if there exist indices i<j<k with nums[i]<nums[j]<nums[k].

Example input/output:
Input: nums=[5,1,6,2,3]

Output: true

Why: 1,2,3 is an increasing triplet.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

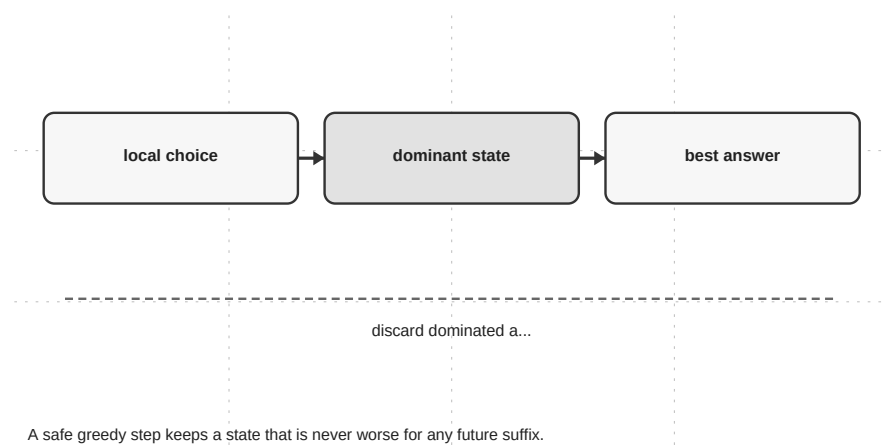
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 334. Increasing Triplet Subsequence
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def increasing_triplet_subsequence(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 009 of 229: 443. String Compression

Pattern: Array / String | Sources: LC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Compress a character array in-place by replacing repeated groups with the character followed by its count digits; return the new length.

Example input/output:
Input: chars=["a","a","b"]

Output: length=3, chars starts ["a","2","b"]

Why: The group "aa" becomes "a2".

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

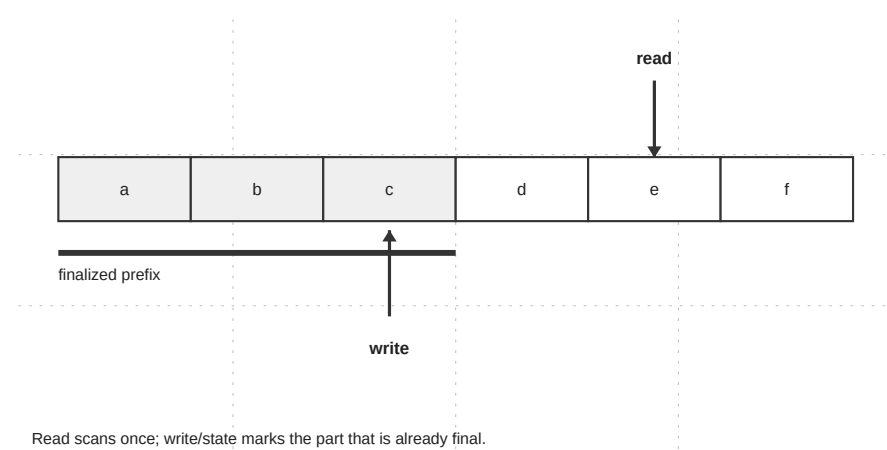
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 443. String Compression
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:
V1: Require in-place edits instead of building a new array/string.
V2: Add Unicode/case-normalization rules before comparing characters.
V3: Stream inputs so only a small buffer may be kept.

Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def string_compression(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 010 of 229: 283. Move Zeroes

Pattern: Two Pointers | Sources: LC75 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: O(n) or O(n log n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Move all zero values to the end of the array in-place while preserving the order of nonzero values.

Example input/output:
Input: nums=[0,1,0,3]

Output: [1,3,0,0]

Why: Nonzero values remain in order.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but O(n^2) or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

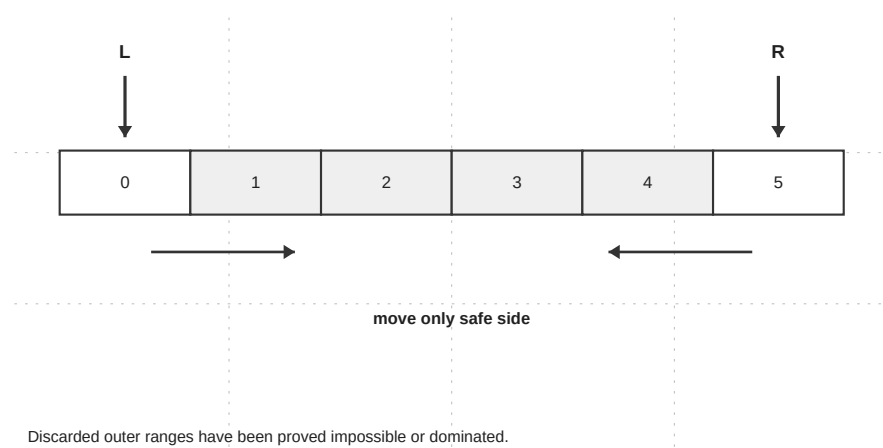
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 283. Move Zeroes
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time O(n) or O(n log n); Space O(1) to O(n)

Why:
Each pointer moves monotonically at most n times, giving O(n) after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def move_zeroes(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 011 of 229: 392. Is Subsequence

Pattern: Two Pointers | Sources: LC75, Top150 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Determine whether *s* can be obtained by deleting characters from *t* without reordering the remaining characters.

Example input/output:
Input: *s*="ace", *t*="abcde"

Output: true

Why: Choose a, c, e from *t* in order.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

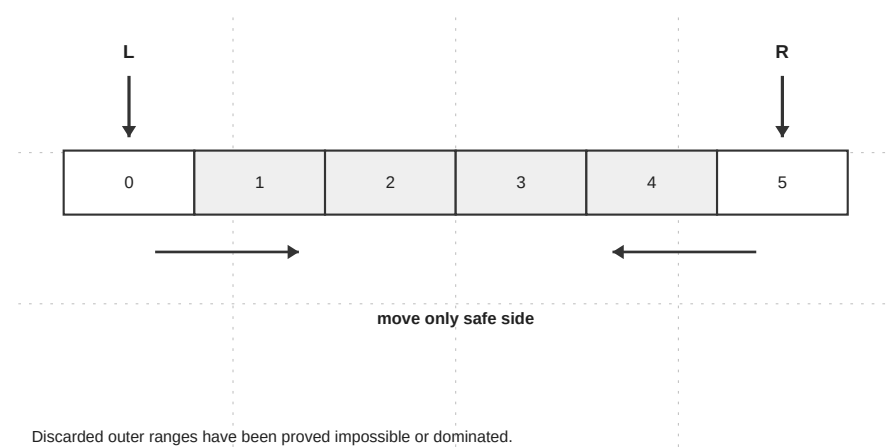
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 392. Is Subsequence
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most *n* times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: *left* <= *right*? *i* < *n*?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def is_subsequence(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 012 of 229: 11. Container With Most Water

Pattern: Two Pointers | Sources: LC75, Top150, NC75 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Choose two vertical lines so the water container area, width times shorter height, is maximized.

Example input/output:
Input: height=[1,7,2,5,4,7,3,6]

Output: 36

Why: Lines at heights 7 and 6 with width 6 give area 36.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

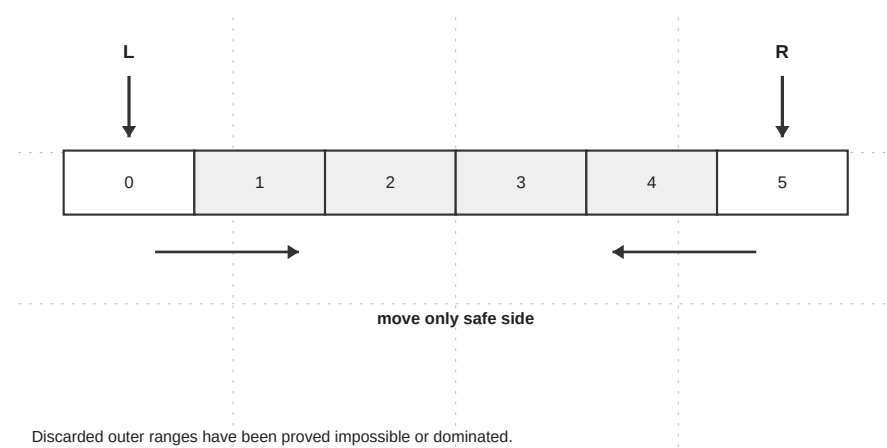
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 11. Container With Most Water
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most n times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def container_with_most_water(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 013 of 229: 1679. Max Number of K-Sum Pairs

Pattern: Two Pointers | Sources: LC75 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Return the maximum number of disjoint pairs whose values sum to k.

Example input/output:
Input: nums=[1,2,3,4], k=5

Output: 2

Why: Pairs are (1,4) and (2,3).

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

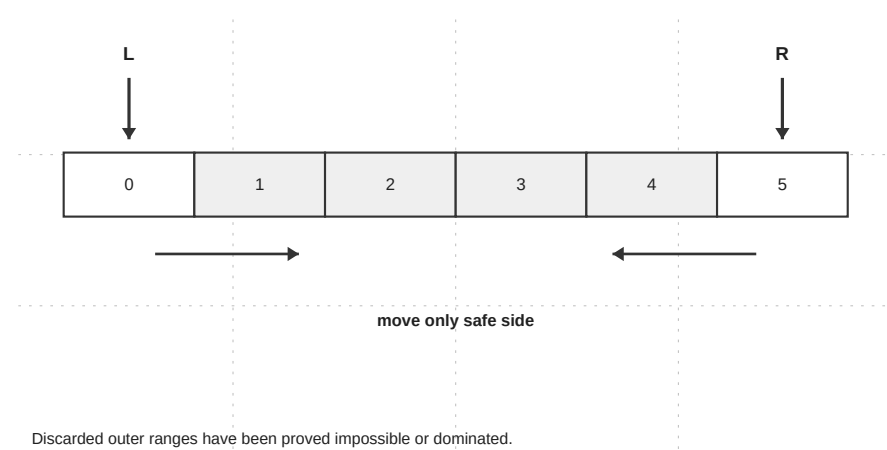
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1679. Max Number of K-Sum Pairs
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most n times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def max_number_of_k_sum_pairs(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 014 of 229: 643. Maximum Average Subarray I

Pattern: Sliding Window | Sources: LC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Find the maximum average value among all contiguous subarrays of length k .

Example input/output:
Input: nums=[1,12,-5,-6,50,3], $k=4$

Output: 12.75

Why: The best length-4 window is [12,-5,-6,50].

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

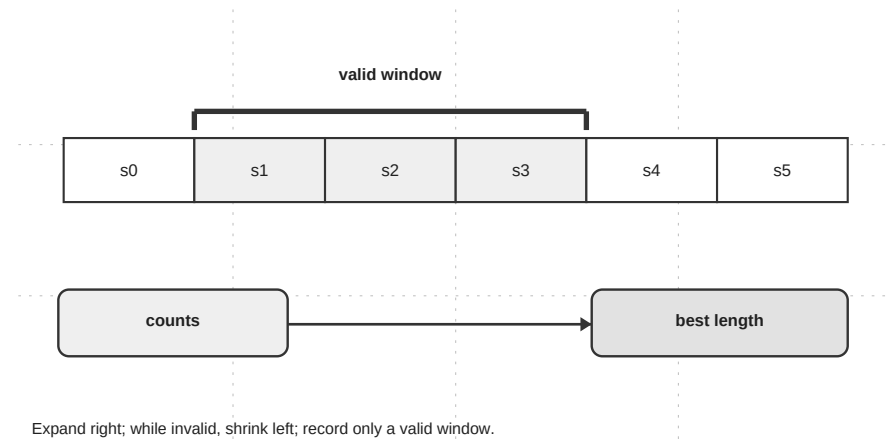
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 643. Maximum Average Subarray I
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Shrinking only once when a while loop is needed; stale max/count variables.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
 $k=0$; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left}+1$.

Variations and follow-ups

- Three drills:
- V1: Switch from at-most- k to exactly- k constraints.
 - V2: Return the smallest valid window instead of the largest.
 - V3: Support multiple character/value constraints at once.

- Follow-ups:
- Can you return the actual window, not only its length/count?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def maximum_average_subarray_i(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1; left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 015 of 229: 1456. Maximum Number of Vowels in a Substring of Given Length

Pattern: Sliding Window | Sources: LC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the largest number of vowels in any substring of length k .

Example input/output:
Input: $s = \text{"abciidef"}$, $k = 3$

Output: 3

Why: Substring "iii" has three vowels.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

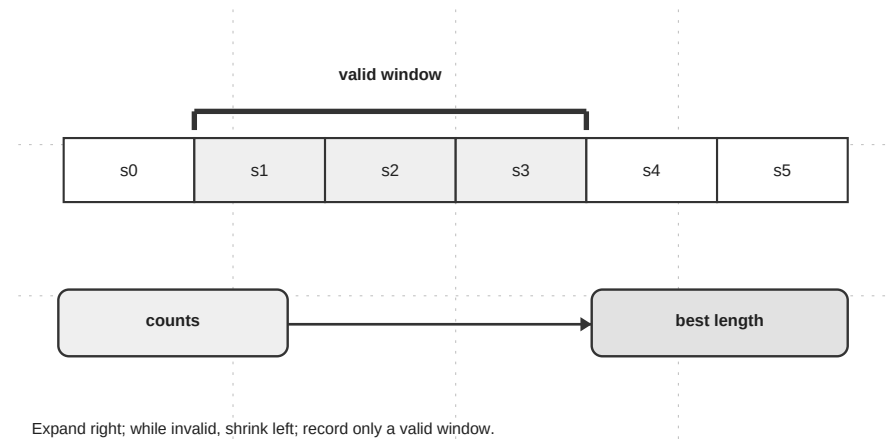
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1456. Maximum Number of Vowels in a Substring of Given Length
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Shrinking only once when a while loop is needed; stale max/count variables.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
 $k = 0$; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left} + 1$.

Variations and follow-ups

- Three drills:
- V1: Switch from at-most- k to exactly- k constraints.
 - V2: Return the smallest valid window instead of the largest.
 - V3: Support multiple character/value constraints at once.

- Follow-ups:
- Can you return the actual window, not only its length/count?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def maximum_number_of_vowels_in_a_substr(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1; left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 016 of 229: 1004. Max Consecutive Ones III

Pattern: Sliding Window | Sources: LC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Given a binary array, return the longest subarray containing only 1s after flipping at most k zeros.

Example input/output:
Input: nums=[1,0,1,1,0,1], k=1

Output: 4

Why: Flip the zero inside [1,1,0,1].

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

Implementation checkpoints:

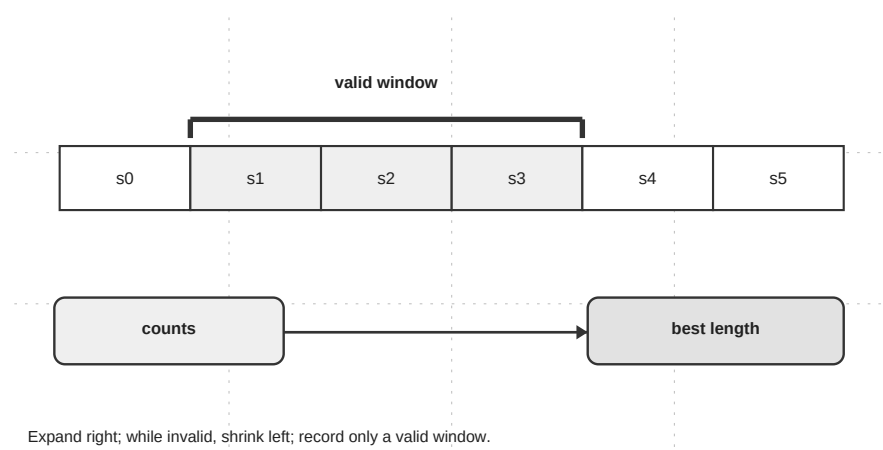
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1004. Max Consecutive Ones III
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Shrinking only once when a while loop is needed; stale max/count variables.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $k=0$; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left}+1$.

Variations and follow-ups

Three drills:

- V1: Switch from at-most- k to exactly- k constraints.
- V2: Return the smallest valid window instead of the largest.
- V3: Support multiple character/value constraints at once.

Follow-ups:

- Can you return the actual window, not only its length/count?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def max_consecutive_ones_iii(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1; left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 017 of 229: 1493. Longest Subarray of 1's After Deleting One Element

Pattern: Sliding Window | Sources: LC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Delete exactly one element from a binary array and return the longest remaining contiguous run of 1s.

Example input/output:

Input: nums=[1,1,0,1]

Output: 3

Why: Delete the zero to join three ones.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

Implementation checkpoints:

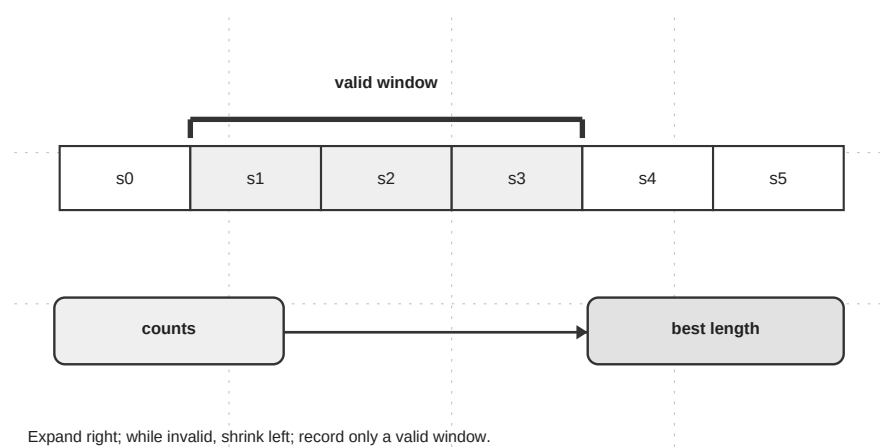
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1493. Longest Subarray of 1's After Deleting One Element
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Shrinking only once when a while loop is needed; stale max/count variables.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $k=0$; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left}+1$.

Variations and follow-ups

Three drills:

- V1: Switch from at-most- k to exactly- k constraints.
- V2: Return the smallest valid window instead of the largest.
- V3: Support multiple character/value constraints at once.

Follow-ups:

- Can you return the actual window, not only its length/count?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_subarray_of_1_s_after_deleti(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1; left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 018 of 229: 1732. Find the Highest Altitude

Pattern: Prefix / Scan | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Starting altitude is 0. Given net gains between points, return the highest altitude reached.

Example input/output:
Input: gain=[-5,1,5]

Output: 1

Why: Altitudes are 0,-5,-4,1.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recompute every left/right sum or product per index. This repeats work and often becomes $O(n^2)$.

Optimized approach:
Carry a running prefix/suffix value. Update answer at the exact point where left and right states are both known.

Alternative:
Two passes are simplest; one pass works when the equation can be rearranged.

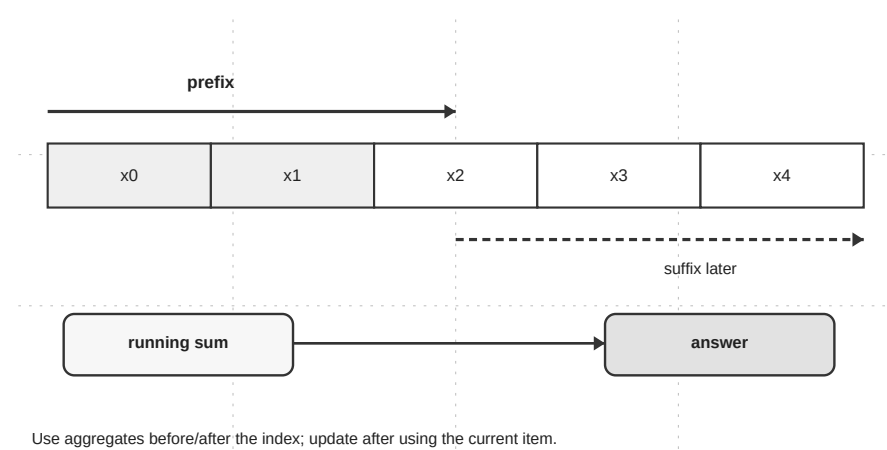
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1732. Find the Highest Altitude
func solvePrefixScan(nums []int) int {
    total := 0
    for _, x := range nums { total += x }
    prefix, ans := 0, defaultAnswer()
    for i, x := range nums {
        suffix := total - prefix - x
        if condition(prefix, x, suffix) {
            ans = updateAnswer(ans, i)
        }
        prefix += x // next index sees this x on its left
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before index i , prefix equals the aggregate of all elements strictly left of i .

Complexity target:
Time $O(n)$; Space $O(1)$ to $O(n)$

Why:
Each element is added/multiplied a constant number of times, so time is $O(n)$; space is $O(1)$ or $O(n)$ for output arrays.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Using updated prefix too early; not handling index 0.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
zeros; negative numbers; overflow in Go int; empty arrays; inclusive/exclusive index boundary.

Variations and follow-ups

Three drills:
V1: Answer many range queries after one preprocessing pass.
V2: Handle zeros/negative numbers without division.
V3: Return both value and index where the best prefix occurs.

Follow-ups:
- Can you answer many range queries after preprocessing?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_the_highest_altitude(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before index i , prefix equals the aggregate of all elements strictly left of i .

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 019 of 229: 724. Find Pivot Index

Pattern: Prefix / Scan | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Return an index where the sum of elements to its left equals the sum to its right, or -1 if none exists.

Example input/output:
Input: nums=[1,7,3,6,5,6]

Output: 3

Why: Left sum and right sum are both 11.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recompute every left/right sum or product per index. This repeats work and often becomes $O(n^2)$.

Optimized approach:
Carry a running prefix/suffix value. Update answer at the exact point where left and right states are both known.

Alternative:
Two passes are simplest; one pass works when the equation can be rearranged.

Implementation checkpoints:

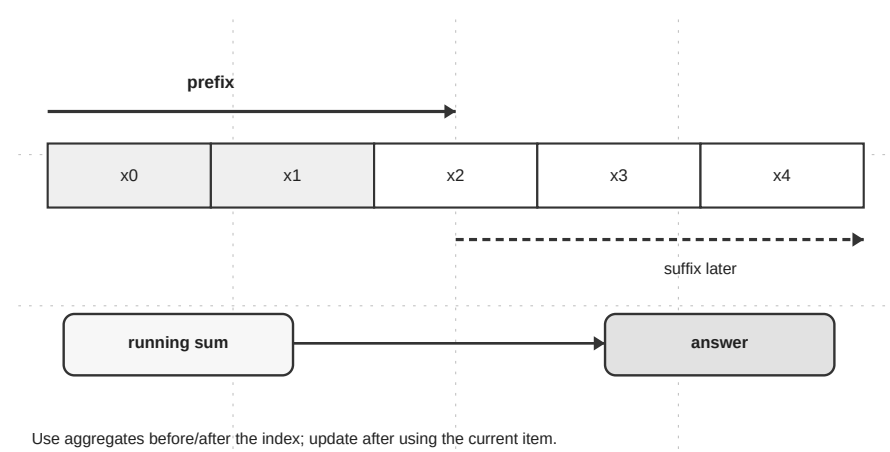
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 724. Find Pivot Index
func solvePrefixScan(nums []int) int {
    total := 0
    for _, x := range nums { total += x }
    prefix, ans := 0, defaultAnswer()
    for i, x := range nums {
        suffix := total - prefix - x
        if condition(prefix, x, suffix) {
            ans = updateAnswer(ans, i)
        }
        prefix += x // next index sees this x on its left
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before index i , prefix equals the aggregate of all elements strictly left of i .

Complexity target:
Time $O(n)$; Space $O(1)$ to $O(n)$

Why:
Each element is added/multiplied a constant number of times, so time is $O(n)$; space is $O(1)$ or $O(n)$ for output arrays.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using updated prefix too early; not handling index 0.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
zeros; negative numbers; overflow in Go int; empty arrays; inclusive/exclusive index boundary.

Variations and follow-ups

Three drills:

- V1: Answer many range queries after one preprocessing pass.
- V2: Handle zeros/negative numbers without division.
- V3: Return both value and index where the best prefix occurs.

Follow-ups:

- Can you answer many range queries after preprocessing?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_pivot_index(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before index i , prefix equals the aggregate of all elements strictly left of i .

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 020 of 229: 2215. Find the Difference of Two Arrays

Pattern: Hash Map / Set | Sources: LC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return two lists: values present in nums1 but not nums2, and values present in nums2 but not nums1.

Example input/output:
Input: nums1=[1,2,3], nums2=[2,4]

Output: [[1,3],[4]]

Why: Only distinct missing values are returned.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

Implementation checkpoints:

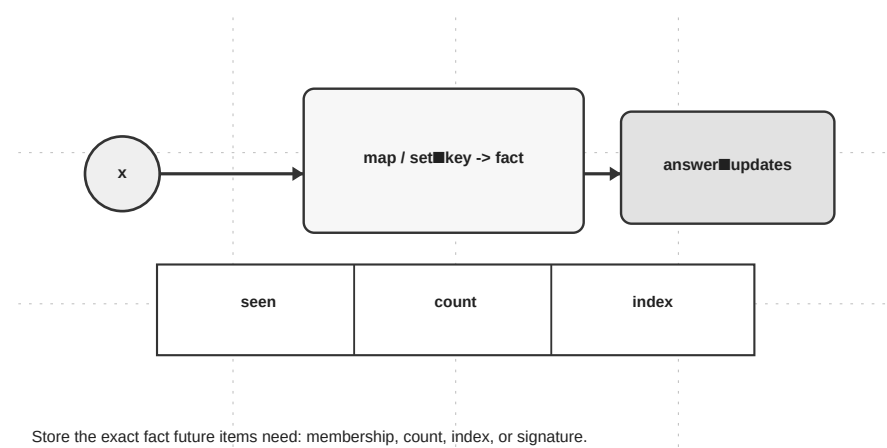
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2215. Find the Difference of Two Arrays
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:

- V1: Return all matching groups instead of a boolean/count.
- V2: Support incremental updates while queries arrive.
- V3: Handle case-insensitive or normalized keys.

Follow-ups:

- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_the_difference_of_two_arrays(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 021 of 229: 1207. Unique Number of Occurrences

Pattern: Hash Map / Set | Sources: LC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return true if every distinct value in the array has a distinct occurrence count.

Example input/output:
Input: nums=[1,2,2,3,3,3]

Output: true

Why: Counts are 1,2,3, all unique.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

Implementation checkpoints:

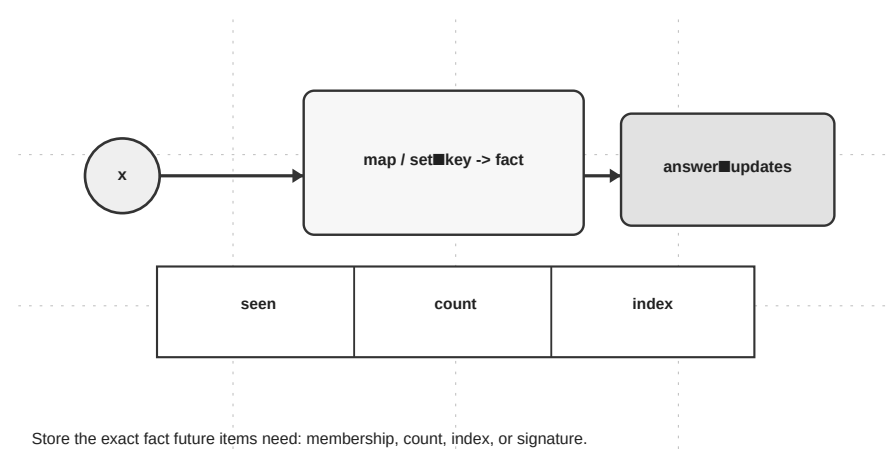
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1207. Unique Number of Occurrences
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: `left <= right?` `i < n?`
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:

- V1: Return all matching groups instead of a boolean/count.
- V2: Support incremental updates while queries arrive.
- V3: Handle case-insensitive or normalized keys.

Follow-ups:

- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def unique_number_of_occurrences(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 022 of 229: 1657. Determine if Two Strings Are Close

Pattern: Hash Map / Set | Sources: LC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Two strings are close if one can be transformed into the other by swapping positions and globally renaming existing characters.

Example input/output:
Input: word1="abbzzca", word2="babzzcz"

Output: true

Why: They share characters and the multiset of frequencies.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

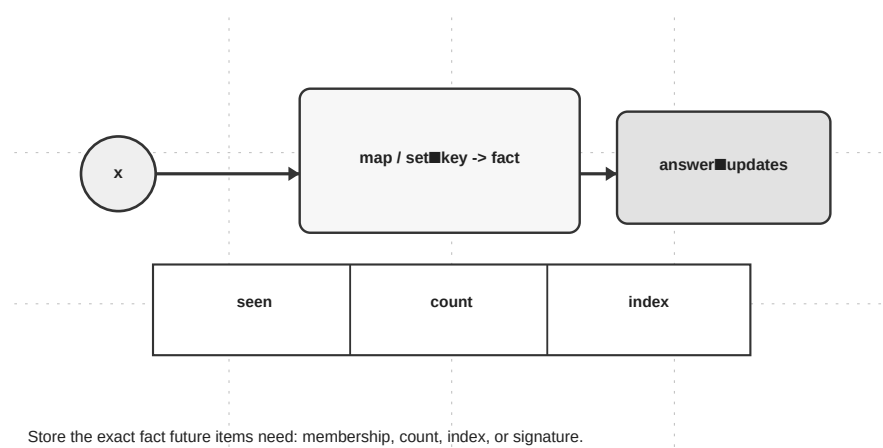
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1657. Determine if Two Strings Are Close
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

- Three drills:
- V1: Return all matching groups instead of a boolean/count.
 - V2: Support incremental updates while queries arrive.
 - V3: Handle case-insensitive or normalized keys.

- Follow-ups:
- How would memory limits change the approach?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def determine_if_two_strings_are_close(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 023 of 229: 2352. Equal Row and Column Pairs

Pattern: Hash Map / Set | Sources: LC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Count pairs (row i, column j) where the row values exactly equal the column values.

Example input/output:
Input: grid=[[3,1,2],[1,7,6],[2,6,7]]

Output: 1

Why: Row [1,7,6] equals column 1.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

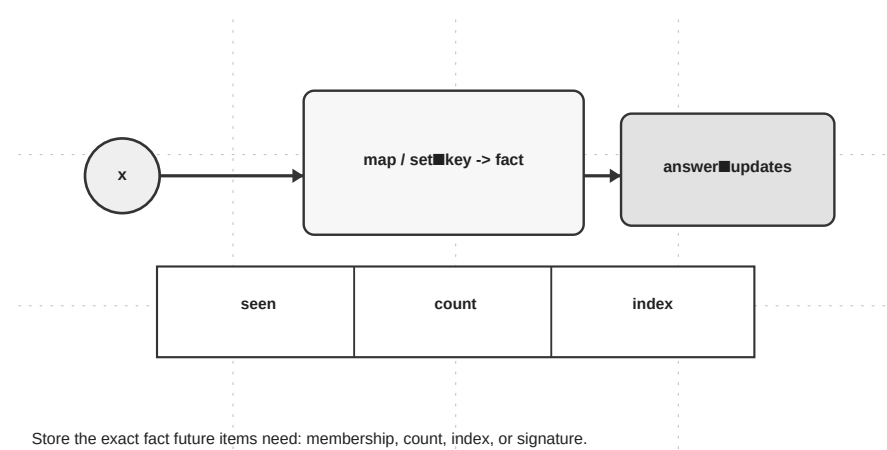
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2352. Equal Row and Column Pairs
func solveWithMap(nums [][]int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

- Three drills:
- V1: Return all matching groups instead of a boolean/count.
 - V2: Support incremental updates while queries arrive.
 - V3: Handle case-insensitive or normalized keys.

- Follow-ups:
- How would memory limits change the approach?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def equal_row_and_column_pairs(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 024 of 229: 2390. Removing Stars From a String

Pattern: Stack | Sources: LC75 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Process a string where each star deletes itself and the closest remaining character to its left.

Example input/output:
Input: s="leet**cod*e"
Output: "lecoe"

Output: "lecoe"

Why: Each star removes one previous character.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become $O(n^2)$.

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

Implementation checkpoints:

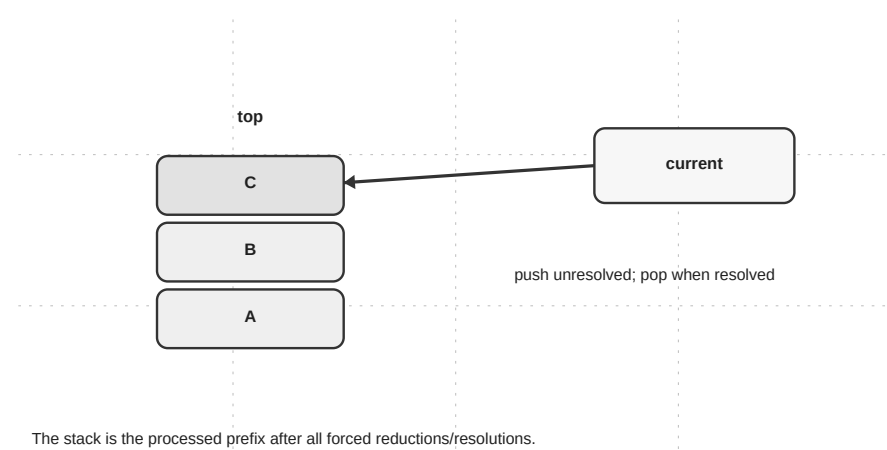
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2390. Removing Stars From a String
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Each item is pushed and popped at most once, so $O(n)$ time and $O(n)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Popping empty stack; forgetting to flush remaining stack entries.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

Three drills:

- V1: Also return the position of the unmatched item.
- V2: Handle nested operators or multiple bracket types.
- V3: Convert next-greater logic into previous-greater logic.

Follow-ups:

- Can the stack be compressed to store counts or indices only?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def removing_stars_from_a_string(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 025 of 229: 735. Asteroid Collision

Pattern: Stack | Sources: LC75 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Simulate asteroid collisions: positive moves right, negative moves left, and smaller colliding asteroids explode.

Example input/output:
Input: asteroids=[5,10,-5]

Output: [5,10]

Why: The -5 collides with 10 and disappears.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become $O(n^2)$.

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

Implementation checkpoints:

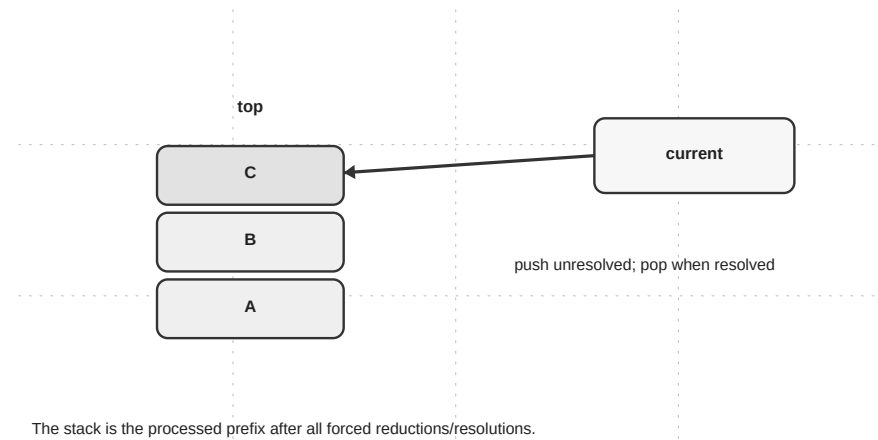
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 735. Asteroid Collision
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Each item is pushed and popped at most once, so $O(n)$ time and $O(n)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Popping empty stack; forgetting to flush remaining stack entries.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

Three drills:

- V1: Also return the position of the unmatched item.
- V2: Handle nested operators or multiple bracket types.
- V3: Convert next-greater logic into previous-greater logic.

Follow-ups:

- Can the stack be compressed to store counts or indices only?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def asteroid_collision(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 026 of 229: 394. Decode String

Pattern: Stack | Sources: LC75 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Decode a string with patterns k[encoded], including nested patterns, by repeating encoded exactly k times.

Example input/output:

Input: s="3[a2[c]]"

Output: "accaccacc"

Why: "a2[c]" becomes "acc", repeated three times.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become O(n^2).

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

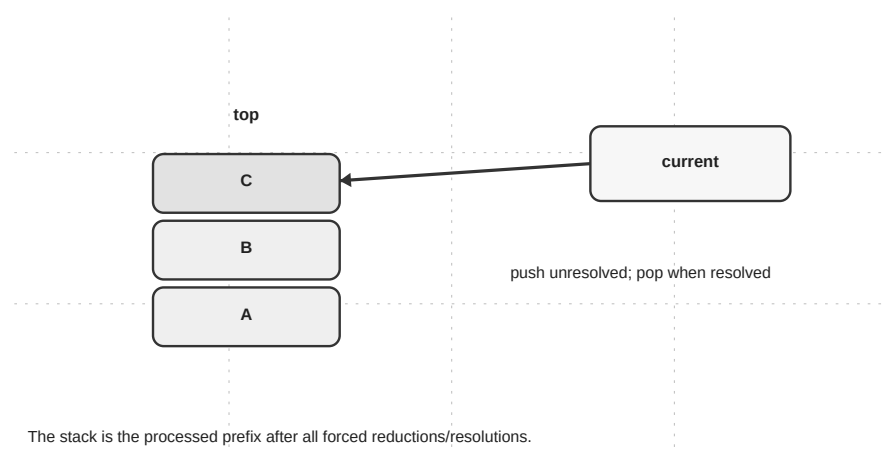
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 394. Decode String
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time O(n); Space O(n)

Why:
Each item is pushed and popped at most once, so O(n) time and O(n) space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Popping empty stack; forgetting to flush remaining stack entries.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

Three drills:
V1: Also return the position of the unmatched item.
V2: Handle nested operators or multiple bracket types.
V3: Convert next-greater logic into previous-greater logic.

Follow-ups:
- Can the stack be compressed to store counts or indices only?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def decode_string(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 027 of 229: 933. Number of Recent Calls

Pattern: Queue / Simulation | Sources: LC75 | Companies: Amazon, Google, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Implement RecentCounter.ping(t), returning how many ping times are in the inclusive range [t-3000,t].

Example input/output:
Input: pings=[1,100,3001,3002]

Output: [1,2,3,3]

Why: Only recent timestamps remain in the queue.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Simulate with repeated full scans of all active entities; too slow when many rounds occur.

Optimized approach:
Use a queue/deque so each event is processed in the order it becomes eligible.

Alternative:
Use two queues when entities belong to two groups competing over time.

Implementation checkpoints:

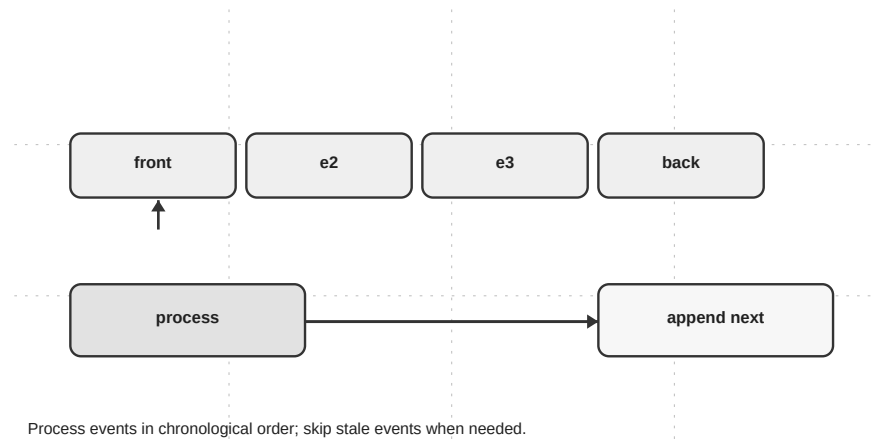
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 933. Number of Recent Calls
func solveQueueSimulation(events []int) int {
    q := append([]int(nil), events...)
    answer := 0
    for len(q) > 0 {
        cur := q[0]; q = q[1:]
        if isStale(cur) { continue }
        answer = update(answer, cur)
        if shouldRepeat(cur) { q = append(q, nextEvent(cur)) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The queue contains pending events in exact chronological order.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Each event is enqueued/dequeued a bounded number of times, usually $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Mixing one round with the next; not updating queue length per layer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty queue; stale events; repeated timestamps; stopping condition; modulo/cyclic turns.

Variations and follow-ups

Three drills:

- V1: Add timestamps and expire old events.
- V2: Process simultaneous events in stable order.
- V3: Return intermediate states after each round.

Follow-ups:

- What if events are timestamped or arrive online?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def number_of_recent_calls(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The queue contains pending events in exact chronological order.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 028 of 229: 649. Dota2 Senate

Pattern: Queue / Simulation | Sources: LC75 | Companies: Amazon, Google, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Given a string of R and D senators, simulate bans in order until only one party can act.

Example input/output:
Input: senate="RDD"

Output: "Dire"

Why: Dire senators eventually ban the remaining Radiant senator.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Simulate with repeated full scans of all active entities; too slow when many rounds occur.

Optimized approach:
Use a queue/deque so each event is processed in the order it becomes eligible.

Alternative:
Use two queues when entities belong to two groups competing over time.

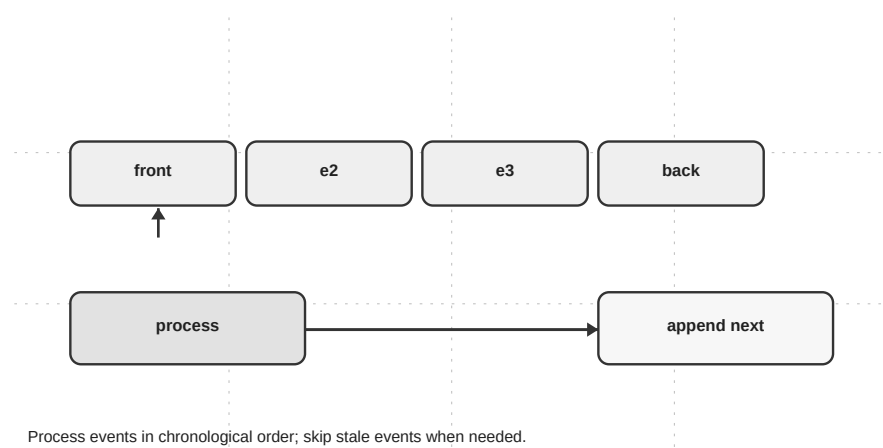
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 649. Dota2 Senate
func solveQueueSimulation(events []int) int {
    q := append([]int(nil), events...)
    answer := 0
    for len(q) > 0 {
        cur := q[0]; q = q[1:]
        if isStale(cur) { continue }
        answer = update(answer, cur)
        if shouldRepeat(cur) { q = append(q, nextEvent(cur)) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The queue contains pending events in exact chronological order.

Complexity target:
Time O(n); Space O(n)

Why:
Each event is enqueued/dequeued a bounded number of times, usually O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Mixing one round with the next; not updating queue length per layer.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty queue; stale events; repeated timestamps; stopping condition; modulo/cyclic turns.

Variations and follow-ups

- Three drills:
- V1: Add timestamps and expire old events.
 - V2: Process simultaneous events in stable order.
 - V3: Return intermediate states after each round.

- Follow-ups:
- What if events are timestamped or arrive online?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def dota2_senate(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The queue contains pending events in exact chronological order.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 029 of 229: 2095. Delete the Middle Node of a Linked List

Pattern: Linked List | Sources: LC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Delete the middle node of a singly linked list, where middle index is $\text{floor}(n/2)$, and return the head.

Example input/output:
Input: head=[1,3,4,7,1,2,6]

Output: [1,3,4,1,2,6]

Why: Node value 7 at index 3 is removed.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

Implementation checkpoints:

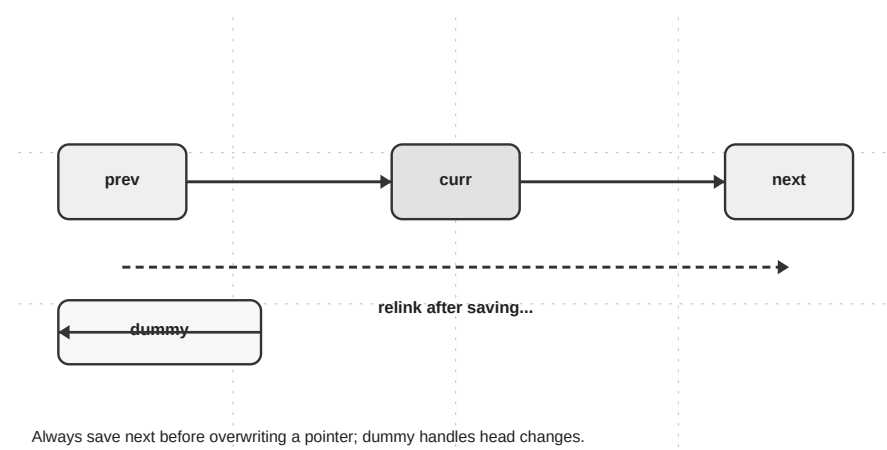
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2095. Delete the Middle Node of a Linked List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:

- V1: Do the operation recursively and compare stack use.
- V2: Handle cycle/random pointers in the same structure.
- V3: Perform the change for every k-sized group.

Follow-ups:

- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def delete_the_middle_node_of_a_linked_l(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 030 of 229: 328. Odd Even Linked List

Pattern: Linked List | Sources: LC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Group nodes at odd positions first, then nodes at even positions, preserving relative order within each group.

Example input/output:
Input: head=[1,2,3,4,5]

Output: [1,3,5,2,4]

Why: Positions 1,3,5 come before 2,4.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

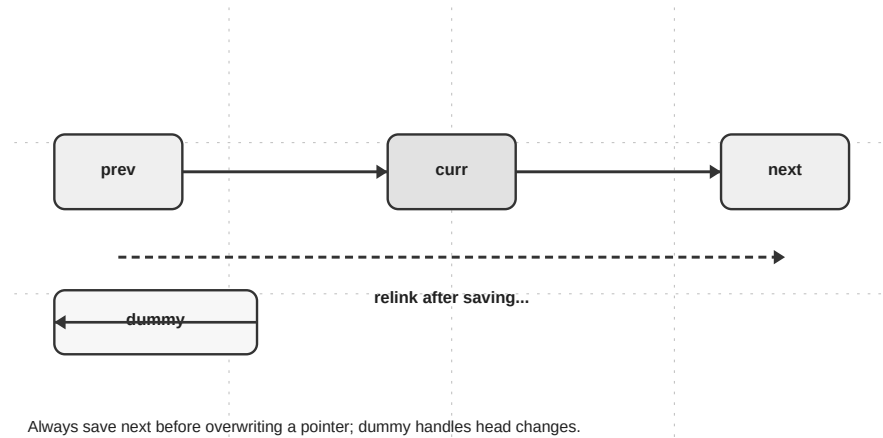
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 328. Odd Even Linked List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:
V1: Do the operation recursively and compare stack use.
V2: Handle cycle/random pointers in the same structure.
V3: Perform the change for every k-sized group.

Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def odd_even_linked_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 031 of 229: 206. Reverse Linked List

Pattern: Linked List | Sources: LC75, NC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: $O(n)$

Space: $O(1)$ unless recursion/map

Problem statement + sample

Prompt:
Reverse a singly linked list and return the new head.

Example input/output:
Input: head=[1,2,3]

Output: [3,2,1]

Why: All next pointers are reversed.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra $O(n)$ space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use $O(n)$ call stack.

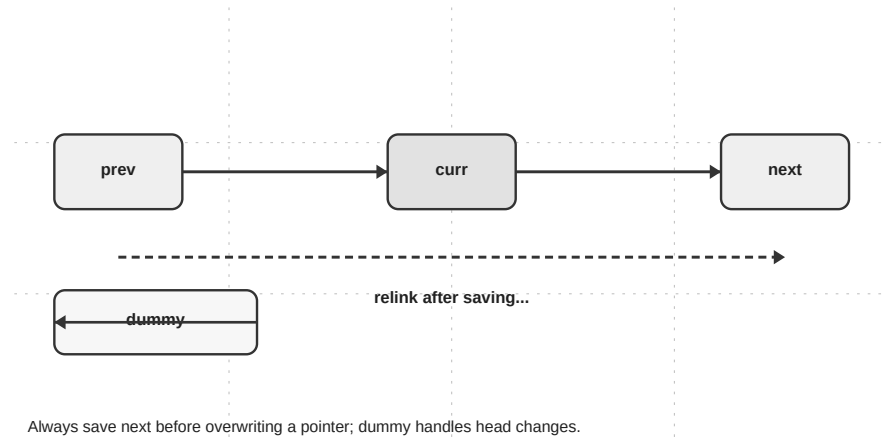
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 206. Reverse Linked List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time $O(n)$; Space $O(1)$ unless recursion/map

Why:
Most list pointer solutions are one or two passes: $O(n)$ time, $O(1)$ extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with $O(1)$ extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def reverse_linked_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 032 of 229: 2130. Maximum Twin Sum of a Linked List

Pattern: Linked List | Sources: LC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
For an even-length linked list, pair the i-th node with the symmetric node from the end and return the maximum twin sum.

Example input/output:
Input: head=[5,4,2,1]

Output: 6

Why: Twin sums are 5+1 and 4+2.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

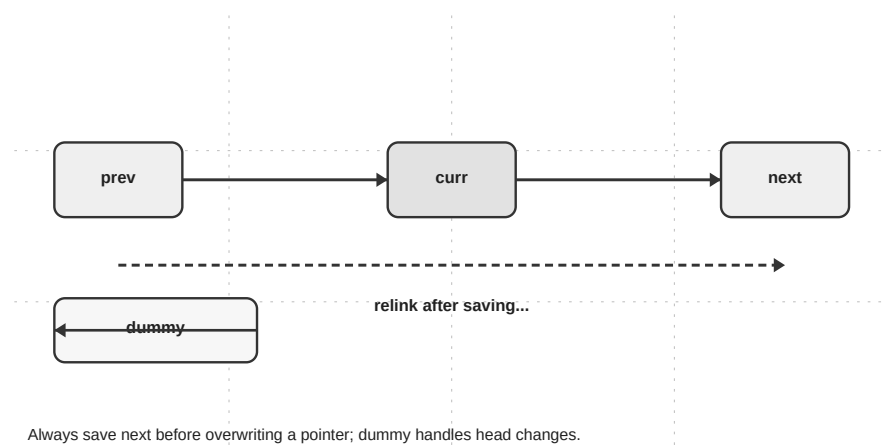
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2130. Maximum Twin Sum of a Linked List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:
V1: Do the operation recursively and compare stack use.
V2: Handle cycle/random pointers in the same structure.
V3: Perform the change for every k-sized group.

Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def maximum_twin_sum_of_a_linked_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 033 of 229: 104. Maximum Depth of Binary Tree

Pattern: Tree DFS | Sources: LC75, Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Return the maximum number of nodes on any root-to-leaf path in a binary tree.

Example input/output:
Input: root=[3,9,20,null,null,15,7]

Output: 3

Why: The longest path has three nodes.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

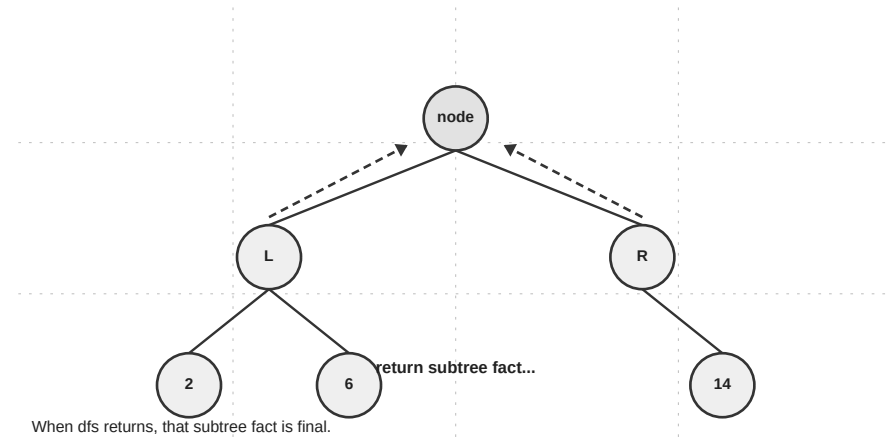
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 104. Maximum Depth of Binary Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def maximum_depth_of_binary_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 034 of 229: 872. Leaf-Similar Trees

Pattern: Tree DFS | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return true if two binary trees have the same leaf values in left-to-right order.

Example input/output:
Input: root1=[1,2,3], root2=[7,2,3]

Output: true

Why: Both leaf sequences are [2,3].

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

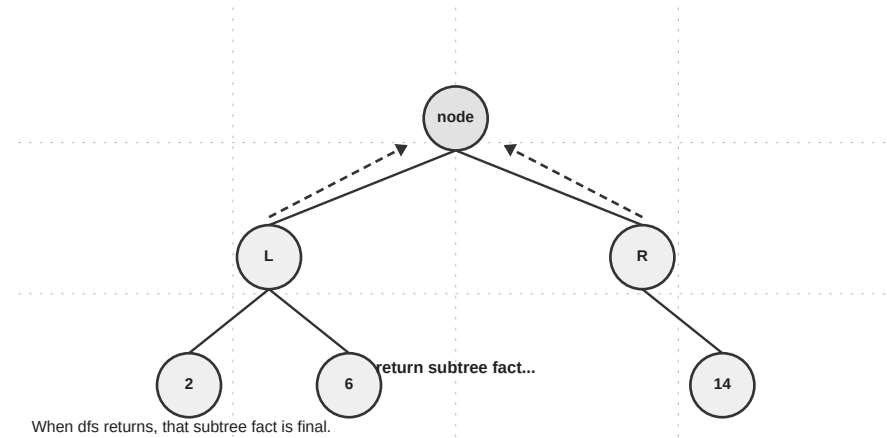
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 872. Leaf-Similar Trees
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:
V1: Return the nodes on the path, not just a count/value.
V2: Make the tree contain negative values or duplicate values.
V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:
- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def leaf_similar_trees(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 035 of 229: 1448. Count Good Nodes in Binary Tree

Pattern: Tree DFS | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Count nodes whose value is at least every value on the path from the root to that node.

Example input/output:
Input: root=[3,1,4,3,null,1,5]

Output: 4

Why: Values 3,3,4,5 are good nodes.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

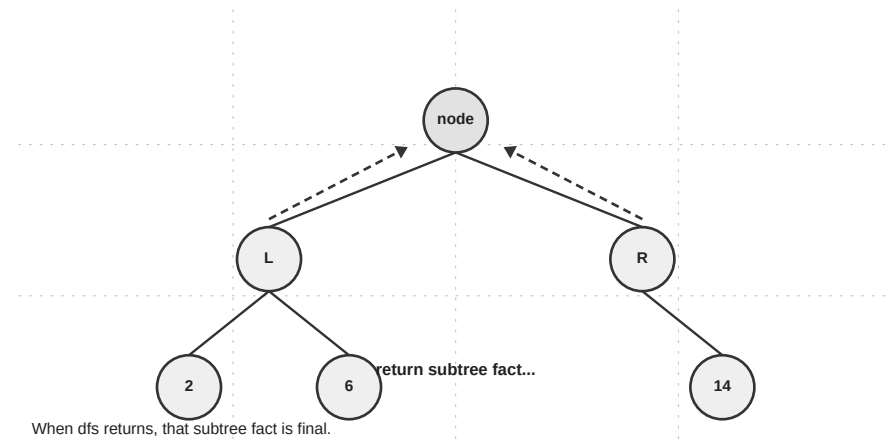
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1448. Count Good Nodes in Binary Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def count_good_nodes_in_binary_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 036 of 229: 437. Path Sum III

Pattern: Tree DFS | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Count all downward paths whose node values sum to targetSum; a path may start and end anywhere but must go parent to child.

Example input/output:
Input: root=[10,5,-3,3,2,null,11,3,-2,null,1], targetSum=8

Output: 3

Why: There are three downward paths summing to 8.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

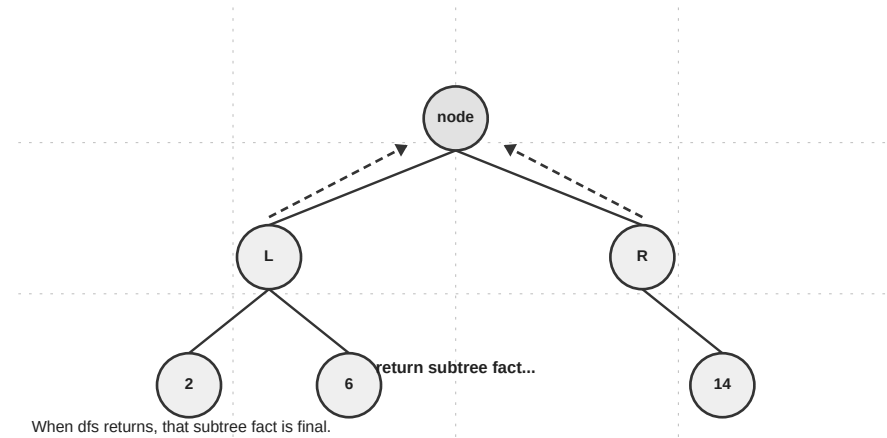
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 437. Path Sum III
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def path_sum_iii(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 037 of 229: 1372. Longest ZigZag Path in a Binary Tree

Pattern: Tree DFS | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return the maximum number of edges in a path that alternates left and right child moves at each step.

Example input/output:
Input: path 1->right 2->left 3->right 4

Output: 3

Why: The shown path has three alternating edges.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

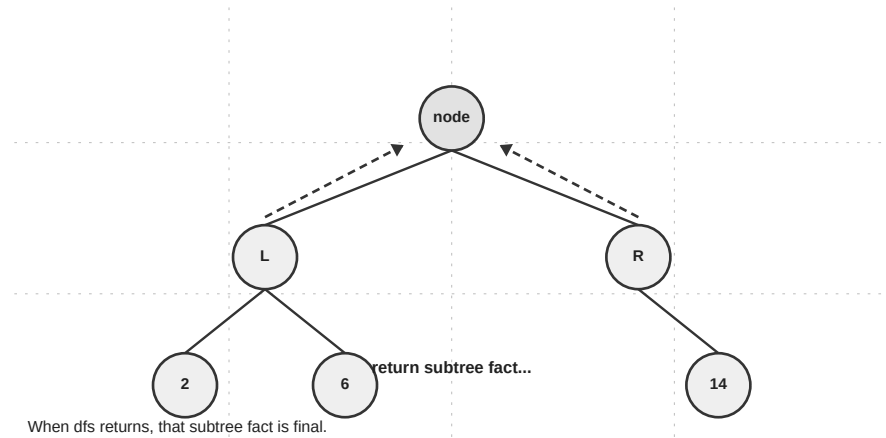
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1372. Longest ZigZag Path in a Binary Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_zigzag_path_in_a_binary_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 038 of 229: 236. Lowest Common Ancestor of a Binary Tree

Pattern: Tree DFS | Sources: LC75, Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return the lowest node in a binary tree that has both p and q as descendants, where a node can be its own descendant.

Example input/output:
Input: root=[3,5,1,6,2,0,8,null,null,7,4], p=5, q=1

Output: 3

Why: Node 3 is the lowest common ancestor.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

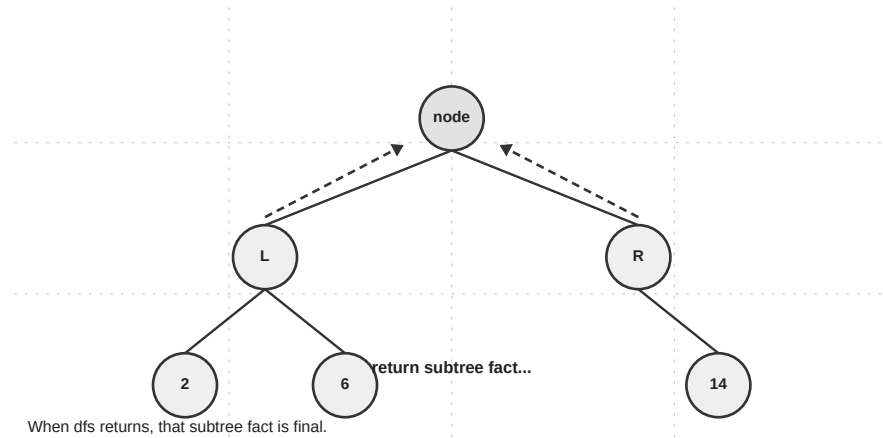
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 236. Lowest Common Ancestor of a Binary Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def lowest_common_ancestor_of_a_binary_t(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 039 of 229: 199. Binary Tree Right Side View

Pattern: Tree BFS | Sources: LC75, Top150 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: O(n)

Space: O(w)

Problem statement + sample

Prompt:
Return the values visible when viewing a binary tree from the right side, top to bottom.

Example input/output:
Input: root=[1,2,3,null,5,null,4]

Output: [1,3,4]

Why: The rightmost node at each level is selected.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
DFS with depth maps or repeated level searches works but is messier.

Optimized approach:
Queue nodes level by level; record level size before processing so levels do not mix.

Alternative:
DFS can collect depth buckets if recursion is preferred.

Implementation checkpoints:

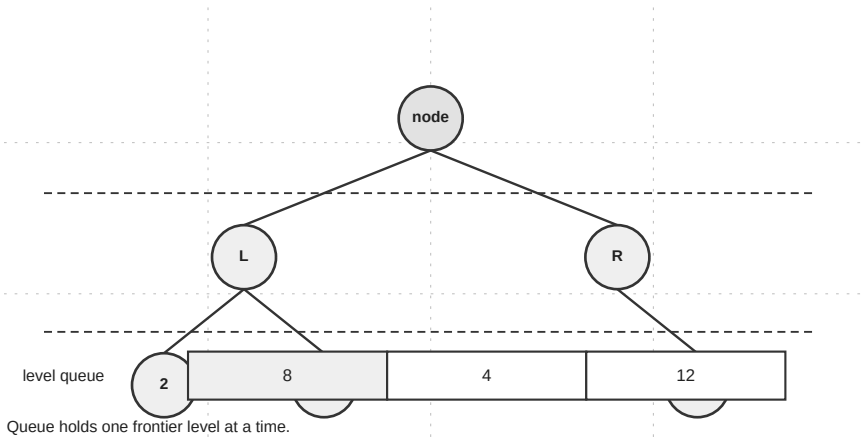
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 199. Binary Tree Right Side View
func solveTreeBFS(root *TreeNode) []int {
    if root == nil { return nil }
    q, ans := []*TreeNode{root}, []int{}
    for len(q) > 0 {
        levelSize := len(q) // freezes current level
        levelValue := 0
        for i := 0; i < levelSize; i++ {
            node := q[0]; q = q[1:]
            levelValue = updateLevel(levelValue, node.Val, i)
            if node.Left != nil { q = append(q, node.Left) }
            if node.Right != nil { q = append(q, node.Right) }
        }
        ans = append(ans, levelValue)
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At the start of each outer loop, queue contains exactly one level.

Complexity target:
Time O(n); Space O(w)

Why:
O(n) time; O(w) queue space where w is tree width.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not separating levels; queueing nil children unnecessarily.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; last node per level; zigzag order; integer averages; queue memory at widest level.

Variations and follow-ups

Three drills:

- V1: Return zigzag/order statistics per level.
- V2: Use right-to-left priority instead of left-to-right.
- V3: Limit search to the nearest target level.

Follow-ups:

- Can you stream one level at a time without storing all levels?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def binary_tree_right_side_view(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At the start of each outer loop, queue contains exactly one level.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 040 of 229: 1161. Maximum Level Sum of a Binary Tree

Pattern: Tree BFS | Sources: LC75 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: $O(n)$

Space: $O(w)$

Problem statement + sample

Prompt:
Return the 1-indexed level of a binary tree with the maximum sum of node values.

Example input/output:
Input: root=[1,7,0,7,-8,null,null]

Output: 2

Why: Level 2 sum is 7, larger than other levels.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
DFS with depth maps or repeated level searches works but is messier.

Optimized approach:
Queue nodes level by level; record level size before processing so levels do not mix.

Alternative:
DFS can collect depth buckets if recursion is preferred.

Implementation checkpoints:

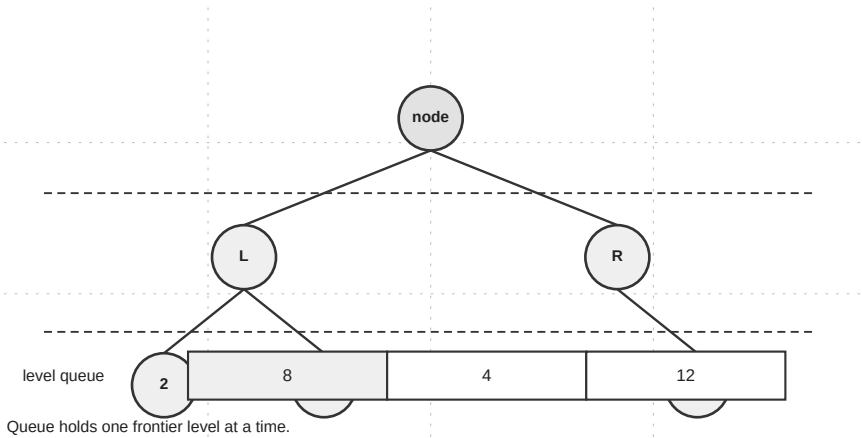
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1161. Maximum Level Sum of a Binary Tree
func solveTreeBFS(root *TreeNode) []int {
    if root == nil { return nil }
    q, ans := []*TreeNode{root}, []int{}
    for len(q) > 0 {
        levelSize := len(q) // freezes current level
        levelValue := 0
        for i := 0; i < levelSize; i++ {
            node := q[0]; q = q[1:]
            levelValue = updateLevel(levelValue, node.Val, i)
            if node.Left != nil { q = append(q, node.Left) }
            if node.Right != nil { q = append(q, node.Right) }
        }
        ans = append(ans, levelValue)
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At the start of each outer loop, queue contains exactly one level.

Complexity target:
Time $O(n)$; Space $O(w)$

Why:
 $O(n)$ time; $O(w)$ queue space where w is tree width.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not separating levels; queueing nil children unnecessarily.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; last node per level; zigzag order; integer averages; queue memory at widest level.

Variations and follow-ups

Three drills:

- V1: Return zigzag/order statistics per level.
- V2: Use right-to-left priority instead of left-to-right.
- V3: Limit search to the nearest target level.

Follow-ups:

- Can you stream one level at a time without storing all levels?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def maximum_level_sum_of_a_binary_tree(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At the start of each outer loop, queue contains exactly one level.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 041 of 229: 700. Search in a Binary Search Tree

Pattern: BST | Sources: LC75 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: $O(h)$ to $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Search a binary search tree for val and return the subtree rooted at that node, or null if absent.

Example input/output:
Input: root=[4,2,7,1,3], val=2

Output: [2,1,3]

Why: The node with value 2 is found.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives $O(h)$ memory and supports lazy iterators.

Implementation checkpoints:

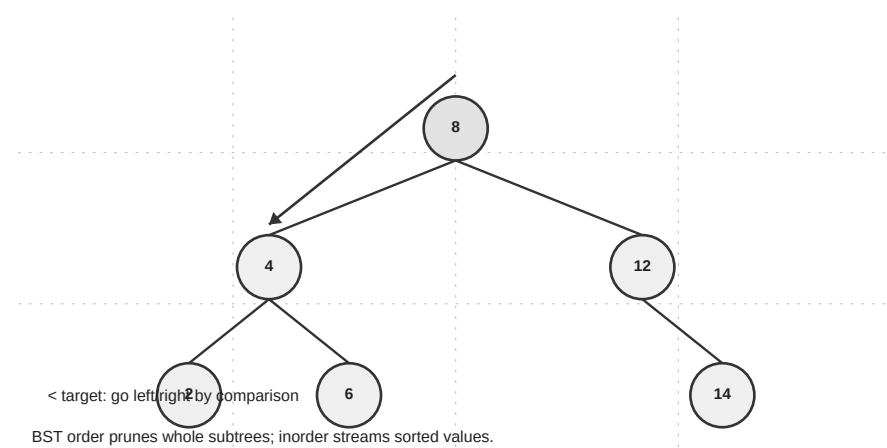
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 700. Search in a Binary Search Tree
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time $O(h)$ to $O(n)$; Space $O(h)$

Why:
Balanced operations are $O(h)=O(\log n)$; worst-case skewed tree is $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using <= bounds incorrectly; not preserving lower and upper constraints.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

Three drills:

- V1: Support duplicate values with a clear side policy.
- V2: Convert to an iterative bounded search.
- V3: Return predecessor/successor around a target.

Follow-ups:

- Can you solve it with inorder iteration and $O(h)$ memory?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def search_in_a_binary_search_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 042 of 229: 450. Delete Node in a BST

Pattern: BST | Sources: LC75 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: O(h) to O(n)

Space: O(h)

Problem statement + sample

Prompt:
Delete a key from a binary search tree and return a valid BST root after deletion.

Example input/output:
Input: root=[5,3,6,2,4,null,7], key=3

Output: [5,4,6,2,null,null,7]

Why: One valid result replaces 3 with its successor 4.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives O(h) memory and supports lazy iterators.

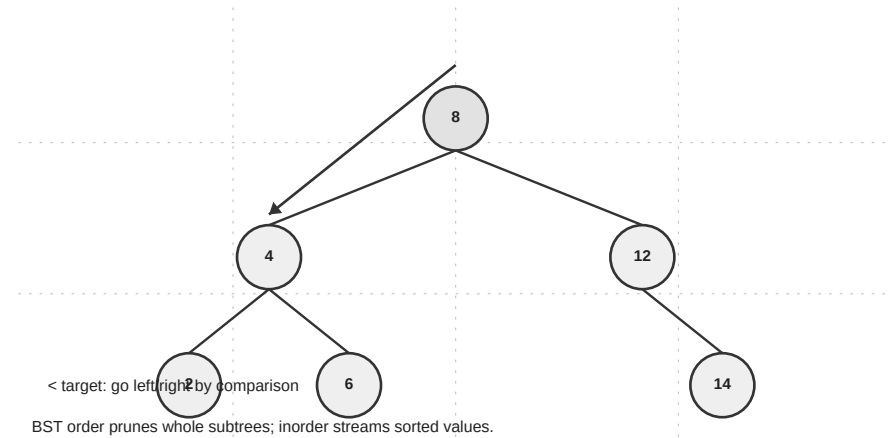
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 450. Delete Node in a BST
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time O(h) to O(n); Space O(h)

Why:
Balanced operations are O(h)=O(log n); worst-case skewed tree is O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using <= bounds incorrectly; not preserving lower and upper constraints.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

- Three drills:
- V1: Support duplicate values with a clear side policy.
 - V2: Convert to an iterative bounded search.
 - V3: Return predecessor/successor around a target.

- Follow-ups:
- Can you solve it with inorder iteration and O(h) memory?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def delete_node_in_a_bst(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 043 of 229: 841. Keys and Rooms

Pattern: Graph | Sources: LC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Each room may contain keys to other rooms. Starting in room 0, return whether all rooms can be visited.

Example input/output:
Input: rooms=[[1],[2],[3],[]]

Output: true

Why: Keys unlock rooms 1,2,3 in order.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

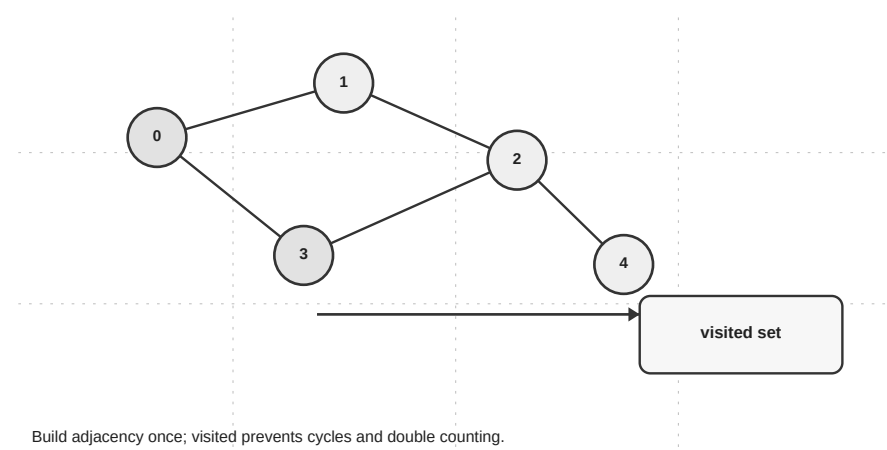
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 841. Keys and Rooms
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def keys_and_rooms(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 044 of 229: 547. Number of Provinces

Pattern: Graph | Sources: LC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Given an adjacency matrix of directly connected cities, return the number of connected components.

Example input/output:
Input: isConnected=[[1,1,0],[1,1,0],[0,0,1]]

Output: 2

Why: Cities 0 and 1 form one province; city 2 another.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

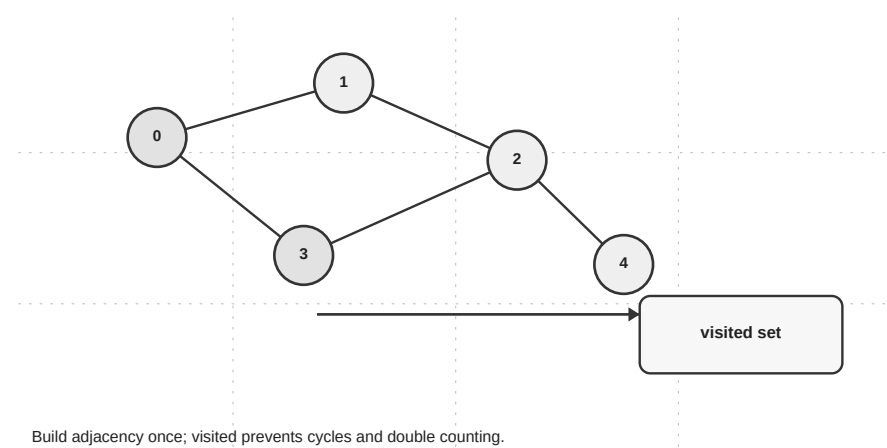
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 547. Number of Provinces
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def number_of_provinces(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 045 of 229: 1466. Reorder Routes to Make All Paths Lead to the City Zero

Pattern: Graph | Sources: LC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Directed roads form a tree. Count the minimum roads to reverse so every city can reach city 0.

Example input/output:
Input: $n=3$, $\text{connections}=[[1,0],[1,2]]$

Output: 1

Why: Reverse road $1 \rightarrow 2$ so city 2 can reach 0.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

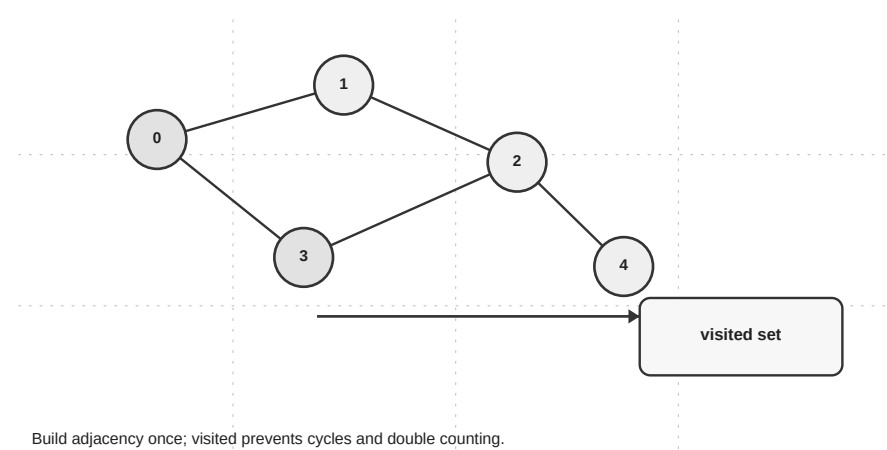
Annotated Go kernel - comments explain the reasoning

```
// 1466. Reorder Routes to Make All Paths Lead to the City Zero
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}
```

Explanation checklist:

- // - Initialization creates the invariant.
- // - Each loop consumes one new fact or safely discards one candidate.
- // - Helper predicates stand for exact prompt logic.
- // - Use int64 when sums/products can overflow int.
- // - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def reorder_routes_to_make_all_paths_lead_to_city_zero(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 046 of 229: 399. Evaluate Division

Pattern: Graph | Sources: LC75, Top150 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Given equations $a/b = \text{value}$, answer division queries or -1.0 when variables are disconnected or unknown.

Example input/output:
Input: equations=[["a","b"],["b","c"]], values=[2,3], queries=[["a","c"],["c","a"],["x","y"]]

Output: [6.0,0.16667,-1.0]

Why: Build a weighted graph of ratios.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

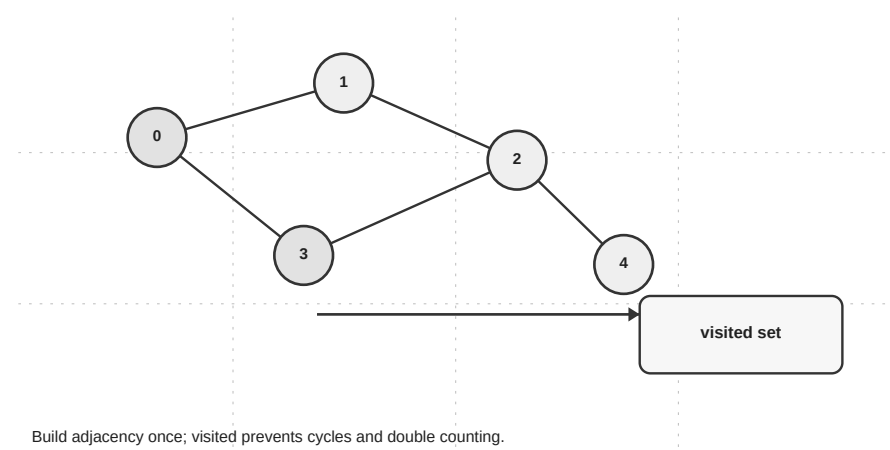
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 399. Evaluate Division
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def evaluate_division(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 047 of 229: 1926. Nearest Exit from Entrance in Maze

Pattern: Graph | Sources: LC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
In a maze of walls plus and open dots, return the shortest steps from entrance to any boundary exit not equal to entrance.

Example input/output:
Input: maze=[["+","+",",",""],[".",",",",",",""],["+",",",",", "+"]], entrance=[1,0]

Output: 2

Why: Move to [1,2], a boundary exit.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

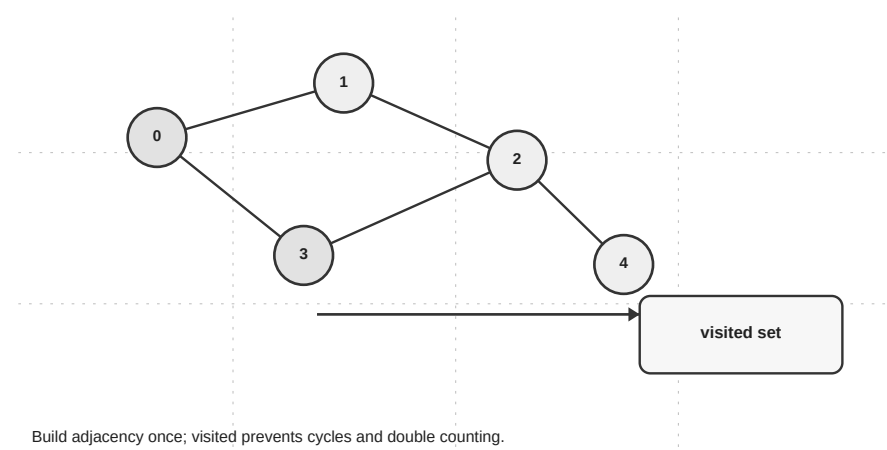
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1926. Nearest Exit from Entrance in Maze
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def nearest_exit_from_entrance_in_maze(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 048 of 229: 994. Rotting Oranges

Pattern: Graph | Sources: LC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Each minute, rotten oranges make adjacent fresh oranges rotten. Return minutes until all fresh rot, or -1 if impossible.

Example input/output:
Input: grid=[[2,1,1],[1,1,0],[0,1,1]]

Output: 4

Why: BFS spreads rot level by level.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

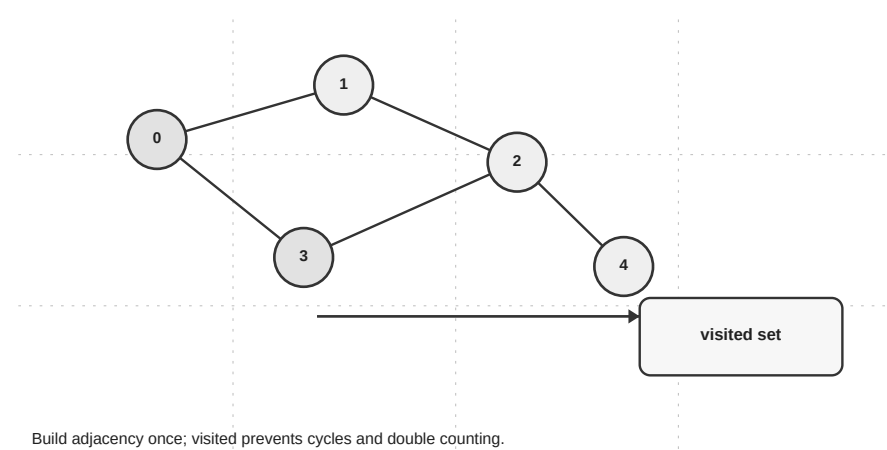
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 994. Rotting Oranges
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def rotting_oranges(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 049 of 229: 215. Kth Largest Element in an Array

Pattern: Heap / Priority Queue | Sources: LC75, Top150 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the k-th largest element in an unsorted array, not the k-th distinct largest.

Example input/output:
Input: nums=[3,2,1,5,6,4], k=2

Output: 5

Why: Sorted descending is [6,5,4,3,2,1].

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

Implementation checkpoints:

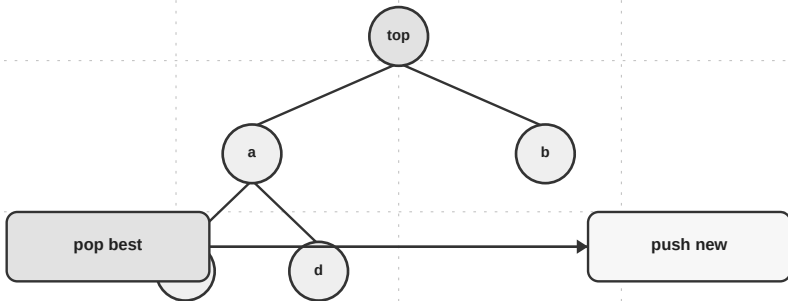
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 215. Kth Largest Element in an Array
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

Three drills:

- V1: Return top-k in sorted order after using the heap.
- V2: Handle ties with a secondary key.
- V3: Support streaming insertions and deletions.

Follow-ups:

- Can you reduce memory from $O(n)$ to $O(k)$?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def kth_largest_element_in_an_array(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 050 of 229: 2336. Smallest Number in Infinite Set

Pattern: Heap / Priority Queue | Sources: LC75 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Design a set initially containing all positive integers, supporting popSmallest and addBack.

Example input/output:
Input: operations=pop,pop,addBack(1),pop

Output: [1,2,null,1]

Why: Adding 1 back makes it smallest again.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

Implementation checkpoints:

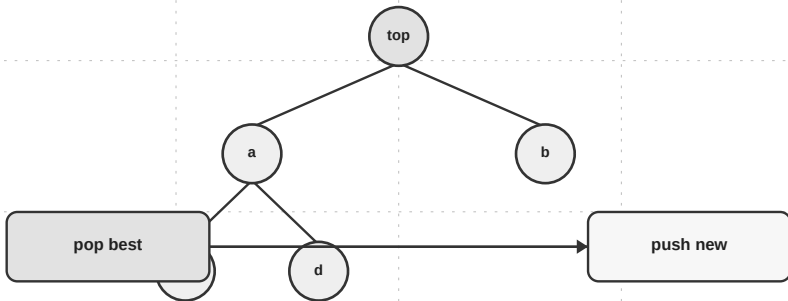
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2336. Smallest Number in Infinite Set
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

Three drills:

- V1: Return top-k in sorted order after using the heap.
- V2: Handle ties with a secondary key.
- V3: Support streaming insertions and deletions.

Follow-ups:

- Can you reduce memory from $O(n)$ to $O(k)$?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def smallest_number_in_infinite_set(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 051 of 229: 2542. Maximum Subsequence Score

Pattern: Heap / Priority Queue | Sources: LC75 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Choose k indices to maximize $\text{sum}(\text{nums1 chosen})$ multiplied by the minimum nums2 among the chosen indices.

Example input/output:
Input: $\text{nums1}=[1,3,3,2]$, $\text{nums2}=[2,1,3,4]$, $k=3$

Output: 12

Why: Choose indices giving sum 6 and minimum nums2 2.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one k th statistic; heap is better for streaming or repeated picks.

Implementation checkpoints:

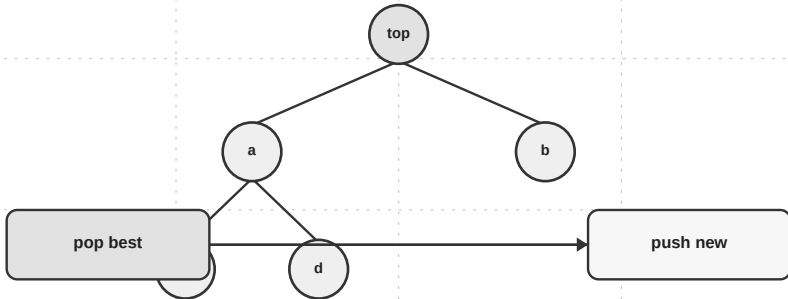
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2542. Maximum Subsequence Score
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using min-heap/max-heap sign incorrectly; heap grows beyond k .
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

Three drills:

- V1: Return top- k in sorted order after using the heap.
- V2: Handle ties with a secondary key.
- V3: Support streaming insertions and deletions.

Follow-ups:

- Can you reduce memory from $O(n)$ to $O(k)$?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def maximum_subsequence_score(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 052 of 229: 2462. Total Cost to Hire K Workers

Pattern: Heap / Priority Queue | Sources: LC75 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Hire exactly k workers. Each round choose the cheapest worker from the first candidates or last candidates available, breaking ties by index.

Example input/output:
Input: costs=[17,12,10,2,7,2,11,20,8], $k=3$, candidates=4

Output: 11

Why: Pick costs 2,2,7.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k \cdot n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one k th statistic; heap is better for streaming or repeated picks.

Implementation checkpoints:

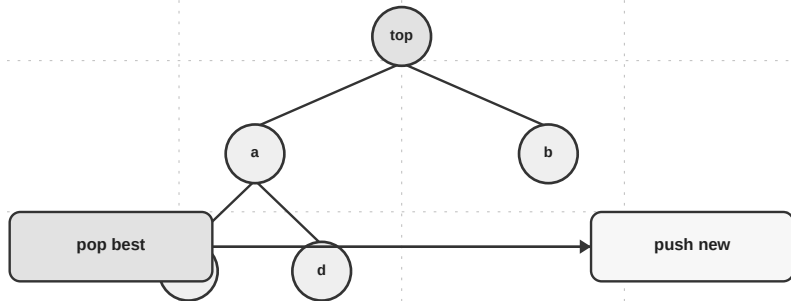
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2462. Total Cost to Hire K Workers
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using min-heap/max-heap sign incorrectly; heap grows beyond k .
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

Three drills:

- V1: Return top- k in sorted order after using the heap.
- V2: Handle ties with a secondary key.
- V3: Support streaming insertions and deletions.

Follow-ups:

- Can you reduce memory from $O(n)$ to $O(k)$?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def total_cost_to_hire_k_workers(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 053 of 229: 374. Guess Number Higher or Lower

Pattern: Binary Search | Sources: LC75 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Using the guess API that says lower, higher, or correct, find the hidden number between 1 and n.

Example input/output:
Input: n=10, pick=6

Output: 6

Why: Binary search finds the pick.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

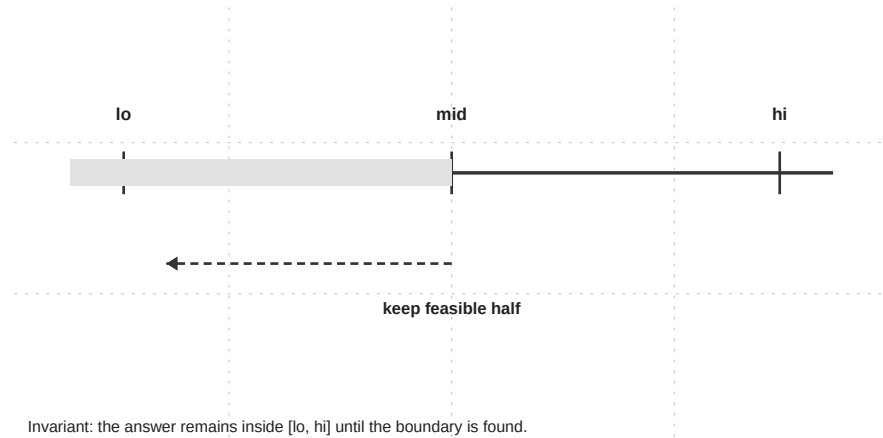
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 374. Guess Number Higher or Lower
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {    // mid works; try to improve boundary
            ans = mid
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Infinite loop from mid/lo/hi update; predicate not monotonic.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

- Three drills:
- V1: Find first true vs last true boundary.
 - V2: Binary-search the answer value instead of an index.
 - V3: Handle duplicates and missing target gracefully.

- Follow-ups:
- What is the exact monotonic predicate?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def guess_number_higher_or_lower(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 054 of 229: 2300. Successful Pairs of Spells and Potions

Pattern: Binary Search | Sources: LC75 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
For each spell, count potions such that spell*potion is at least success.

Example input/output:
Input: spells=[5,1,3], potions=[1,2,3,4,5], success=7

Output: [4,0,3]

Why: Sort potions and binary-search the first sufficient potion.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

Implementation checkpoints:

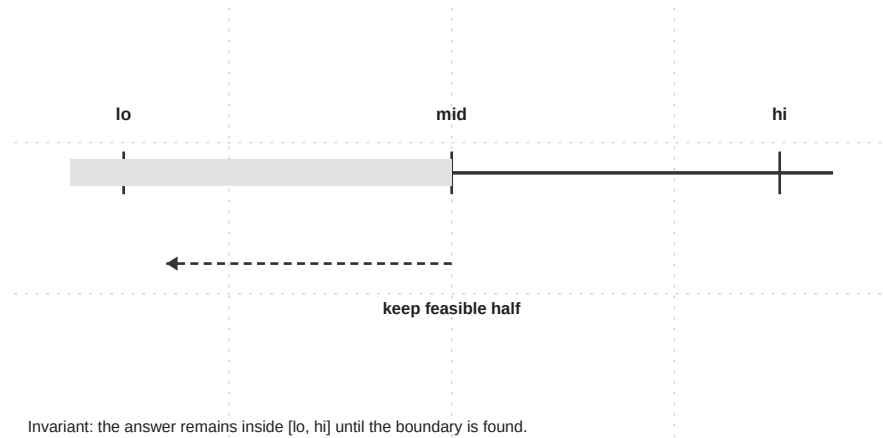
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2300. Successful Pairs of Spells and Potions
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:

- V1: Find first true vs last true boundary.
- V2: Binary-search the answer value instead of an index.
- V3: Handle duplicates and missing target gracefully.

Follow-ups:

- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def successful_pairs_of_spells_and_potio(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 055 of 229: 162. Find Peak Element

Pattern: Binary Search | Sources: LC75, Top150 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return any index i such that $\text{nums}[i]$ is greater than its neighbors; outside the array counts as negative infinity.

Example input/output:

Input: $\text{nums}=[1,2,3,1]$

Output: 2

Why: Value 3 is a peak.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow $[\text{lo}, \text{hi}]$ until the boundary is found.

Alternative:
Use `lower_bound`/`upper_bound` variants for first true, last true, or exact match.

Implementation checkpoints:

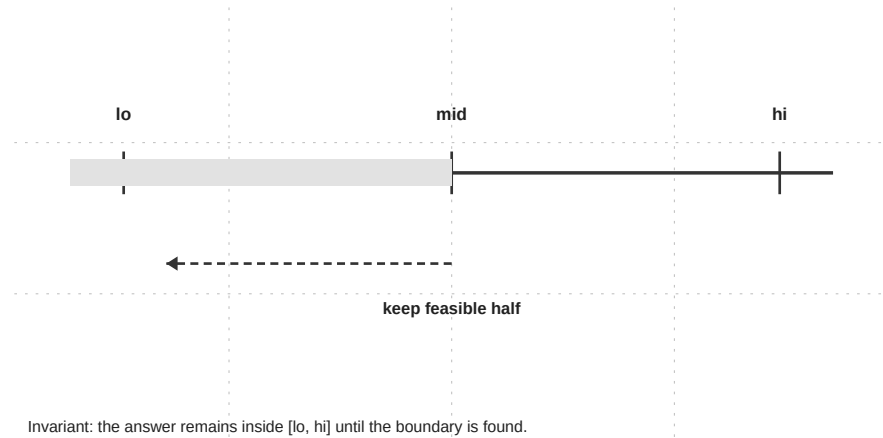
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 162. Find Peak Element
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {    // mid works; try to improve boundary
            ans = mid
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Infinite loop from `mid/lo/hi` update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing $<$ vs $\leq$ correctly.

Variations and follow-ups

Three drills:

- V1: Find first true vs last true boundary.
- V2: Binary-search the answer value instead of an index.
- V3: Handle duplicates and missing target gracefully.

Follow-ups:

- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_peak_element(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 056 of 229: 875. Koko Eating Bananas

Pattern: Binary Search | Sources: LC75 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Find the minimum integer banana-eating speed k so all piles are eaten within h hours.

Example input/output:
Input: piles=[3,6,7,11], h=8

Output: 4

Why: Speed 4 finishes in 8 hours.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow $[lo, hi]$ until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

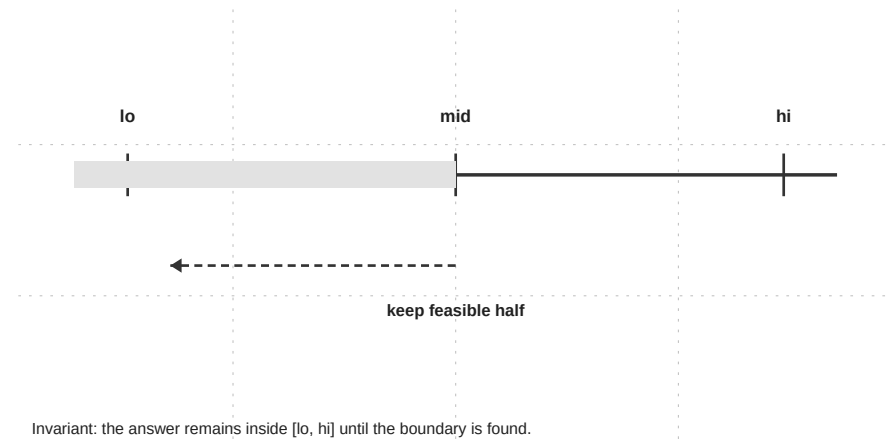
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 875. Koko Eating Bananas
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {    // mid works; try to improve boundary
            ans = mid
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Infinite loop from mid/lo/hi update; predicate not monotonic.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

- Three drills:
- V1: Find first true vs last true boundary.
 - V2: Binary-search the answer value instead of an index.
 - V3: Handle duplicates and missing target gracefully.

- Follow-ups:
- What is the exact monotonic predicate?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def koko_eating_bananas(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 057 of 229: 17. Letter Combinations of a Phone Number

Pattern: Backtracking | Sources: LC75, Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Map phone digits 2-9 to letters and return all possible letter strings in digit order.

Example input/output:
Input: digits="23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Why: Combine letters for 2 and 3.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

Implementation checkpoints:

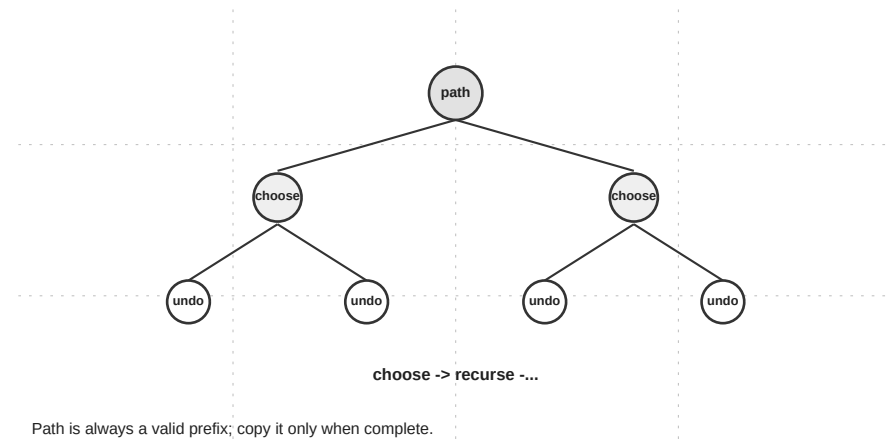
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 17. Letter Combinations of a Phone Number
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def letter_combinations_of_a_phone_numbe(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 058 of 229: 216. Combination Sum III

Pattern: Backtracking | Sources: LC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Return all combinations of k distinct numbers from 1..9 that sum to n; each number used at most once.

Example input/output:

Input: k=3, n=7

Output: [[1,2,4]]

Why: Only 1+2+4 equals 7.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

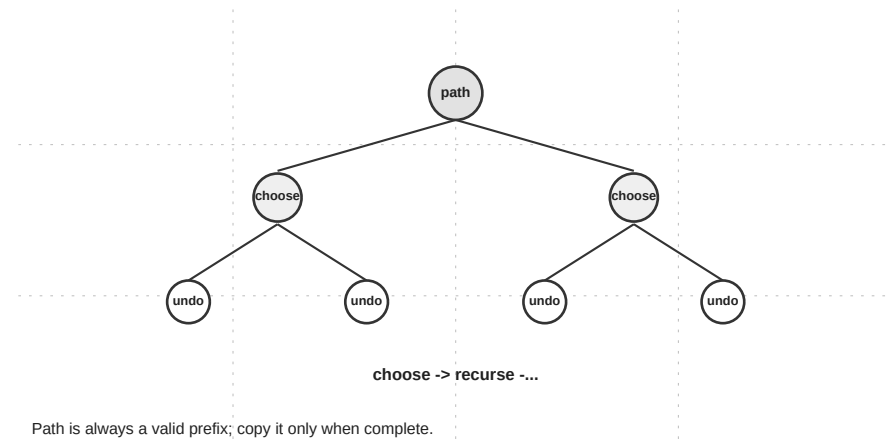
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 216. Combination Sum III
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def combination_sum_iii(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 059 of 229: 1137. N-th Tribonacci Number

Pattern: DP 1D | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n * state)

Space: O(state)

Problem statement + sample

Prompt:
Compute the n-th Tribonacci number with T0=0, T1=1, T2=1, and Tn=Tn-1+Tn-2+Tn-3.

Example input/output:
Input: n=4

Output: 4

Why: Sequence is 0,1,1,2,4.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

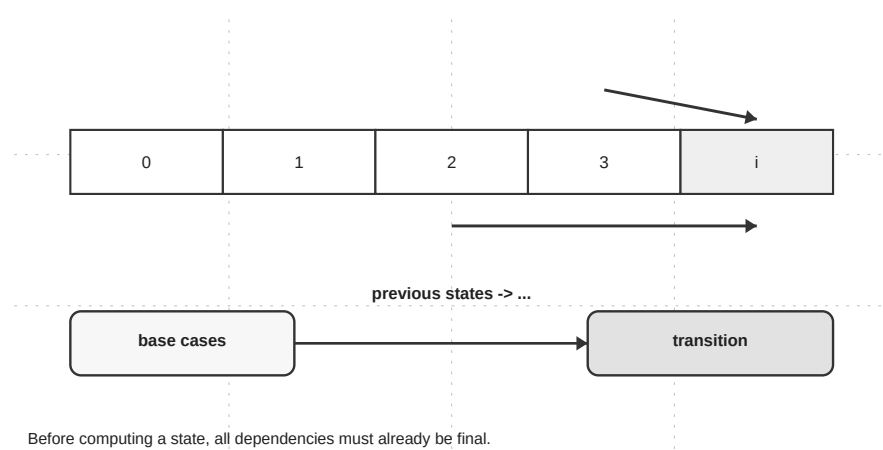
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1137. N-th Tribonacci Number
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing dp[i], every state it depends on has already been finalized.

Complexity target:
Time O(n * state); Space O(state)

Why:
States times transition cost. Usually O(n) or O(n*k), with O(n) or compressed O(1) space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Wrong base cases; overwriting compressed state in the wrong order.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
n=0/1; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

- Three drills:
- V1: Return the chosen items/path, not only the optimum value.
 - V2: Compress memory and explain update direction.
 - V3: Add modulo or large-number constraints.

- Follow-ups:
- Can you reconstruct the chosen path, not just the value?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def n_th_tribonacci_number(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing dp[i], every state it depends on has already been finalized.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 060 of 229: 746. Min Cost Climbing Stairs

Pattern: DP 1D | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Given cost per stair, return the minimum cost to reach just beyond the last stair, starting from step 0 or 1.

Example input/output:
Input: cost=[10,15,20]

Output: 15

Why: Start at step 1, then climb to the top.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

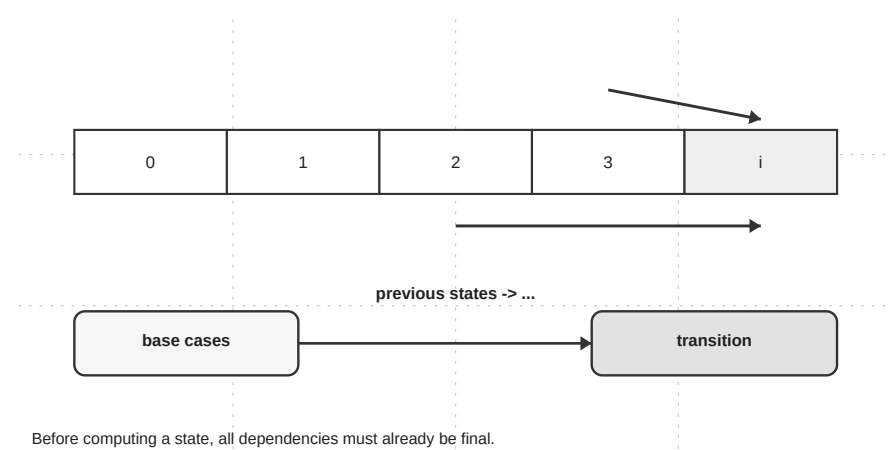
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 746. Min Cost Climbing Stairs
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def min_cost_climbing_stairs(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 061 of 229: 198. House Robber

Pattern: DP 1D | Sources: LC75, Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Maximize money robbed from houses in a line without robbing adjacent houses.

Example input/output:
Input: nums=[2,7,9,3,1]

Output: 12

Why: Rob houses with values 2,9,1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

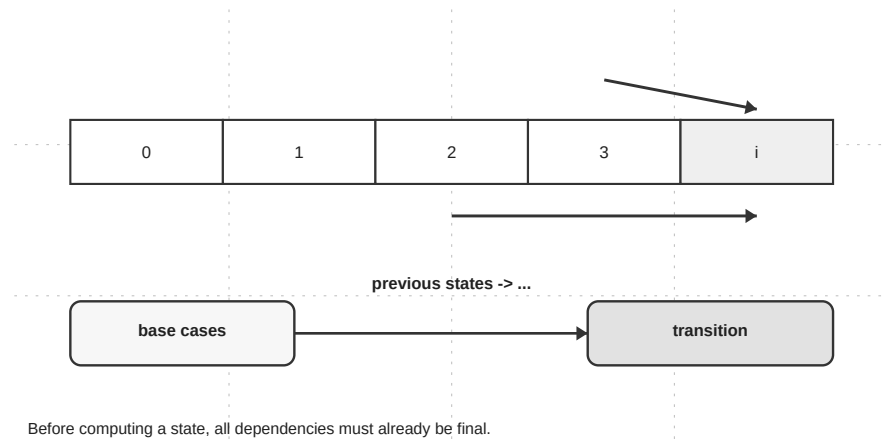
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 198. House Robber
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def house_robber(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 062 of 229: 790. Domino and Tromino Tiling

Pattern: DP 1D | Sources: LC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Count ways to tile a 2 by n board using dominoes and trominoes; return the count modulo $1e9+7$.

Example input/output:
Input: n=3

Output: 5

Why: There are five tilings for width 3.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

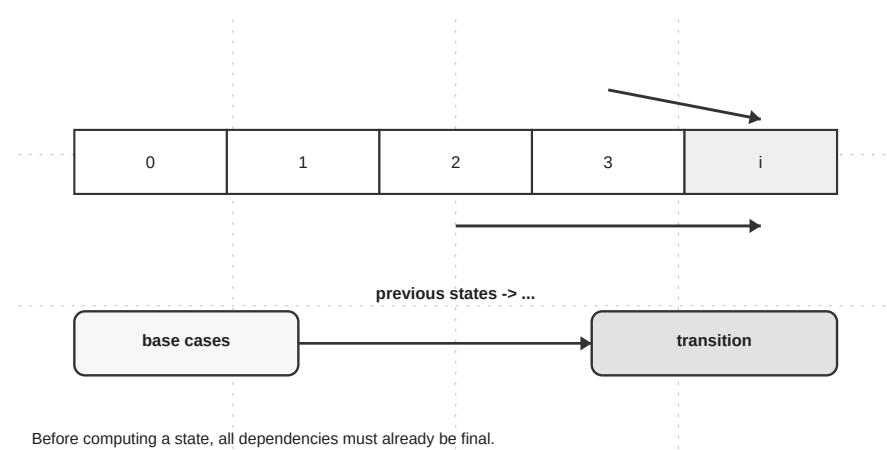
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 790. Domino and Tromino Tiling
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def domino_and_tromino_tiling(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 063 of 229: 62. Unique Paths

Pattern: DP 2D / Kadane | Sources: LC75, NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Count unique paths from top-left to bottom-right of an m by n grid using only right and down moves.

Example input/output:
Input: $m=3, n=7$

Output: 28

Why: Combinatorial DP gives 28 paths.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

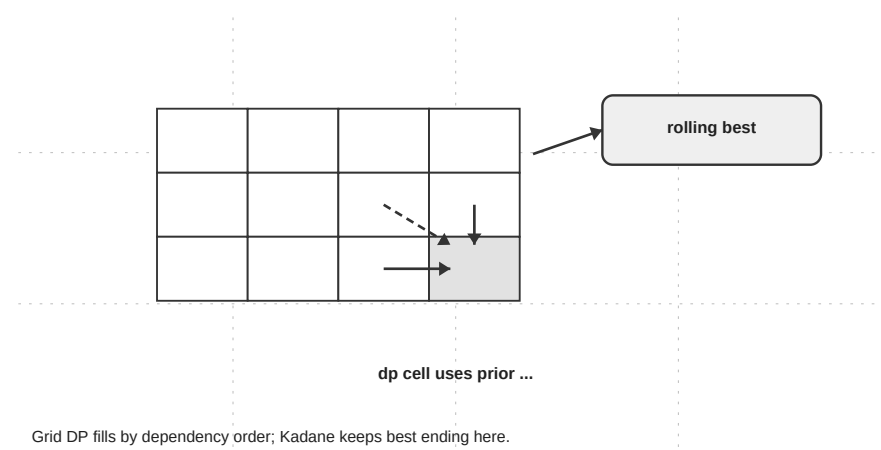
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 62. Unique Paths
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def unique_paths(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 064 of 229: 1143. Longest Common Subsequence

Pattern: DP 2D / Kadane | Sources: LC75, NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the length of the longest sequence that appears in both strings in the same relative order.

Example input/output:
Input: text1="abcde", text2="ace"

Output: 3

Why: "ace" is common.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

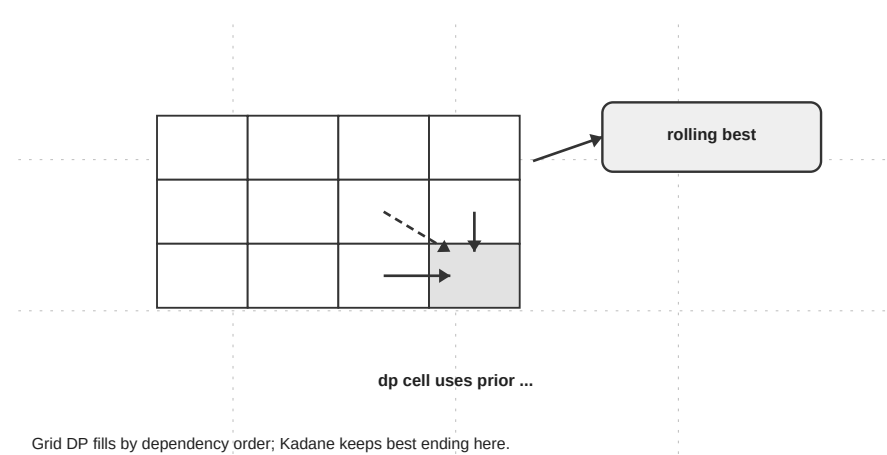
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1143. Longest Common Subsequence
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_common_subsequence(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 065 of 229: 714. Best Time to Buy and Sell Stock with Transaction Fee

Pattern: DP 2D / Kadane | Sources: LC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Maximize stock profit with unlimited transactions, paying a fixed fee for each completed sale.

Example input/output:
Input: prices=[1,3,2,8,4,9], fee=2

Output: 8

Why: Buy/sell around 1->8 and 4->9 after fees.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

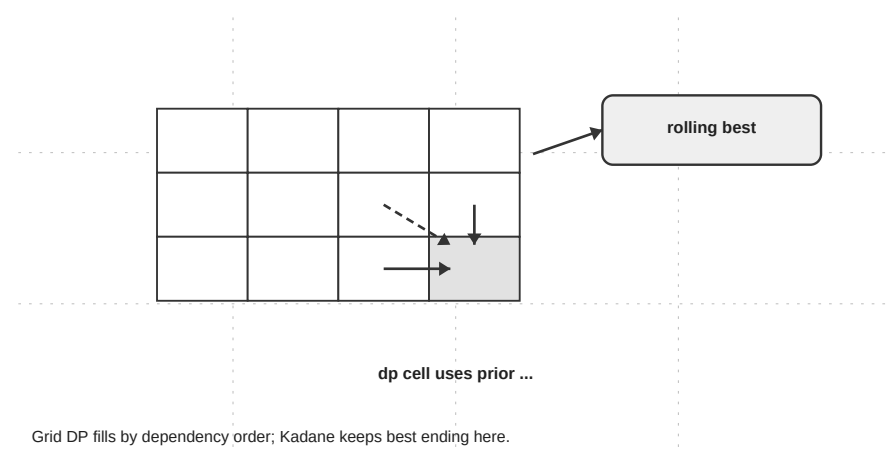
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 714. Best Time to Buy and Sell Stock with Transaction Fee
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def best_time_to_buy_and_sell_stock_with(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 066 of 229: 72. Edit Distance

Pattern: DP 2D / Kadane | Sources: LC75, Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the minimum number of insertions, deletions, or replacements needed to transform word1 into word2.

Example input/output:
Input: word1="horse", word2="ros"

Output: 3

Why: One optimal edit distance is 3.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

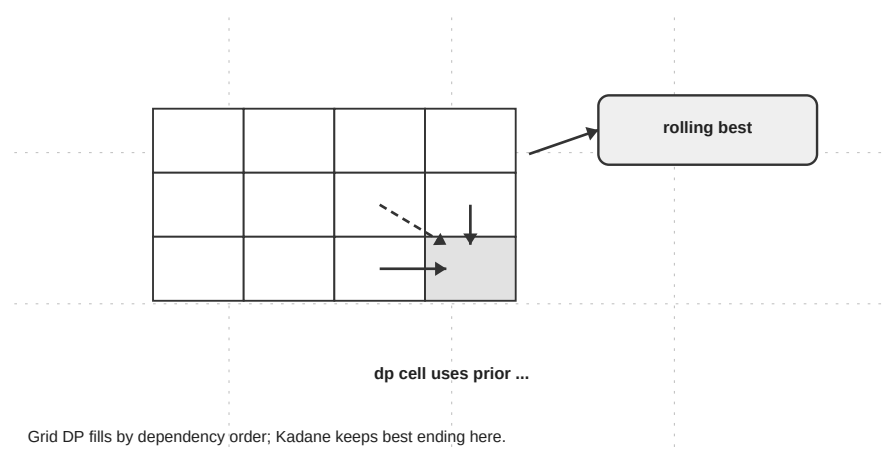
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 72. Edit Distance
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Bad initialization; mixing row/column dependencies.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

- Three drills:
- V1: Return the actual subarray/subsequence.
 - V2: Reduce 2D memory to rolling rows where safe.
 - V3: Handle all-negative values or empty inputs explicitly.

- Follow-ups:
- Can the DP be space compressed safely?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def edit_distance(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 067 of 229: 338. Counting Bits

Pattern: Bit Manipulation | Sources: LC75, NC75 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
For every integer from 0 to n, return the number of 1 bits in its binary representation.

Example input/output:
Input: n=5

Output: [0,1,1,2,1,2]

Why: Counts for 0,1,2,3,4,5.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

Implementation checkpoints:

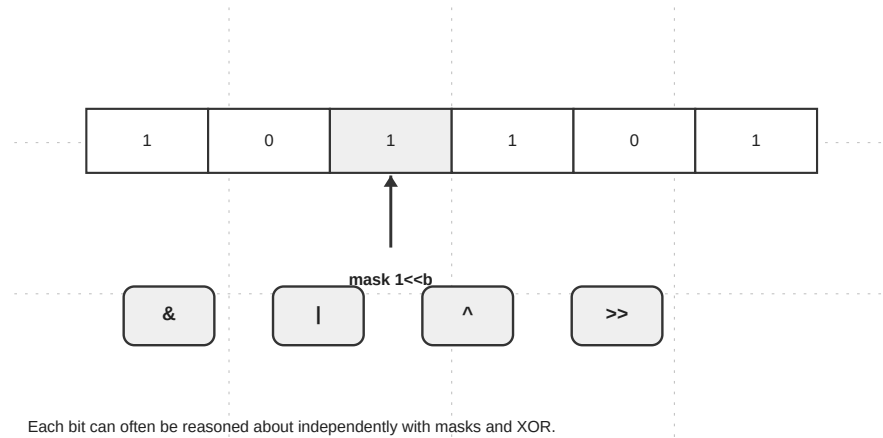
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 338. Counting Bits
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:

- V1: Generalize from one missing/unique value to two.
- V2: Use fixed 32-bit signed behavior.
- V3: Count set bits across a range instead of one value.

Follow-ups:

- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def counting_bits(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 068 of 229: 136. Single Number

Pattern: Bit Manipulation | Sources: LC75, Top150 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Every value appears twice except one value that appears once; return the single value.

Example input/output:
Input: nums=[4,1,2,1,2]

Output: 4

Why: Duplicates cancel by XOR.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

Implementation checkpoints:

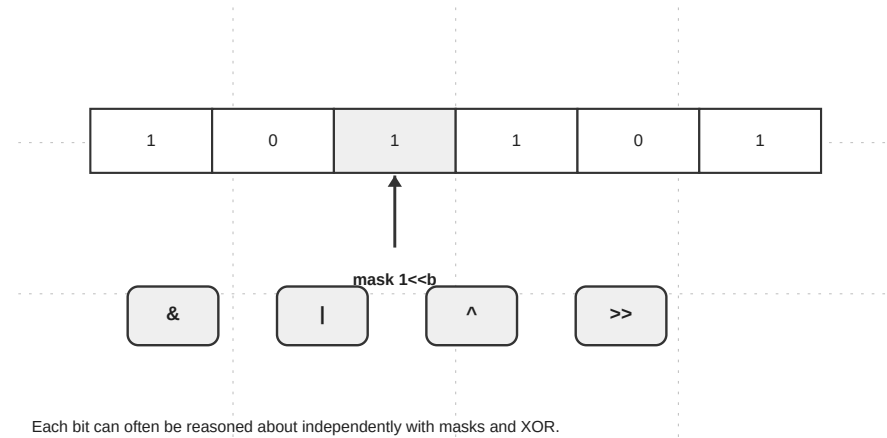
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 136. Single Number
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:

- V1: Generalize from one missing/unique value to two.
- V2: Use fixed 32-bit signed behavior.
- V3: Count set bits across a range instead of one value.

Follow-ups:

- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def single_number(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 069 of 229: 1318. Minimum Flips to Make a OR b Equal to c

Pattern: Bit Manipulation | Sources: LC75 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Return the minimum bit flips in a and b needed so the bitwise OR of a and b equals c.

Example input/output:
Input: a=2, b=6, c=5

Output: 3

Why: Three bit positions must change.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

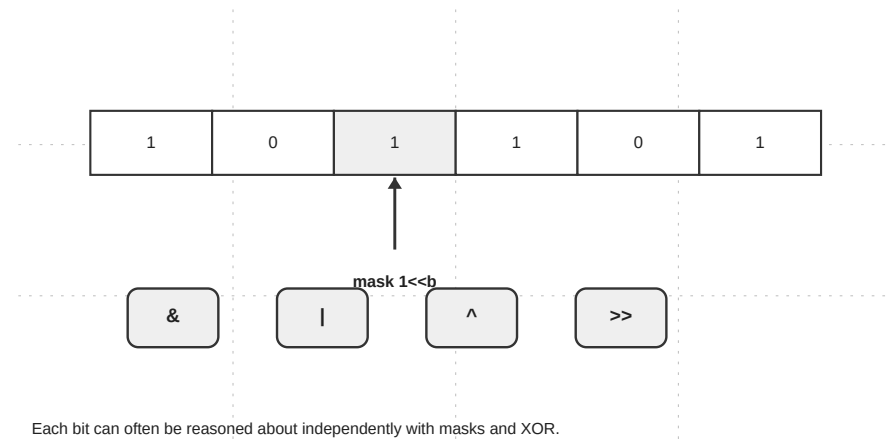
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1318. Minimum Flips to Make a OR b Equal to c
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:
V1: Generalize from one missing/unique value to two.
V2: Use fixed 32-bit signed behavior.
V3: Count set bits across a range instead of one value.

Follow-ups:
- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def minimum_flips_to_make_a_or_b_equal_t(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 070 of 229: 208. Implement Trie (Prefix Tree)

Pattern: Trie / Encoding | Sources: LC75, Top150, NC75 | Companies: Google, Amazon, Bloomberg, Microsoft (public samples)

Time: O(total characters)

Space: O(total characters)

Problem statement + sample

Prompt:
Implement a trie with insert, search for whole word, and startsWith for prefixes.

Example input/output:
Input: insert("apple"), search("apple"), startsWith("app")

Output: [null,true,true]

Why: The inserted word creates the prefix path.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Compare every string with every query or split by ambiguous delimiters.

Optimized approach:
Store characters in a trie for prefix search, or encode lengths so decoding is unambiguous.

Alternative:
Hash maps can replace trie when only whole-word lookup is needed.

Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

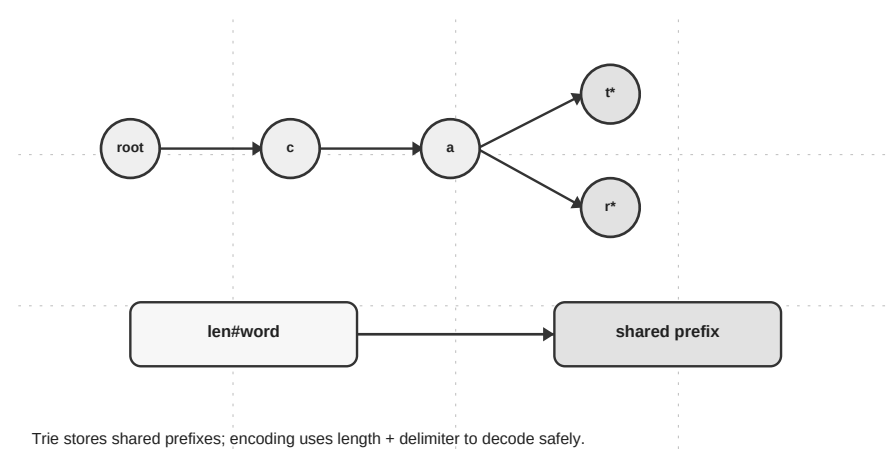
Annotated Go kernel - comments explain the reasoning

```
// 208. Implement Trie (Prefix Tree)
type TrieNode struct {
    child map[byte]*TrieNode
    end bool
}

func solveTrieEncoding(words []string) *TrieNode {
    root := &TrieNode{child: map[byte]*TrieNode{}}
    for _, w := range words {
        cur := root
        for i := 0; i < len(w); i++ {
            if cur.child[w[i]] == nil { cur.child[w[i]] = &TrieNode{child: map[byte]*TrieNode{}} }
            cur = cur.child[w[i]]
        }
        cur.end = true // complete word marker
    }
    return root
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every path from root represents exactly one prefix; terminal marks complete words.

Complexity target:
Time O(total characters); Space O(total characters)

Why:
O(total characters) to build; query cost is O(length of query/prefix).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Terminal marker missing; ambiguous delimiter.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty strings; delimiter appearing in input; duplicate words; memory blowup; lexicographic ordering.

Variations and follow-ups

Three drills:
V1: Support delete/update operations.
V2: Return top-k suggestions with lexicographic ties.
V3: Use length-prefix encoding when delimiter may appear in input.

Follow-ups:
- Can the trie support delete or wildcard search?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def implement_trie_prefix_tree(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every path from root represents exactly one prefix; terminal marks complete words.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 071 of 229: 1268. Search Suggestions System

Pattern: Trie / Encoding | Sources: LC75 | Companies: Google, Amazon, Bloomberg, Microsoft (public samples)

Time: $O(\text{total characters})$

Space: $O(\text{total characters})$

Problem statement + sample

Prompt:
After each typed prefix of searchWord, return up to three lexicographically smallest products starting with that prefix.

Example input/output:
Input: products=["mobile","mouse","moneypot","monitor","mousepad"], searchWord="mouse"

Output:
[["mobile","moneypot","monitor"],["mobile","moneypot","monitor"],["mouse","mousepad"],["mouse","

Why: Suggestions shrink as prefixes grow.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Compare every string with every query or split by ambiguous delimiters.

Optimized approach:
Store characters in a trie for prefix search, or encode lengths so decoding is unambiguous.

Alternative:
Hash maps can replace trie when only whole-word lookup is needed.

Implementation checkpoints:

- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

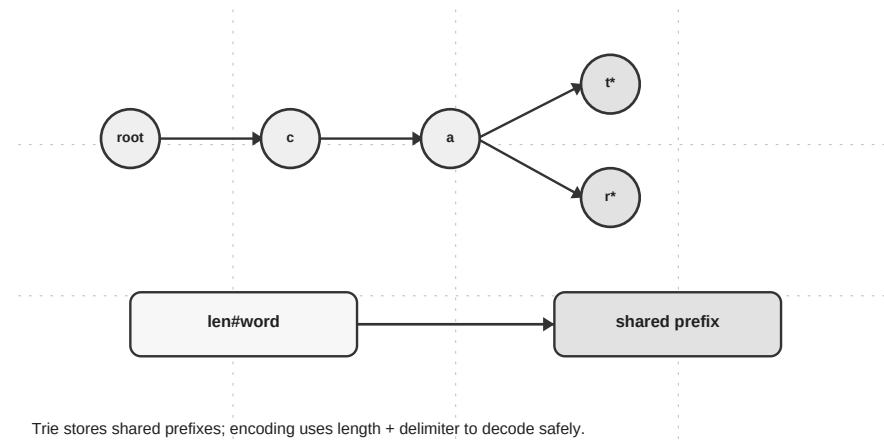
Annotated Go kernel - comments explain the reasoning

```
// 1268. Search Suggestions System
type TrieNode struct {
    child map[byte]*TrieNode
    end bool
}

func solveTrieEncoding(words []string) *TrieNode {
    root := &TrieNode{child: map[byte]*TrieNode{}}
    for _, w := range words {
        cur := root
        for i := 0; i < len(w); i++ {
            if cur.child[w[i]] == nil { cur.child[w[i]] = &TrieNode{child: map[byte]*TrieNode{}} }
            cur = cur.child[w[i]]
        }
        cur.end = true // complete word marker
    }
    return root
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every path from root represents exactly one prefix; terminal marks complete words.

Complexity target:
Time $O(\text{total characters})$; Space $O(\text{total characters})$

Why:
 $O(\text{total characters})$ to build; query cost is $O(\text{length of query/prefix})$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Terminal marker missing; ambiguous delimiter.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty strings; delimiter appearing in input; duplicate words; memory blowup; lexicographic ordering.

Variations and follow-ups

Three drills:

- V1: Support delete/update operations.
- V2: Return top-k suggestions with lexicographic ties.
- V3: Use length-prefix encoding when delimiter may appear in input.

Follow-ups:

- Can the trie support delete or wildcard search?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def search_suggestions_system(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every path from root represents exactly one prefix; terminal marks complete words.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 072 of 229: 435. Non-overlapping Intervals

Pattern: Intervals | Sources: LC75, NC75 | Companies: Google, Meta, Amazon, Bloomberg (public samples)

Time: O(n log n)

Space: O(n)

Problem statement + sample

Prompt:
Return the minimum number of intervals to remove so the remaining intervals are non-overlapping.

Example input/output:
Input: intervals=[[1,2],[2,3],[3,4],[1,3]]

Output: 1

Why: Remove [1,3].

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Compare every interval pair repeatedly. O(n^2).

Optimized approach:
Sort once, then maintain the active end or merged interval while scanning.

Alternative:
Sweep line with events is useful for counting overlaps/rooms.

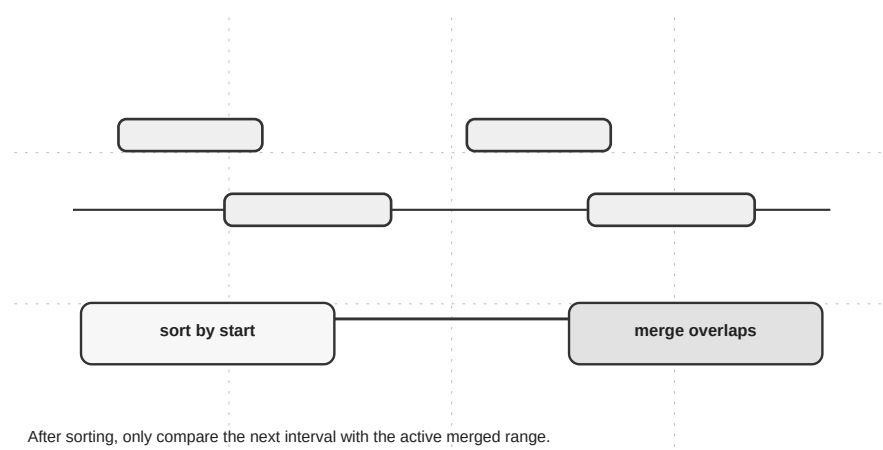
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 435. Non-overlapping Intervals
func solveIntervals(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, in := range intervals {
        if len(merged) == 0 || merged[len(merged)-1][1] < in[0] {
            merged = append(merged, append([]int(nil), in...))
        } else if in[1] > merged[len(merged)-1][1] {
            merged[len(merged)-1][1] = in[1] // extend overlap
        }
    }
    return merged
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Complexity target:
Time O(n log n); Space O(n)

Why:
Sorting dominates at O(n log n); the scan is O(n). Space is O(1) to O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Sorting by wrong key; not merging touching intervals when required.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
touching endpoints; inclusive vs exclusive; empty list; sorting tie-breakers; nested intervals.

Variations and follow-ups

Three drills:
V1: Distinguish touching endpoints from overlapping endpoints.
V2: Return removed intervals as well as the merged result.
V3: Process online insertions into an existing schedule.

Follow-ups:
- Can you process intervals online?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def non_overlapping_intervals(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 073 of 229: 452. Minimum Number of Arrows to Burst Balloons

Pattern: Intervals | Sources: LC75, Top150 | Companies: Google, Meta, Amazon, Bloomberg (public samples)

Time: O(n log n)

Space: O(n)

Problem statement + sample

Prompt:
Each balloon is an interval on the x-axis. Return the fewest arrows needed to burst all balloons, where one arrow hits all intervals containing its x.

Example input/output:
Input: points=[[10,16],[2,8],[1,6],[7,12]]

Output: 2

Why: One arrow can hit [1,6],[2,8]; another hits [7,12],[10,16].

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every interval pair repeatedly. O(n^2).

Optimized approach:
Sort once, then maintain the active end or merged interval while scanning.

Alternative:
Sweep line with events is useful for counting overlaps/rooms.

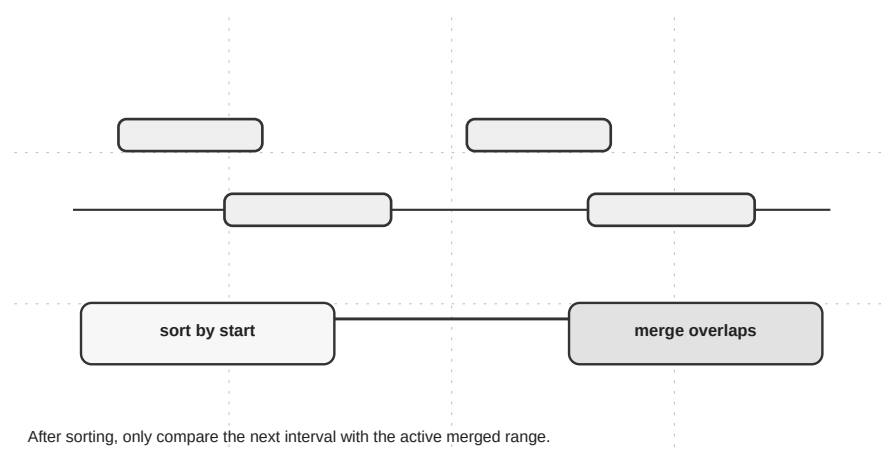
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 452. Minimum Number of Arrows to Burst Balloons
func solveIntervals(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, in := range intervals {
        if len(merged) == 0 || merged[len(merged)-1][1] < in[0] {
            merged = append(merged, append([]int(nil), in...))
        } else if in[1] > merged[len(merged)-1][1] {
            merged[len(merged)-1][1] = in[1] // extend overlap
        }
    }
    return merged
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Complexity target:
Time O(n log n); Space O(n)

Why:
Sorting dominates at O(n log n); the scan is O(n). Space is O(1) to O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Sorting by wrong key; not merging touching intervals when required.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
touching endpoints; inclusive vs exclusive; empty list; sorting tie-breakers; nested intervals.

Variations and follow-ups

Three drills:
V1: Distinguish touching endpoints from overlapping endpoints.
V2: Return removed intervals as well as the merged result.
V3: Process online insertions into an existing schedule.

- Follow-ups:
- Can you process intervals online?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def minimum_number_of_arrows_to_burst_ba(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All intervals before current have been merged or counted consistently with the chosen end boundary.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 074 of 229: 739. Daily Temperatures

Pattern: Monotonic Stack | Sources: LC75 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
For each day, return how many days until a warmer temperature; use 0 when none occurs.

Example input/output:
Input: temperatures=[73,74,75,71,69,72,76,73]

Output: [1,1,4,2,1,1,0,0]

Why: Next warmer positions determine distances.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
For each index, scan forward/backward to find the next greater/smaller item. O(n^2).

Optimized approach:
Maintain a stack of unresolved indices in monotonic order; resolve them when a stronger value appears.

Alternative:
Use decreasing stack for next greater, increasing stack for next smaller, or scan from the opposite side.

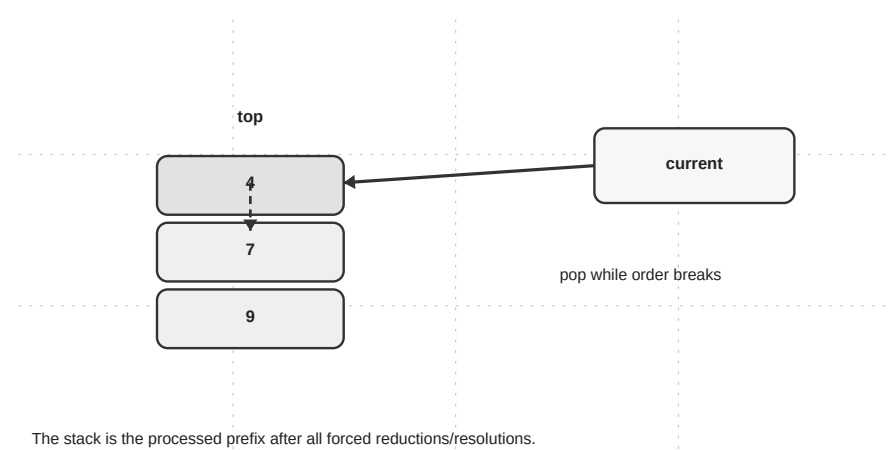
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 739. Daily Temperatures
func solveMonotonicStack(nums []int) []int {
    ans := make([]int, len(nums))
    st := []int{} // unresolved indices, values kept monotonic
    for i, x := range nums {
        for len(st) > 0 && nums[st[len(st)-1]] < x {
            j := st[len(st)-1]
            st = st[:len(st)-1]
            ans[j] = i - j // x answers index j
        }
        st = append(st, i)
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Stack indices have not found their answer and their values remain monotonic.

Complexity target:
Time O(n); Space O(n)

Why:
Each index enters and leaves the stack once: O(n) time, O(n) space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using < instead of <=; storing values when indices are needed.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
equal values; direction of scan; circular arrays; storing values vs indices.

Variations and follow-ups

- Three drills:
- V1: Find previous instead of next warmer/greater item.
 - V2: Use circular-array behavior.
 - V3: Return spans/counts rather than target index.

- Follow-ups:
- Do you need next greater value, index, or distance?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def daily_temperatures(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Stack indices have not found their answer and their values remain monotonic.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 075 of 229: 901. Online Stock Span

Pattern: Monotonic Stack | Sources: LC75 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Design StockSpanner.next(price), returning consecutive days ending today with price less than or equal to today's price.

Example input/output:
Input: prices=[100,80,60,70,60,75,85]

Output: [1,1,1,2,1,4,6]

Why: A monotonic stack merges previous spans.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each index, scan forward/backward to find the next greater/smaller item. O(n^2).

Optimized approach:
Maintain a stack of unresolved indices in monotonic order; resolve them when a stronger value appears.

Alternative:
Use decreasing stack for next greater, increasing stack for next smaller, or scan from the opposite side.

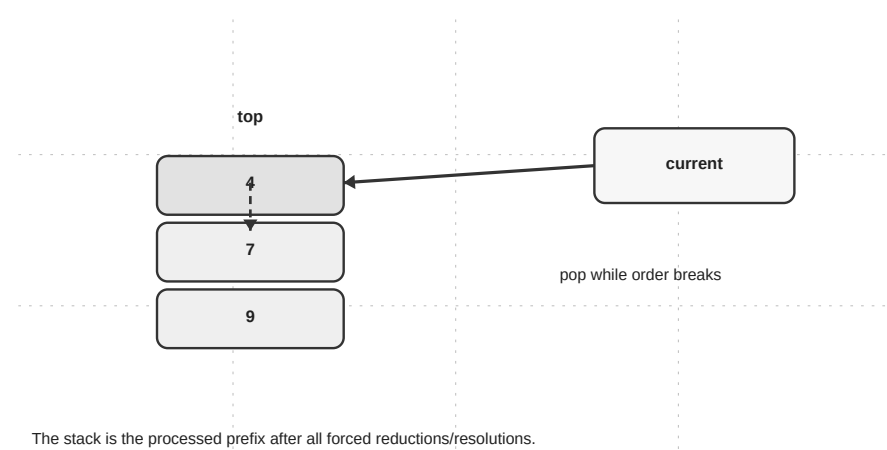
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 901. Online Stock Span
func solveMonotonicStack(nums []int) []int {
    ans := make([]int, len(nums))
    st := []int{} // unresolved indices, values kept monotonic
    for i, x := range nums {
        for len(st) > 0 && nums[st[len(st)-1]] < x {
            j := st[len(st)-1]
            st = st[:len(st)-1]
            ans[j] = i - j // x answers index j
        }
        st = append(st, i)
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Stack indices have not found their answer and their values remain monotonic.

Complexity target:
Time O(n); Space O(n)

Why:
Each index enters and leaves the stack once: O(n) time, O(n) space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Using < instead of <=; storing values when indices are needed.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
equal values; direction of scan; circular arrays; storing values vs indices.

Variations and follow-ups

Three drills:
V1: Find previous instead of next warmer/greater item.
V2: Use circular-array behavior.
V3: Return spans/counts rather than target index.

Follow-ups:
- Do you need next greater value, index, or distance?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def online_stock_span(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Stack indices have not found their answer and their values remain monotonic.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 076 of 229: 88. Merge Sorted Array

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Merge sorted nums2 into sorted nums1 in-place, where nums1 has trailing space for all elements.

Example input/output:
Input: nums1=[1,2,3,0,0,0], m=3, nums2=[2,5,6], n=3

Output: [1,2,2,3,5,6]

Why: Fill from the end to avoid overwriting.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

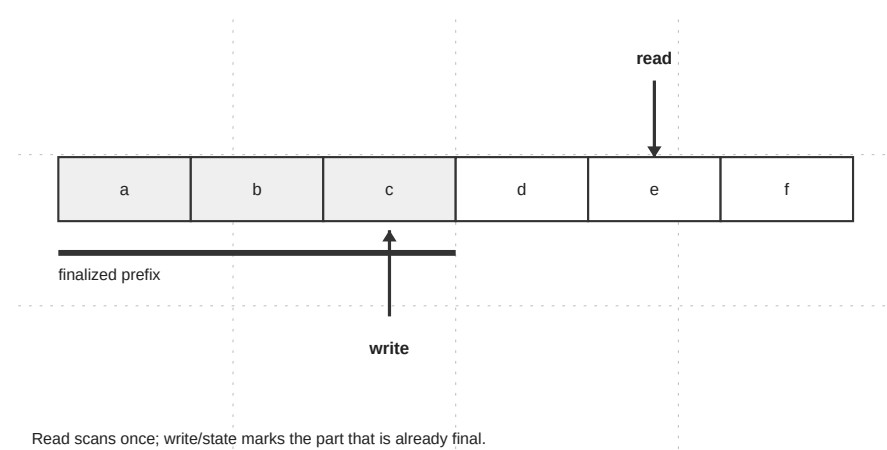
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 88. Merge Sorted Array
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def merge_sorted_array(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 077 of 229: 27. Remove Element

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Remove all occurrences of val from nums in-place and return the new length; order of remaining values may change.

Example input/output:
Input: nums=[3,2,2,3], val=3

Output: k=2, first values [2,2]

Why: Only values not equal to 3 remain.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

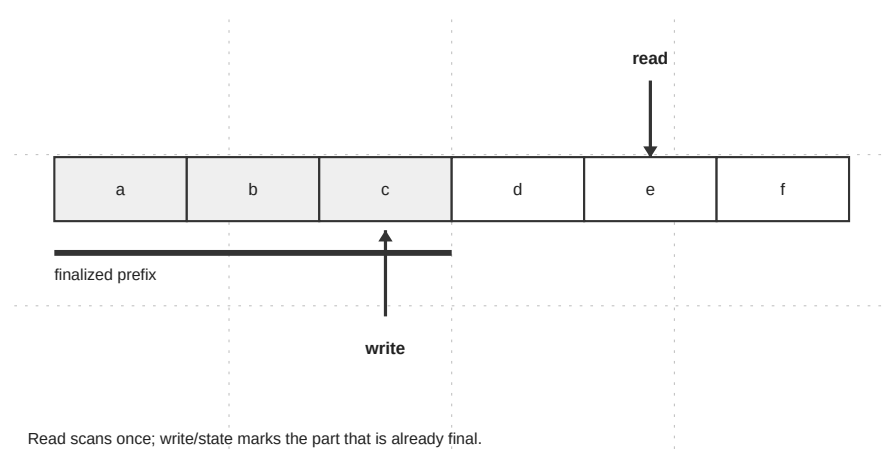
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 27. Remove Element
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def remove_element(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 078 of 229: 26. Remove Duplicates from Sorted Array

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Remove duplicates from a sorted array in-place so each value appears once; return the count of kept values.

Example input/output:

Input: nums=[1,1,2]

Output: k=2, first values [1,2]

Why: Two unique values remain.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

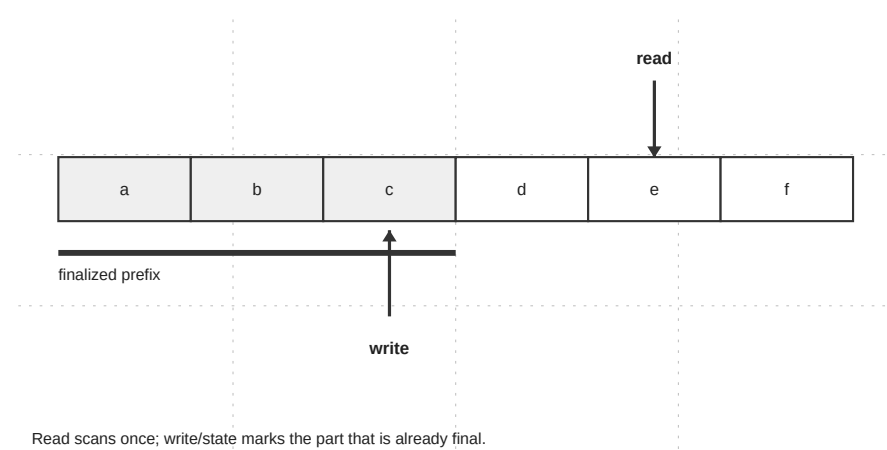
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 26. Remove Duplicates from Sorted Array
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:

empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

- V1: Require in-place edits instead of building a new array/string.
- V2: Add Unicode/case-normalization rules before comparing characters.
- V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def remove_duplicates_from_sorted_array(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:

Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 079 of 229: 80. Remove Duplicates from Sorted Array II

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Remove duplicates from a sorted array in-place so each value appears at most twice; return the kept length.

Example input/output:

Input: nums=[0,0,1,1,1,1,2]

Output: k=5, first values [0,0,1,1,2]

Why: Extra copies of 1 are skipped.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

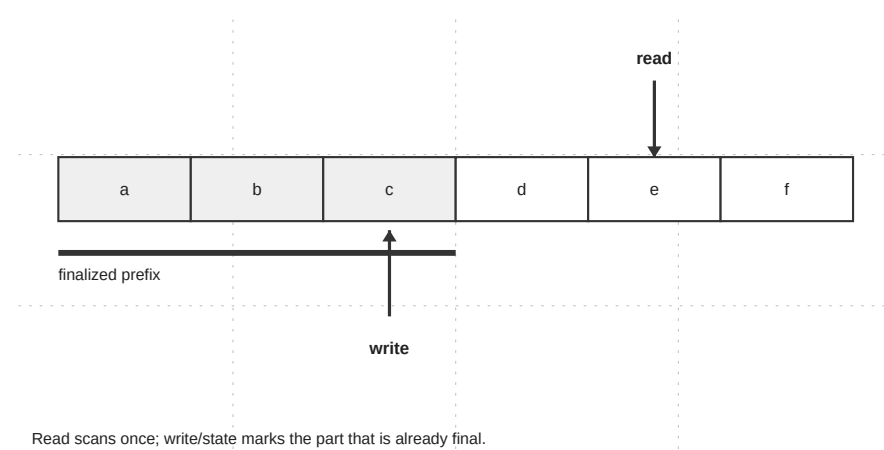
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 80. Remove Duplicates from Sorted Array II
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

- V1: Require in-place edits instead of building a new array/string.
- V2: Add Unicode/case-normalization rules before comparing characters.
- V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def remove_duplicates_from_sorted_array(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 080 of 229: 169. Majority Element

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return the majority element that appears more than floor(n/2) times.

Example input/output:
Input: nums=[2,2,1,1,1,2,2]

Output: 2

Why: 2 appears four times out of seven.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

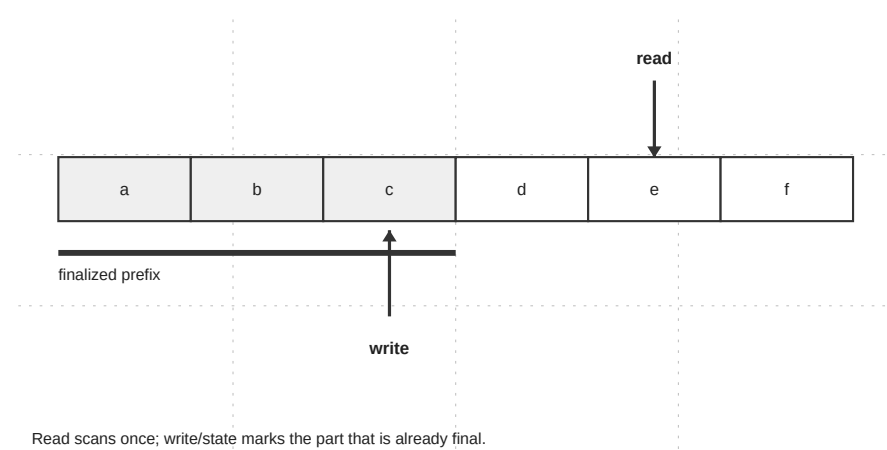
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 169. Majority Element
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def majority_element(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 081 of 229: 189. Rotate Array

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Rotate an array to the right by k steps in-place.

Example input/output:
Input: nums=[1,2,3,4,5], k=2

Output: [4,5,1,2,3]

Why: The last two elements move to the front.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

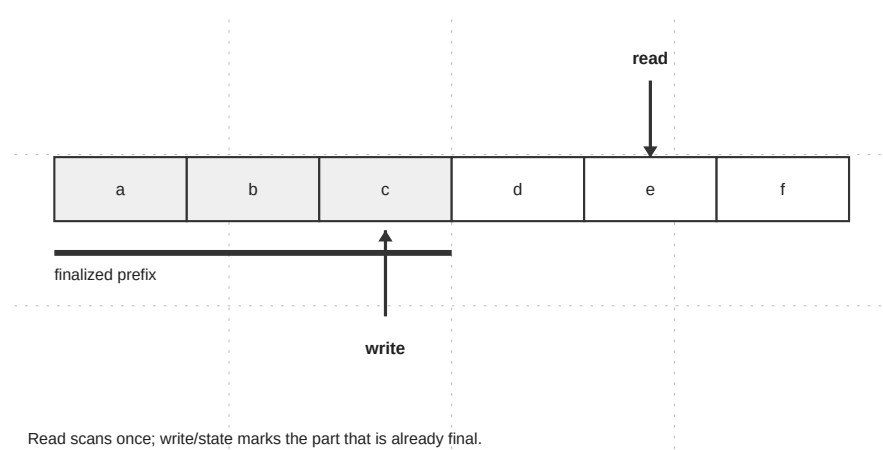
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 189. Rotate Array
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def rotate_array(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 082 of 229: 121. Best Time to Buy and Sell Stock

Pattern: Math / Greedy | Sources: Top150, NC75 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: $O(n)$ or $O(\log n)$

Space: $O(1)$

Problem statement + sample

Prompt:
Choose one buy day and one later sell day to maximize stock profit; return 0 if no profit is possible.

Example input/output:
Input: prices=[7,1,5,3,6,4]

Output: 5

Why: Buy at 1 and sell at 6.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

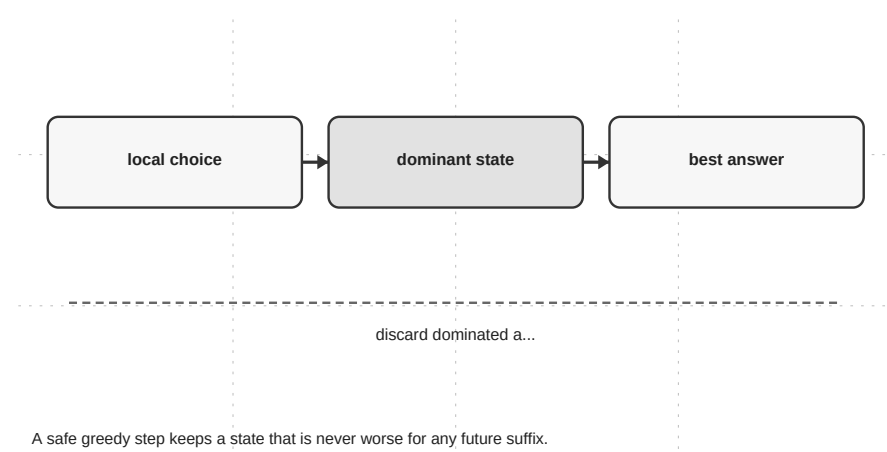
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 121. Best Time to Buy and Sell Stock
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time $O(n)$ or $O(\log n)$; Space $O(1)$

Why:
Usually $O(n)$ or $O(n \log n)$ if sorting is needed. Space is often $O(1)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

Three drills:
V1: Prove the greedy choice with an exchange argument.
V2: Return the chosen sequence of actions.
V3: Handle overflow and impossible cases explicitly.

Follow-ups:
- What invariant proves the greedy choice is safe?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def best_time_to_buy_and_sell_stock(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 083 of 229: 122. Best Time to Buy and Sell Stock II

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: $O(n)$ or $O(\log n)$

Space: $O(1)$

Problem statement + sample

Prompt:
Maximize stock profit with as many buy-sell transactions as desired, but hold at most one share at a time.

Example input/output:
Input: prices=[7,1,5,3,6,4]

Output: 7

Why: Take gains 1->5 and 3->6.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

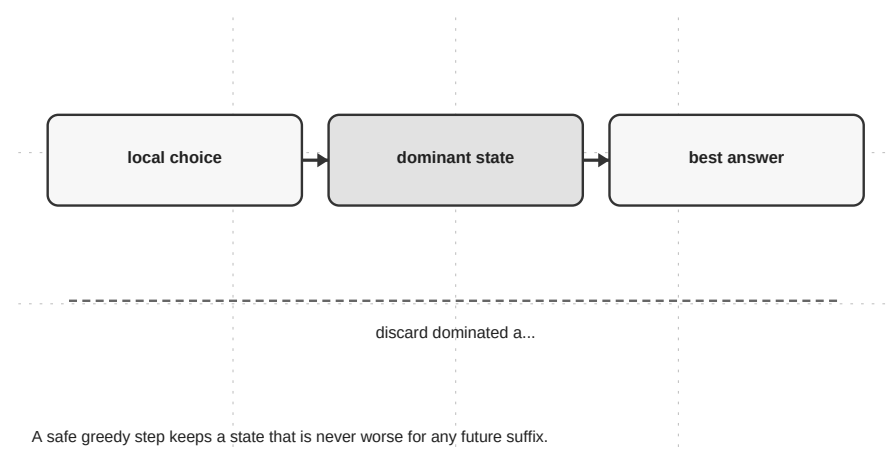
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 122. Best Time to Buy and Sell Stock II
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time $O(n)$ or $O(\log n)$; Space $O(1)$

Why:
Usually $O(n)$ or $O(n \log n)$ if sorting is needed. Space is often $O(1)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

Three drills:
V1: Prove the greedy choice with an exchange argument.
V2: Return the chosen sequence of actions.
V3: Handle overflow and impossible cases explicitly.

Follow-ups:
- What invariant proves the greedy choice is safe?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def best_time_to_buy_and_sell_stock_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 084 of 229: 55. Jump Game

Pattern: Math / Greedy | Sources: Top150, NC75 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Given maximum jump lengths at each index, return whether the last index is reachable from index 0.

Example input/output:
Input: nums=[2,3,1,1,4]

Output: true

Why: Reach index 1, then jump to the end.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

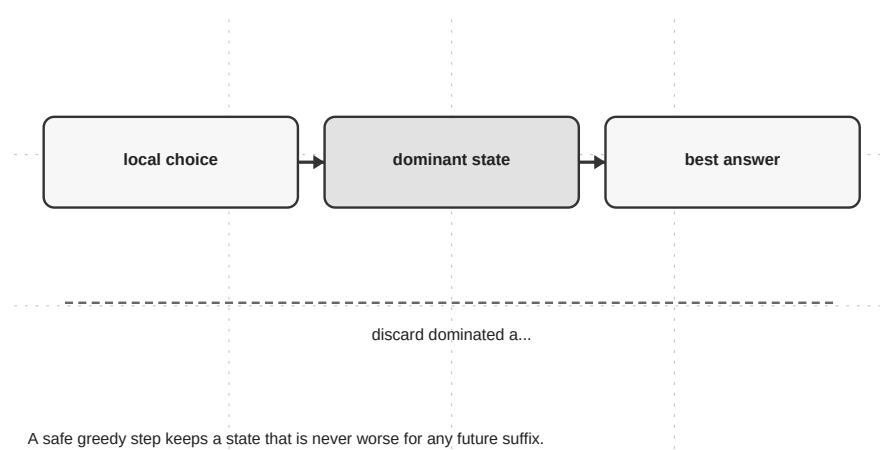
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 55. Jump Game
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def jump_game(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 085 of 229: 45. Jump Game II

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: $O(n)$ or $O(\log n)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return the minimum jumps needed to reach the last index, assuming it is reachable.

Example input/output:
Input: nums=[2,3,1,1,4]

Output: 2

Why: Jump 0->1->4.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

Implementation checkpoints:

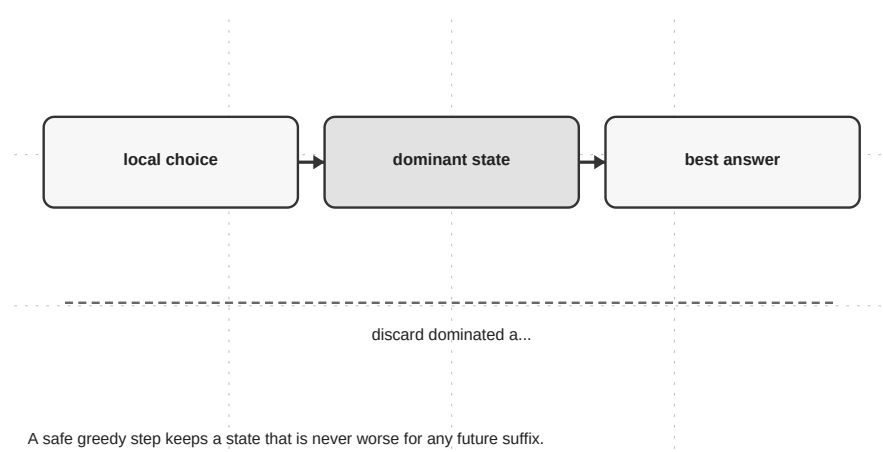
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 45. Jump Game II
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time $O(n)$ or $O(\log n)$; Space $O(1)$

Why:
Usually $O(n)$ or $O(n \log n)$ if sorting is needed. Space is often $O(1)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Assuming local greedy without proof; integer division edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

Three drills:

V1: Prove the greedy choice with an exchange argument.
V2: Return the chosen sequence of actions.
V3: Handle overflow and impossible cases explicitly.

Follow-ups:

- What invariant proves the greedy choice is safe?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def jump_game_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 086 of 229: 274. H-Index

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Given citation counts, return the largest h such that at least h papers have at least h citations.

Example input/output:
Input: citations=[3,0,6,1,5]

Output: 3

Why: Three papers have at least three citations.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

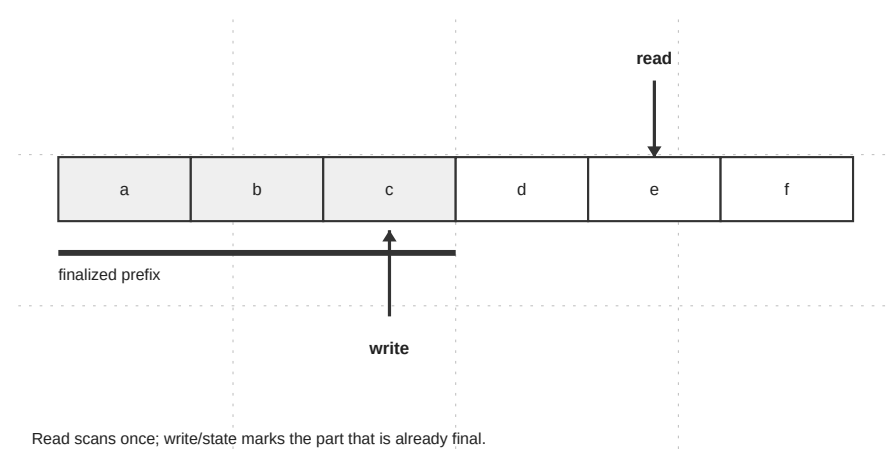
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 274. H-Index
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def h_index(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 087 of 229: 380. Insert Delete GetRandom O(1)

Pattern: Heap / Priority Queue | Sources: Top150 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Design a set supporting insert, remove, and getRandom in average $O(1)$ time.

Example input/output:
Input: insert(1), remove(2), insert(2), getRandom(), remove(1), insert(2), getRandom()

Output: [true,false,true,1 or 2,true,false,2]

Why: The final set contains only 2.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k \cdot n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

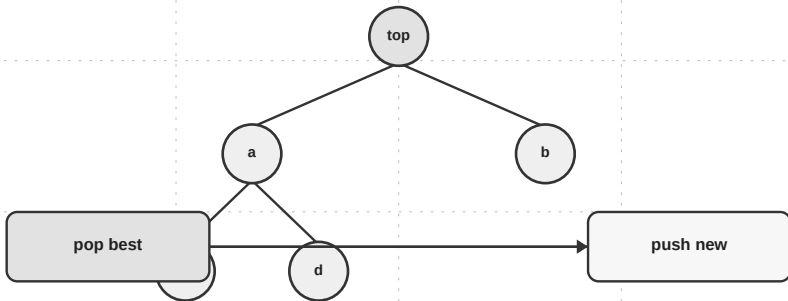
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 380. Insert Delete GetRandom O(1)
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

- Three drills:
- V1: Return top-k in sorted order after using the heap.
 - V2: Handle ties with a secondary key.
 - V3: Support streaming insertions and deletions.

- Follow-ups:
- Can you reduce memory from $O(n)$ to $O(k)$?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def insert_delete_getrandom_o_1(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 088 of 229: 134. Gas Station

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Given gas gained and travel cost at each station on a circle, return the start index that completes the circuit, or -1.

Example input/output:
Input: gas=[1,2,3,4,5], cost=[3,4,5,1,2]

Output: 3

Why: Starting at station 3 finishes the loop.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

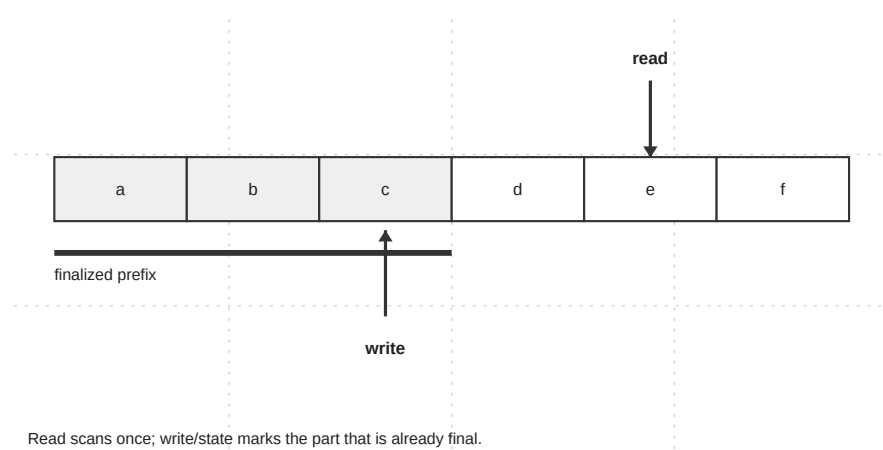
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 134. Gas Station
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def gas_station(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 089 of 229: 135. Candy

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Distribute candies so every child gets at least one and higher-rated children than neighbors get more; minimize total.

Example input/output:
Input: ratings=[1,0,2]

Output: 5

Why: Candies [2,1,2] is minimal.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

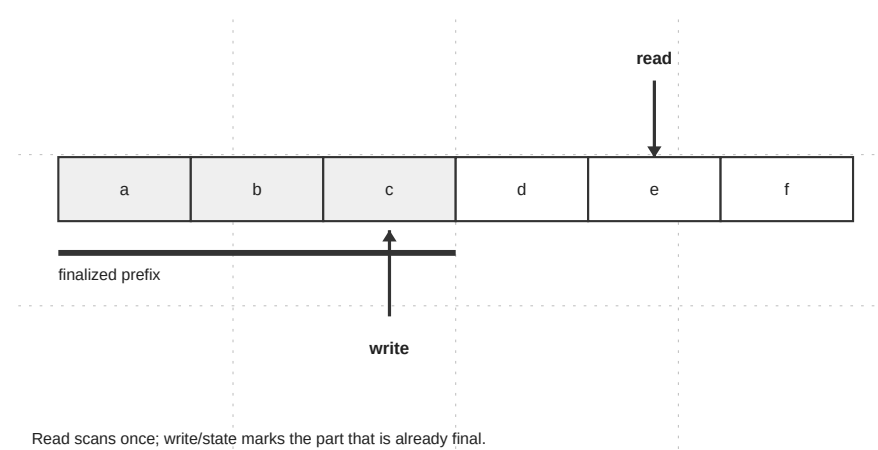
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 135. Candy
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:
V1: Require in-place edits instead of building a new array/string.
V2: Add Unicode/case-normalization rules before comparing characters.
V3: Stream inputs so only a small buffer may be kept.

Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def candy(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 090 of 229: 42. Trapping Rain Water

Pattern: Two Pointers | Sources: Top150 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Given bar heights, compute total rain water trapped between bars.

Example input/output:
Input: height=[0,2,0,3,0,1]

Output: 3

Why: Water accumulates above low bars between taller boundaries.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

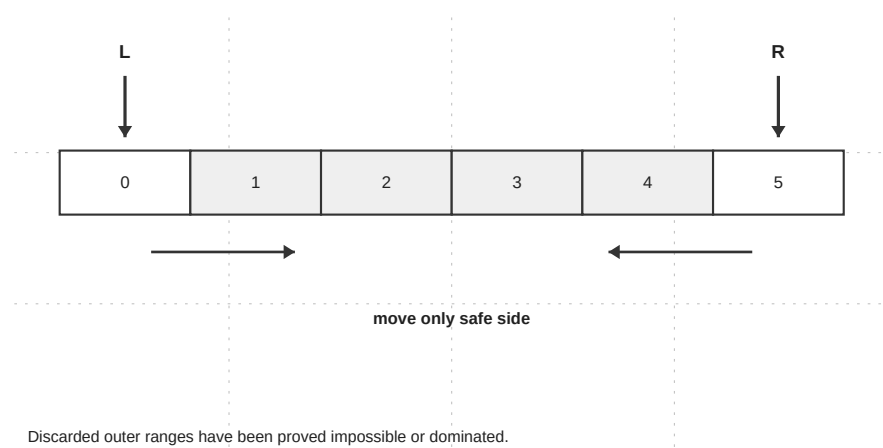
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 42. Trapping Rain Water
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most n times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def trapping_rain_water(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 091 of 229: 13. Roman to Integer

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Convert a valid Roman numeral to its integer value.

Example input/output:
Input: s="MCMXCIV"

Output: 1994

Why: M=1000, CM=900, XC=90, IV=4.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

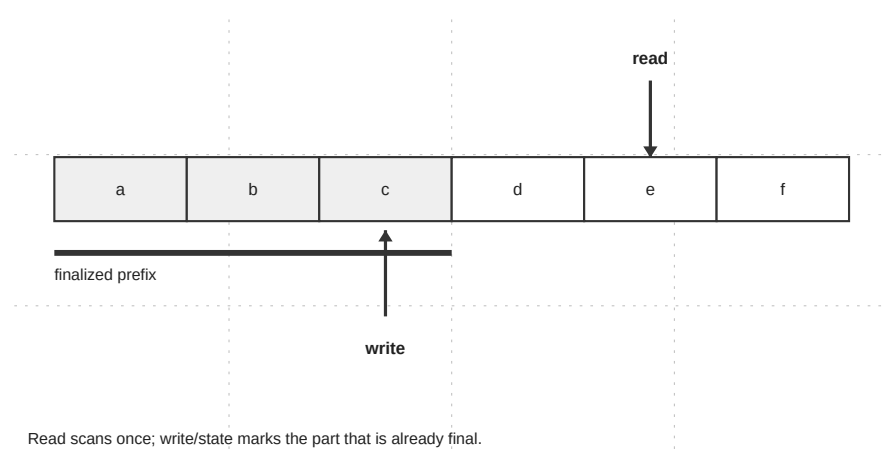
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 13. Roman to Integer
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

- V1: Require in-place edits instead of building a new array/string.
- V2: Add Unicode/case-normalization rules before comparing characters.
- V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def roman_to_integer(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 092 of 229: 12. Integer to Roman

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Convert an integer to a Roman numeral using standard subtractive notation.

Example input/output:
Input: num=58

Output: "LVIII"

Why: 50+5+3 is L+V+III.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

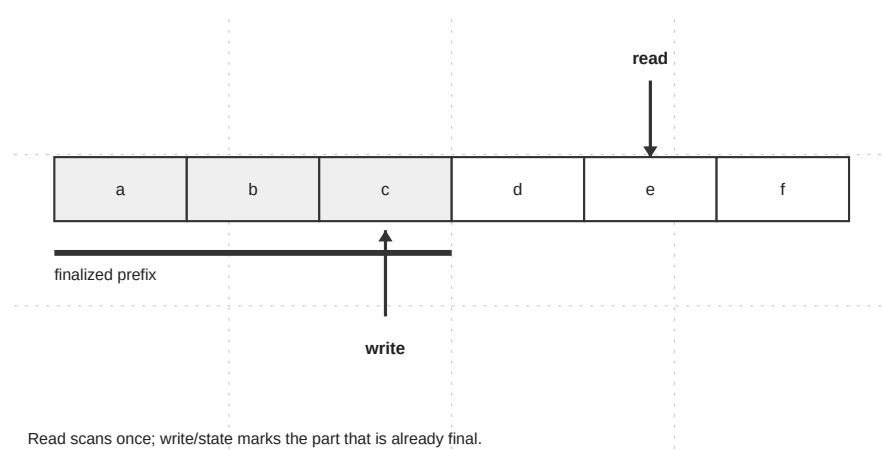
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 12. Integer to Roman
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def integer_to_roman(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 093 of 229: 58. Length of Last Word

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return the length of the last word in a string, ignoring trailing spaces.

Example input/output:
Input: s="fly me to the moon"

Output: 4

Why: The last word is "moon".

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

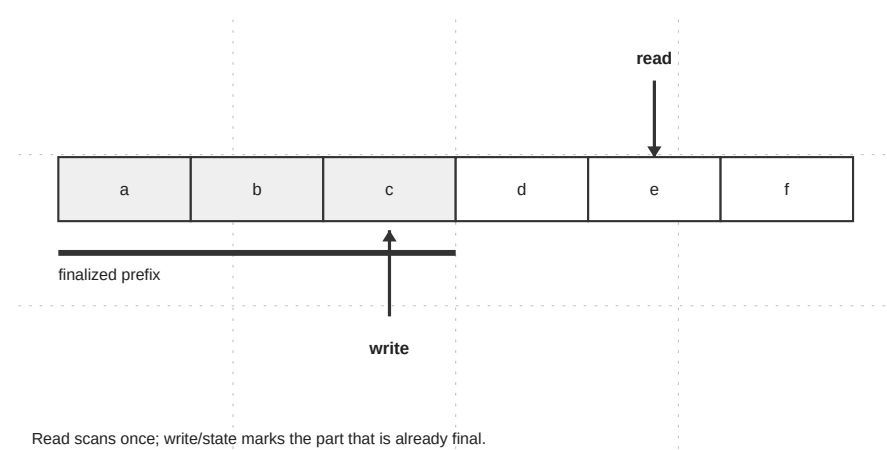
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 58. Length of Last Word
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:
V1: Require in-place edits instead of building a new array/string.
V2: Add Unicode/case-normalization rules before comparing characters.
V3: Stream inputs so only a small buffer may be kept.

Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def length_of_last_word(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 094 of 229: 14. Longest Common Prefix

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return the longest common prefix string among all input strings.

Example input/output:
Input: strs=["flower","flow","flight"]

Output: "fl"

Why: All words start with "fl".

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

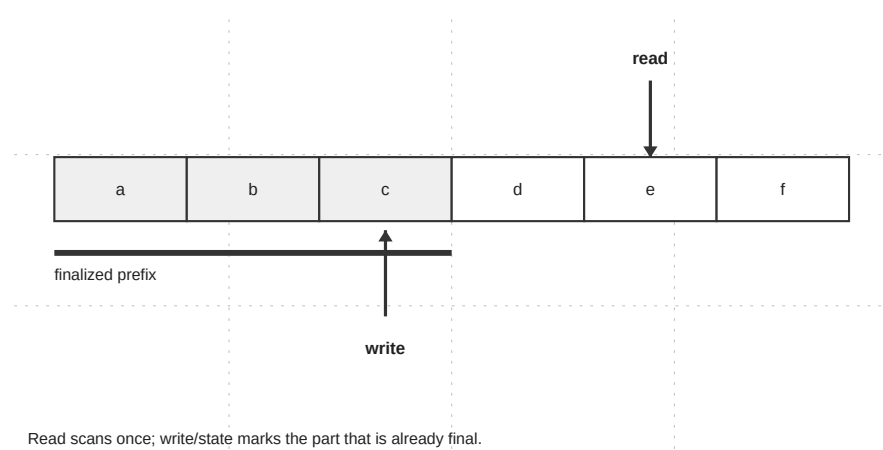
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 14. Longest Common Prefix
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

- V1: Require in-place edits instead of building a new array/string.
- V2: Add Unicode/case-normalization rules before comparing characters.
- V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_common_prefix(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 095 of 229: 6. Zigzag Conversion

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Write characters in a zigzag over numRows rows, then read row by row.

Example input/output:
Input: s="ABCDE", numRows=2

Output: "ACEBD"

Why: Rows are A,C,E and B,D.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

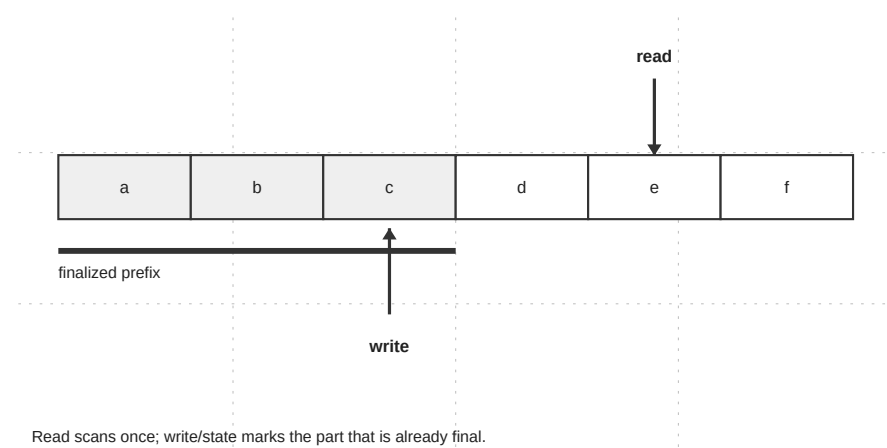
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 6. Zigzag Conversion
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def zigzag_conversion(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 096 of 229: 28. Find the Index of the First Occurrence in a String

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return the index of the first occurrence of needle in haystack, or -1 if absent.

Example input/output:
Input: haystack="sadbutsad", needle="sad"

Output: 0

Why: The first occurrence starts at index 0.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

Implementation checkpoints:

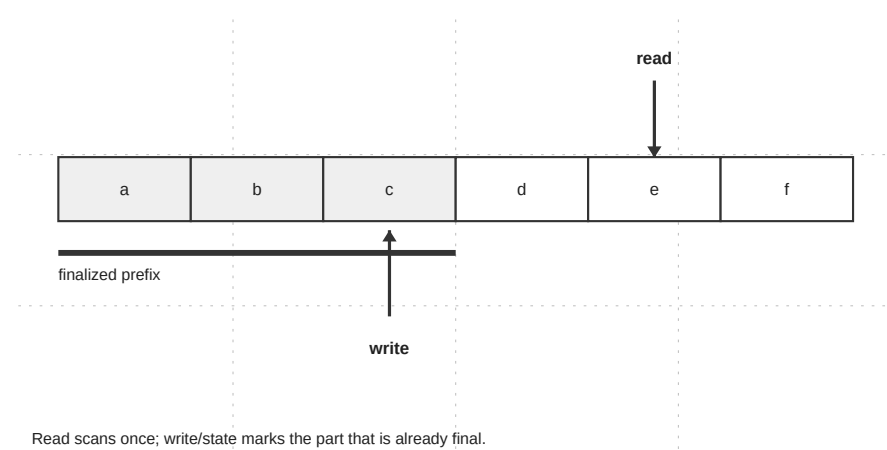
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 28. Find the Index of the First Occurrence in a String
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:

- V1: Require in-place edits instead of building a new array/string.
- V2: Add Unicode/case-normalization rules before comparing characters.
- V3: Stream inputs so only a small buffer may be kept.

Follow-ups:

- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_the_index_of_the_first_occurren(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 097 of 229: 68. Text Justification

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Pack words into lines of maxWidth with spaces distributed as evenly as possible; last line is left-justified.

Example input/output:
Input: words=["This","is","an","example"], maxWidth=16

Output: ["This is an","example "]

Why: First line spreads spaces; final line pads right.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

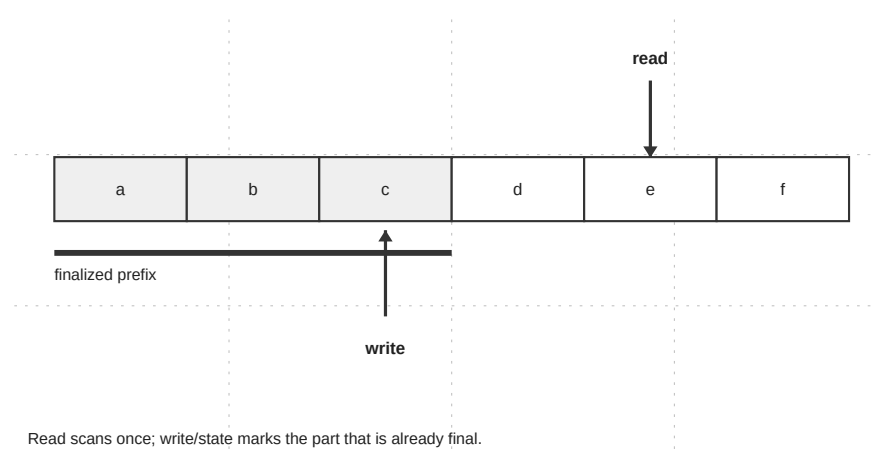
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 68. Text Justification
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

Three drills:
V1: Require in-place edits instead of building a new array/string.
V2: Add Unicode/case-normalization rules before comparing characters.
V3: Stream inputs so only a small buffer may be kept.

Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def text_justification(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 098 of 229: 125. Valid Palindrome

Pattern: Two Pointers | Sources: Top150, NC75 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: O(n) or O(n log n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return whether a string is a palindrome after removing non-alphanumeric characters and ignoring case.

Example input/output:
Input: s="A man, a plan, a canal: Panama"

Output: true

Why: Cleaned string reads the same both ways.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but O(n^2) or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

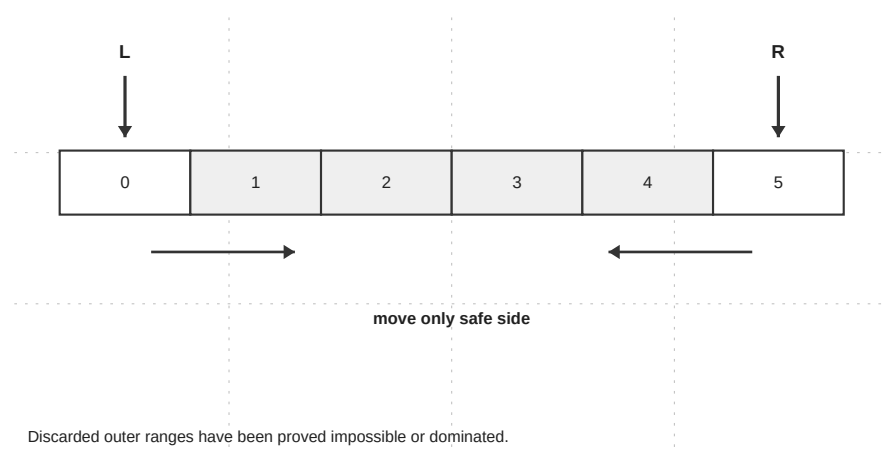
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 125. Valid Palindrome
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time O(n) or O(n log n); Space O(1) to O(n)

Why:
Each pointer moves monotonically at most n times, giving O(n) after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:
V1: Return the actual pair/window, not only the score.
V2: Handle duplicates and ask for all unique answers.
V3: Work after sorting while preserving original indices.

Follow-ups:
- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def valid_palindrome(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 099 of 229: 167. Two Sum II - Input Array Is Sorted

Pattern: Two Pointers | Sources: Top150 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
In a 1-indexed sorted array, return the two indices whose values sum to target.

Example input/output:
Input: numbers=[2,7,11,15], target=9

Output: [1,2]

Why: $2+7=9$.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

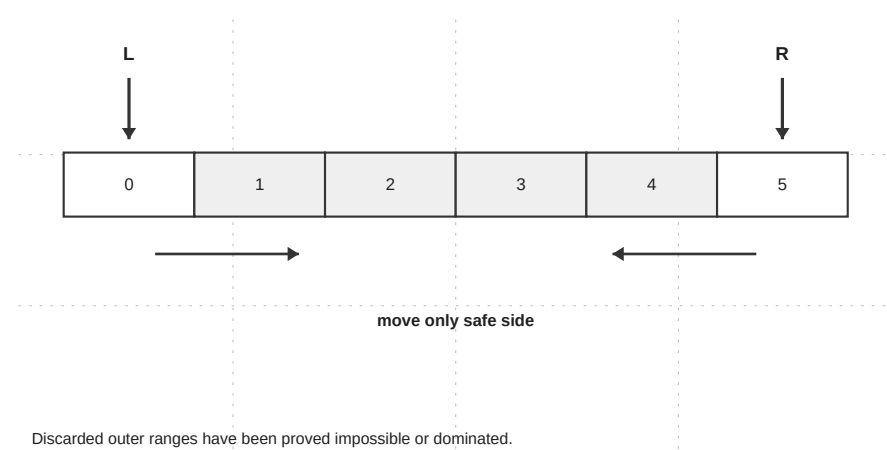
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 167. Two Sum II - Input Array Is Sorted
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most n times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def two_sum_ii_input_array_is_sorted(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 100 of 229: 15. 3Sum

Pattern: Two Pointers | Sources: Top150, NC75 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: O(n) or O(n log n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return all unique triplets whose sum is zero.

Example input/output:
Input: nums=[-1,0,1,2,-1,-4]

Output: [[-1,-1,2],[-1,0,1]]

Why: Sort and skip duplicates.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but O(n^2) or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

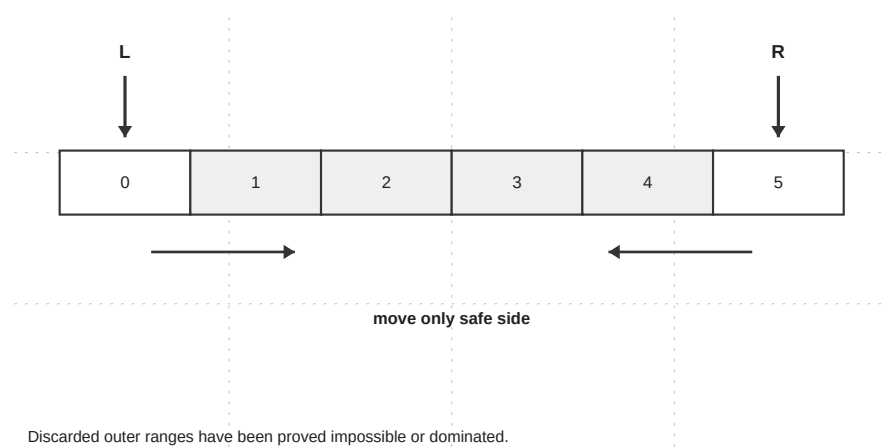
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 15. 3Sum
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time O(n) or O(n log n); Space O(1) to O(n)

Why:
Each pointer moves monotonically at most n times, giving O(n) after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def 3sum(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 101 of 229: 209. Minimum Size Subarray Sum

Pattern: Two Pointers | Sources: Top150 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Return the minimum length of a contiguous subarray with sum at least target, or 0 if none exists.

Example input/output:
Input: target=7, nums=[2,3,1,2,4,3]

Output: 2

Why: Subarray [4,3] reaches 7.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

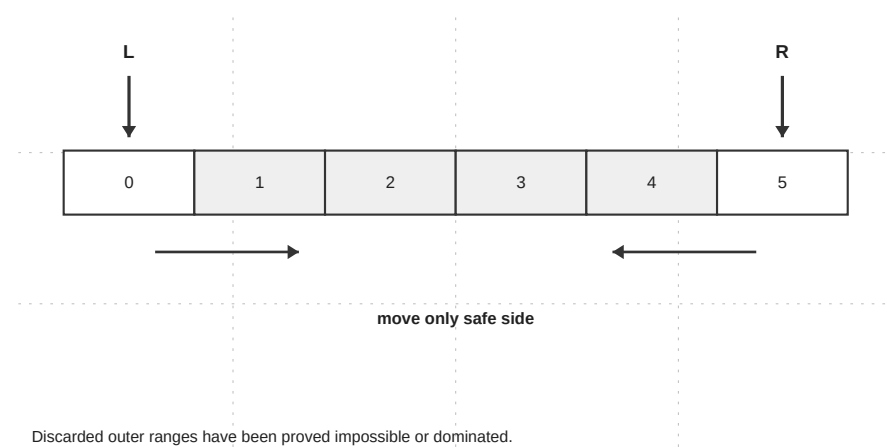
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 209. Minimum Size Subarray Sum
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most n times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def minimum_size_subarray_sum(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 102 of 229: 3. Longest Substring Without Repeating Characters

Pattern: Sliding Window | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the length of the longest substring without repeated characters.

Example input/output:
Input: s="abcabcbb"

Output: 3

Why: "abc" is the longest unique-character window.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

Implementation checkpoints:

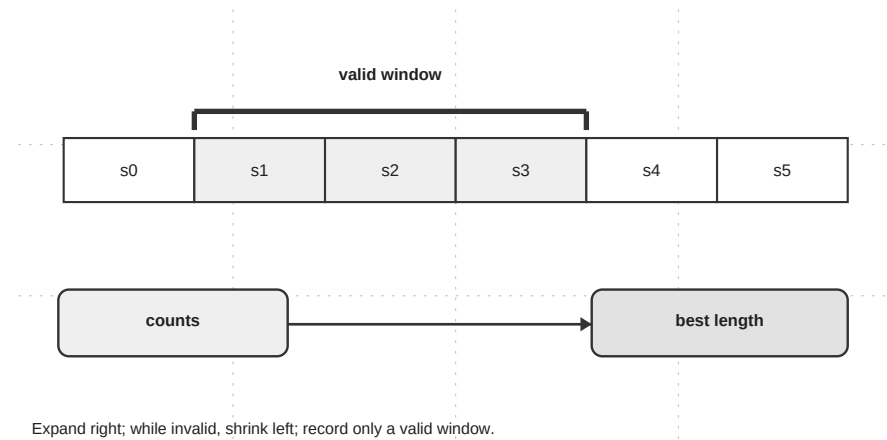
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 3. Longest Substring Without Repeating Characters
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Shrinking only once when a while loop is needed; stale max/count variables.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $k=0$; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left}+1$.

Variations and follow-ups

Three drills:

- V1: Switch from at-most- k to exactly- k constraints.
- V2: Return the smallest valid window instead of the largest.
- V3: Support multiple character/value constraints at once.

Follow-ups:

- Can you return the actual window, not only its length/count?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_substring_without_repeating_(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1
            left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 103 of 229: 30. Substring with Concatenation of All Words

Pattern: Two Pointers | Sources: Top150 | Companies: Amazon, Meta, Apple, Microsoft (public samples)

Time: $O(n)$ or $O(n \log n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Find all start indices where a substring is the concatenation of each word exactly once, with no gaps.

Example input/output:
Input: s="barfoothefoobarman", words=["foo","bar"]

Output: [0,9]

Why: "barfoo" and "foobar" match.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Check all pairs/subarrays. This is clear but $O(n^2)$ or worse.

Optimized approach:
Maintain two indices. Move only the side that cannot produce a better result if kept fixed.

Alternative:
Sort first for unordered inputs; use fast/slow pointers for linked lists or cycle questions.

Implementation checkpoints:

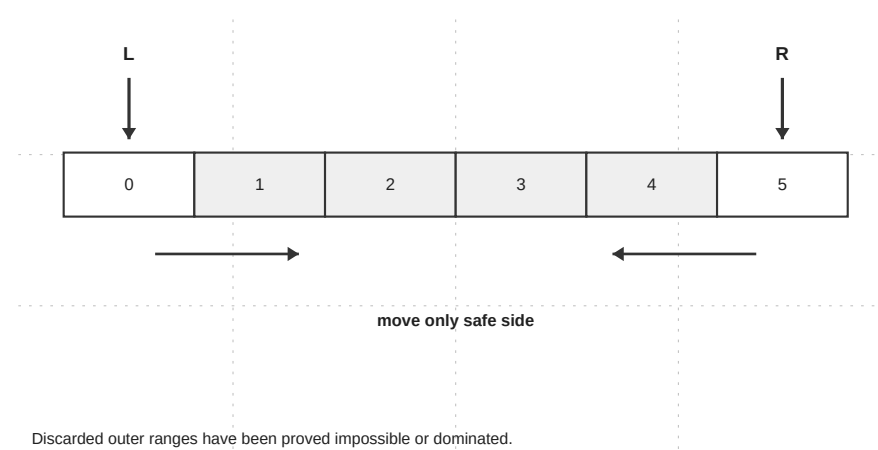
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 30. Substring with Concatenation of All Words
func solveTwoPointers(nums []int, target int) int {
    left, right := 0, len(nums)-1
    best := 0
    for left < right {
        // Evaluate the pair/window formed by the two borders.
        score := evaluate(nums, left, right, target)
        best = max(best, score)
        // Move the side that cannot improve if it stays fixed.
        if shouldMoveLeft(nums, left, right, target) {
            left++
        } else {
            right--
        }
    }
    return best
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All discarded pairs are provably dominated or already evaluated.

Complexity target:
Time $O(n)$ or $O(n \log n)$; Space $O(1)$ to $O(n)$

Why:
Each pointer moves monotonically at most n times, giving $O(n)$ after any required sort.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Moving both pointers when only one should move; duplicate skipping before recording answer.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; crossing pointers; arrays of length 0/1; sorted vs unsorted; inclusive loop conditions.

Variations and follow-ups

Three drills:

- V1: Return the actual pair/window, not only the score.
- V2: Handle duplicates and ask for all unique answers.
- V3: Work after sorting while preserving original indices.

Follow-ups:

- What changes if input is unsorted or has duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def substring_with_concatenation_of_all(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All discarded pairs are provably dominated or already evaluated.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 104 of 229: 76. Minimum Window Substring

Pattern: Sliding Window | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the smallest substring of s that contains every character of t with required multiplicity.

Example input/output:
Input: s="ADOBECODEBANC", t="ABC"

Output: "BANC"

Why: It is the shortest window containing A, B, and C.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

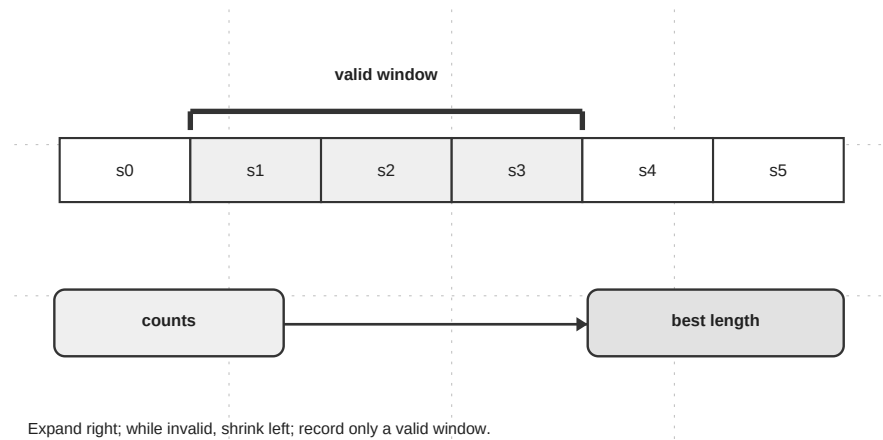
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 76. Minimum Window Substring
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Shrinking only once when a while loop is needed; stale max/count variables.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
k=0; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left}+1$.

Variations and follow-ups

- Three drills:
- V1: Switch from at-most-k to exactly-k constraints.
 - V2: Return the smallest valid window instead of the largest.
 - V3: Support multiple character/value constraints at once.

- Follow-ups:
- Can you return the actual window, not only its length/count?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def minimum_window_substring(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1; left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 105 of 229: 36. Valid Sudoku

Pattern: Hash Map / Set | Sources: Top150 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Return whether a partially filled 9x9 Sudoku board has no repeated digit in any row, column, or 3x3 box.

Example input/output:
Input:
board=["53..7....","6..195...",".98....6.","8...6...3","4..8.3..1","7...2...6",".6....28.","...419..5","....8..79"]

Output: true

Why: All filled digits obey Sudoku rules.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

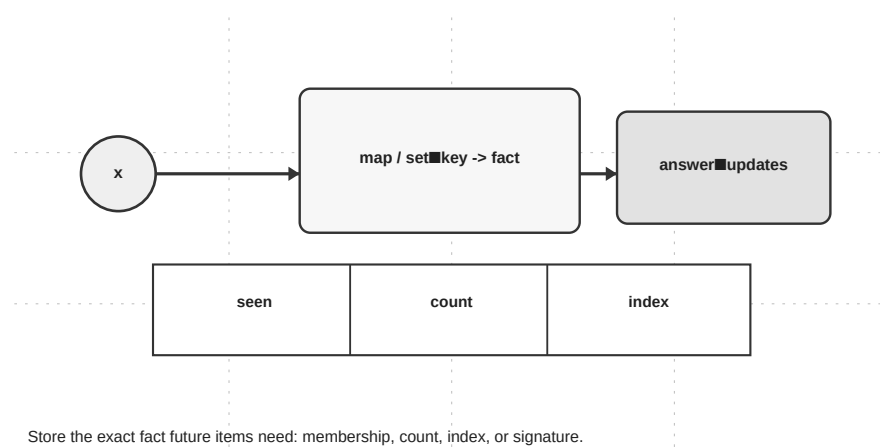
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 36. Valid Sudoku
func solveWithMap(nums [][]int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

- Three drills:
- V1: Return all matching groups instead of a boolean/count.
 - V2: Support incremental updates while queries arrive.
 - V3: Handle case-insensitive or normalized keys.

- Follow-ups:
- How would memory limits change the approach?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def valid_sudoku(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 106 of 229: 54. Spiral Matrix

Pattern: Array / String | Sources: Top150, NC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return all matrix elements in clockwise spiral order.

Example input/output:
Input: matrix=[[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

Why: Traverse top row, right column, bottom row, left column, inward.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

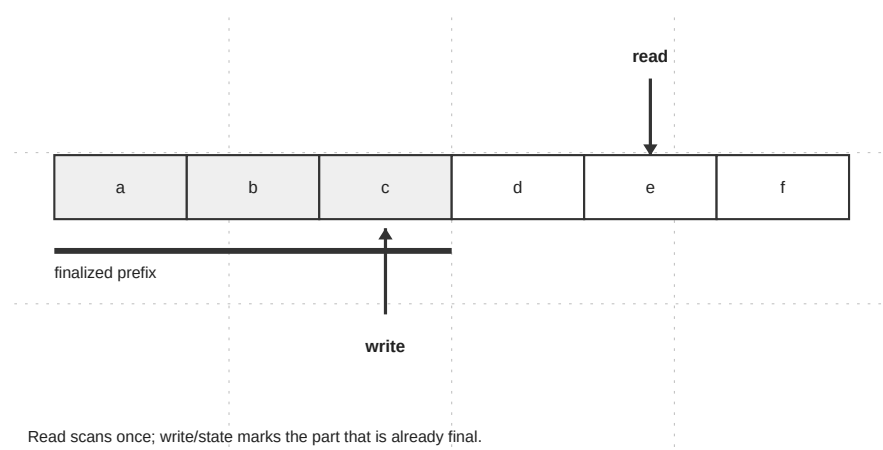
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 54. Spiral Matrix
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def spiral_matrix(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 107 of 229: 48. Rotate Image

Pattern: Array / String | Sources: Top150, NC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Rotate an n by n matrix 90 degrees clockwise in-place.

Example input/output:
Input: matrix=[[1,2],[3,4]]

Output: [[3,1],[4,2]]

Why: Transpose then reverse rows.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

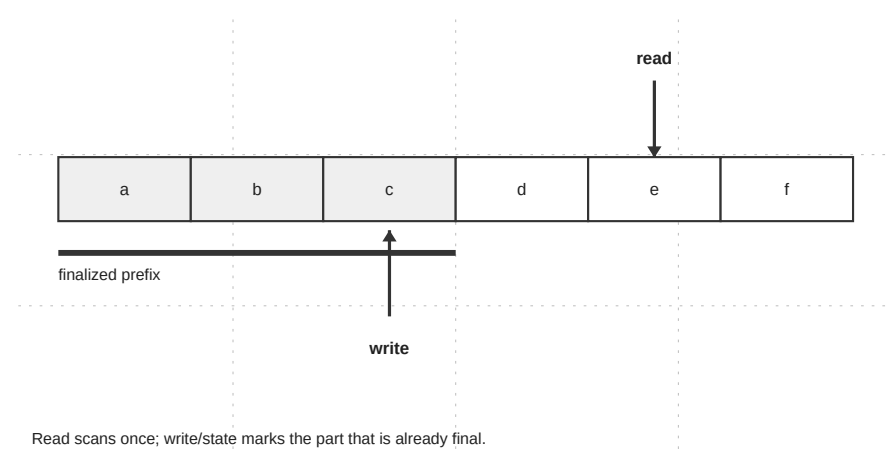
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 48. Rotate Image
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def rotate_image(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 108 of 229: 73. Set Matrix Zeroes

Pattern: Array / String | Sources: Top150, NC75 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
If any matrix cell is zero, set its entire row and column to zero in-place.

Example input/output:
Input: matrix=[[1,1,1],[1,0,1],[1,1,1]]

Output: [[1,0,1],[0,0,0],[1,0,1]]

Why: The zero at center clears row 1 and column 1.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

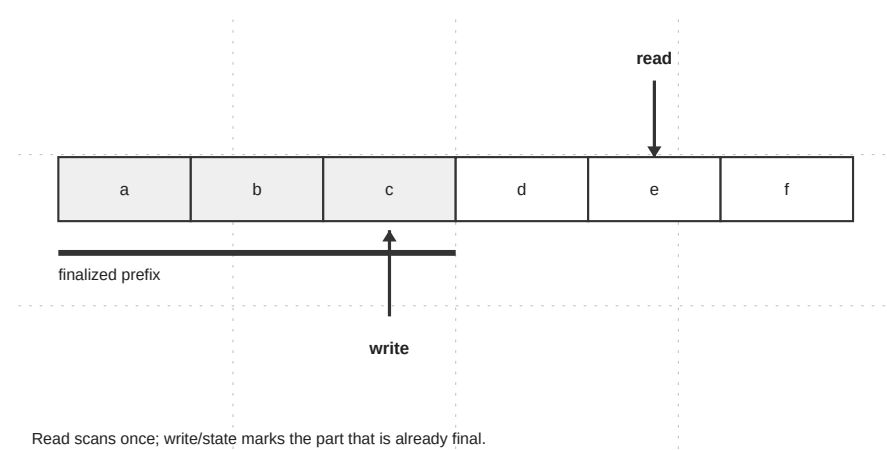
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 73. Set Matrix Zeroes
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def set_matrix_zeroes(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 109 of 229: 289. Game of Life

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Update the Game of Life board one step in-place using the eight-neighbor rules for birth, survival, and death.

Example input/output:
Input: board=[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]

Why: Apply all state changes simultaneously.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

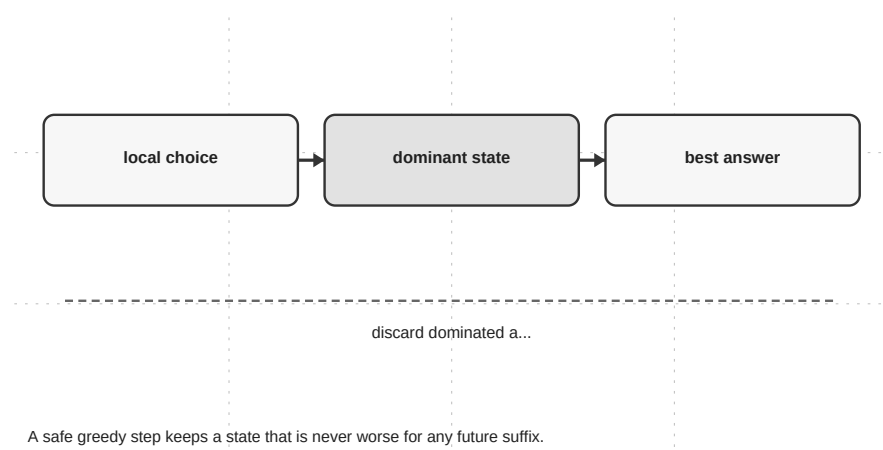
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 289. Game of Life
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def game_of_life(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 110 of 229: 383. Ransom Note

Pattern: Array / String | Sources: Top150 | Companies: Amazon, Microsoft, Apple, Bloomberg (public samples)

Time: O(n)

Space: O(1) to O(n)

Problem statement + sample

Prompt:
Return whether ransomNote can be built using each character from magazine at most once.

Example input/output:
Input: ransomNote="aa", magazine="aab"

Output: true

Why: Magazine has two a characters.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Build every possible candidate or repeatedly scan the string/array. Easy to reason about but often O(n^2) or allocates too much.

Optimized approach:
Use one forward scan, a write pointer, or a small frequency/count state. Mutate only when the prompt explicitly allows it.

Alternative:
When order is irrelevant, sort or count first; when order matters, preserve stable scan order.

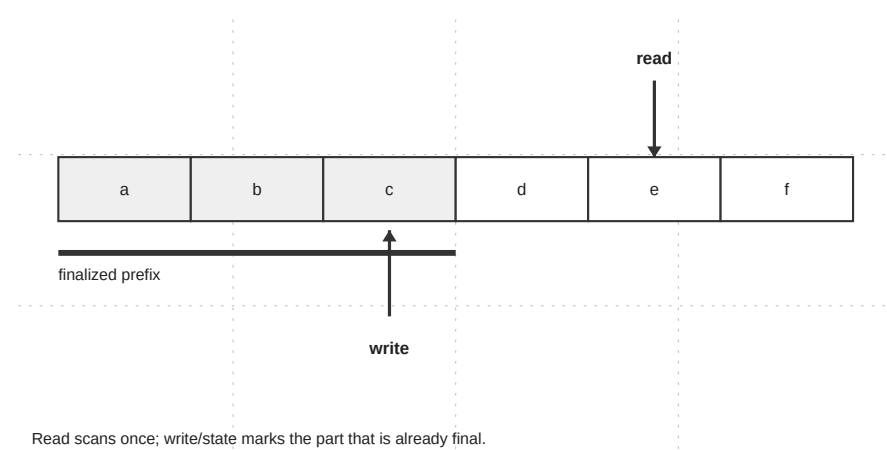
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 383. Ransom Note
func solveArrayString(chars []rune) int {
    write, ans := 0, 0
    for read, ch := range chars {
        // read scans every item once; write marks the kept/mutated prefix.
        if keepOrUse(ch, read) {
            chars[write] = ch
            write++
            ans = update(ans, write, ch)
        }
    }
    // Return ans, write, or string(chars[:write]) depending on prompt.
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
After processing index i, all positions before write already contain the final kept prefix or answer state.

Complexity target:
Time O(n); Space O(1) to O(n)

Why:
One pass touches each element once. Extra space is O(1) unless the returned output/frequency table is required.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Off-by-one on in-place writes; forgetting empty strings/arrays.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty input; one item; all equal values; long strings; in-place vs copy; Unicode/byte assumptions.

Variations and follow-ups

- Three drills:
- V1: Require in-place edits instead of building a new array/string.
 - V2: Add Unicode/case-normalization rules before comparing characters.
 - V3: Stream inputs so only a small buffer may be kept.

- Follow-ups:
- Can it be done in-place, streaming, or with Unicode/non-ASCII input?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def ransom_note(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
After processing index i, all positions before write already contain the final kept prefix or answer state.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 111 of 229: 205. Isomorphic Strings

Pattern: Hash Map / Set | Sources: Top150 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Return whether characters in s can be replaced one-to-one to create t, preserving order.

Example input/output:
Input: s="egg", t="add"

Output: true

Why: e maps to a and g maps to d.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

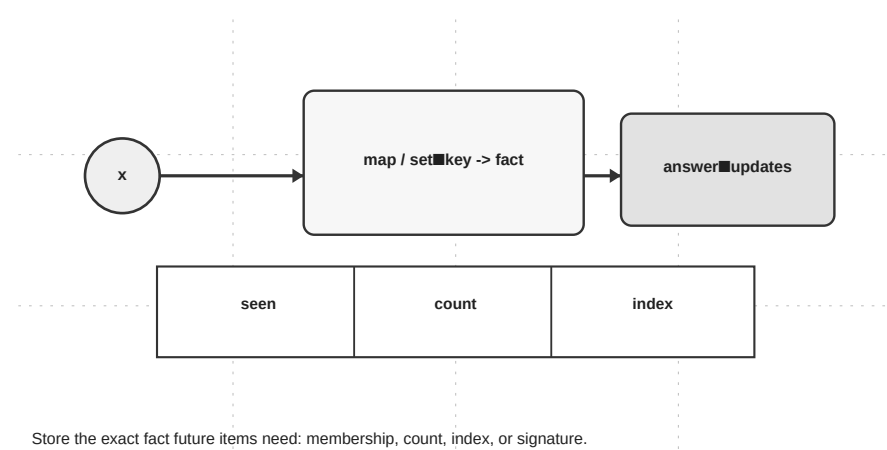
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 205. Isomorphic Strings
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

- Three drills:
- V1: Return all matching groups instead of a boolean/count.
 - V2: Support incremental updates while queries arrive.
 - V3: Handle case-insensitive or normalized keys.

- Follow-ups:
- How would memory limits change the approach?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def isomorphic_strings(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 112 of 229: 290. Word Pattern

Pattern: Hash Map / Set | Sources: Top150 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Return whether a pattern's characters map one-to-one to words in the string.

Example input/output:
Input: pattern="abba", s="dog cat cat dog"

Output: true

Why: a maps to dog and b maps to cat.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

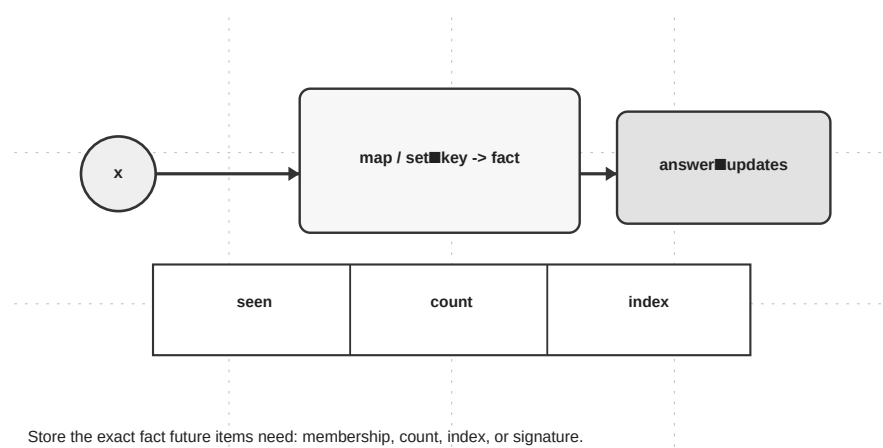
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 290. Word Pattern
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

- Three drills:
- V1: Return all matching groups instead of a boolean/count.
 - V2: Support incremental updates while queries arrive.
 - V3: Handle case-insensitive or normalized keys.

- Follow-ups:
- How would memory limits change the approach?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def word_pattern(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 113 of 229: 242. Valid Anagram

Pattern: Hash Map / Set | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return whether two strings contain exactly the same character counts.

Example input/output:
Input: s="anagram", t="nagaram"

Output: true

Why: Both have the same letters.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

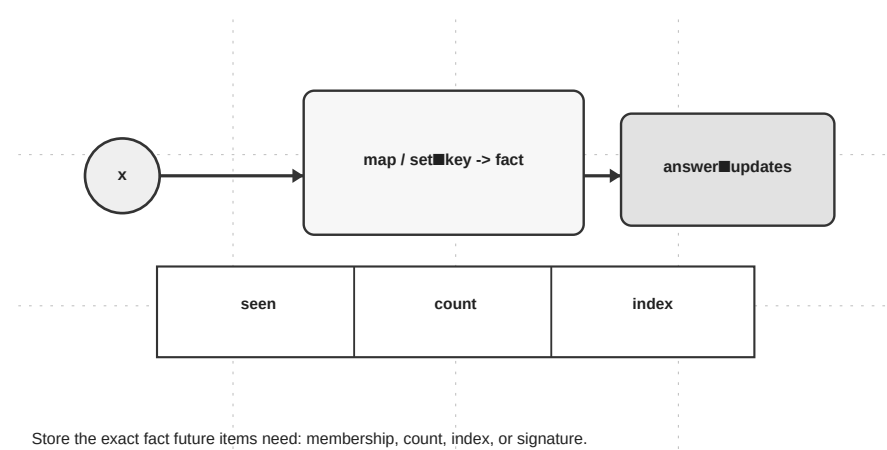
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 242. Valid Anagram
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:
V1: Return all matching groups instead of a boolean/count.
V2: Support incremental updates while queries arrive.
V3: Handle case-insensitive or normalized keys.

Follow-ups:
- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def valid_anagram(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 114 of 229: 49. Group Anagrams

Pattern: Hash Map / Set | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Group strings that are anagrams of each other.

Example input/output:
Input: strs=["eat","tea","tan","ate","nat","bat"]

Output: [["eat","tea","ate"],["tan","nat"],["bat"]]

Why: Anagrams share sorted letters or counts.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

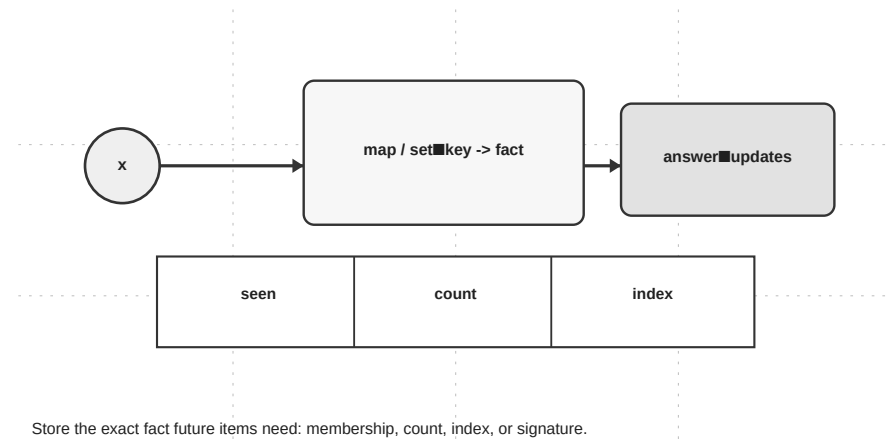
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 49. Group Anagrams
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

- Three drills:
- V1: Return all matching groups instead of a boolean/count.
 - V2: Support incremental updates while queries arrive.
 - V3: Handle case-insensitive or normalized keys.

- Follow-ups:
- How would memory limits change the approach?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def group_anagrams(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 115 of 229: 1. Two Sum

Pattern: Hash Map / Set | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Return indices of the two numbers that add to target; assume exactly one answer and do not reuse an element.

Example input/output:
Input: nums=[2,7,11,15], target=9

Output: [0,1]

Why: 2+7=9.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

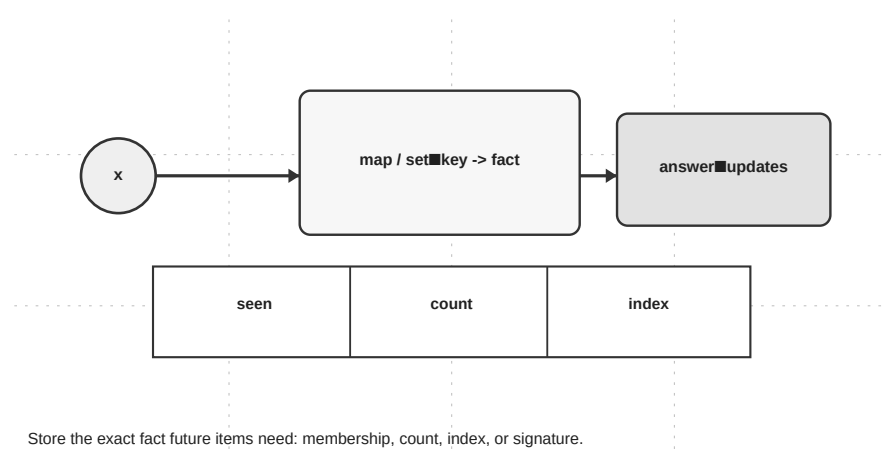
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 1. Two Sum
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:
V1: Return all matching groups instead of a boolean/count.
V2: Support incremental updates while queries arrive.
V3: Handle case-insensitive or normalized keys.

Follow-ups:
- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def two_sum(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 116 of 229: 202. Happy Number

Pattern: Hash Map / Set | Sources: Top150 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Repeatedly replace n by the sum of squares of its digits; return true if the process reaches 1.

Example input/output:
Input: n=19

Output: true

Why: 19 eventually reaches 1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

Implementation checkpoints:

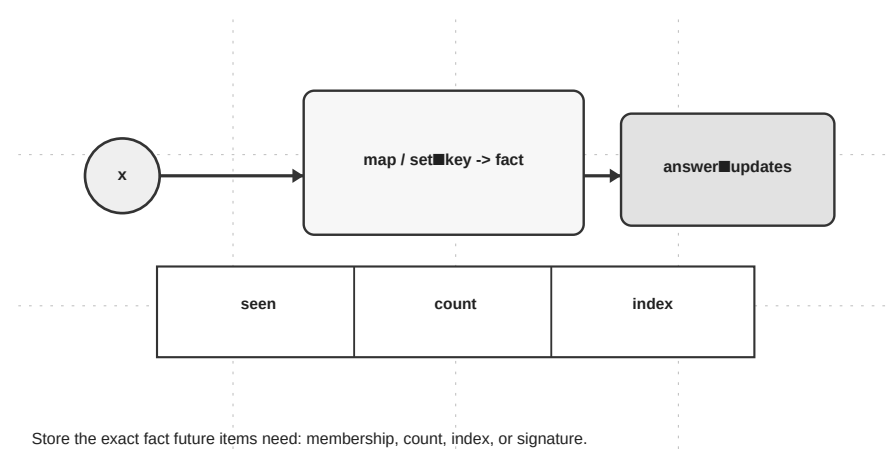
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 202. Happy Number
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:

- V1: Return all matching groups instead of a boolean/count.
- V2: Support incremental updates while queries arrive.
- V3: Handle case-insensitive or normalized keys.

Follow-ups:

- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def happy_number(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 117 of 229: 219. Contains Duplicate II

Pattern: Hash Map / Set | Sources: Top150 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Return true if the same value appears at two indices whose distance is at most k.

Example input/output:
Input: nums=[1,2,3,1], k=3

Output: true

Why: The two 1 values are distance 3 apart.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually O(n^2).

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

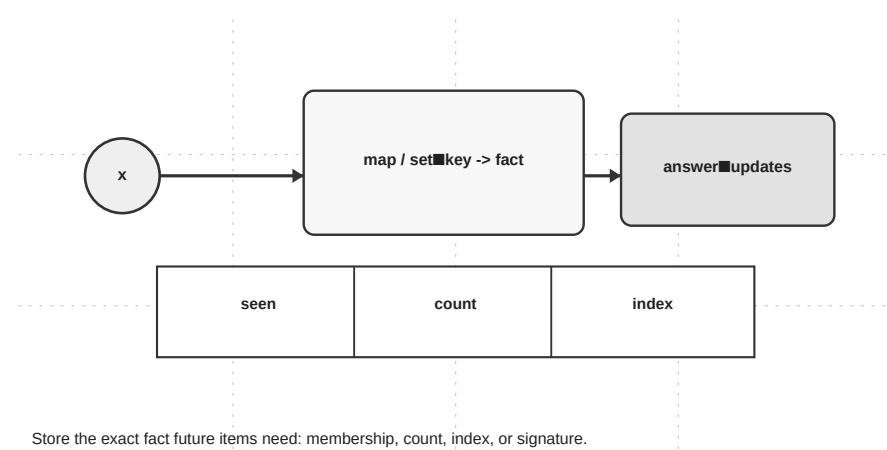
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 219. Contains Duplicate II
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time O(n); Space O(n)

Why:
Expected O(n) lookups/inserts. Space is O(k), where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:
V1: Return all matching groups instead of a boolean/count.
V2: Support incremental updates while queries arrive.
V3: Handle case-insensitive or normalized keys.

Follow-ups:
- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def contains_duplicate_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 118 of 229: 128. Longest Consecutive Sequence

Pattern: Hash Map / Set | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return the length of the longest consecutive integer sequence in an unsorted array.

Example input/output:
Input: nums=[100,4,200,1,3,2]

Output: 4

Why: Sequence 1,2,3,4 has length 4.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

Implementation checkpoints:

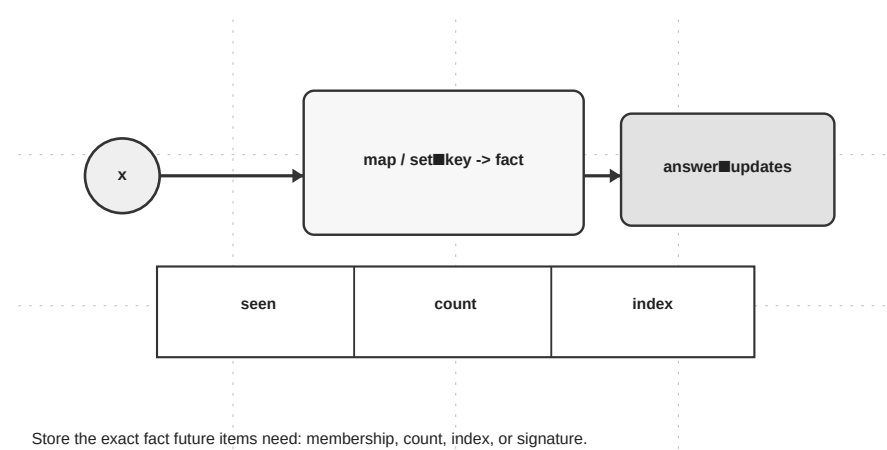
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 128. Longest Consecutive Sequence
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:

- V1: Return all matching groups instead of a boolean/count.
- V2: Support incremental updates while queries arrive.
- V3: Handle case-insensitive or normalized keys.

Follow-ups:

- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_consecutive_sequence(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 119 of 229: 228. Summary Ranges

Pattern: Prefix / Scan | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(1)$ to $O(n)$

Problem statement + sample

Prompt:
Summarize sorted unique integers into minimal ranges.

Example input/output:
Input: nums=[0,1,2,4,5,7]

Output: ["0->2","4->5","7"]

Why: Consecutive runs become ranges.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recompute every left/right sum or product per index. This repeats work and often becomes $O(n^2)$.

Optimized approach:
Carry a running prefix/suffix value. Update answer at the exact point where left and right states are both known.

Alternative:
Two passes are simplest; one pass works when the equation can be rearranged.

Implementation checkpoints:

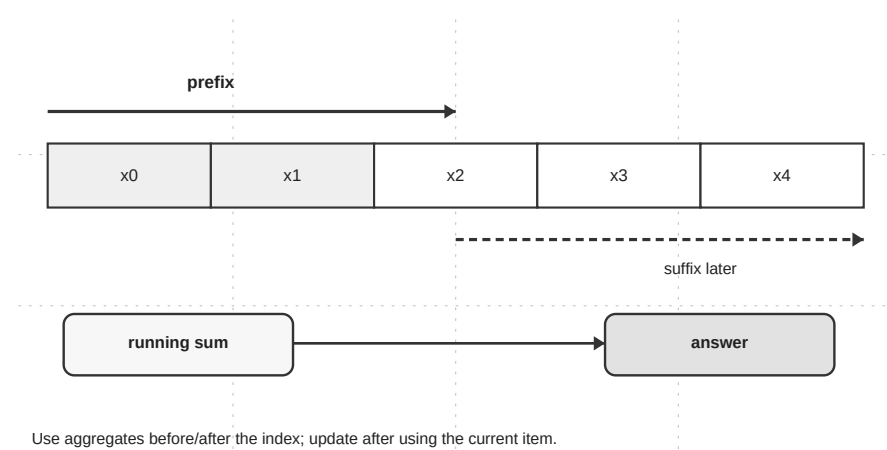
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 228. Summary Ranges
func solvePrefixScan(nums []int) int {
    total := 0
    for _, x := range nums { total += x }
    prefix, ans := 0, defaultAnswer()
    for i, x := range nums {
        suffix := total - prefix - x
        if condition(prefix, x, suffix) {
            ans = updateAnswer(ans, i)
        }
        prefix += x // next index sees this x on its left
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before index i , prefix equals the aggregate of all elements strictly left of i .

Complexity target:
Time $O(n)$; Space $O(1)$ to $O(n)$

Why:
Each element is added/multiplied a constant number of times, so time is $O(n)$; space is $O(1)$ or $O(n)$ for output arrays.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using updated prefix too early; not handling index 0.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
zeros; negative numbers; overflow in Go int; empty arrays; inclusive/exclusive index boundary.

Variations and follow-ups

Three drills:

- V1: Answer many range queries after one preprocessing pass.
- V2: Handle zeros/negative numbers without division.
- V3: Return both value and index where the best prefix occurs.

Follow-ups:

- Can you answer many range queries after preprocessing?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def summary_ranges(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before index i , prefix equals the aggregate of all elements strictly left of i .

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 120 of 229: 56. Merge Intervals

Pattern: Intervals | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Bloomberg (public samples)

Time: $O(n \log n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Merge all overlapping intervals and return the merged intervals.

Example input/output:
Input: intervals=[[1,3],[2,6],[8,10]]

Output: [[1,6],[8,10]]

Why: The first two intervals overlap.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every interval pair repeatedly. $O(n^2)$.

Optimized approach:
Sort once, then maintain the active end or merged interval while scanning.

Alternative:
Sweep line with events is useful for counting overlaps/rooms.

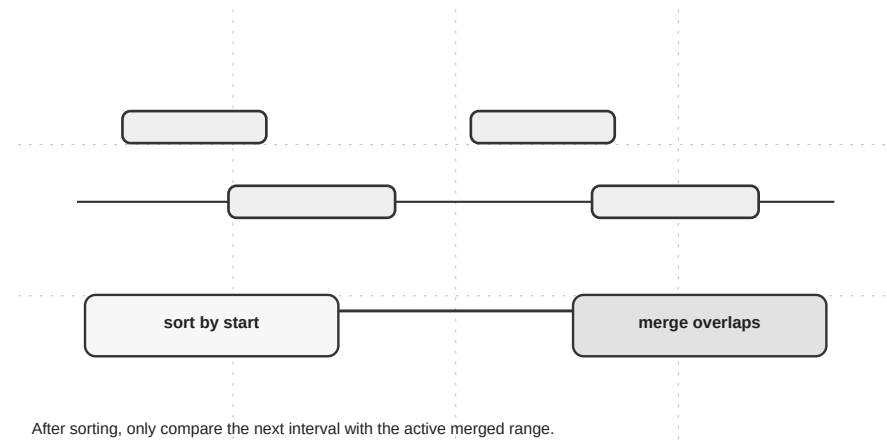
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 56. Merge Intervals
func solveIntervals(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, in := range intervals {
        if len(merged) == 0 || merged[len(merged)-1][1] < in[0] {
            merged = append(merged, append([]int(nil), in...))
        } else if in[1] > merged[len(merged)-1][1] {
            merged[len(merged)-1][1] = in[1] // extend overlap
        }
    }
    return merged
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Complexity target:
Time $O(n \log n)$; Space $O(n)$

Why:
Sorting dominates at $O(n \log n)$; the scan is $O(n)$. Space is $O(1)$ to $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Sorting by wrong key; not merging touching intervals when required.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
touching endpoints; inclusive vs exclusive; empty list; sorting tie-breakers; nested intervals.

Variations and follow-ups

Three drills:
V1: Distinguish touching endpoints from overlapping endpoints.
V2: Return removed intervals as well as the merged result.
V3: Process online insertions into an existing schedule.

- Follow-ups:
- Can you process intervals online?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def merge_intervals(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All intervals before current have been merged or counted consistently with the chosen end boundary.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 121 of 229: 57. Insert Interval

Pattern: Intervals | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Bloomberg (public samples)

Time: $O(n \log n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Insert a new interval into sorted non-overlapping intervals and merge if needed.

Example input/output:
Input: intervals=[[1,3],[6,9]], newInterval=[2,5]

Output: [[1,5],[6,9]]

Why: The new interval overlaps [1,3].

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every interval pair repeatedly. $O(n^2)$.

Optimized approach:
Sort once, then maintain the active end or merged interval while scanning.

Alternative:
Sweep line with events is useful for counting overlaps/rooms.

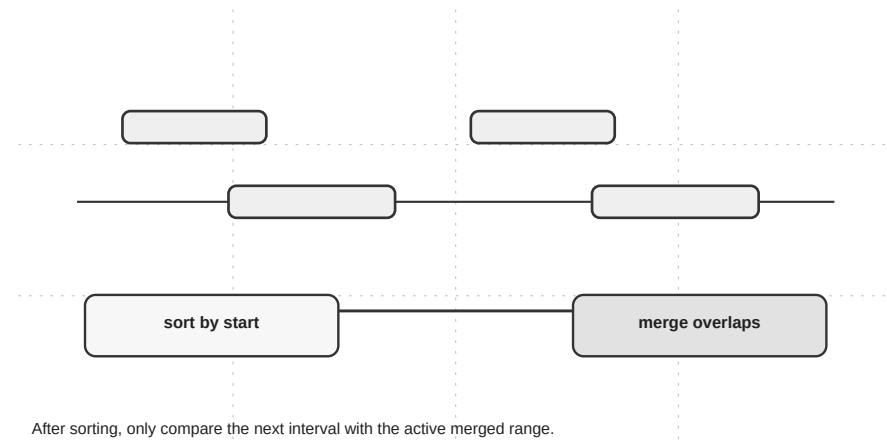
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 57. Insert Interval
func solveIntervals(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, in := range intervals {
        if len(merged) == 0 || merged[len(merged)-1][1] < in[0] {
            merged = append(merged, append([]int(nil), in...))
        } else if in[1] > merged[len(merged)-1][1] {
            merged[len(merged)-1][1] = in[1] // extend overlap
        }
    }
    return merged
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Complexity target:
Time $O(n \log n)$; Space $O(n)$

Why:
Sorting dominates at $O(n \log n)$; the scan is $O(n)$. Space is $O(1)$ to $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Sorting by wrong key; not merging touching intervals when required.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
touching endpoints; inclusive vs exclusive; empty list; sorting tie-breakers; nested intervals.

Variations and follow-ups

- Three drills:
- V1: Distinguish touching endpoints from overlapping endpoints.
 - V2: Return removed intervals as well as the merged result.
 - V3: Process online insertions into an existing schedule.

- Follow-ups:
- Can you process intervals online?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def insert_interval(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All intervals before current have been merged or counted consistently with the chosen end boundary.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 122 of 229: 20. Valid Parentheses

Pattern: Stack | Sources: Top150, NC75 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return whether a bracket string is valid: each opening bracket is closed by the same type in correct order.

Example input/output:

Input: `s="({})"`

Output: `true`

Why: The stack closes brackets in reverse order.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become $O(n^2)$.

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

Implementation checkpoints:

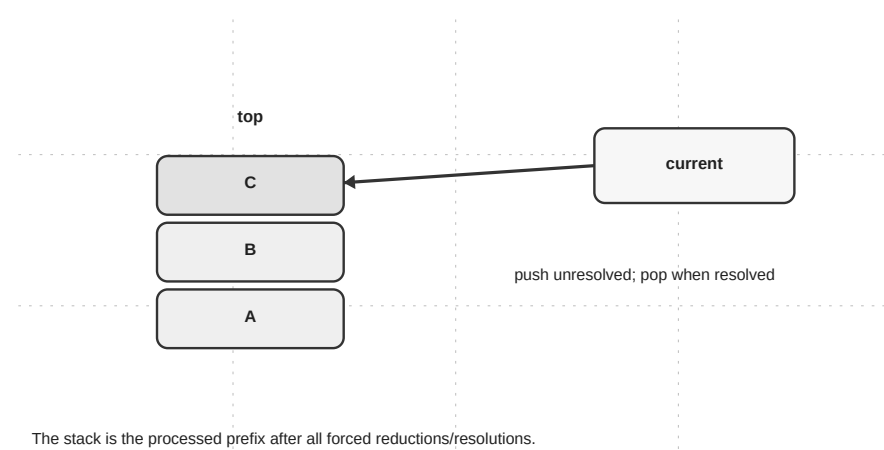
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 20. Valid Parentheses
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Each item is pushed and popped at most once, so $O(n)$ time and $O(n)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: `left <= right?` `i < n?`
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Popping empty stack; forgetting to flush remaining stack entries.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

Three drills:

V1: Also return the position of the unmatched item.
V2: Handle nested operators or multiple bracket types.
V3: Convert next-greater logic into previous-greater logic.

Follow-ups:

- Can the stack be compressed to store counts or indices only?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def valid_parentheses(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 123 of 229: 71. Simplify Path

Pattern: Stack | Sources: Top150 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Simplify an absolute Unix path using '.', '..', and repeated slashes.

Example input/output:
Input: path="/a/b/./c/"

Output: "/a/c"

Why: Move into a, b, back to a, then c.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become $O(n^2)$.

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

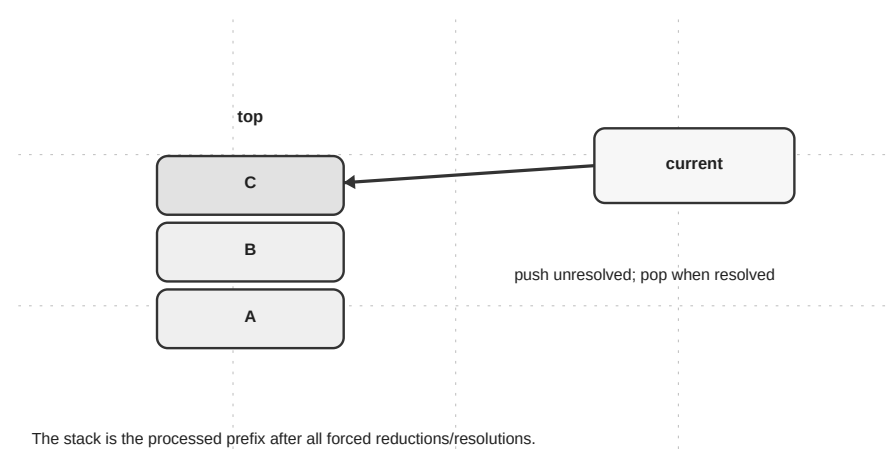
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 71. Simplify Path
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Each item is pushed and popped at most once, so $O(n)$ time and $O(n)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Popping empty stack; forgetting to flush remaining stack entries.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

Three drills:
V1: Also return the position of the unmatched item.
V2: Handle nested operators or multiple bracket types.
V3: Convert next-greater logic into previous-greater logic.

Follow-ups:
- Can the stack be compressed to store counts or indices only?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def simplify_path(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 124 of 229: 155. Min Stack

Pattern: Stack | Sources: Top150 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Design a stack supporting push, pop, top, and getMin, all in O(1) time.

Example input/output:
Input: push(-2),push(0),push(-3),getMin,pop,top,getMin

Output: [null,null,null,-3,null,0,-2]

Why: Track the running minimum.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become O(n^2).

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

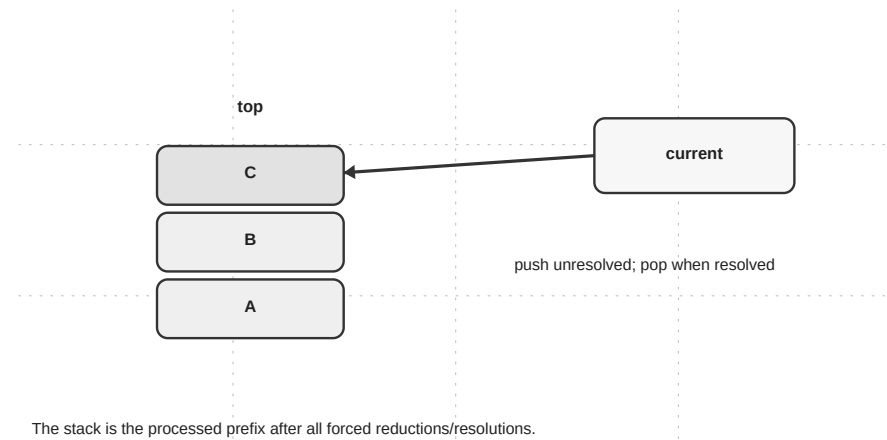
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 155. Min Stack
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time O(n); Space O(n)

Why:
Each item is pushed and popped at most once, so O(n) time and O(n) space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Popping empty stack; forgetting to flush remaining stack entries.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

- Three drills:
- V1: Also return the position of the unmatched item.
 - V2: Handle nested operators or multiple bracket types.
 - V3: Convert next-greater logic into previous-greater logic.

- Follow-ups:
- Can the stack be compressed to store counts or indices only?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def min_stack(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 125 of 229: 150. Evaluate Reverse Polish Notation

Pattern: Stack | Sources: Top150 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: O(n)

Space: O(n)

Problem statement + sample

Prompt:
Evaluate an arithmetic expression in Reverse Polish Notation with integer division truncating toward zero.

Example input/output:
Input: tokens=["2","1","+","3","*"]

Output: 9

Why: (2+1)*3=9.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become O(n^2).

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

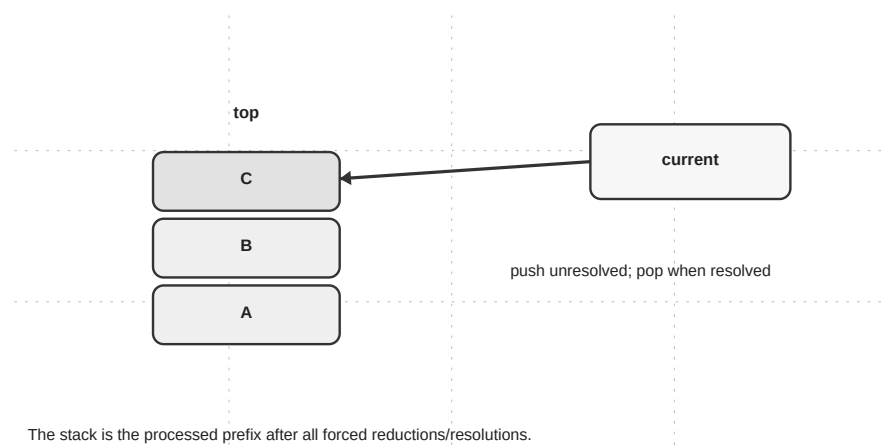
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 150. Evaluate Reverse Polish Notation
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time O(n); Space O(n)

Why:
Each item is pushed and popped at most once, so O(n) time and O(n) space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Popping empty stack; forgetting to flush remaining stack entries.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

- Three drills:
- V1: Also return the position of the unmatched item.
 - V2: Handle nested operators or multiple bracket types.
 - V3: Convert next-greater logic into previous-greater logic.

- Follow-ups:
- Can the stack be compressed to store counts or indices only?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def evaluate_reverse_polish_notation(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 126 of 229: 224. Basic Calculator

Pattern: Stack | Sources: Top150 | Companies: Amazon, Bloomberg, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Evaluate a string expression containing nonnegative integers, plus, minus, parentheses, and spaces.

Example input/output:
Input: s="(1+(4+5+2)-3)+(6+8)"

Output: 23

Why: Parentheses determine signs and grouping.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly scan and delete/merge adjacent conflicts until stable; can become $O(n^2)$.

Optimized approach:
Use a stack to keep unresolved items. The current item only interacts with the top until stable.

Alternative:
For arithmetic parsing, keep operator/context stacks or fold high-precedence operations immediately.

Implementation checkpoints:

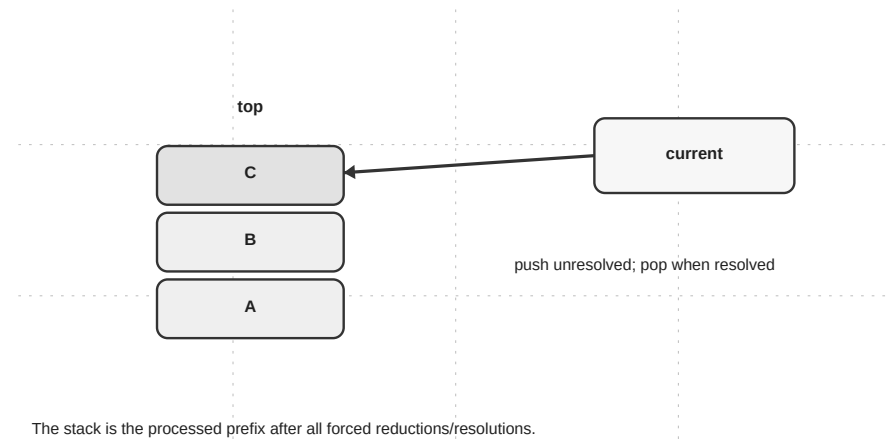
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 224. Basic Calculator
func solveStack(items []int) []int {
    st := []int{}
    for _, x := range items {
        // Resolve all conflicts with the current top before pushing x.
        for len(st) > 0 && shouldPop(st[len(st)-1], x) {
            st = st[:len(st)-1]
        }
        if shouldKeep(x) {
            st = append(st, x)
        }
    }
    return st // stack is the reduced processed prefix
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The stack is exactly the processed prefix after all forced reductions have been applied.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Each item is pushed and popped at most once, so $O(n)$ time and $O(n)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Popping empty stack; forgetting to flush remaining stack entries.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty stack; nested brackets; negative numbers; multi-digit tokens; final flush after scan.

Variations and follow-ups

Three drills:

- V1: Also return the position of the unmatched item.
- V2: Handle nested operators or multiple bracket types.
- V3: Convert next-greater logic into previous-greater logic.

Follow-ups:

- Can the stack be compressed to store counts or indices only?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def basic_calculator(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The stack is exactly the processed prefix after all forced reductions have been applied.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 127 of 229: 141. Linked List Cycle

Pattern: Linked List | Sources: Top150, NC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Return whether a singly linked list contains a cycle.

Example input/output:
Input: head=[3,2,0,-4], tail connects to index 1

Output: true

Why: Fast and slow pointers meet inside the cycle.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

Implementation checkpoints:

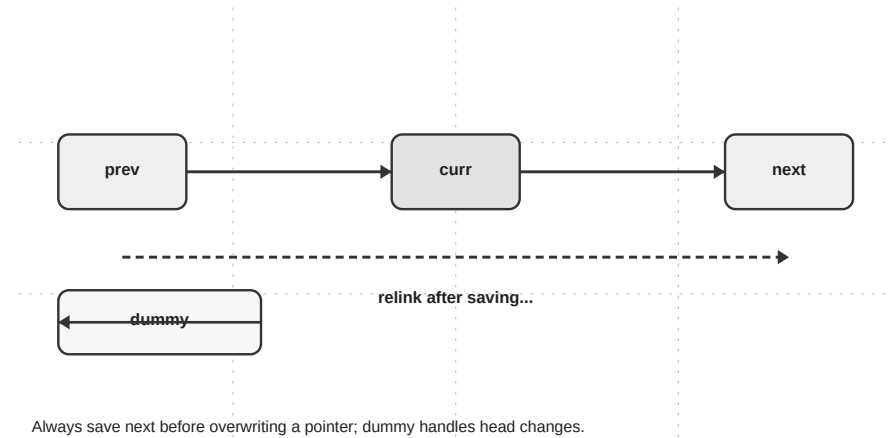
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 141. Linked List Cycle
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:

- V1: Do the operation recursively and compare stack use.
- V2: Handle cycle/random pointers in the same structure.
- V3: Perform the change for every k-sized group.

Follow-ups:

- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def linked_list_cycle(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 128 of 229: 2. Add Two Numbers

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Add two nonnegative integers represented by reversed linked-list digits and return the sum in the same format.

Example input/output:
Input: l1=[2,4,3], l2=[5,6,4]

Output: [7,0,8]

Why: 342+465=807.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

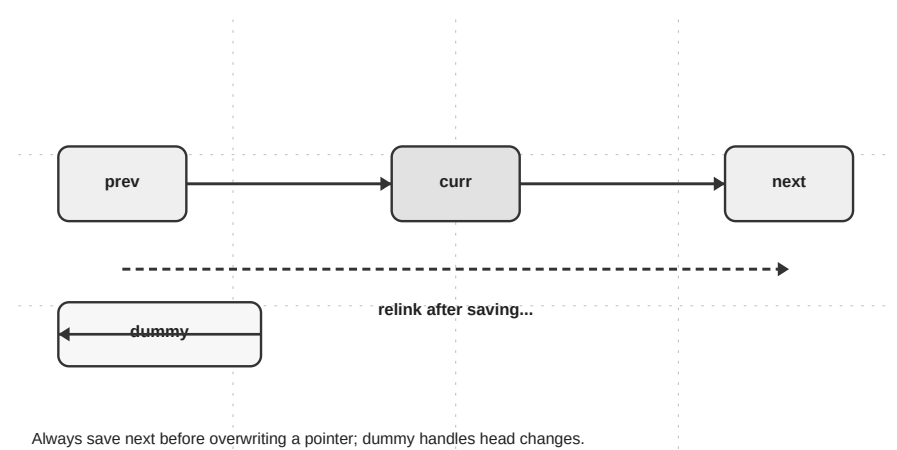
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 2. Add Two Numbers
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def add_two_numbers(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 129 of 229: 21. Merge Two Sorted Lists

Pattern: Linked List | Sources: Top150, NC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: $O(n)$

Space: $O(1)$ unless recursion/map

Problem statement + sample

Prompt:
Merge two sorted linked lists into one sorted linked list.

Example input/output:
Input: list1=[1,2,4], list2=[1,3,4]

Output: [1,1,2,3,4,4]

Why: Choose the smaller current node each time.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra $O(n)$ space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use $O(n)$ call stack.

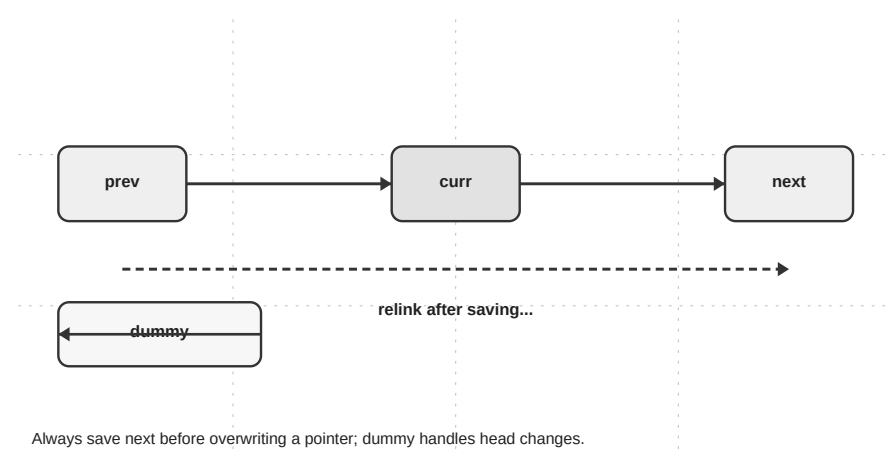
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 21. Merge Two Sorted Lists
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time $O(n)$; Space $O(1)$ unless recursion/map

Why:
Most list pointer solutions are one or two passes: $O(n)$ time, $O(1)$ extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:
V1: Do the operation recursively and compare stack use.
V2: Handle cycle/random pointers in the same structure.
V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with $O(1)$ extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def merge_two_sorted_lists(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 130 of 229: 138. Copy List with Random Pointer

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Deep-copy a linked list where each node has next and random pointers.

Example input/output:
Input: nodes=[[7,null],[13,0],[11,0]]

Output: deep copy with same values and random links

Why: The copy must not share original nodes.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

Implementation checkpoints:

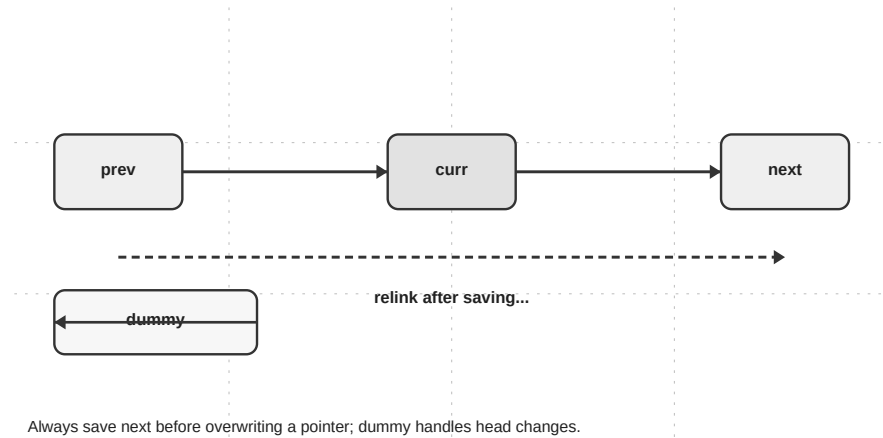
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 138. Copy List with Random Pointer
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:

- V1: Do the operation recursively and compare stack use.
- V2: Handle cycle/random pointers in the same structure.
- V3: Perform the change for every k-sized group.

Follow-ups:

- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def copy_list_with_random_pointer(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 131 of 229: 92. Reverse Linked List II

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Reverse nodes from position left to right in one linked list pass if possible.

Example input/output:
Input: head=[1,2,3,4,5], left=2, right=4

Output: [1,4,3,2,5]

Why: Only the middle segment is reversed.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

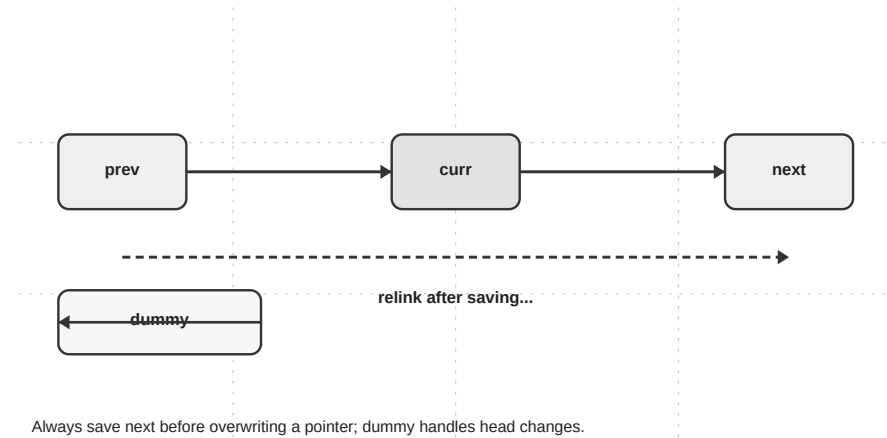
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 92. Reverse Linked List II
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def reverse_linked_list_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 132 of 229: 25. Reverse Nodes in k-Group

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Reverse every consecutive group of k nodes; leave the final smaller group unchanged.

Example input/output:
Input: head=[1,2,3,4,5], k=2

Output: [2,1,4,3,5]

Why: Groups [1,2] and [3,4] reverse.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

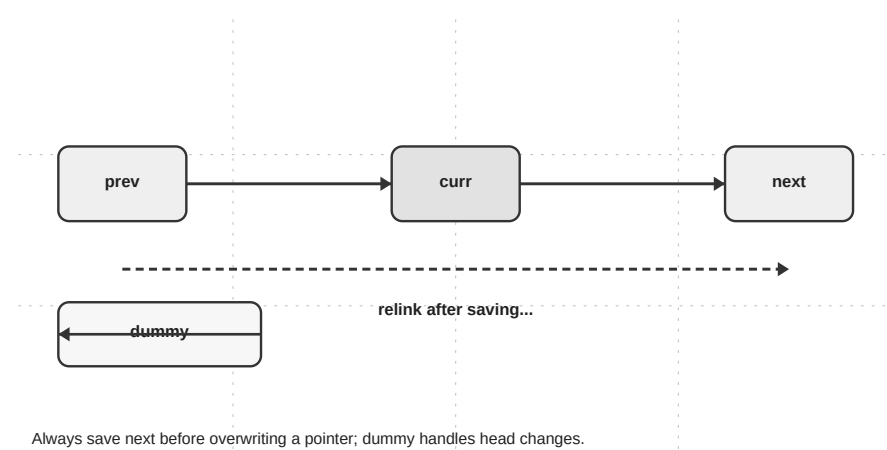
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 25. Reverse Nodes in k-Group
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def reverse_nodes_in_k_group(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 133 of 229: 19. Remove Nth Node From End of List

Pattern: Linked List | Sources: Top150, NC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: $O(n)$

Space: $O(1)$ unless recursion/map

Problem statement + sample

Prompt:
Remove the n -th node from the end of a linked list and return the head.

Example input/output:
Input: head=[1,2,3,4,5], $n=2$

Output: [1,2,3,5]

Why: The second node from the end is 4.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra $O(n)$ space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use $O(n)$ call stack.

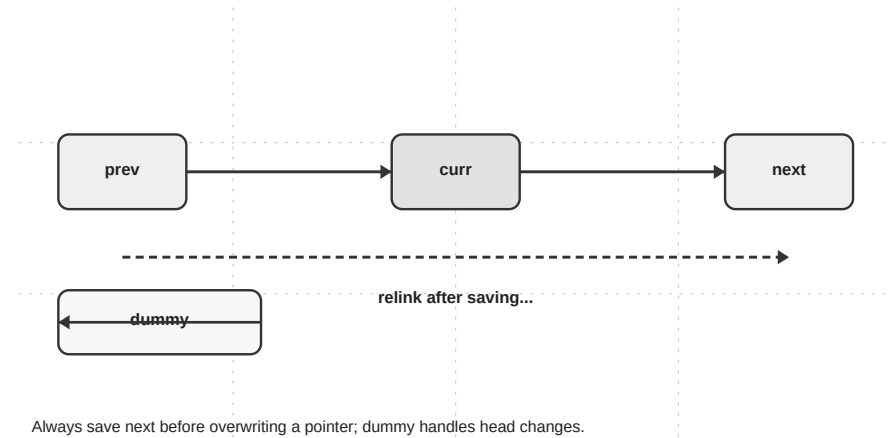
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 19. Remove Nth Node From End of List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev  // reverse or relink current edge
        prev = curr       // processed prefix grows
        curr = next        // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time $O(n)$; Space $O(1)$ unless recursion/map

Why:
Most list pointer solutions are one or two passes: $O(n)$ time, $O(1)$ extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k -sized group.

- Follow-ups:
- Can you solve it with $O(1)$ extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def remove_nth_node_from_end_of_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 134 of 229: 82. Remove Duplicates from Sorted List II

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: $O(n)$

Space: $O(1)$ unless recursion/map

Problem statement + sample

Prompt:
Delete all nodes that have duplicate numbers from a sorted linked list, leaving only values that occurred once.

Example input/output:
Input: head=[1,2,3,3,4,4,5]

Output: [1,2,5]

Why: Values 3 and 4 are removed entirely.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra $O(n)$ space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use $O(n)$ call stack.

Implementation checkpoints:

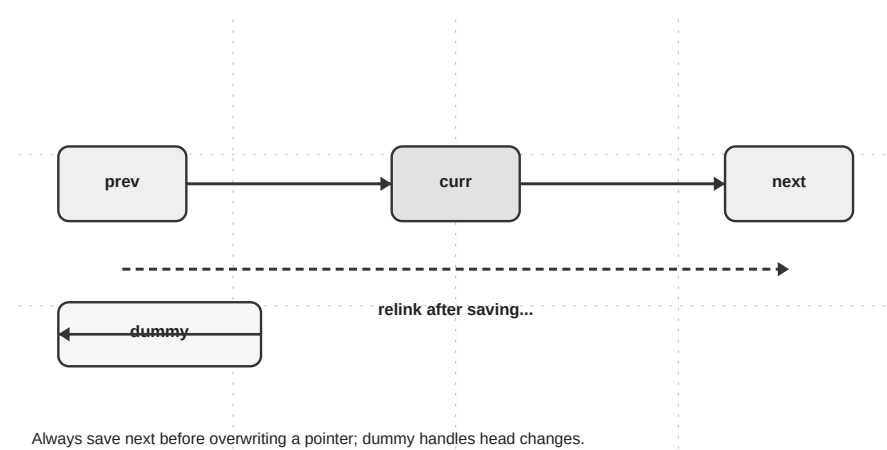
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 82. Remove Duplicates from Sorted List II
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time $O(n)$; Space $O(1)$ unless recursion/map

Why:
Most list pointer solutions are one or two passes: $O(n)$ time, $O(1)$ extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:

- V1: Do the operation recursively and compare stack use.
- V2: Handle cycle/random pointers in the same structure.
- V3: Perform the change for every k-sized group.

Follow-ups:

- Can you solve it with $O(1)$ extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def remove_duplicates_from_sorted_list_i(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 135 of 229: 61. Rotate List

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Rotate a linked list right by k places.

Example input/output:
Input: head=[1,2,3,4,5], k=2

Output: [4,5,1,2,3]

Why: Last two nodes move to the front.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

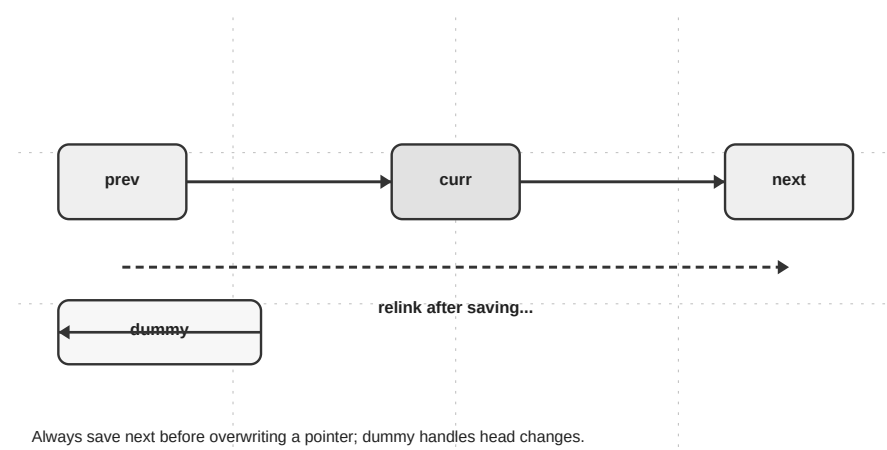
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 61. Rotate List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def rotate_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 136 of 229: 86. Partition List

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Partition a linked list so nodes less than x come before nodes greater than or equal to x, preserving relative order.

Example input/output:
Input: head=[1,4,3,2,5,2], x=3

Output: [1,2,2,4,3,5]

Why: Stable partition into two chains.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

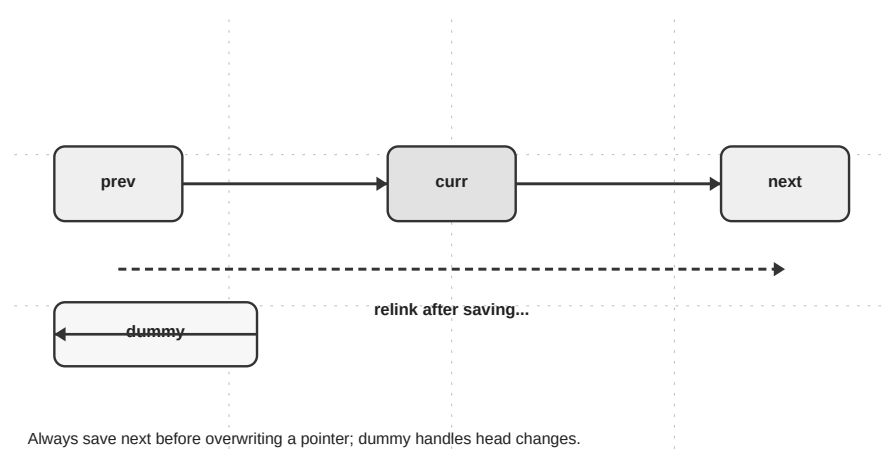
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 86. Partition List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:
V1: Do the operation recursively and compare stack use.
V2: Handle cycle/random pointers in the same structure.
V3: Perform the change for every k-sized group.

Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def partition_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 137 of 229: 146. LRU Cache

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Design an LRU cache with get and put in O(1); when capacity is exceeded, evict the least recently used key.

Example input/output:
Input: capacity=2; put(1,1),put(2,2),get(1),put(3,3),get(2)

Output: [null,null,1,null,-1]

Why: Key 2 is evicted after key 1 is used.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

Implementation checkpoints:

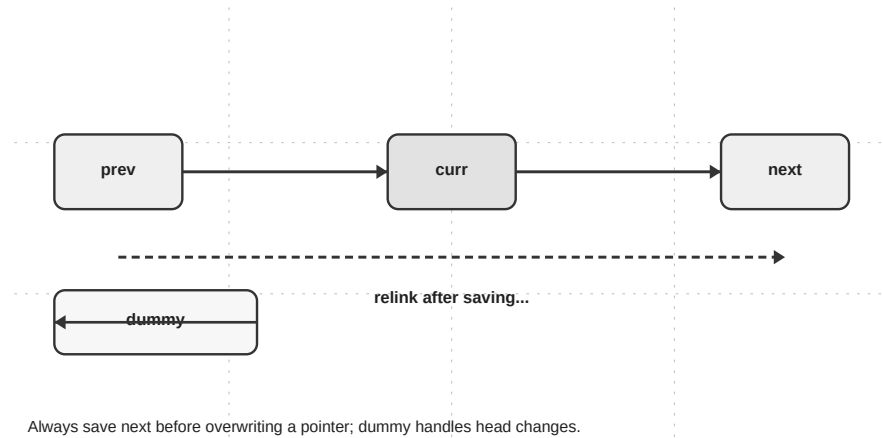
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 146. LRU Cache
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

Three drills:

- V1: Do the operation recursively and compare stack use.
- V2: Handle cycle/random pointers in the same structure.
- V3: Perform the change for every k-sized group.

Follow-ups:

- Can you solve it with O(1) extra memory and one pass?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def lru_cache(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 138 of 229: 100. Same Tree

Pattern: Tree DFS | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return whether two binary trees are structurally identical and have equal node values.

Example input/output:
Input: p=[1,2,3], q=[1,2,3]

Output: true

Why: Shape and values match.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

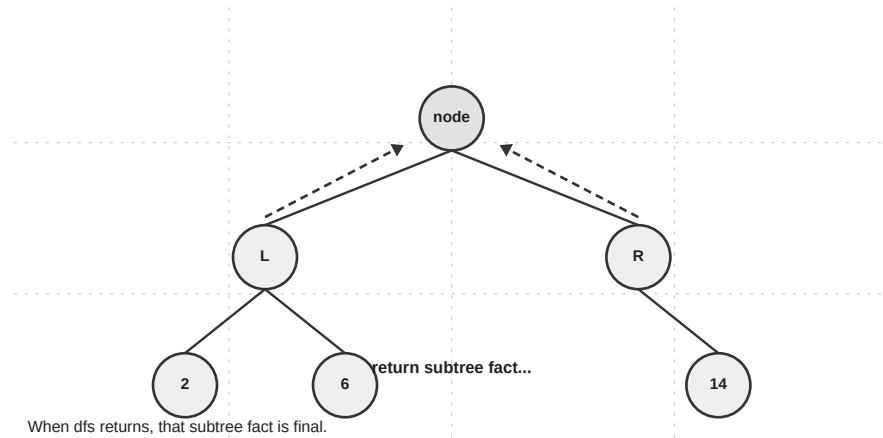
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 100. Same Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:
V1: Return the nodes on the path, not just a count/value.
V2: Make the tree contain negative values or duplicate values.
V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:
- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def same_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 139 of 229: 226. Invert Binary Tree

Pattern: Tree DFS | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Invert a binary tree by swapping every node's left and right children.

Example input/output:
Input: root=[4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Why: Every child pair is swapped.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

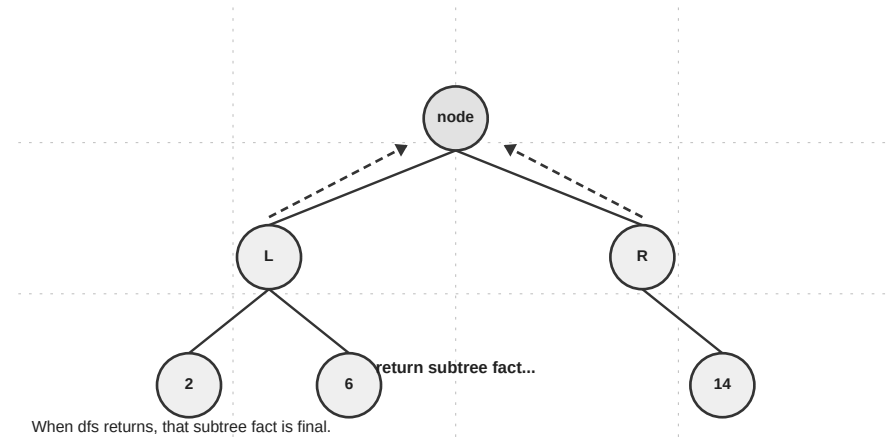
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 226. Invert Binary Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def invert_binary_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 140 of 229: 101. Symmetric Tree

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Return whether a binary tree is a mirror of itself around its center.

Example input/output:
Input: root=[1,2,2,3,4,4,3]

Output: true

Why: Left and right subtrees mirror each other.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

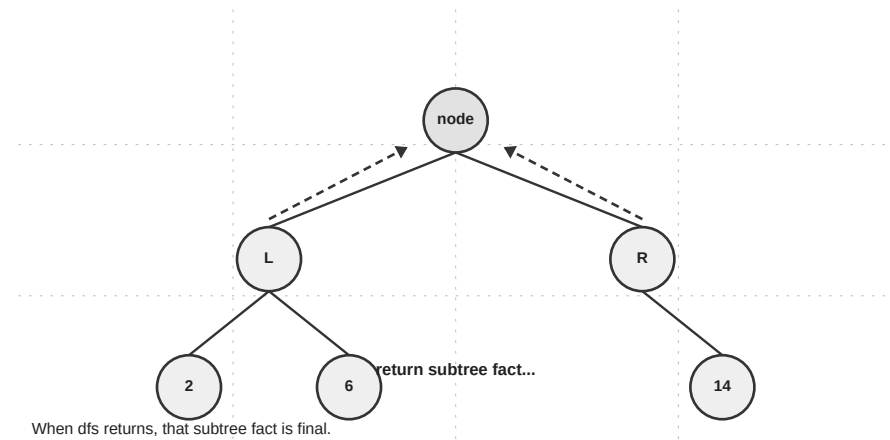
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 101. Symmetric Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def symmetric_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 141 of 229: 105. Construct Binary Tree from Preorder and Inorder Traversal

Pattern: Tree DFS | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Build a binary tree from preorder and inorder traversal arrays with unique values.

Example input/output:
Input: preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]

Output: [3,9,20,null,null,15,7]

Why: Preorder chooses roots; inorder splits subtrees.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

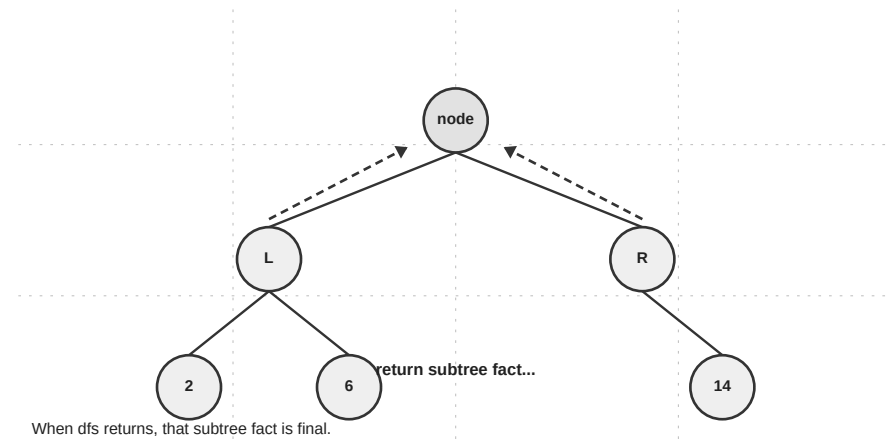
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 105. Construct Binary Tree from Preorder and Inorder Traversal
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def construct_binary_tree_from_preorder_(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 142 of 229: 106. Construct Binary Tree from Inorder and Postorder Traversal

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Build a binary tree from inorder and postorder traversal arrays with unique values.

Example input/output:
Input: inorder=[9,3,15,20,7], postorder=[9,15,7,20,3]

Output: [3,9,20,null,null,15,7]

Why: Postorder's last value is the root.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

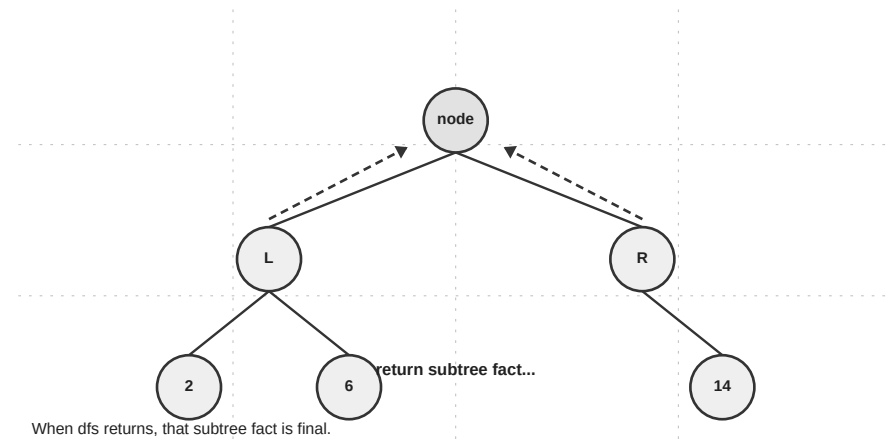
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 106. Construct Binary Tree from Inorder and Postorder Traversal
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def construct_binary_tree_from_inorder_a(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 143 of 229: 117. Populating Next Right Pointers in Each Node II

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Populate each node's next pointer to its neighbor on the same level, or null if none, in any binary tree.

Example input/output:
Input: root=[1,2,3,4,5,null,7]

Output: levels linked as 1->null, 2->3->null, 4->5->7->null

Why: Next pointers connect across each level.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

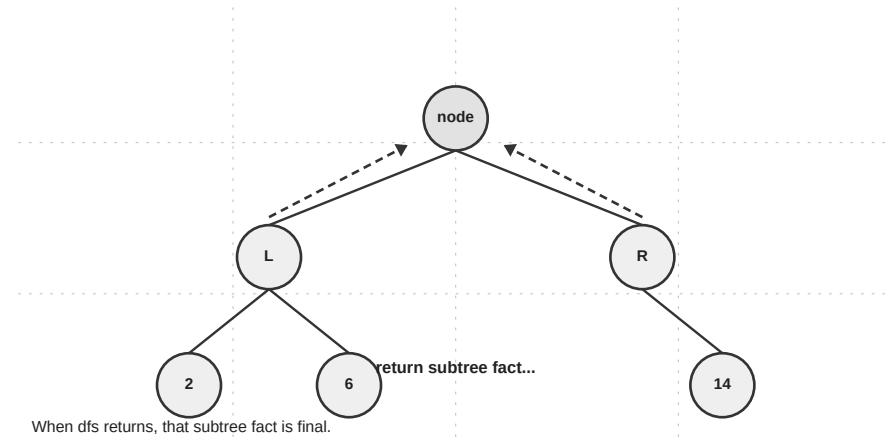
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 117. Populating Next Right Pointers in Each Node II
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def populating_next_right_pointers_in_ea(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 144 of 229: 114. Flatten Binary Tree to Linked List

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Flatten a binary tree to a right-child-only linked list following preorder traversal.

Example input/output:
Input: root=[1,2,5,3,4,null,6]

Output: [1,2,3,4,5,6]

Why: The right chain is preorder.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

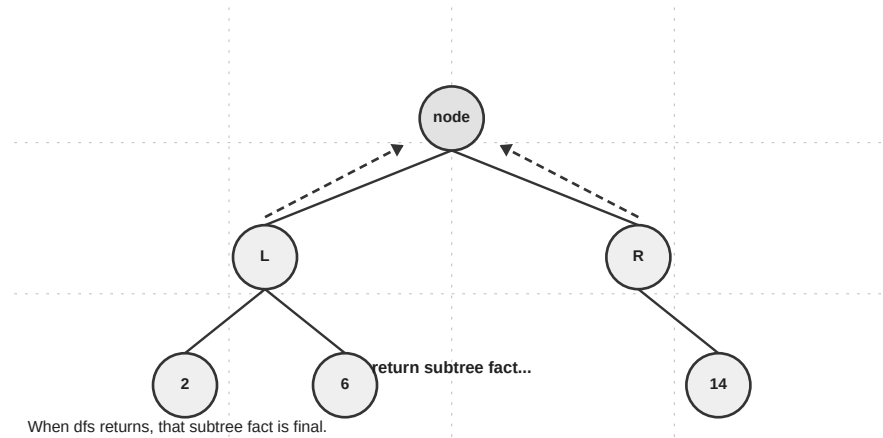
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 114. Flatten Binary Tree to Linked List
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def flatten_binary_tree_to_linked_list(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 145 of 229: 112. Path Sum

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Return true if a root-to-leaf path has node values summing to targetSum.

Example input/output:
Input: root=[5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum=22

Output: true

Why: Path 5->4->11->2 sums to 22.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

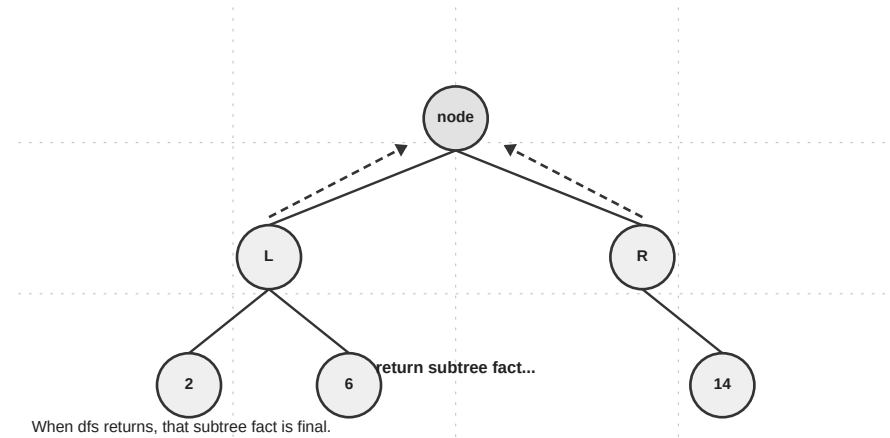
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 112. Path Sum
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $left \leq right?$ $i < n?$
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Returning answer instead of path contribution; wrong null base case.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

- Three drills:
- V1: Return the nodes on the path, not just a count/value.
 - V2: Make the tree contain negative values or duplicate values.
 - V3: Convert recursive DFS to iterative stack DFS.

- Follow-ups:
- Can you write iterative DFS? What value does each call return?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def path_sum(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 146 of 229: 129. Sum Root to Leaf Numbers

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Each root-to-leaf path forms a number by concatenating digits; return the sum of all such numbers.

Example input/output:

Input: root=[1,2,3]

Output: 25

Why: Numbers are 12 and 13.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

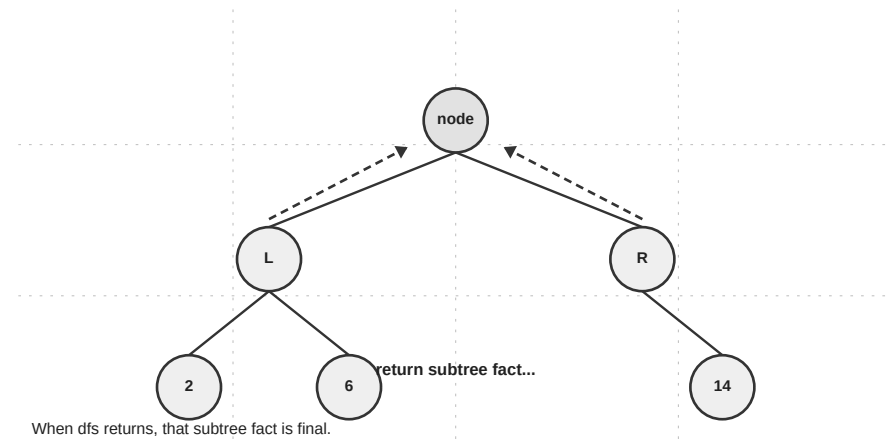
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 129. Sum Root to Leaf Numbers
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}
```

```
// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:
V1: Return the nodes on the path, not just a count/value.
V2: Make the tree contain negative values or duplicate values.
V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:
- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def sum_root_to_leaf_numbers(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 147 of 229: 124. Binary Tree Maximum Path Sum

Pattern: Tree DFS | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return the maximum path sum in a binary tree; a path may start and end at any nodes but must be connected.

Example input/output:
Input: root=[-10,9,20,null,null,15,7]

Output: 42

Why: Path 15->20->7 sums to 42.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

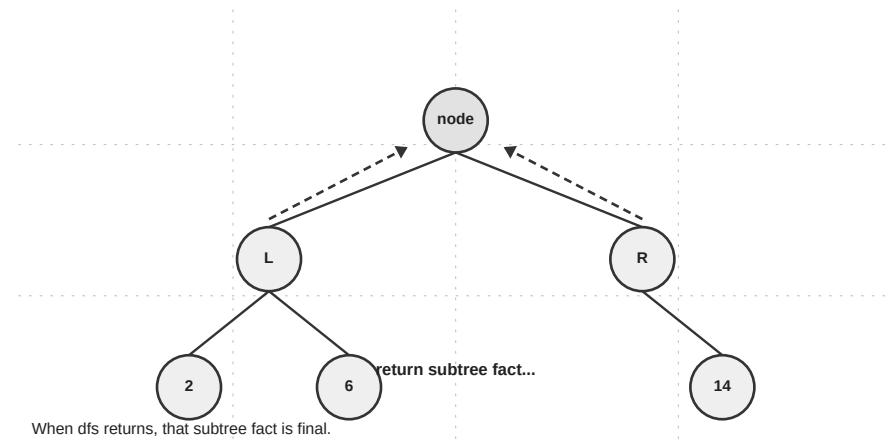
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 124. Binary Tree Maximum Path Sum
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def binary_tree_maximum_path_sum(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 148 of 229: 173. Binary Search Tree Iterator

Pattern: BST | Sources: Top150 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: O(h) to O(n)

Space: O(h)

Problem statement + sample

Prompt:
Implement a BST iterator with next returning the next smallest value and hasNext reporting availability.

Example input/output:
Input: BST=[7,3,15,null,null,9,20], calls=next,next,hasNext

Output: [3,7,true]

Why: Inorder traversal yields sorted values.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives O(h) memory and supports lazy iterators.

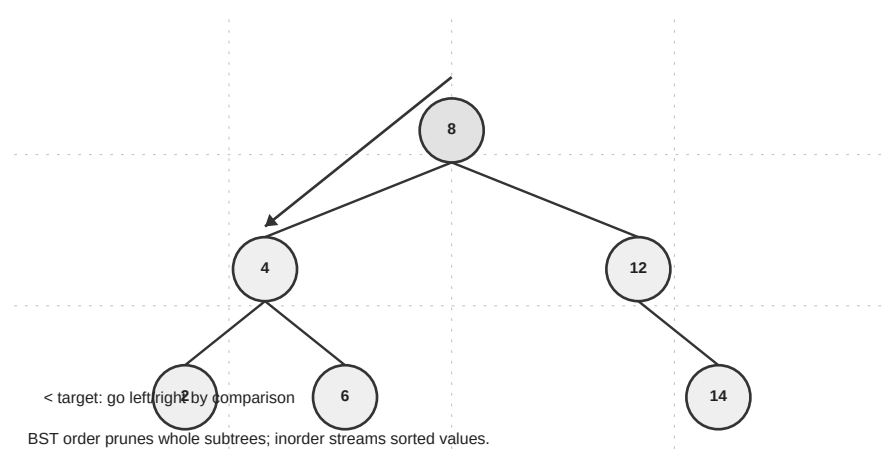
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 173. Binary Search Tree Iterator
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time O(h) to O(n); Space O(h)

Why:
Balanced operations are O(h)=O(log n); worst-case skewed tree is O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using <= bounds incorrectly; not preserving lower and upper constraints.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

- Three drills:
- V1: Support duplicate values with a clear side policy.
 - V2: Convert to an iterative bounded search.
 - V3: Return predecessor/successor around a target.

- Follow-ups:
- Can you solve it with inorder iteration and O(h) memory?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def binary_search_tree_iterator(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 149 of 229: 222. Count Complete Tree Nodes

Pattern: Tree DFS | Sources: Top150 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Count the nodes in a complete binary tree.

Example input/output:
Input: root=[1,2,3,4,5,6]

Output: 6

Why: All six listed nodes exist.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

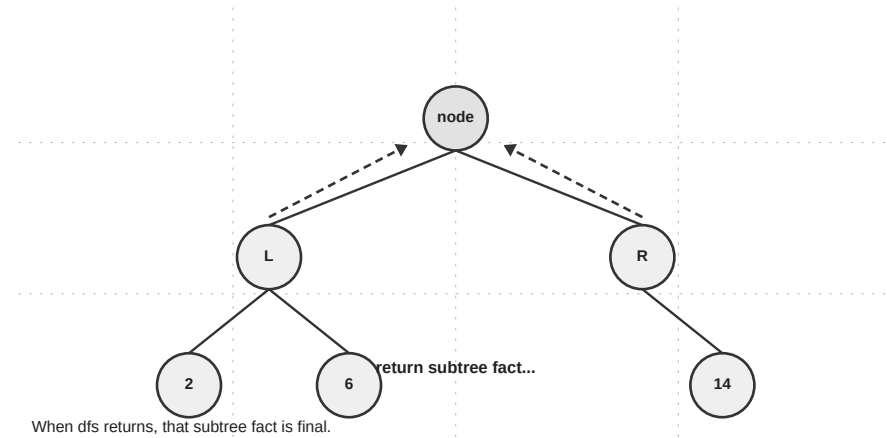
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 222. Count Complete Tree Nodes
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def count_complete_tree_nodes(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 150 of 229: 637. Average of Levels in Binary Tree

Pattern: Tree BFS | Sources: Top150 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: O(n)

Space: O(w)

Problem statement + sample

Prompt:
Return the average node value at each binary-tree level.

Example input/output:
Input: root=[3,9,20,null,null,15,7]

Output: [3,14.5,11]

Why: Level averages are 3, (9+20)/2, (15+7)/2.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
DFS with depth maps or repeated level searches works but is messier.

Optimized approach:
Queue nodes level by level; record level size before processing so levels do not mix.

Alternative:
DFS can collect depth buckets if recursion is preferred.

Implementation checkpoints:

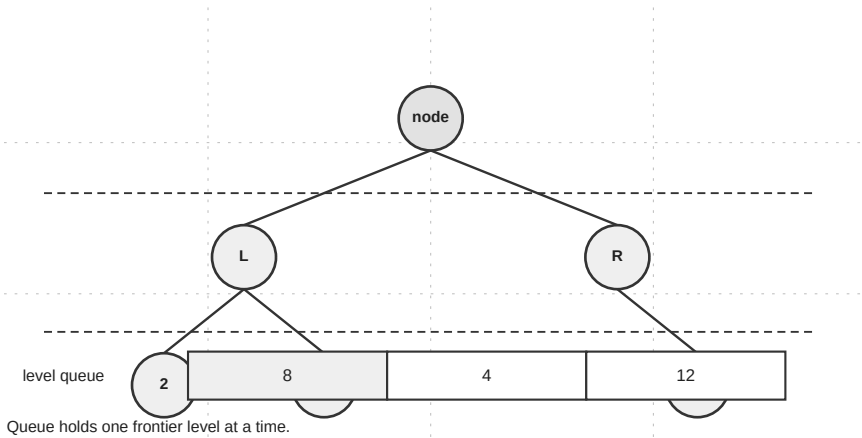
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 637. Average of Levels in Binary Tree
func solveTreeBFS(root *TreeNode) []int {
    if root == nil { return nil }
    q, ans := []*TreeNode{root}, []int{}
    for len(q) > 0 {
        levelSize := len(q) // freezes current level
        levelValue := 0
        for i := 0; i < levelSize; i++ {
            node := q[0]; q = q[1:]
            levelValue = updateLevel(levelValue, node.Val, i)
            if node.Left != nil { q = append(q, node.Left) }
            if node.Right != nil { q = append(q, node.Right) }
        }
        ans = append(ans, levelValue)
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At the start of each outer loop, queue contains exactly one level.

Complexity target:
Time O(n); Space O(w)

Why:
O(n) time; O(w) queue space where w is tree width.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not separating levels; queueing nil children unnecessarily.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; last node per level; zigzag order; integer averages; queue memory at widest level.

Variations and follow-ups

Three drills:

- V1: Return zigzag/order statistics per level.
- V2: Use right-to-left priority instead of left-to-right.
- V3: Limit search to the nearest target level.

Follow-ups:

- Can you stream one level at a time without storing all levels?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def average_of_levels_in_binary_tree(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At the start of each outer loop, queue contains exactly one level.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 151 of 229: 102. Binary Tree Level Order Traversal

Pattern: Tree BFS | Sources: Top150, NC75 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: $O(n)$

Space: $O(w)$

Problem statement + sample

Prompt:
Return the level-order traversal of a binary tree as lists per level.

Example input/output:
Input: root=[3,9,20,null,null,15,7]

Output: [[3],[9,20],[15,7]]

Why: BFS visits level by level.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
DFS with depth maps or repeated level searches works but is messier.

Optimized approach:
Queue nodes level by level; record level size before processing so levels do not mix.

Alternative:
DFS can collect depth buckets if recursion is preferred.

Implementation checkpoints:

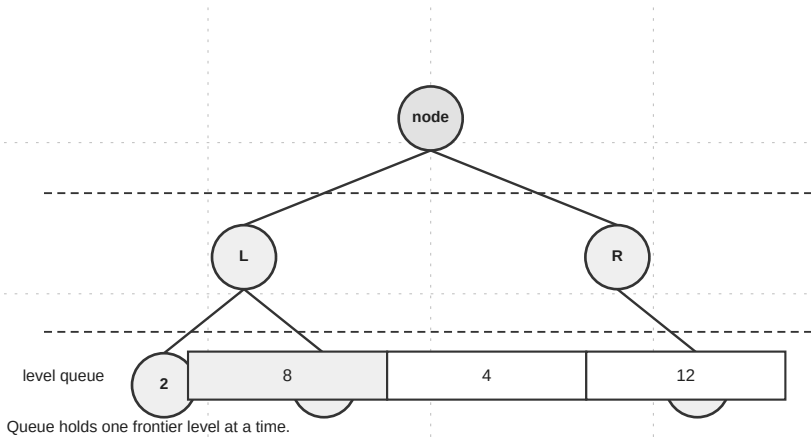
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 102. Binary Tree Level Order Traversal
func solveTreeBFS(root *TreeNode) [][]int {
    if root == nil { return nil }
    q, ans := []*TreeNode{root}, [][]int{}
    for len(q) > 0 {
        levelSize := len(q) // freezes current level
        levelValue := 0
        for i := 0; i < levelSize; i++ {
            node := q[0]; q = q[1:]
            levelValue = updateLevel(levelValue, node.Val, i)
            if node.Left != nil { q = append(q, node.Left) }
            if node.Right != nil { q = append(q, node.Right) }
        }
        ans = append(ans, levelValue)
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At the start of each outer loop, queue contains exactly one level.

Complexity target:
Time $O(n)$; Space $O(w)$

Why:
 $O(n)$ time; $O(w)$ queue space where w is tree width.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not separating levels; queueing nil children unnecessarily.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; last node per level; zigzag order; integer averages; queue memory at widest level.

Variations and follow-ups

Three drills:

- V1: Return zigzag/order statistics per level.
- V2: Use right-to-left priority instead of left-to-right.
- V3: Limit search to the nearest target level.

Follow-ups:

- Can you stream one level at a time without storing all levels?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def binary_tree_level_order_traversal(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At the start of each outer loop, queue contains exactly one level.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 152 of 229: 103. Binary Tree Zigzag Level Order Traversal

Pattern: Tree BFS | Sources: Top150 | Companies: Amazon, Google, Bloomberg, Meta (public samples)

Time: $O(n)$

Space: $O(w)$

Problem statement + sample

Prompt:
Return binary-tree levels alternating left-to-right then right-to-left.

Example input/output:
Input: root=[3,9,20,null,null,15,7]

Output: [[3],[20,9],[15,7]]

Why: Level 2 is reversed.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
DFS with depth maps or repeated level searches works but is messier.

Optimized approach:
Queue nodes level by level; record level size before processing so levels do not mix.

Alternative:
DFS can collect depth buckets if recursion is preferred.

Implementation checkpoints:

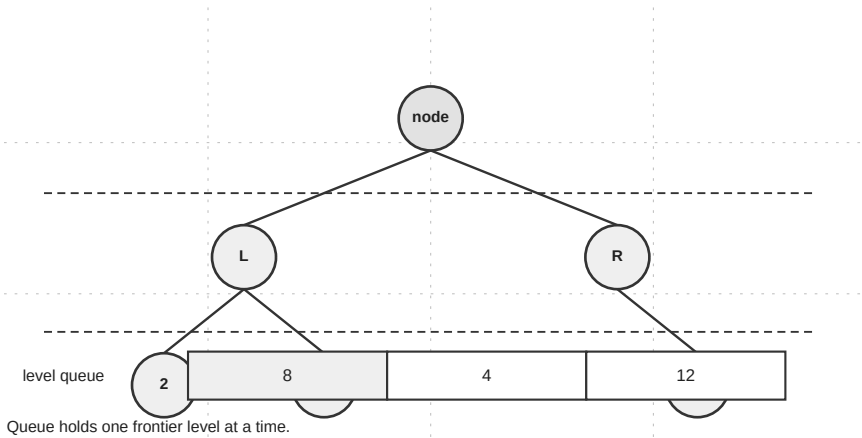
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 103. Binary Tree Zigzag Level Order Traversal
func solveTreeBFS(root *TreeNode) [][]int {
    if root == nil { return nil }
    q, ans := []*TreeNode{root}, []int{}
    for len(q) > 0 {
        levelSize := len(q) // freezes current level
        levelValue := 0
        for i := 0; i < levelSize; i++ {
            node := q[0]; q = q[1:]
            levelValue = updateLevel(levelValue, node.Val, i)
            if node.Left != nil { q = append(q, node.Left) }
            if node.Right != nil { q = append(q, node.Right) }
        }
        ans = append(ans, levelValue)
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At the start of each outer loop, queue contains exactly one level.

Complexity target:
Time $O(n)$; Space $O(w)$

Why:
 $O(n)$ time; $O(w)$ queue space where w is tree width.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not separating levels; queueing nil children unnecessarily.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; last node per level; zigzag order; integer averages; queue memory at widest level.

Variations and follow-ups

Three drills:

- V1: Return zigzag/order statistics per level.
- V2: Use right-to-left priority instead of left-to-right.
- V3: Limit search to the nearest target level.

Follow-ups:

- Can you stream one level at a time without storing all levels?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def binary_tree_zigzag_level_order_trave(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At the start of each outer loop, queue contains exactly one level.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 153 of 229: 530. Minimum Absolute Difference in BST

Pattern: BST | Sources: Top150 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: O(h) to O(n)

Space: O(h)

Problem statement + sample

Prompt:
In a BST, return the minimum absolute difference between values of any two nodes.

Example input/output:
Input: root=[4,2,6,1,3]

Output: 1

Why: Adjacent inorder values differ by 1.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives O(h) memory and supports lazy iterators.

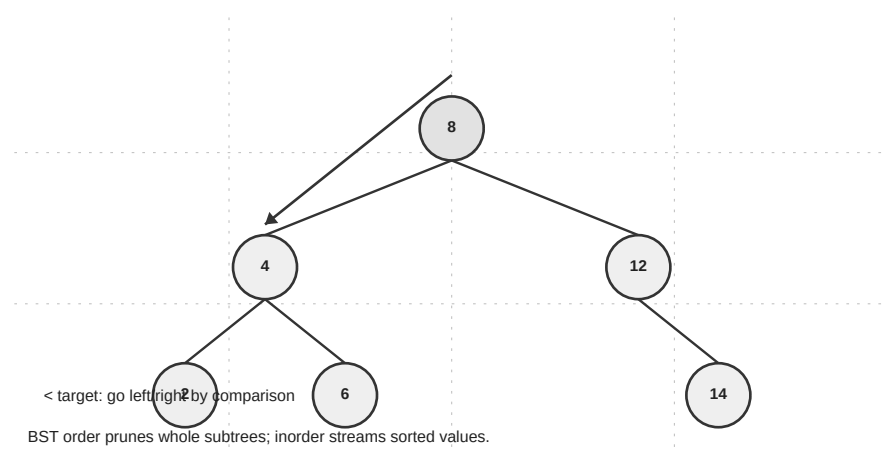
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 530. Minimum Absolute Difference in BST
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time O(h) to O(n); Space O(h)

Why:
Balanced operations are O(h)=O(log n); worst-case skewed tree is O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using <= bounds incorrectly; not preserving lower and upper constraints.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

- Three drills:
- V1: Support duplicate values with a clear side policy.
 - V2: Convert to an iterative bounded search.
 - V3: Return predecessor/successor around a target.

- Follow-ups:
- Can you solve it with inorder iteration and O(h) memory?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def minimum_absolute_difference_in_bst(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 154 of 229: 230. Kth Smallest Element in a BST

Pattern: BST | Sources: Top150, NC75 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: O(h) to O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return the k-th smallest value in a BST.

Example input/output:
Input: root=[3,1,4,null,2], k=1

Output: 1

Why: Inorder order is [1,2,3,4].

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives O(h) memory and supports lazy iterators.

Implementation checkpoints:

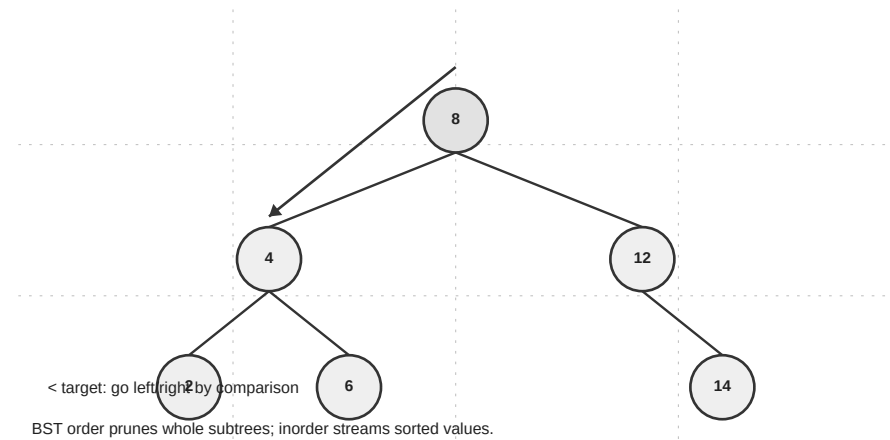
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 230. Kth Smallest Element in a BST
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time O(h) to O(n); Space O(h)

Why:
Balanced operations are O(h)=O(log n); worst-case skewed tree is O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using <= bounds incorrectly; not preserving lower and upper constraints.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

Three drills:

- V1: Support duplicate values with a clear side policy.
- V2: Convert to an iterative bounded search.
- V3: Return predecessor/successor around a target.

Follow-ups:

- Can you solve it with inorder iteration and O(h) memory?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def kth_smallest_element_in_a_bst(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 155 of 229: 98. Validate Binary Search Tree

Pattern: BST | Sources: Top150, NC75 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: O(h) to O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return whether a binary tree satisfies the BST property with strict lower and upper bounds at every node.

Example input/output:

Input: root=[2,1,3]

Output: true

Why: All left values are smaller and all right values larger.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives O(h) memory and supports lazy iterators.

Implementation checkpoints:

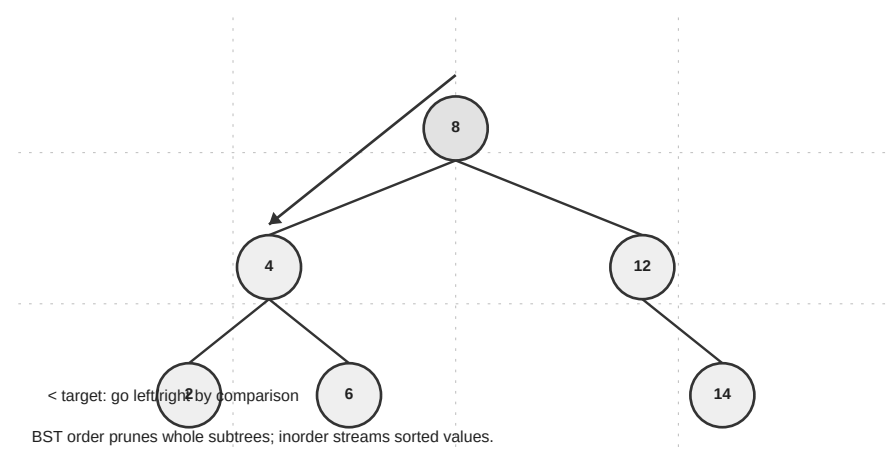
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 98. Validate Binary Search Tree
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time O(h) to O(n); Space O(h)

Why:
Balanced operations are O(h)=O(log n); worst-case skewed tree is O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using <= bounds incorrectly; not preserving lower and upper constraints.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

Three drills:

- V1: Support duplicate values with a clear side policy.
- V2: Convert to an iterative bounded search.
- V3: Return predecessor/successor around a target.

Follow-ups:

- Can you solve it with inorder iteration and O(h) memory?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def validate_binary_search_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 156 of 229: 200. Number of Islands

Pattern: Graph | Sources: Top150, NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Count connected groups of '1' cells in a grid, connected horizontally or vertically.

Example input/output:
Input: grid=`[["1","1","0"],["0","1","0"],["1","0","1"]]`

Output: 3

Why: There are three islands.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

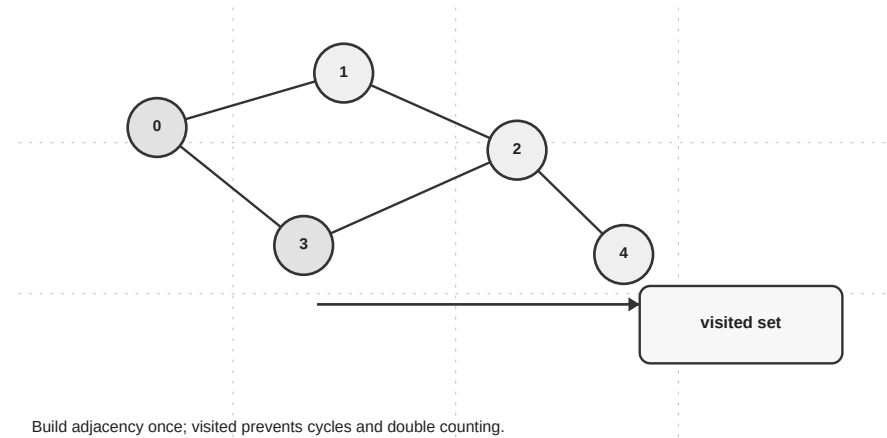
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 200. Number of Islands
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def number_of_islands(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 157 of 229: 130. Surrounded Regions

Pattern: Graph | Sources: Top150 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Capture all 'O' regions fully surrounded by 'X'; border-connected 'O' cells remain unchanged.

Example input/output:
Input: board=[["X","X","X","X"],["X","O","X"],["X","X","O","X"],["X","O","X","X"]]

Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]

Why: Only the border-connected O survives.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

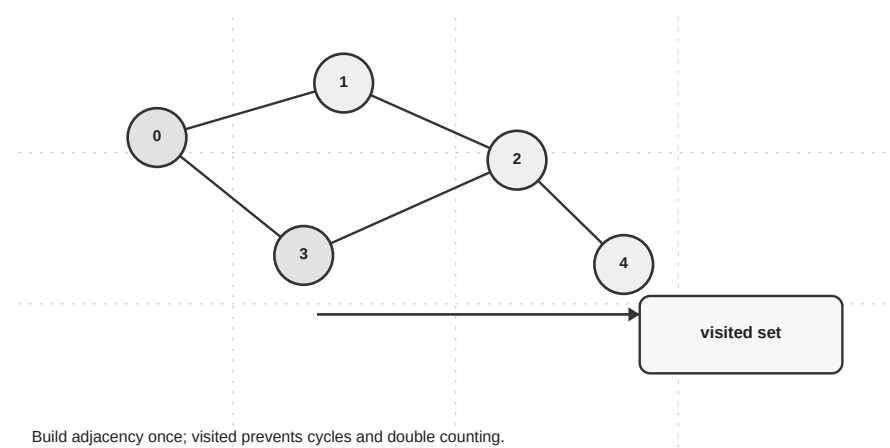
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 130. Surrounded Regions
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Not marking visited before enqueue; confusing directed and undirected edges.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

- Three drills:
- V1: Detect cycle vs produce traversal order.
 - V2: Handle disconnected components.
 - V3: Convert unweighted BFS to weighted shortest path.

- Follow-ups:
- What happens with disconnected components or cycles?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def surrounded_regions(n, edges):
    adj = [[] for _ in range(n)]
    for a,b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 158 of 229: 133. Clone Graph

Pattern: Graph | Sources: Top150, NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Deep-copy an undirected connected graph where each node stores a value and neighbor list.

Example input/output:
Input: adjList=[[2,4],[1,3],[2,4],[1,3]]

Output: deep copy with same adjacency

Why: The clone must use new nodes.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

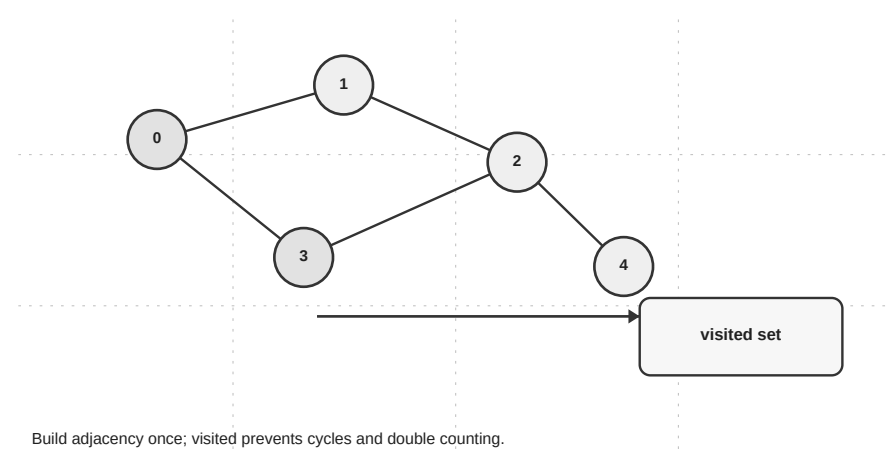
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 133. Clone Graph
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def clone_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 159 of 229: 207. Course Schedule

Pattern: Graph | Sources: Top150, NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Given prerequisite pairs, return whether all courses can be finished.

Example input/output:
Input: numCourses=2, prerequisites=[[1,0]]

Output: true

Why: Take course 0 before course 1.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

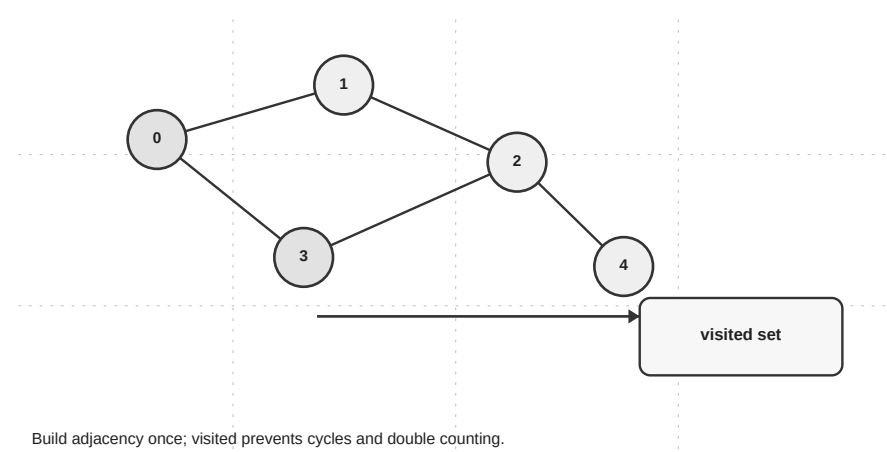
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 207. Course Schedule
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Not marking visited before enqueue; confusing directed and undirected edges.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

- Three drills:
- V1: Detect cycle vs produce traversal order.
 - V2: Handle disconnected components.
 - V3: Convert unweighted BFS to weighted shortest path.

- Follow-ups:
- What happens with disconnected components or cycles?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def course_schedule(n, edges):
    adj = [[] for _ in range(n)]
    for a,b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 160 of 229: 210. Course Schedule II

Pattern: Graph | Sources: Top150 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Return any valid course order satisfying prerequisites, or empty list if impossible.

Example input/output:
Input: numCourses=2, prerequisites=[[1,0]]

Output: [0,1]

Why: Course 0 must precede 1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

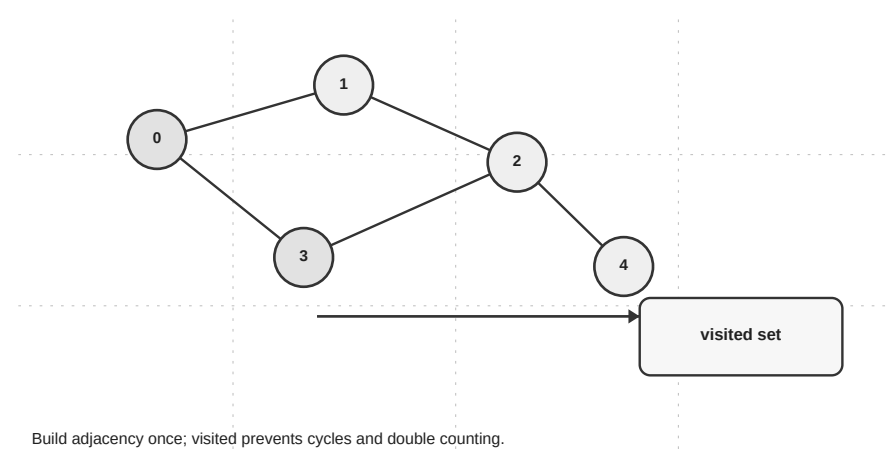
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 210. Course Schedule II
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def course_schedule_ii(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 161 of 229: 909. Snakes and Ladders

Pattern: Graph | Sources: Top150 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
On a snakes-and-ladders board labeled boustrophedon from 1 to n^2 , return minimum dice throws to reach n^2 .

Example input/output:
Input: board=[[-1,-1],[-1,-1]]

Output: 1

Why: From square 1, one roll can reach square 4.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

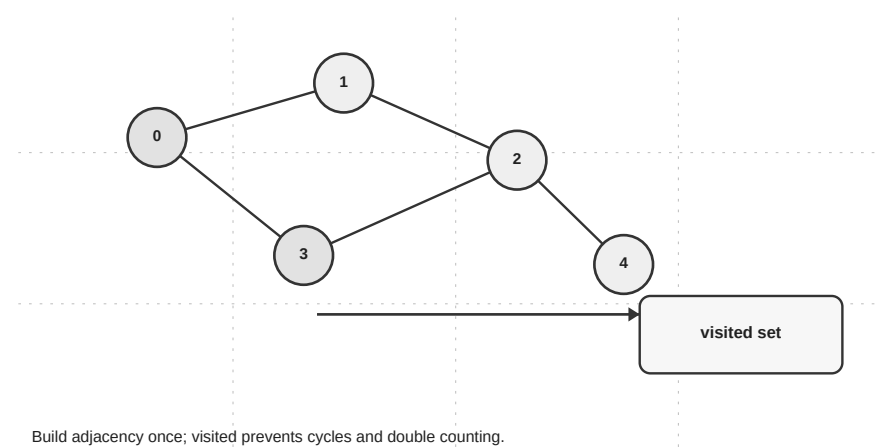
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 909. Snakes and Ladders
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:
V1: Detect cycle vs produce traversal order.
V2: Handle disconnected components.
V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:
- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def snakes_and_ladders(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 162 of 229: 433. Minimum Genetic Mutation

Pattern: Graph | Sources: Top150 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Return the minimum one-character gene mutations from startGene to endGene, with every intermediate gene in bank.

Example input/output:
Input: start="AACCGGTT", end="AACCGGTA", bank=["AACCGGTA"]

Output: 1

Why: One mutation changes the final T to A.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

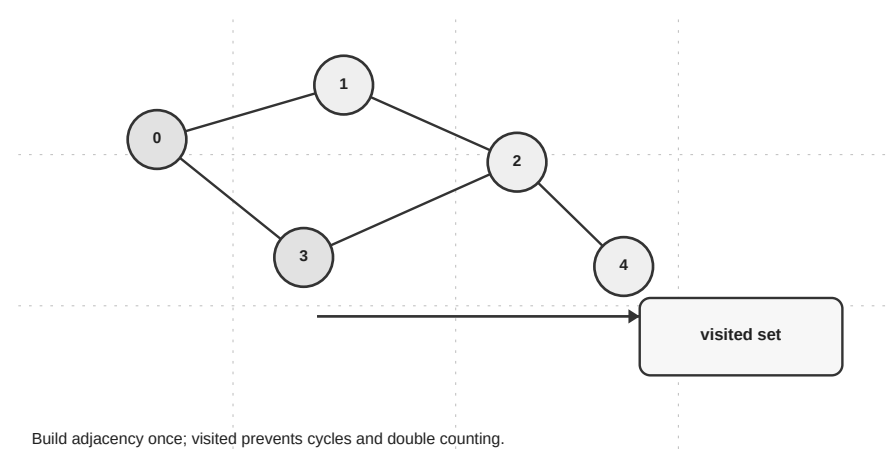
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 433. Minimum Genetic Mutation
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def minimum_genetic_mutation(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 163 of 229: 127. Word Ladder

Pattern: Graph | Sources: Top150 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Return the length of the shortest word transformation from beginWord to endWord, changing one letter at a time and using only wordList words.

Example input/output:
Input: beginWord="hit", endWord="cog", wordList=["hot","dot","dog","lot","log","cog"]

Output: 5

Why: hit->hot->dot->dog->cog.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

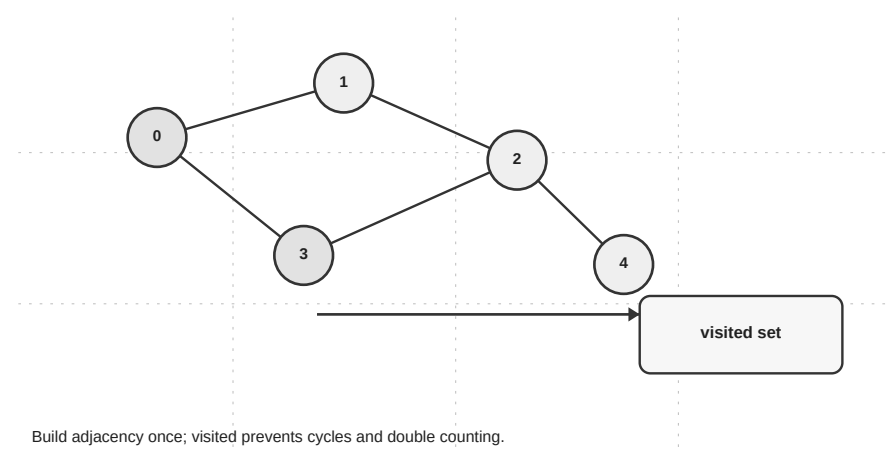
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 127. Word Ladder
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:
V1: Detect cycle vs produce traversal order.
V2: Handle disconnected components.
V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:
- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def word_ladder(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 164 of 229: 211. Design Add and Search Words Data Structure

Pattern: Trie / Encoding | Sources: Top150, NC75 | Companies: Google, Amazon, Bloomberg, Microsoft (public samples)

Time: O(total characters)

Space: O(total characters)

Problem statement + sample

Prompt:
Design a word dictionary with addWord and search, where '.' in search matches any single character.

Example input/output:
Input: add bad,dad,mad; search pad,bad,.ad,b..

Output: [false,true,true,true]

Why: Dot wildcards match any character.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every string with every query or split by ambiguous delimiters.

Optimized approach:
Store characters in a trie for prefix search, or encode lengths so decoding is unambiguous.

Alternative:
Hash maps can replace trie when only whole-word lookup is needed.

Implementation checkpoints:

- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

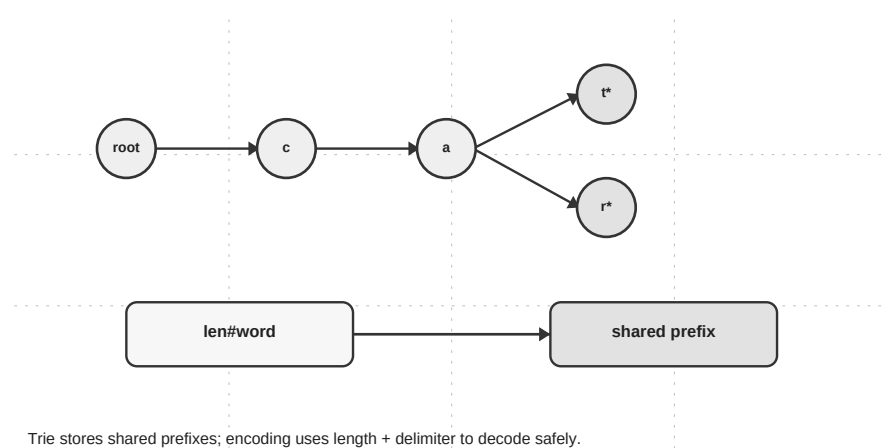
Annotated Go kernel - comments explain the reasoning

```
// 211. Design Add and Search Words Data Structure
type TrieNode struct {
    child map[byte]*TrieNode
    end bool
}

func solveTrieEncoding(words []string) *TrieNode {
    root := &TrieNode{child: map[byte]*TrieNode{}}
    for _, w := range words {
        cur := root
        for i := 0; i < len(w); i++ {
            if cur.child[w[i]] == nil { cur.child[w[i]] = &TrieNode{child: map[byte]*TrieNode{}} }
            cur = cur.child[w[i]]
        }
        cur.end = true // complete word marker
    }
    return root
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every path from root represents exactly one prefix; terminal marks complete words.

Complexity target:
Time O(total characters); Space O(total characters)

Why:
O(total characters) to build; query cost is O(length of query/prefix).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Terminal marker missing; ambiguous delimiter.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty strings; delimiter appearing in input; duplicate words; memory blowup; lexicographic ordering.

Variations and follow-ups

Three drills:

V1: Support delete/update operations.

V2: Return top-k suggestions with lexicographic ties.

V3: Use length-prefix encoding when delimiter may appear in input.

Follow-ups:

- Can the trie support delete or wildcard search?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def design_add_and_search_words_data_str(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every path from root represents exactly one prefix; terminal marks complete words.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 165 of 229: 212. Word Search II

Pattern: Trie / Encoding | Sources: Top150, NC75 | Companies: Google, Amazon, Bloomberg, Microsoft (public samples)

Time: $O(\text{total characters})$

Space: $O(\text{total characters})$

Problem statement + sample

Prompt:
Find all words from a list that can be formed in a board by adjacent horizontal or vertical cells without reusing a cell.

Example input/output:
Input: board=[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words=["oath","pea","eat","rain"]

Output: ["eat","oath"]

Why: Both words can be traced in the board.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every string with every query or split by ambiguous delimiters.

Optimized approach:
Store characters in a trie for prefix search, or encode lengths so decoding is unambiguous.

Alternative:
Hash maps can replace trie when only whole-word lookup is needed.

Implementation checkpoints:

- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

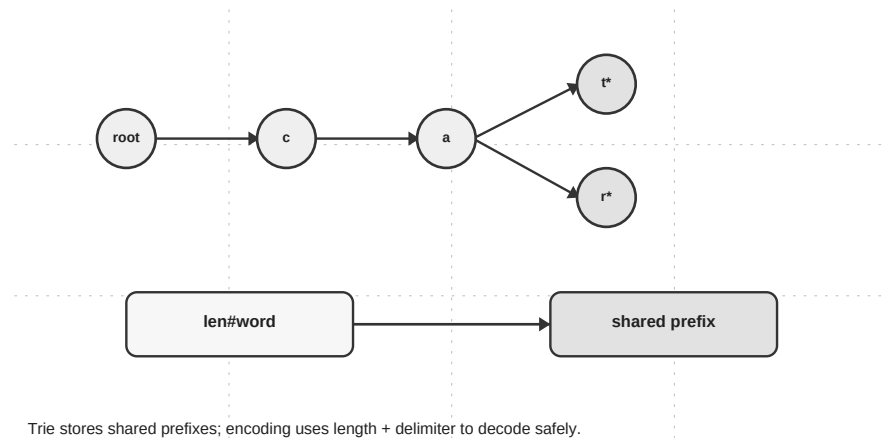
Annotated Go kernel - comments explain the reasoning

```
// 212. Word Search II
type TrieNode struct {
    child map[byte]*TrieNode
    end bool
}

func solveTrieEncoding(words []string) *TrieNode {
    root := &TrieNode{child: map[byte]*TrieNode{}}
    for _, w := range words {
        cur := root
        for i := 0; i < len(w); i++ {
            if cur.child[w[i]] == nil { cur.child[w[i]] = &TrieNode{child: map[byte]*TrieNode{}} }
            cur = cur.child[w[i]]
        }
        cur.end = true // complete word marker
    }
    return root
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every path from root represents exactly one prefix; terminal marks complete words.

Complexity target:
Time $O(\text{total characters})$; Space $O(\text{total characters})$

Why:
 $O(\text{total characters})$ to build; query cost is $O(\text{length of query/prefix})$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Terminal marker missing; ambiguous delimiter.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty strings; delimiter appearing in input; duplicate words; memory blowup; lexicographic ordering.

Variations and follow-ups

Three drills:

- V1: Support delete/update operations.
- V2: Return top-k suggestions with lexicographic ties.
- V3: Use length-prefix encoding when delimiter may appear in input.

Follow-ups:

- Can the trie support delete or wildcard search?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def word_search_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every path from root represents exactly one prefix; terminal marks complete words.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 166 of 229: 77. Combinations

Pattern: Backtracking | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Return all combinations of k numbers chosen from 1..n.

Example input/output:
Input: n=4, k=2

Output: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]

Why: All six 2-combinations are listed.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

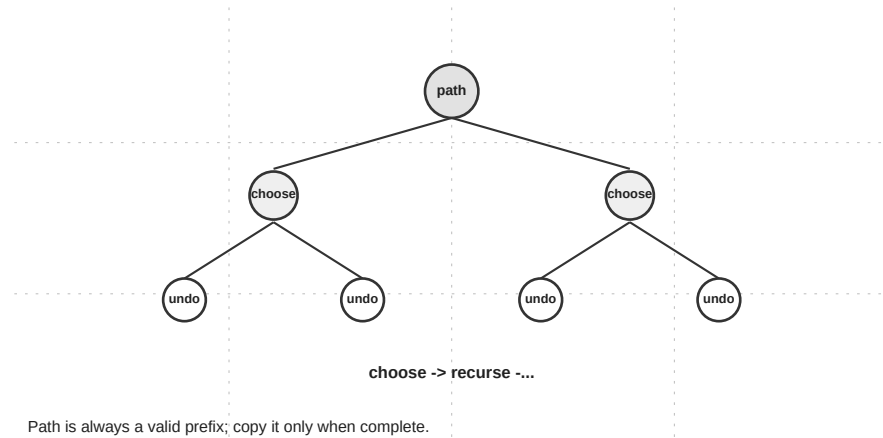
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 77. Combinations
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def combinations(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 167 of 229: 46. Permutations

Pattern: Backtracking | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: O(branch^depth)

Space: O(depth + output)

Problem statement + sample

Prompt:
Return all permutations of distinct input numbers.

Example input/output:
Input: nums=[1,2,3]

Output: 6 permutations

Why: There are 3! orderings.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

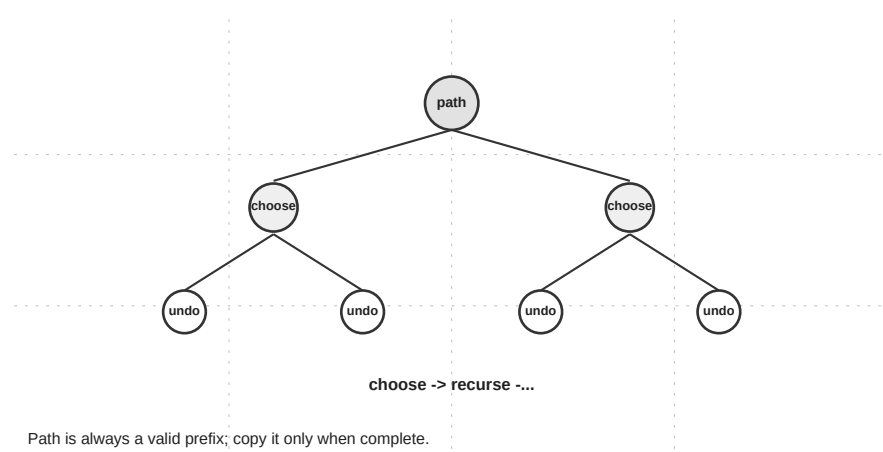
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 46. Permutations
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time O(branch^depth); Space O(depth + output)

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

- Three drills:
- V1: Return count only instead of all paths.
 - V2: Skip duplicate candidates after sorting.
 - V3: Add a max-length or lexicographic-order constraint.

- Follow-ups:
- How do you prune or avoid duplicates?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def permutations(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 168 of 229: 39. Combination Sum

Pattern: Backtracking | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Return all unique combinations where candidates may be reused unlimited times and the sum equals target.

Example input/output:
Input: candidates=[2,3,6,7], target=7

Output: [[2,2,3],[7]]

Why: 2+2+3 and 7 work.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

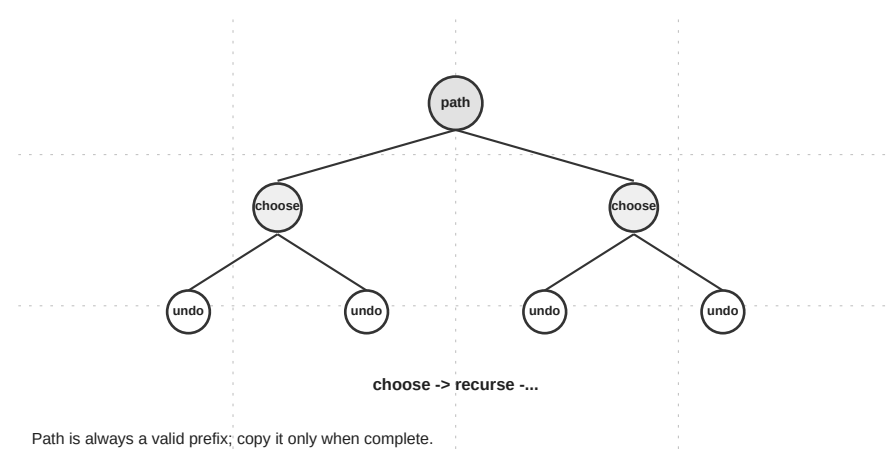
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 39. Combination Sum
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def combination_sum(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 169 of 229: 52. N-Queens II

Pattern: Backtracking | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Return the number of ways to place n queens on an n by n board so no two attack.

Example input/output:
Input: n=4

Output: 2

Why: There are two valid 4-queen boards.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

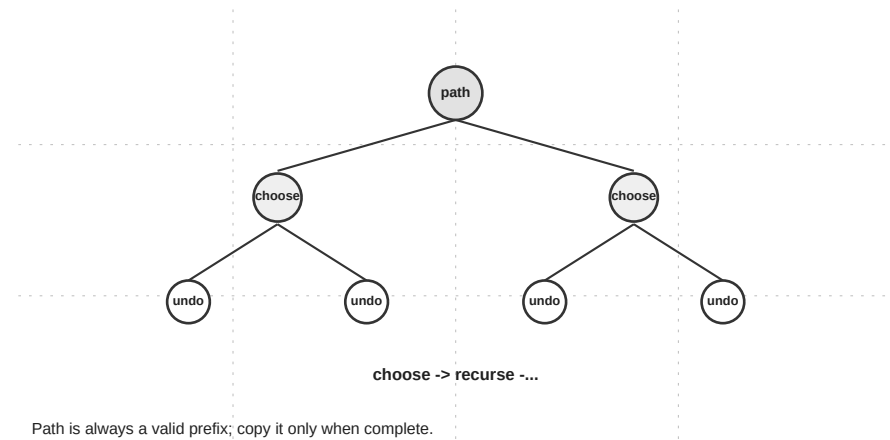
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 52. N-Queens II
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def n_queens_ii(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 170 of 229: 22. Generate Parentheses

Pattern: Backtracking | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Generate all valid parentheses strings containing n pairs.

Example input/output:
Input: n=3

Output: ["((()))","(())","()()","()()","()()"]

Why: All strings are balanced.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

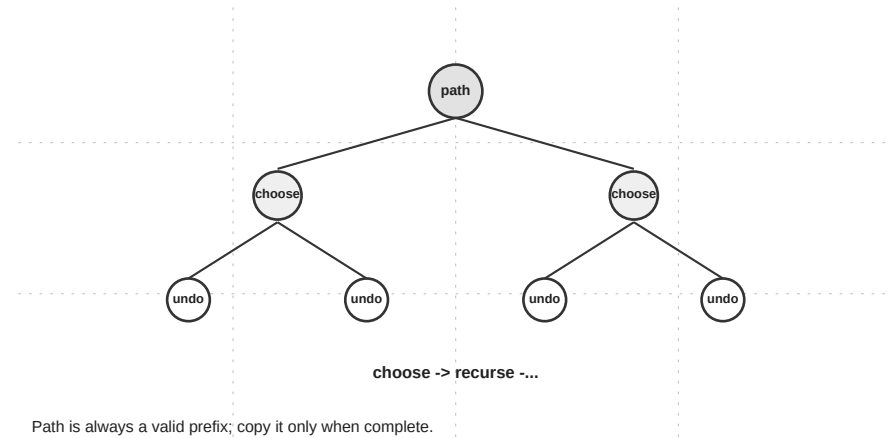
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 22. Generate Parentheses
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def generate_parentheses(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 171 of 229: 79. Word Search

Pattern: Backtracking | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(\text{branch}^{\text{depth}})$

Space: $O(\text{depth} + \text{output})$

Problem statement + sample

Prompt:
Return whether word exists in a board by moving horizontally or vertically without reusing cells.

Example input/output:
Input: board=[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word="ABCCED"

Output: true

Why: The word can be traced through adjacent cells.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Generate all raw permutations/subsets and filter after completion; wastes branches.

Optimized approach:
Build only valid prefixes, prune by remaining capacity/constraints, and undo choices exactly.

Alternative:
Bitmasks can replace used arrays for small n.

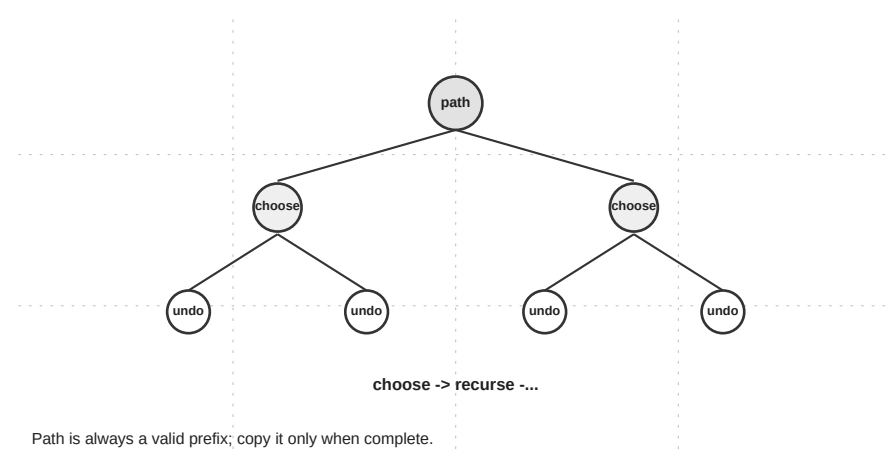
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 79. Word Search
func solveBacktracking(choices []int) [][]int {
    ans, path := [][]int{}, []int{}
    used := make([]bool, len(choices))
    var dfs func()
    dfs = func() {
        if isComplete(path) {
            ans = append(ans, append([]int(nil), path...)) // copy path
            return
        }
        for i, x := range choices {
            if used[i] || !allowed(path, x) { continue }
            used[i] = true; path = append(path, x)
            dfs()
            path = path[:len(path)-1]; used[i] = false // undo
        }
    }
    dfs()
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The path is always a valid prefix that can still be extended to a solution.

Complexity target:
Time $O(\text{branch}^{\text{depth}})$; Space $O(\text{depth} + \text{output})$

Why:
Exponential by output size; pruning lowers constants but not worst-case class.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Not undoing choice; duplicate outputs from unsorted inputs.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicate choices; forgetting undo; shallow copy path before appending; pruning too aggressively.

Variations and follow-ups

Three drills:
V1: Return count only instead of all paths.
V2: Skip duplicate candidates after sorting.
V3: Add a max-length or lexicographic-order constraint.

Follow-ups:
- How do you prune or avoid duplicates?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def word_search(choices):
    ans, path, used = [], [], [False]*len(choices)
    def dfs():
        if complete(path): ans.append(path.copy()); return
        for i, x in enumerate(choices):
            if used[i] or not allowed(path, x): continue
            used[i]=True; path.append(x); dfs(); path.pop(); used[i]=False
    dfs(); return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The path is always a valid prefix that can still be extended to a solution.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 172 of 229: 108. Convert Sorted Array to Binary Search Tree

Pattern: BST | Sources: Top150 | Companies: Google, Amazon, Microsoft, Bloomberg (public samples)

Time: O(h) to O(n)

Space: O(h)

Problem statement + sample

Prompt:
Convert a sorted array into a height-balanced binary search tree.

Example input/output:
Input: nums=[-10,-3,0,5,9]

Output: balanced BST such as [0,-3,9,-10,null,5]

Why: Middle elements become roots.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Traverse the full tree and filter/sort values. Correct but ignores BST ordering.

Optimized approach:
Use the BST invariant to prune a side or stream values in sorted order with inorder.

Alternative:
Iterative inorder gives O(h) memory and supports lazy iterators.

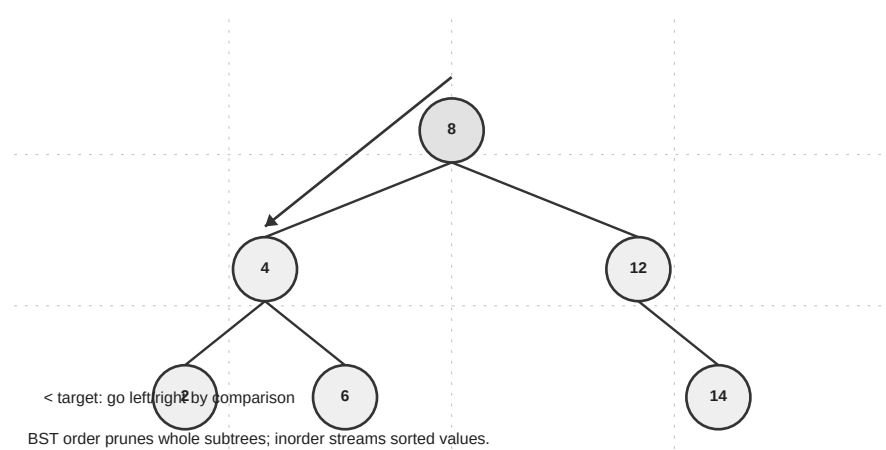
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 108. Convert Sorted Array to Binary Search Tree
func solveBST(root *TreeNode, target int) *TreeNode {
    cur := root
    for cur != nil {
        if target < cur.Val {
            cur = cur.Left // entire right side is too large
        } else if target > cur.Val {
            cur = cur.Right // entire left side is too small
        } else {
            return cur
        }
    }
    return nil
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every recursive call receives a valid range or search interval for that subtree.

Complexity target:
Time O(h) to O(n); Space O(h)

Why:
Balanced operations are O(h)=O(log n); worst-case skewed tree is O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using <= bounds incorrectly; not preserving lower and upper constraints.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
duplicates policy; min/max int bounds; nil child; deleting root; successor/predecessor replacement.

Variations and follow-ups

- Three drills:
- V1: Support duplicate values with a clear side policy.
 - V2: Convert to an iterative bounded search.
 - V3: Return predecessor/successor around a target.

- Follow-ups:
- Can you solve it with inorder iteration and O(h) memory?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def convert_sorted_array_to_binary_search(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every recursive call receives a valid range or search interval for that subtree.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 173 of 229: 148. Sort List

Pattern: Linked List | Sources: Top150 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Sort a linked list in ascending order.

Example input/output:
Input: head=[4,2,1,3]

Output: [1,2,3,4]

Why: Merge sort keeps O(n log n) time.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

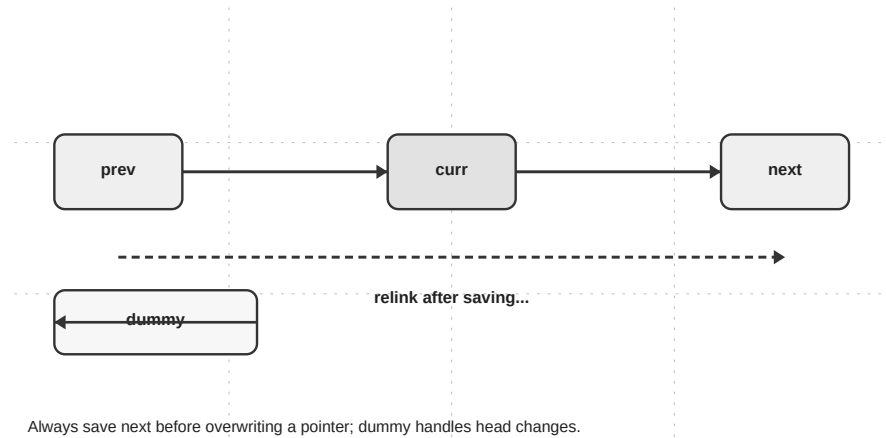
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 148. Sort List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev  // reverse or relink current edge
        prev = curr       // processed prefix grows
        curr = next       // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def sort_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 174 of 229: 427. Construct Quad Tree

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: $O(n)$ or $O(\log n)$

Space: $O(1)$

Problem statement + sample

Prompt:
Construct a quad tree from a square binary grid, merging uniform regions into leaf nodes.

Example input/output:
Input: grid=[[1,1],[1,1]]

Output: leaf node with val=true

Why: The whole grid is uniform.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

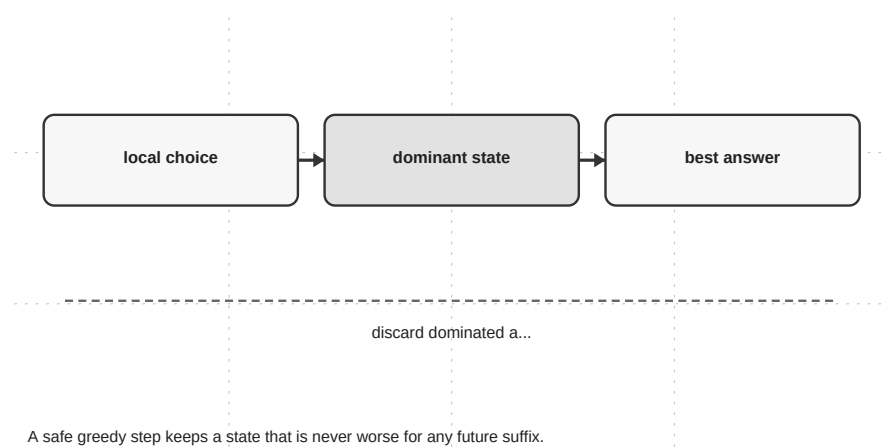
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 427. Construct Quad Tree
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time $O(n)$ or $O(\log n)$; Space $O(1)$

Why:
Usually $O(n)$ or $O(n \log n)$ if sorting is needed. Space is often $O(1)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

Three drills:
V1: Prove the greedy choice with an exchange argument.
V2: Return the chosen sequence of actions.
V3: Handle overflow and impossible cases explicitly.

Follow-ups:
- What invariant proves the greedy choice is safe?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def construct_quad_tree(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 175 of 229: 23. Merge k Sorted Lists

Pattern: Heap / Priority Queue | Sources: Top150, NC75 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Merge k sorted linked lists into one sorted linked list.

Example input/output:
Input: lists=[[1,4,5],[1,3,4],[2,6]]

Output: [1,1,2,3,4,4,5,6]

Why: Repeatedly take the smallest current node.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

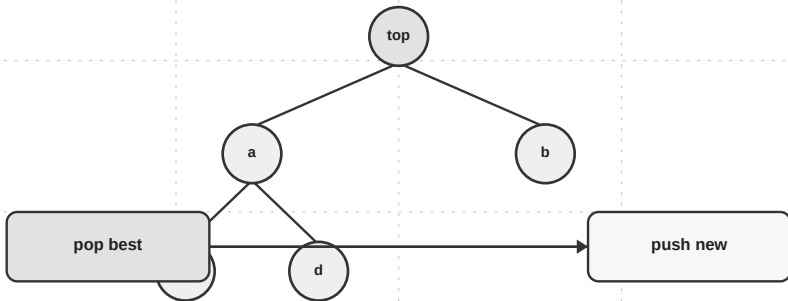
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 23. Merge k Sorted Lists
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

- Three drills:
- V1: Return top-k in sorted order after using the heap.
 - V2: Handle ties with a secondary key.
 - V3: Support streaming insertions and deletions.

- Follow-ups:
- Can you reduce memory from $O(n)$ to $O(k)$?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def merge_k_sorted_lists(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 176 of 229: 53. Maximum Subarray

Pattern: DP 2D / Kadane | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the maximum possible sum of a non-empty contiguous subarray.

Example input/output:
Input: nums=[-2,1,-3,4,-1,2,1,-5,4]

Output: 6

Why: Subarray [4,-1,2,1] sums to 6.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

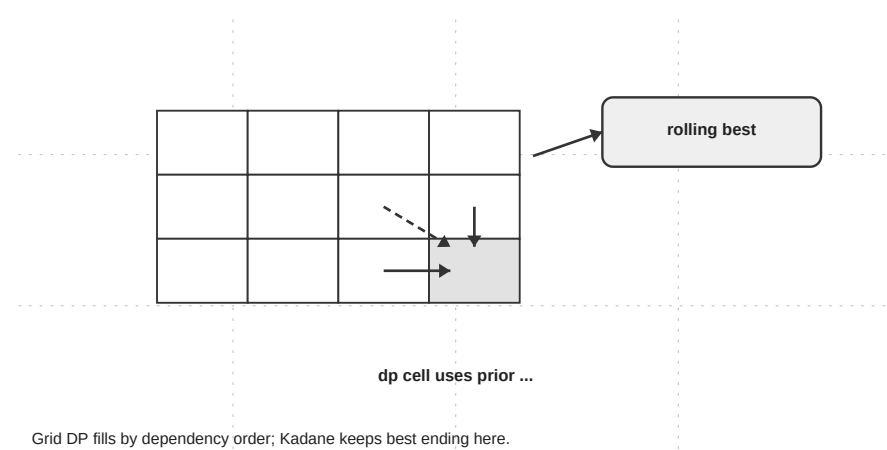
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 53. Maximum Subarray
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Bad initialization; mixing row/column dependencies.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

- Three drills:
- V1: Return the actual subarray/subsequence.
 - V2: Reduce 2D memory to rolling rows where safe.
 - V3: Handle all-negative values or empty inputs explicitly.

- Follow-ups:
- Can the DP be space compressed safely?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def maximum_subarray(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 177 of 229: 918. Maximum Sum Circular Subarray

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the maximum non-empty subarray sum in a circular array.

Example input/output:
Input: nums=[5,-3,5]

Output: 10

Why: Wraparound subarray [5,5] gives 10.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

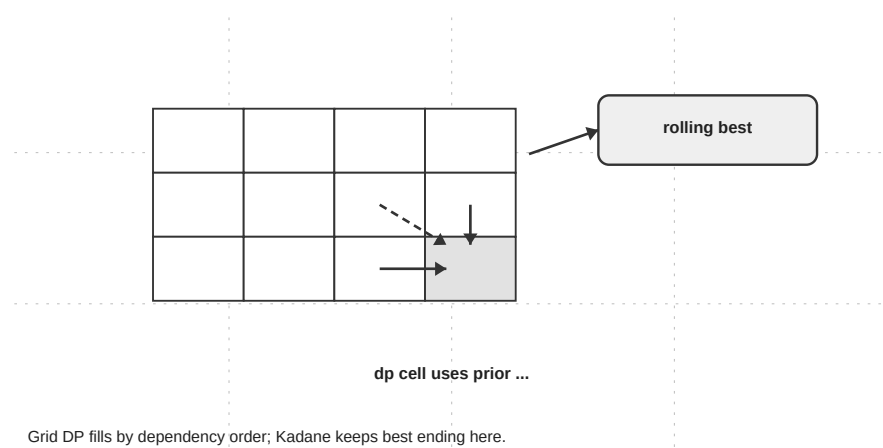
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 918. Maximum Sum Circular Subarray
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Bad initialization; mixing row/column dependencies.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

- Three drills:
- V1: Return the actual subarray/subsequence.
 - V2: Reduce 2D memory to rolling rows where safe.
 - V3: Handle all-negative values or empty inputs explicitly.

- Follow-ups:
- Can the DP be space compressed safely?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def maximum_sum_circular_subarray(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 178 of 229: 35. Search Insert Position

Pattern: Binary Search | Sources: Top150 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return the index of target in a sorted array, or the index where it should be inserted to maintain order.

Example input/output:
Input: nums=[1,3,5,6], target=2

Output: 1

Why: 2 belongs between 1 and 3.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

Implementation checkpoints:

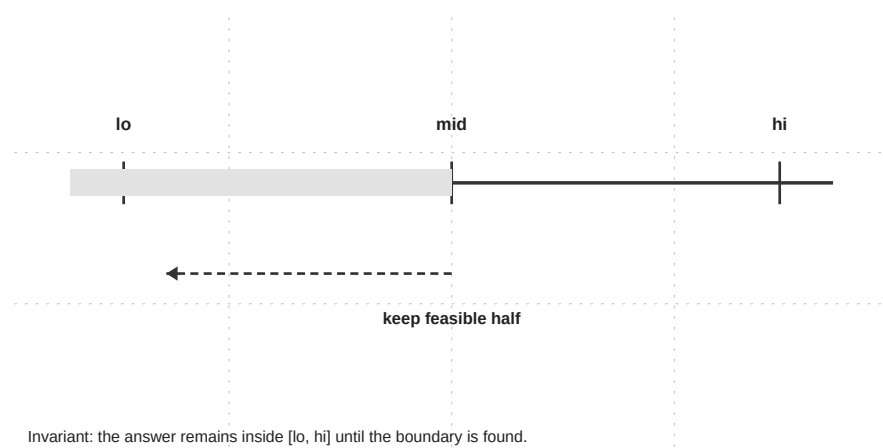
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 35. Search Insert Position
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:

- V1: Find first true vs last true boundary.
- V2: Binary-search the answer value instead of an index.
- V3: Handle duplicates and missing target gracefully.

Follow-ups:

- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def search_insert_position(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 179 of 229: 74. Search a 2D Matrix

Pattern: Binary Search | Sources: Top150 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Search a matrix where each row is sorted and the first value of each row is greater than the previous row's last value.

Example input/output:
Input: matrix=[[1,3,5],[7,9,11]], target=9

Output: true

Why: Treat the matrix as one sorted array.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

Implementation checkpoints:

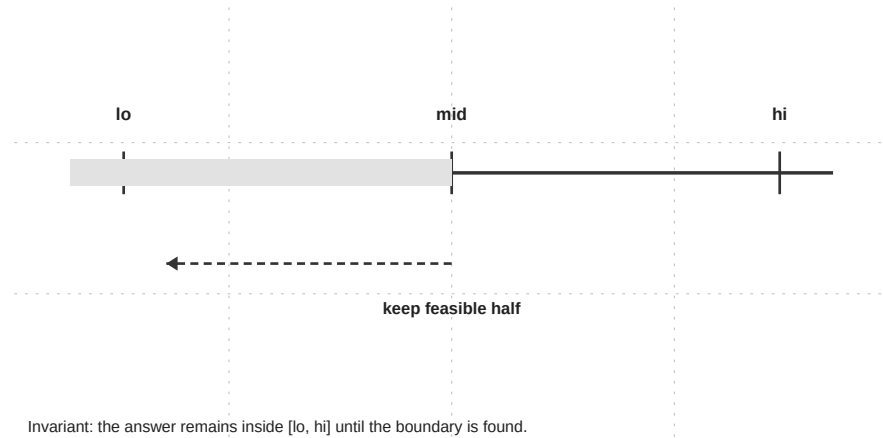
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 74. Search a 2D Matrix
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:

- V1: Find first true vs last true boundary.
- V2: Binary-search the answer value instead of an index.
- V3: Handle duplicates and missing target gracefully.

Follow-ups:

- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def search_a_2d_matrix(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 180 of 229: 33. Search in Rotated Sorted Array

Pattern: Binary Search | Sources: Top150, NC75 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Search a target in a rotated sorted array with distinct values; return its index or -1.

Example input/output:
Input: nums=[4,5,6,7,0,1,2], target=0

Output: 4

Why: 0 is at index 4.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

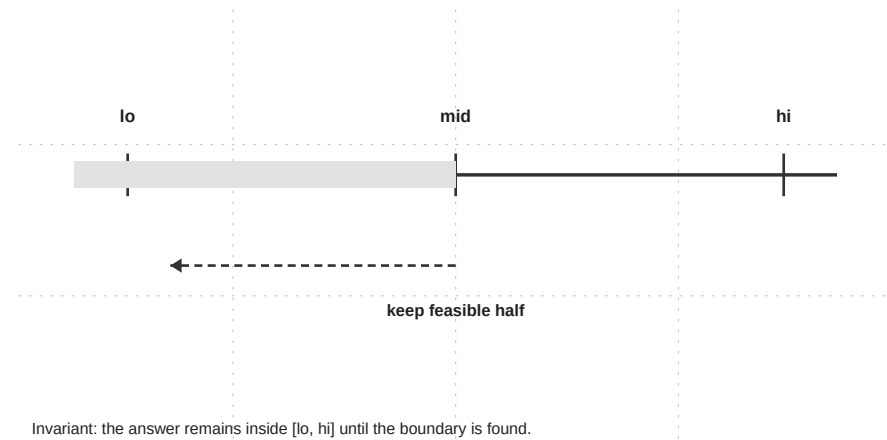
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 33. Search in Rotated Sorted Array
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:
V1: Find first true vs last true boundary.
V2: Binary-search the answer value instead of an index.
V3: Handle duplicates and missing target gracefully.

Follow-ups:
- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def search_in_rotated_sorted_array(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 181 of 229: 34. Find First and Last Position of Element in Sorted Array

Pattern: Binary Search | Sources: Top150 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return the first and last index of target in a sorted array, or [-1,-1].

Example input/output:
Input: nums=[5,7,7,8,8,10], target=8

Output: [3,4]

Why: 8 occupies indices 3 and 4.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

Implementation checkpoints:

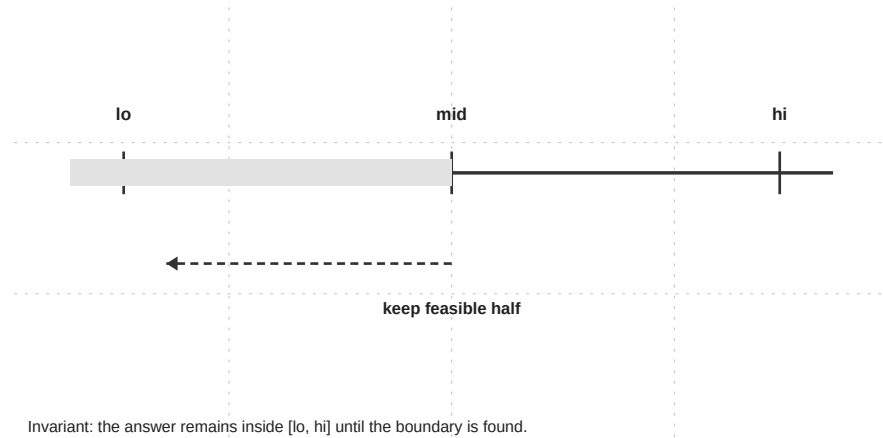
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 34. Find First and Last Position of Element in Sorted Array
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:

- V1: Find first true vs last true boundary.
- V2: Binary-search the answer value instead of an index.
- V3: Handle duplicates and missing target gracefully.

Follow-ups:

- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_first_and_last_position_of_elem(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 182 of 229: 153. Find Minimum in Rotated Sorted Array

Pattern: Binary Search | Sources: Top150, NC75 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return the minimum element in a rotated sorted array with distinct values.

Example input/output:
Input: nums=[3,4,5,1,2]

Output: 1

Why: The rotation pivot is at value 1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

Implementation checkpoints:

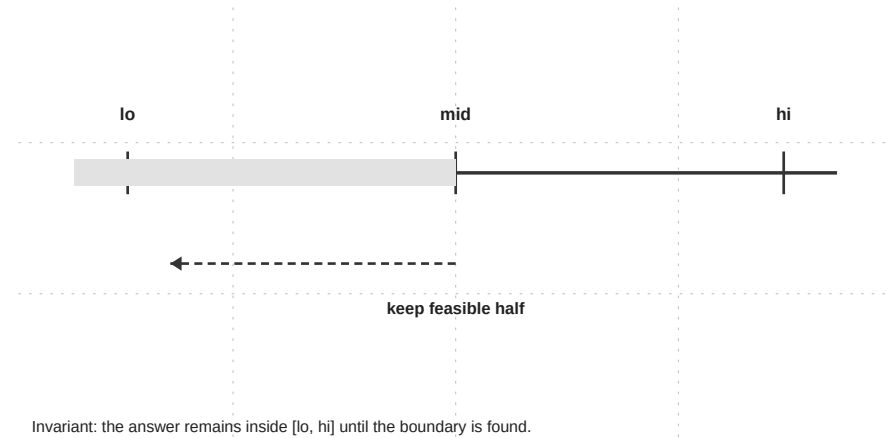
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 153. Find Minimum in Rotated Sorted Array
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:

- V1: Find first true vs last true boundary.
- V2: Binary-search the answer value instead of an index.
- V3: Handle duplicates and missing target gracefully.

Follow-ups:

- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def find_minimum_in_rotated_sorted_array(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 183 of 229: 4. Median of Two Sorted Arrays

Pattern: Binary Search | Sources: Top150 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return the median of two sorted arrays in $O(\log(m+n))$ time.

Example input/output:
Input: nums1=[1,3], nums2=[2]

Output: 2.0

Why: Merged order is [1,2,3].

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow $[lo, hi]$ until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

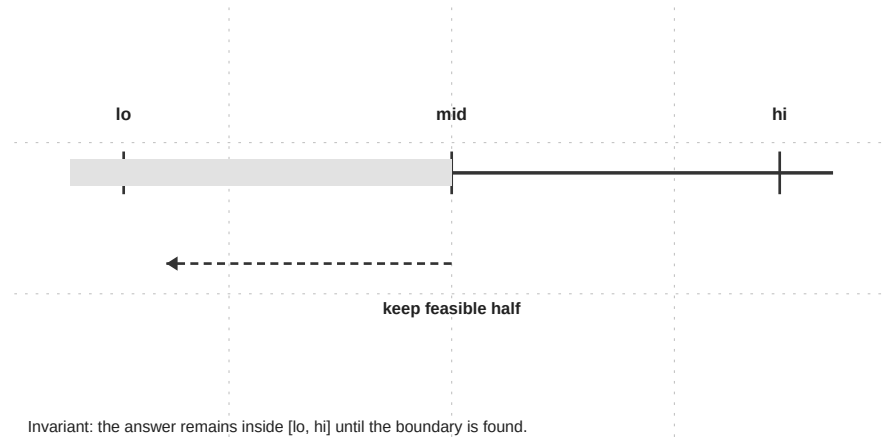
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 4. Median of Two Sorted Arrays
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Infinite loop from mid/lo/hi update; predicate not monotonic.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

Three drills:
V1: Find first true vs last true boundary.
V2: Binary-search the answer value instead of an index.
V3: Handle duplicates and missing target gracefully.

Follow-ups:
- What is the exact monotonic predicate?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def median_of_two_sorted_arrays(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 184 of 229: 502. IPO

Pattern: Heap / Priority Queue | Sources: Top150 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
With starting capital w , choose at most k projects whose capital requirement is met to maximize final capital.

Example input/output:
Input: $k=2$, $w=0$, $\text{profits}=[1,2,3]$, $\text{capital}=[0,1,1]$

Output: 4

Why: Do profit 1, then profit 3.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k \cdot n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one k th statistic; heap is better for streaming or repeated picks.

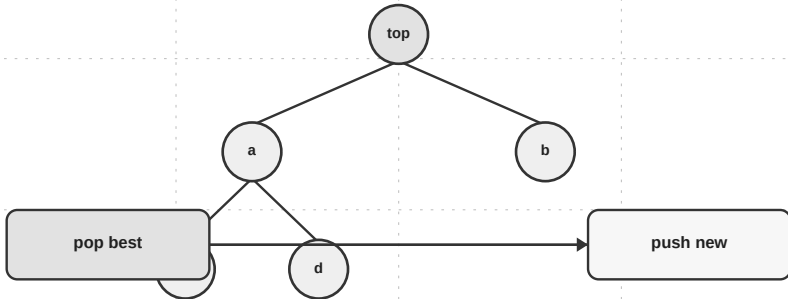
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 502. IPO
func solveHeap(items [][]int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Using min-heap/max-heap sign incorrectly; heap grows beyond k .
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

Three drills:
V1: Return top- k in sorted order after using the heap.
V2: Handle ties with a secondary key.
V3: Support streaming insertions and deletions.

Follow-ups:
- Can you reduce memory from $O(n)$ to $O(k)$?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def ipo(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 185 of 229: 373. Find K Pairs with Smallest Sums

Pattern: Heap / Priority Queue | Sources: Top150 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the k pairs (one from each sorted array) with the smallest sums.

Example input/output:
Input: nums1=[1,7,11], nums2=[2,4,6], k=3

Output: [[1,2],[1,4],[1,6]]

Why: All three smallest pairs use 1.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

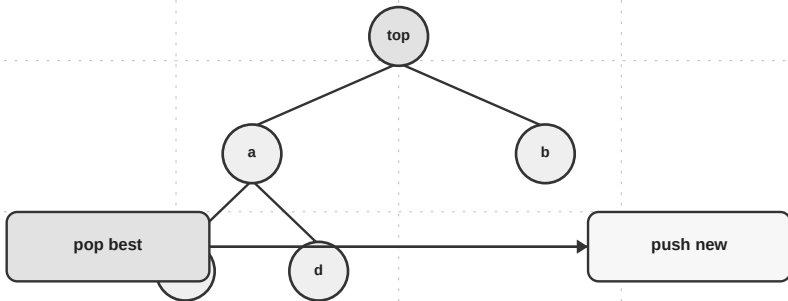
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 373. Find K Pairs with Smallest Sums
func solveHeap(items [][]int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

- Three drills:
- V1: Return top-k in sorted order after using the heap.
 - V2: Handle ties with a secondary key.
 - V3: Support streaming insertions and deletions.

- Follow-ups:
- Can you reduce memory from $O(n)$ to $O(k)$?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def find_k_pairs_with_smallest_sums(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 186 of 229: 295. Find Median from Data Stream

Pattern: Heap / Priority Queue | Sources: Top150, NC75 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Design a data structure supporting addNum and findMedian for a stream of numbers.

Example input/output:
Input: add 1, add 2, find, add 3, find

Output: [null,null,1.5,null,2]

Why: Two heaps maintain lower and upper halves.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

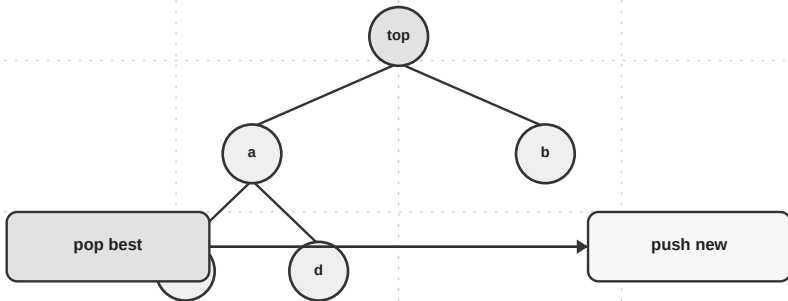
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 295. Find Median from Data Stream
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

- Three drills:
- V1: Return top-k in sorted order after using the heap.
 - V2: Handle ties with a secondary key.
 - V3: Support streaming insertions and deletions.

- Follow-ups:
- Can you reduce memory from $O(n)$ to $O(k)$?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def find_median_from_data_stream(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 187 of 229: 67. Add Binary

Pattern: Bit Manipulation | Sources: Top150 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Add two binary strings and return the binary sum as a string.

Example input/output:
Input: a="1010", b="1011"

Output: "10101"

Why: 10+11 in decimal is 21.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

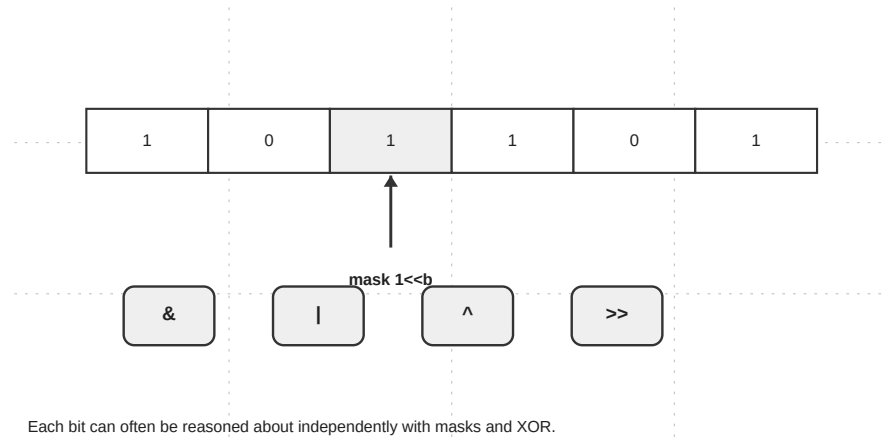
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 67. Add Binary
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:
V1: Generalize from one missing/unique value to two.
V2: Use fixed 32-bit signed behavior.
V3: Count set bits across a range instead of one value.

Follow-ups:
- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def add_binary(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 188 of 229: 190. Reverse Bits

Pattern: Bit Manipulation | Sources: Top150, NC75 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Reverse the bits of a 32-bit unsigned integer.

Example input/output:
Input: n=43261596

Output: 964176192

Why: The 32-bit pattern is reversed.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

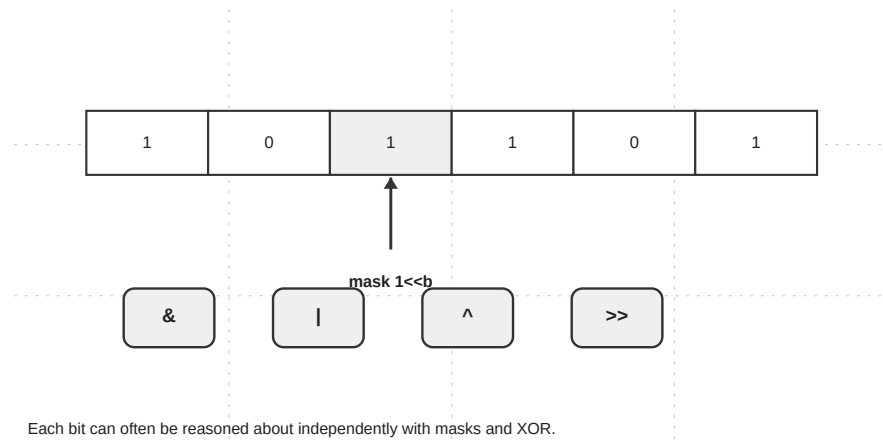
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 190. Reverse Bits
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Signed shifts and negative numbers; forgetting mask width.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

- Three drills:
- V1: Generalize from one missing/unique value to two.
 - V2: Use fixed 32-bit signed behavior.
 - V3: Count set bits across a range instead of one value.

- Follow-ups:
- How do you handle signed 32-bit behavior?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def reverse_bits(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 189 of 229: 191. Number of 1 Bits

Pattern: Bit Manipulation | Sources: Top150, NC75 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Return the number of 1 bits in an unsigned integer.

Example input/output:
Input: n=11

Output: 3

Why: Binary 1011 has three ones.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

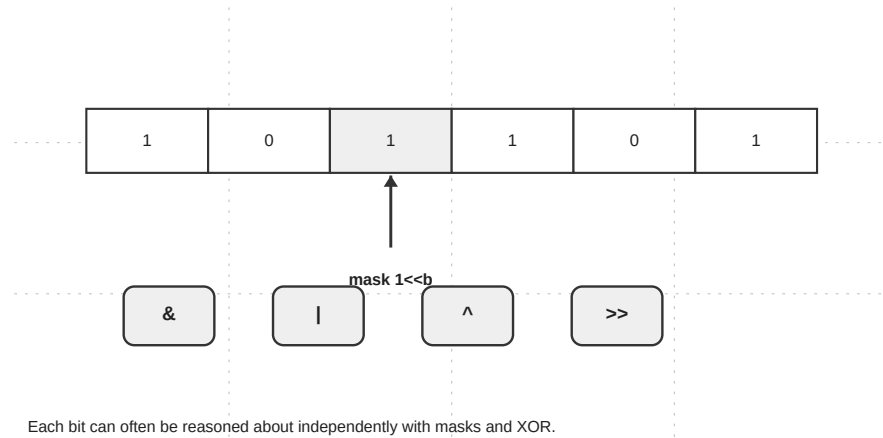
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 191. Number of 1 Bits
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Signed shifts and negative numbers; forgetting mask width.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

- Three drills:
- V1: Generalize from one missing/unique value to two.
 - V2: Use fixed 32-bit signed behavior.
 - V3: Count set bits across a range instead of one value.

- Follow-ups:
- How do you handle signed 32-bit behavior?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def number_of_1_bits(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 190 of 229: 137. Single Number II

Pattern: Bit Manipulation | Sources: Top150 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Every value appears three times except one value that appears once; return the single value.

Example input/output:
Input: nums=[2,2,3,2]

Output: 3

Why: Bit counts modulo 3 isolate 3.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

Implementation checkpoints:

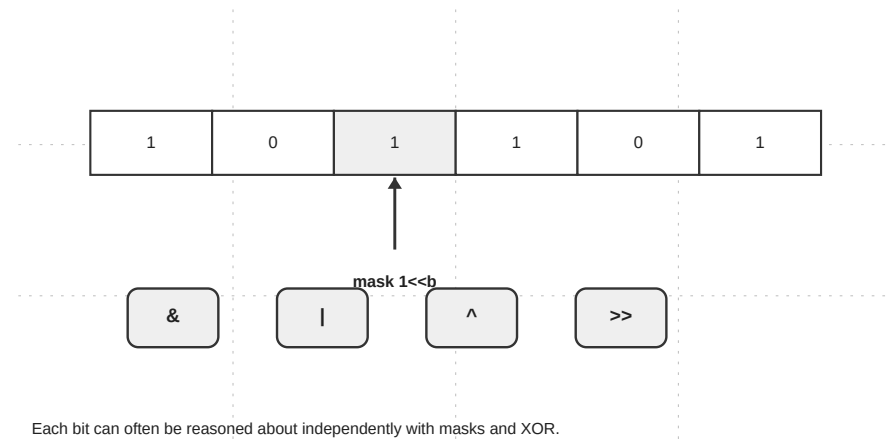
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 137. Single Number II
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:

- V1: Generalize from one missing/unique value to two.
- V2: Use fixed 32-bit signed behavior.
- V3: Count set bits across a range instead of one value.

Follow-ups:

- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def single_number_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 191 of 229: 201. Bitwise AND of Numbers Range

Pattern: Bit Manipulation | Sources: Top150 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Return the bitwise AND of every integer in the inclusive range [left,right].

Example input/output:
Input: left=5, right=7

Output: 4

Why: 5&6&7 equals 4.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

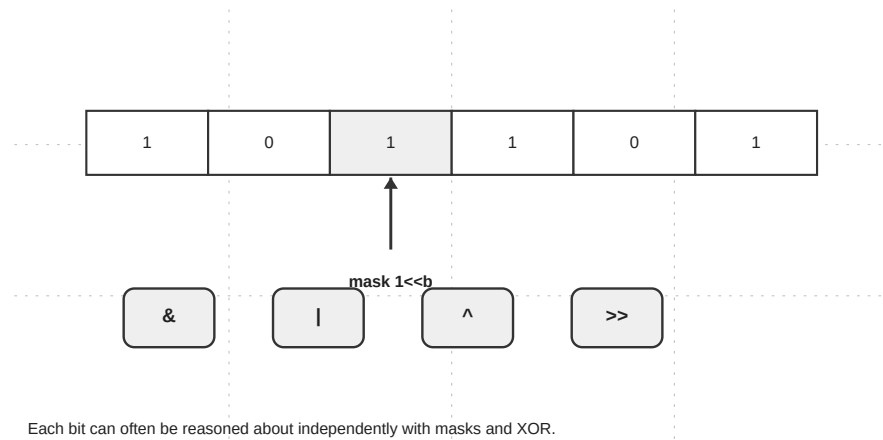
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 201. Bitwise AND of Numbers Range
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:
V1: Generalize from one missing/unique value to two.
V2: Use fixed 32-bit signed behavior.
V3: Count set bits across a range instead of one value.

Follow-ups:
- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def bitwise_and_of_numbers_range(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 192 of 229: 9. Palindrome Number

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Return whether an integer reads the same forward and backward without converting if possible.

Example input/output:
Input: x=121

Output: true

Why: 121 is symmetric.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

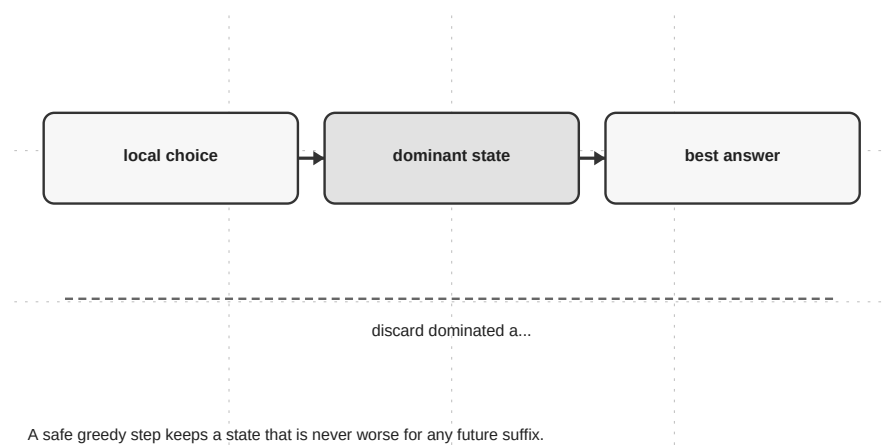
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 9. Palindrome Number
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def palindrome_number(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 193 of 229: 66. Plus One

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Add one to a nonnegative integer represented as an array of decimal digits.

Example input/output:
Input: digits=[9,9]

Output: [1,0,0]

Why: 99+1=100.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

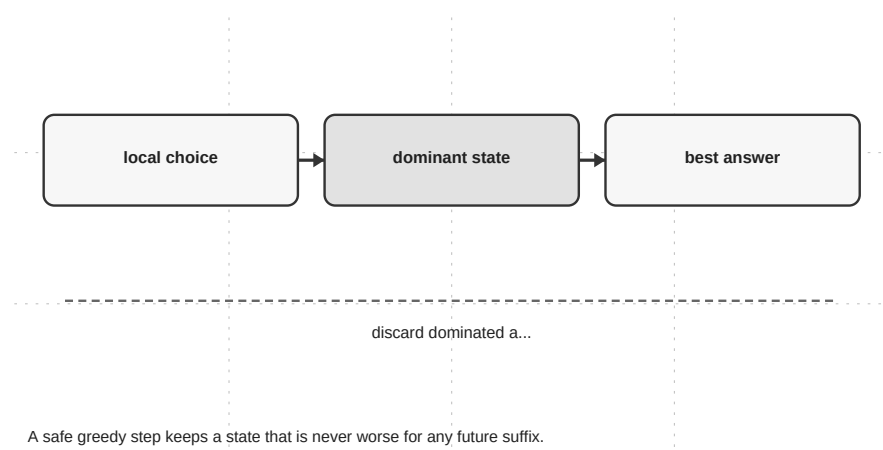
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 66. Plus One
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def plus_one(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 194 of 229: 172. Factorial Trailing Zeroes

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Return the number of trailing zeroes in n factorial.

Example input/output:
Input: n=25

Output: 6

Why: Factors of 5 contribute six zeroes.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

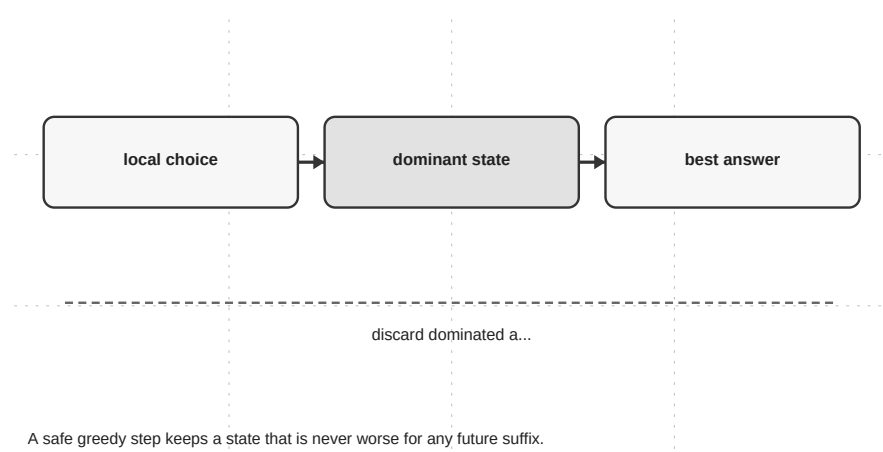
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 172. Factorial Trailing Zeroes
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def factorial_trailing_zeroes(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 195 of 229: 69. Sqrt(x)

Pattern: Binary Search | Sources: Top150 | Companies: Google, Amazon, Microsoft, Meta (public samples)

Time: $O(\log n)$ or $O(n \log R)$

Space: $O(1)$

Problem statement + sample

Prompt:
Return floor(sqrt(x)) for a nonnegative integer x.

Example input/output:
Input: x=8

Output: 2

Why: sqrt(8) is between 2 and 3.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Scan every possible index/value. Works only when range is small.

Optimized approach:
Exploit sorted order or monotonic feasibility. Narrow [lo, hi] until the boundary is found.

Alternative:
Use lower_bound/upper_bound variants for first true, last true, or exact match.

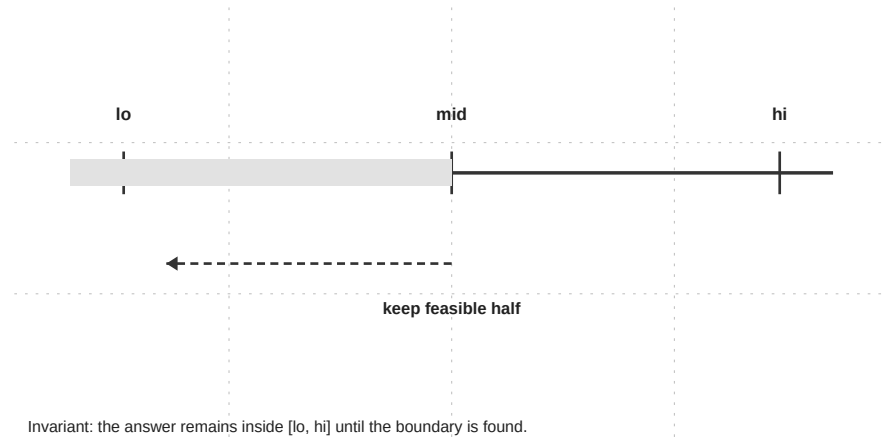
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 69. Sqrt(x)
func solveBinarySearch(lo, hi int) int {
    ans := -1
    for lo <= hi {
        mid := lo + (hi-lo)/2 // avoids overflow
        if feasible(mid) {
            ans = mid // mid works; try to improve boundary
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The answer, if it exists, remains inside the current search interval.

Complexity target:
Time $O(\log n)$ or $O(n \log R)$; Space $O(1)$

Why:
 $O(\log n)$ iterations for index search; $O(n \log R)$ for binary search on answer with $O(n)$ feasibility.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Infinite loop from mid/lo/hi update; predicate not monotonic.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
overflow mid; infinite loops; duplicates; rotated arrays; choosing < vs <= correctly.

Variations and follow-ups

- Three drills:
- V1: Find first true vs last true boundary.
 - V2: Binary-search the answer value instead of an index.
 - V3: Handle duplicates and missing target gracefully.

- Follow-ups:
- What is the exact monotonic predicate?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def sqrt_x(lo, hi):
    ans = -1
    while lo <= hi:
        mid = lo + (hi-lo)//2
        if feasible(mid):
            ans = mid; hi = mid - 1
        else:
            lo = mid + 1
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The answer, if it exists, remains inside the current search interval.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 196 of 229: 50. Pow(x, n)

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Compute x raised to integer power n.

Example input/output:
Input: x=2.0, n=-2

Output: 0.25

Why: 2^-2 is 1/4.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

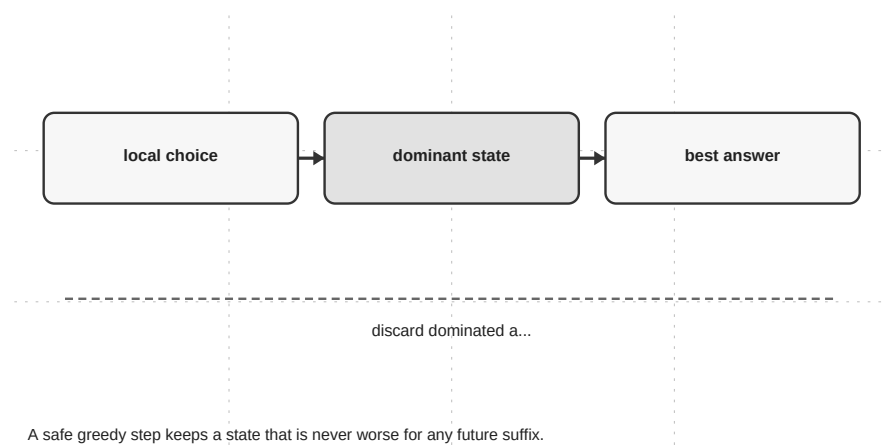
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 50. Pow(x, n)
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def pow_x_n(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 197 of 229: 149. Max Points on a Line

Pattern: Math / Greedy | Sources: Top150 | Companies: Amazon, Google, Microsoft, Bloomberg (public samples)

Time: O(n) or O(log n)

Space: O(1)

Problem statement + sample

Prompt:
Given points in the plane, return the maximum number that lie on the same straight line.

Example input/output:
Input: points=[[1,1],[2,2],[3,3],[3,4]]

Output: 3

Why: The first three points are collinear.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try all choices recursively or simulate every path. Correct but exponential or quadratic.

Optimized approach:
Identify the state that dominates future decisions, then greedily update it while scanning once.

Alternative:
Some greedy tasks can also be solved by DP first; convert to greedy after finding the redundant state.

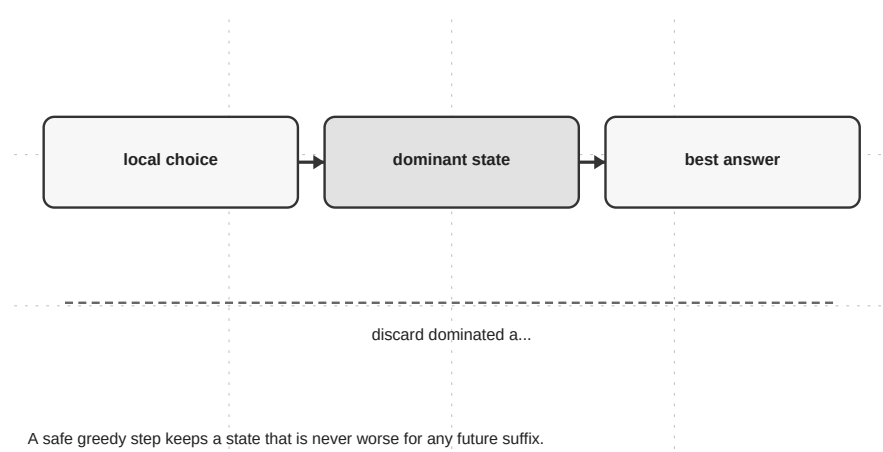
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 149. Max Points on a Line
func solveGreedy(nums []int) int {
    best, state := initialBest(), initialState()
    for i, x := range nums {
        // Keep the state that dominates all alternatives for prefix [0..i].
        state = greedyUpdate(state, x, i)
        best = max(best, score(state))
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The maintained state is at least as good as any alternative state with the same processed prefix.

Complexity target:
Time O(n) or O(log n); Space O(1)

Why:
Usually O(n) or O(n log n) if sorting is needed. Space is often O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Assuming local greedy without proof; integer division edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
ties; impossible states; negative values; very large values; integer overflow; proof of local choice.

Variations and follow-ups

- Three drills:
- V1: Prove the greedy choice with an exchange argument.
 - V2: Return the chosen sequence of actions.
 - V3: Handle overflow and impossible cases explicitly.

- Follow-ups:
- What invariant proves the greedy choice is safe?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def max_points_on_a_line(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The maintained state is at least as good as any alternative state with the same processed prefix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 198 of 229: 70. Climbing Stairs

Pattern: DP 1D | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Count distinct ways to climb n stairs taking 1 or 2 steps at a time.

Example input/output:
Input: $n=3$

Output: 3

Why: Ways are 1+1+1, 1+2, 2+1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

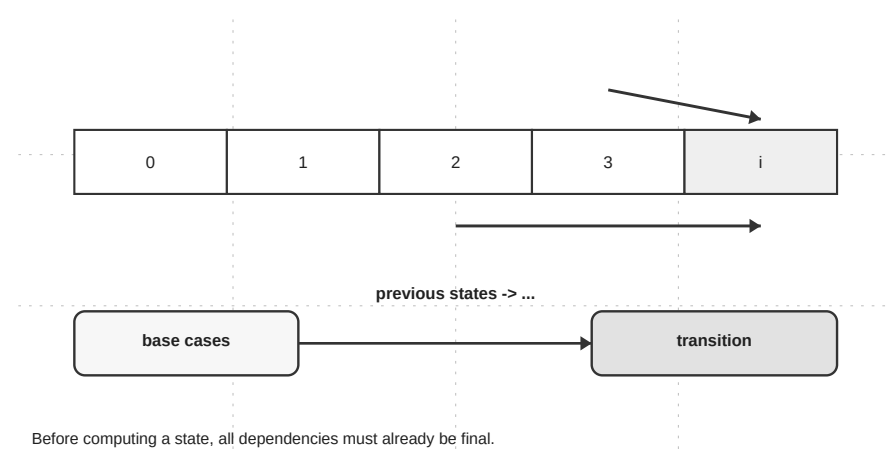
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 70. Climbing Stairs
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def climbing_stairs(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 199 of 229: 139. Word Break

Pattern: DP 1D | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Return whether s can be segmented into a sequence of dictionary words.

Example input/output:
Input: s="leetcode", wordDict=["leet","code"]

Output: true

Why: Split as "leet" + "code".

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

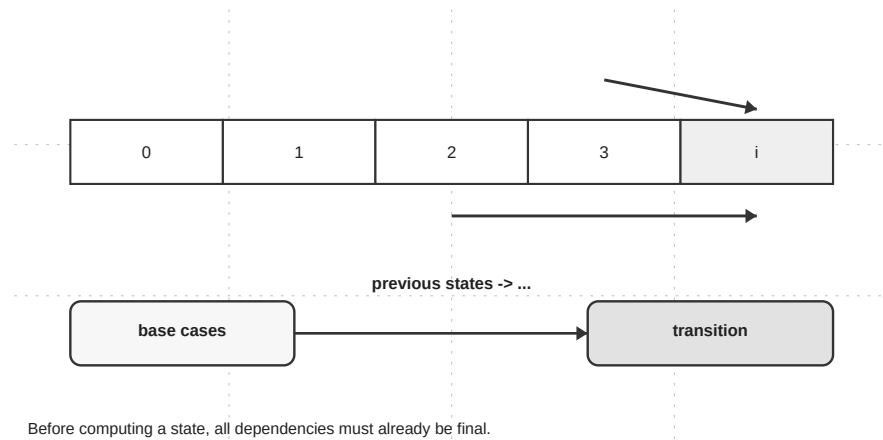
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 139. Word Break
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def word_break(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 200 of 229: 322. Coin Change

Pattern: DP 1D | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Return the fewest coins needed to make amount using unlimited coins, or -1 if impossible.

Example input/output:
Input: coins=[1,2,5], amount=11

Output: 3

Why: Use 5+5+1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

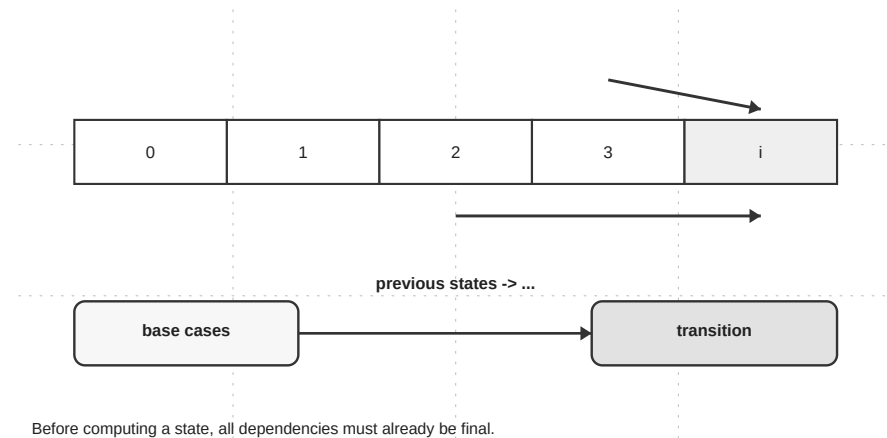
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 322. Coin Change
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}
```

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def coin_change(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 201 of 229: 300. Longest Increasing Subsequence

Pattern: DP 1D | Sources: Top150, NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Return the length of the longest strictly increasing subsequence.

Example input/output:
Input: nums=[10,9,2,5,3,7,101,18]

Output: 4

Why: One LIS is [2,3,7,101].

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

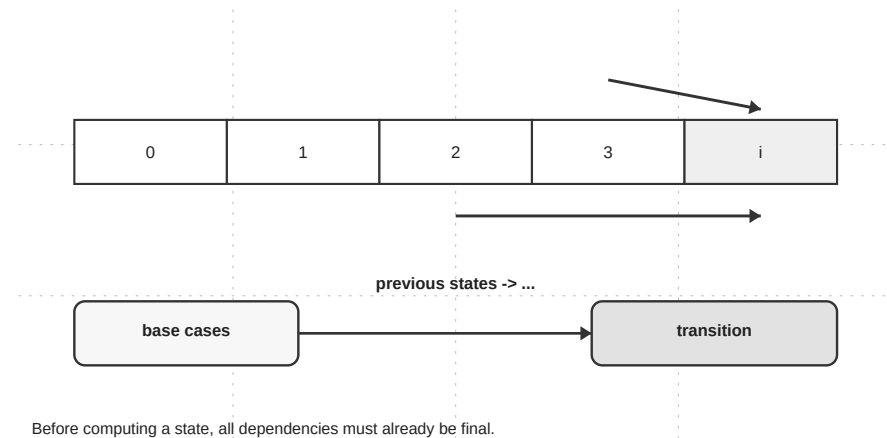
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 300. Longest Increasing Subsequence
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_increasing_subsequence(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 202 of 229: 120. Triangle

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Given a triangle of numbers, return the minimum path sum from top to bottom moving to adjacent positions.

Example input/output:
Input: triangle=[[2],[3,4],[6,5,7],[4,1,8,3]]

Output: 11

Why: Path 2->3->5->1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

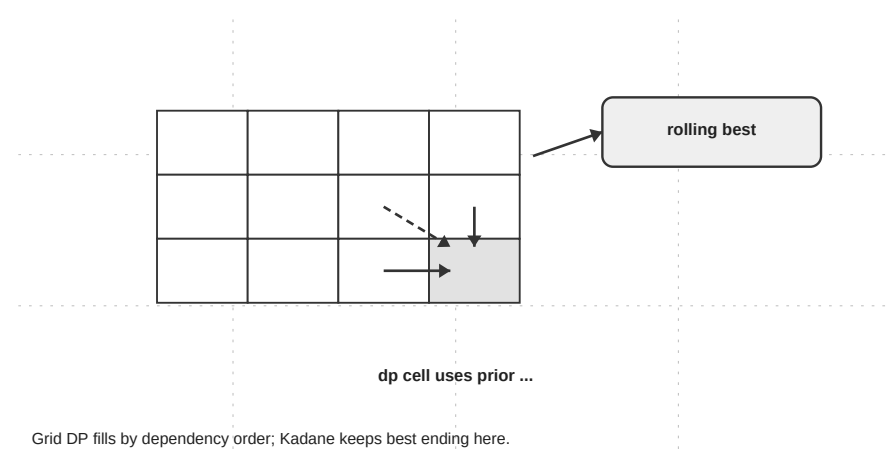
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 120. Triangle
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def triangle(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 203 of 229: 64. Minimum Path Sum

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the minimum path sum from top-left to bottom-right moving only right or down.

Example input/output:
Input: grid=[[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Why: Path 1->3->1->1->1.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

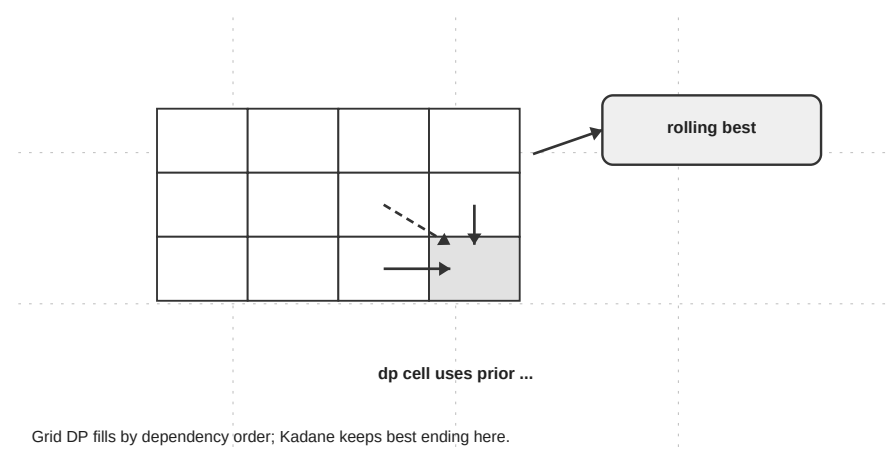
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 64. Minimum Path Sum
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def minimum_path_sum(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 204 of 229: 63. Unique Paths II

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Count unique paths in a grid with obstacles, moving only right or down.

Example input/output:
Input: obstacleGrid=[[0,0,0],[0,1,0],[0,0,0]]

Output: 2

Why: Two paths avoid the obstacle.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

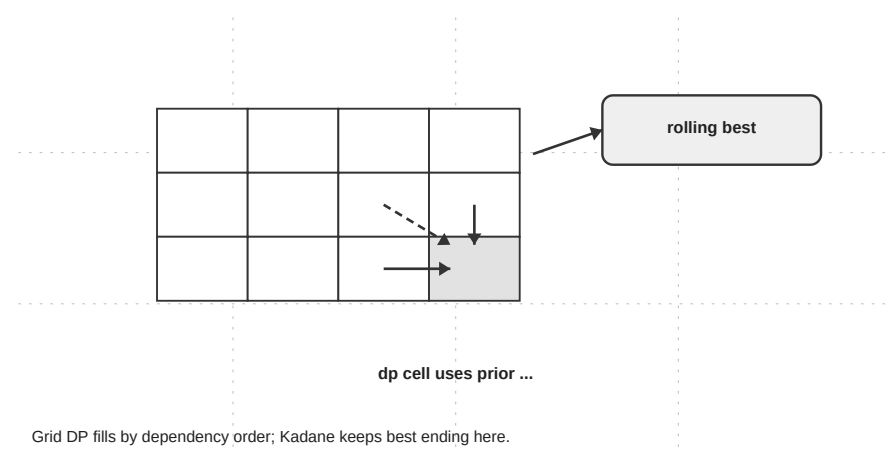
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 63. Unique Paths II
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:
V1: Return the actual subarray/subsequence.
V2: Reduce 2D memory to rolling rows where safe.
V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:
- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def unique_paths_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 205 of 229: 5. Longest Palindromic Substring

Pattern: DP 2D / Kadane | Sources: Top150, NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return any longest palindromic substring of s.

Example input/output:
Input: s="babad"

Output: "bab" or "aba"

Why: Both length-3 substrings are valid.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

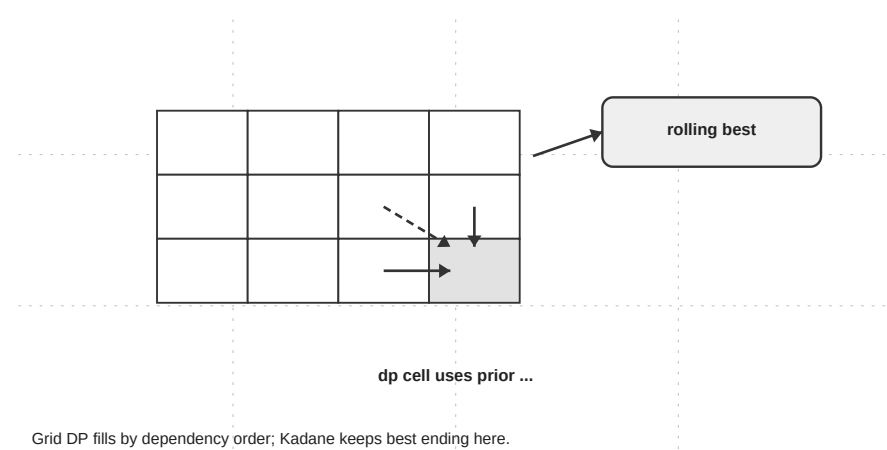
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 5. Longest Palindromic Substring
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_palindromic_substring(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 206 of 229: 97. Interleaving String

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return whether s3 is formed by interleaving s1 and s2 while preserving the order within each string.

Example input/output:
Input: s1="aabcc", s2="dbbca", s3="aadbcbcbcac"

Output: true

Why: Characters can be assigned to s1/s2 in order.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

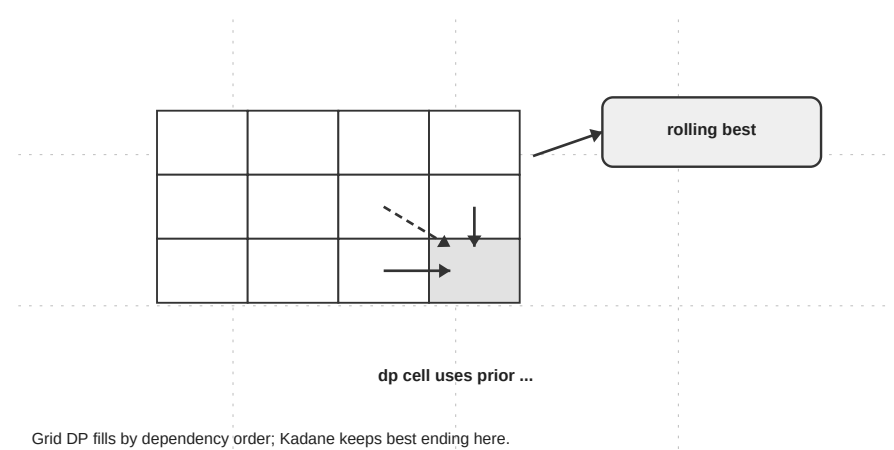
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 97. Interleaving String
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Bad initialization; mixing row/column dependencies.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

- Three drills:
- V1: Return the actual subarray/subsequence.
 - V2: Reduce 2D memory to rolling rows where safe.
 - V3: Handle all-negative values or empty inputs explicitly.

- Follow-ups:
- Can the DP be space compressed safely?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def interleaving_string(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 207 of 229: 123. Best Time to Buy and Sell Stock III

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Maximize stock profit with at most two transactions.

Example input/output:
Input: prices=[3,3,5,0,0,3,1,4]

Output: 6

Why: Buy at 0 sell at 3, buy at 1 sell at 4.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

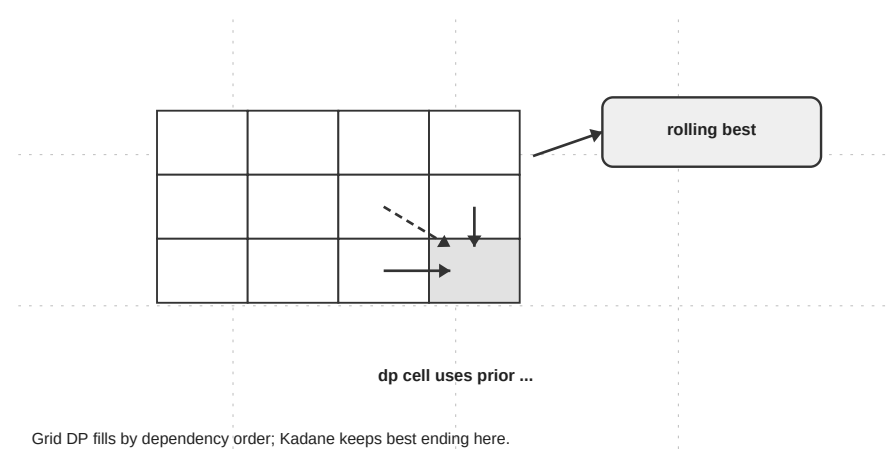
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 123. Best Time to Buy and Sell Stock III
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:
V1: Return the actual subarray/subsequence.
V2: Reduce 2D memory to rolling rows where safe.
V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:
- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def best_time_to_buy_and_sell_stock_iii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 208 of 229: 188. Best Time to Buy and Sell Stock IV

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Maximize stock profit with at most k transactions.

Example input/output:
Input: k=2, prices=[2,4,1]

Output: 2

Why: Buy at 2 and sell at 4.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

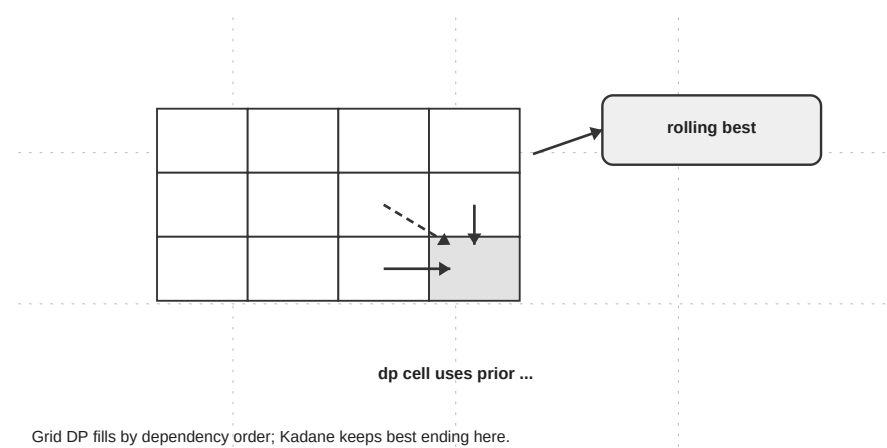
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 188. Best Time to Buy and Sell Stock IV
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:
V1: Return the actual subarray/subsequence.
V2: Reduce 2D memory to rolling rows where safe.
V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:
- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def best_time_to_buy_and_sell_stock_iv(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 209 of 229: 221. Maximal Square

Pattern: DP 2D / Kadane | Sources: Top150 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
In a binary matrix, return the area of the largest square containing only 1s.

Example input/output:
Input: matrix=[["1","0","1","0"],["1","0","1","1"],["1","1","1","1"]]

Output: 4

Why: A 2 by 2 square of ones exists.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

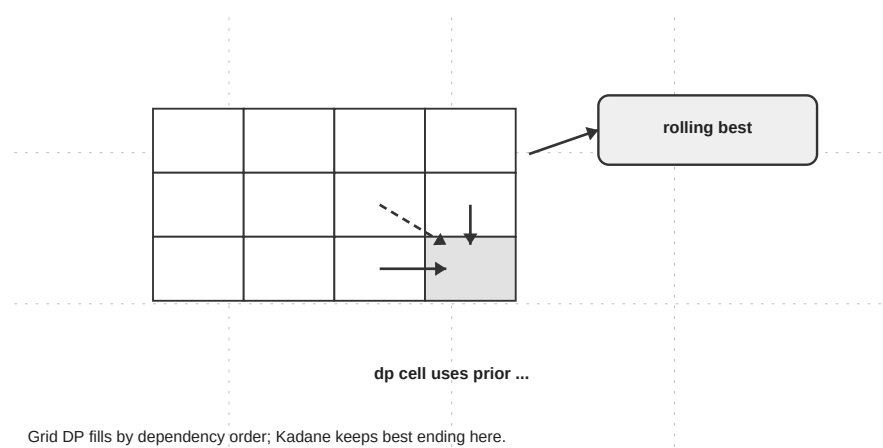
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 221. Maximal Square
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: $\text{left} \leq \text{right}$? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Bad initialization; mixing row/column dependencies.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

- Three drills:
- V1: Return the actual subarray/subsequence.
 - V2: Reduce 2D memory to rolling rows where safe.
 - V3: Handle all-negative values or empty inputs explicitly.

- Follow-ups:
- Can the DP be space compressed safely?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def maximal_square(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 210 of 229: 217. Contains Duplicate

Pattern: Hash Map / Set | Sources: NC75 | Companies: Amazon, Google, Meta, Bloomberg (public samples)

Time: $O(n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return true if any value appears at least twice in the array.

Example input/output:
Input: nums=[1,2,3,1]

Output: true

Why: Value 1 repeats.

Clarify before coding:
- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Nested scans compare every item with every other item. Usually $O(n^2)$.

Optimized approach:
Store exactly the fact needed for future lookups: index, count, membership, or normalized signature.

Alternative:
Sorting can reduce memory and expose groups but may lose original index order.

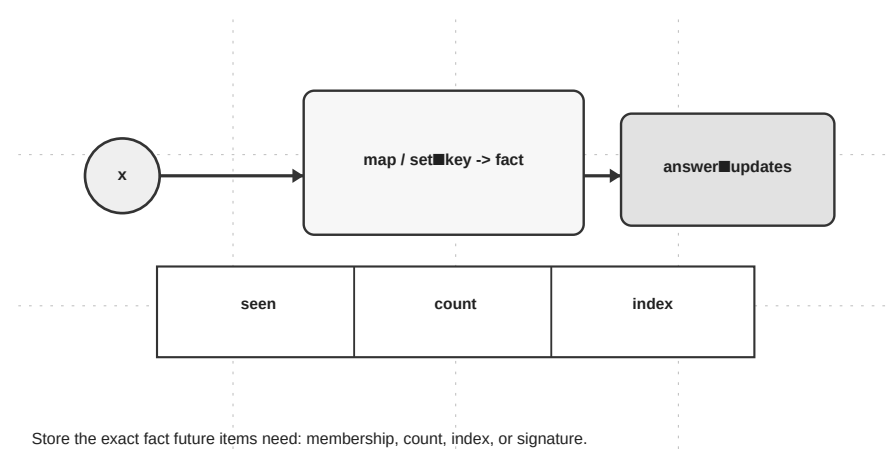
Implementation checkpoints:
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 217. Contains Duplicate
func solveWithMap(nums []int, target int) int {
    seen := map[int]int{} // value/signature -> index/count
    for i, x := range nums {
        need := target - x
        if j, ok := seen[need]; ok {
            return combineAnswer(j, i) // current item matches stored fact
        }
        seen[x] = i // store after lookup to avoid using same index twice
    }
    return defaultAnswer()
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The map contains all and only facts from the processed prefix needed by future elements.

Complexity target:
Time $O(n)$; Space $O(n)$

Why:
Expected $O(n)$ lookups/inserts. Space is $O(k)$, where k is distinct keys or signatures.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:
1) Inputs and base cases first.
2) State variables match the invariant.
3) The loop moves one pointer/index/event at a time.
4) Record the answer only after the state is valid.
5) Return the exact requested value, not debug state.

Implementation cautions:
- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:
- Counting after checking in the wrong order; ignoring duplicates.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
duplicates; missing keys; decrement-to-zero cleanup; hashable signature construction; order dependence.

Variations and follow-ups

Three drills:
V1: Return all matching groups instead of a boolean/count.
V2: Support incremental updates while queries arrive.
V3: Handle case-insensitive or normalized keys.

Follow-ups:
- How would memory limits change the approach?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def contains_duplicate(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The map contains all and only facts from the processed prefix needed by future elements.

Unit-test matrix:
- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 211 of 229: 152. Maximum Product Subarray

Pattern: DP 2D / Kadane | Sources: NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the maximum product of a non-empty contiguous subarray.

Example input/output:
Input: nums=[2,3,-2,4]

Output: 6

Why: Subarray [2,3] has product 6.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

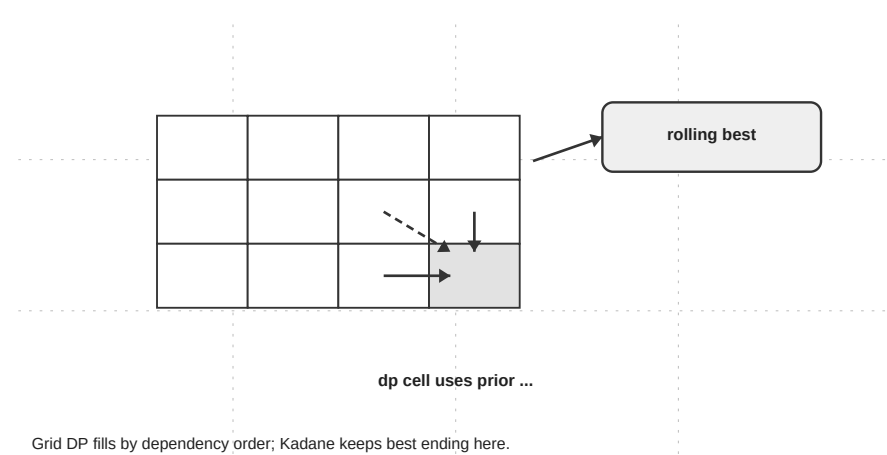
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 152. Maximum Product Subarray
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def maximum_product_subarray(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 212 of 229: 371. Sum of Two Integers

Pattern: Bit Manipulation | Sources: NC75 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: O(n) or O(bits)

Space: O(1)

Problem statement + sample

Prompt:
Return the sum of two integers without using the plus or minus operators.

Example input/output:
Input: a=1, b=2

Output: 3

Why: Use bitwise carry and xor addition.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

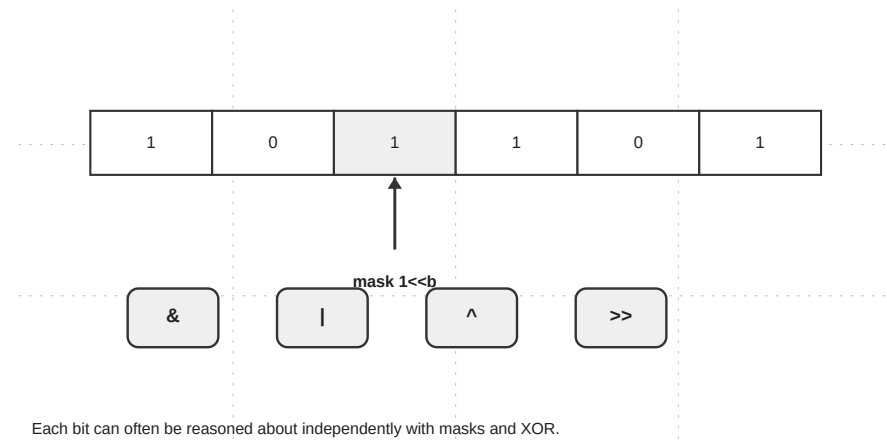
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 371. Sum of Two Integers
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time O(n) or O(bits); Space O(1)

Why:
Often O(n) for arrays or O(1) over fixed 32 bits. Space is O(1).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Signed shifts and negative numbers; forgetting mask width.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

- Three drills:
- V1: Generalize from one missing/unique value to two.
 - V2: Use fixed 32-bit signed behavior.
 - V3: Count set bits across a range instead of one value.

- Follow-ups:
- How do you handle signed 32-bit behavior?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def sum_of_two_integers(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 213 of 229: 268. Missing Number

Pattern: Bit Manipulation | Sources: NC75 | Companies: Google, Amazon, Microsoft, Apple (public samples)

Time: $O(n)$ or $O(\text{bits})$

Space: $O(1)$

Problem statement + sample

Prompt:
An array contains n distinct numbers from $0..n$ with one missing; return the missing number.

Example input/output:
Input: `nums=[3,0,1]`

Output: 2

Why: Range $0..3$ is missing 2.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Convert to strings or simulate arithmetic slowly; simpler but less direct.

Optimized approach:
Use XOR cancellation, bit counts, masks, and shift loops to isolate independent bit decisions.

Alternative:
Math identities sometimes replace bit loops, but bit reasoning is more robust.

Implementation checkpoints:

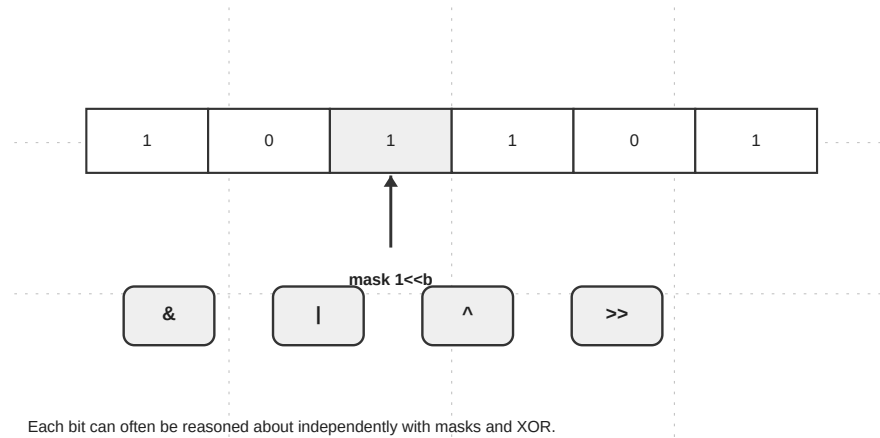
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 268. Missing Number
func solveBits(nums []int) int {
    ans := 0
    for _, x := range nums {
        // XOR cancels pairs; masks isolate independent bits.
        ans ^= x
    }
    for bit := 0; bit < 32; bit++ {
        if ans&(1<<bit) != 0 { /* bit contributes to answer */ }
    }
    return ans
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Complexity target:
Time $O(n)$ or $O(\text{bits})$; Space $O(1)$

Why:
Often $O(n)$ for arrays or $O(1)$ over fixed 32 bits. Space is $O(1)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: `left <= right?` `i < n?`
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Signed shifts and negative numbers; forgetting mask width.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
signed shifts; 32-bit vs 64-bit; negative numbers; overflow; clearing lowest set bit.

Variations and follow-ups

Three drills:

- V1: Generalize from one missing/unique value to two.
- V2: Use fixed 32-bit signed behavior.
- V3: Count set bits across a range instead of one value.

Follow-ups:

- How do you handle signed 32-bit behavior?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def missing_number(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each processed bit contributes independently or the mask preserves exactly the needed prefix/suffix.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 214 of 229: 213. House Robber II

Pattern: DP 1D | Sources: NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Maximize robbery in a circular row of houses where first and last houses are adjacent.

Example input/output:
Input: nums=[2,3,2]

Output: 3

Why: Rob the middle house only.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

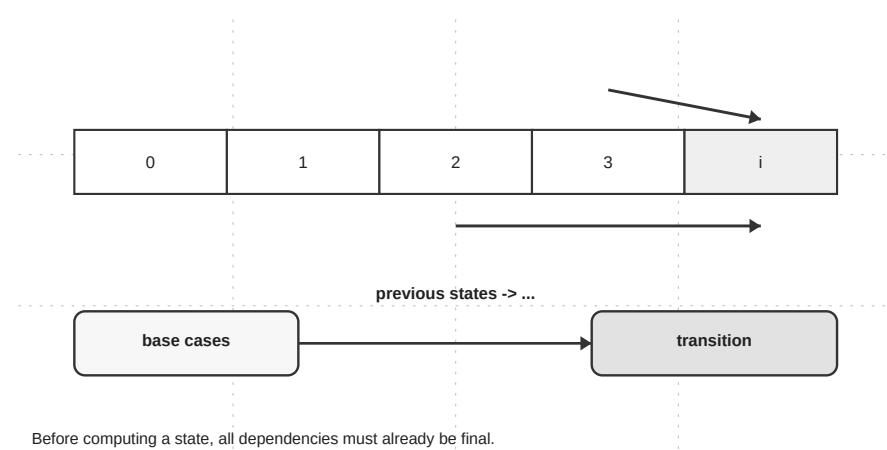
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 213. House Robber II
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def house_robber_ii(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 215 of 229: 91. Decode Ways

Pattern: DP 1D | Sources: NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n * \text{state})$

Space: $O(\text{state})$

Problem statement + sample

Prompt:
Return the number of ways to decode a digit string where 1->A through 26->Z.

Example input/output:
Input: s="226"

Output: 3

Why: It decodes as 2-2-6, 22-6, or 2-26.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Recursive try-all choices recomputes the same suffix/prefix many times.

Optimized approach:
Define state, base case, transition, and iteration order. Compress space when only recent states are needed.

Alternative:
Top-down memoization is easier first; bottom-up is usually interview-cleaner.

Implementation checkpoints:

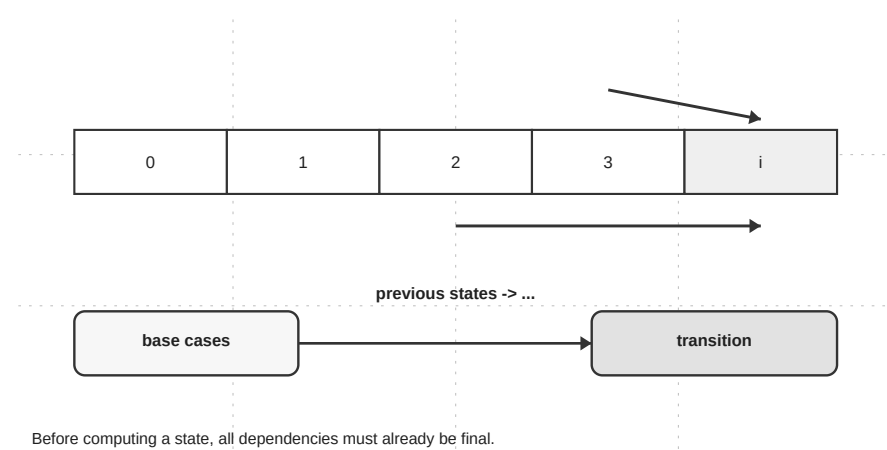
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 91. Decode Ways
func solveDP1D(nums []int) int {
    n := len(nums)
    dp := make([]int, n+1)
    dp[0] = baseValue()
    for i := 1; i <= n; i++ {
        // Transition only from states already finalized.
        dp[i] = transition(dp, nums, i)
    }
    return dp[n]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Before computing $dp[i]$, every state it depends on has already been finalized.

Complexity target:
Time $O(n * \text{state})$; Space $O(\text{state})$

Why:
States times transition cost. Usually $O(n)$ or $O(n*k)$, with $O(n)$ or compressed $O(1)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Wrong base cases; overwriting compressed state in the wrong order.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $n=0/1$; impossible sentinel values; negative costs; modulo; off-by-one state indexing.

Variations and follow-ups

Three drills:

- V1: Return the chosen items/path, not only the optimum value.
- V2: Compress memory and explain update direction.
- V3: Add modulo or large-number constraints.

Follow-ups:

- Can you reconstruct the chosen path, not just the value?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def decode_ways(nums):
    dp = [0] * (len(nums)+1)
    dp[0] = base_value()
    for i in range(1, len(dp)):
        dp[i] = transition(dp, nums, i)
    return dp[-1]

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Before computing $dp[i]$, every state it depends on has already been finalized.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 216 of 229: 417. Pacific Atlantic Water Flow

Pattern: Graph | Sources: NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Return coordinates that can flow to both the Pacific top/left edges and Atlantic bottom/right edges, moving from high to low or equal height.

Example input/output:
Input: heights=[[1,2],[4,3]]

Output: [[0,1],[1,0],[1,1]]

Why: These cells can reach both ocean borders.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

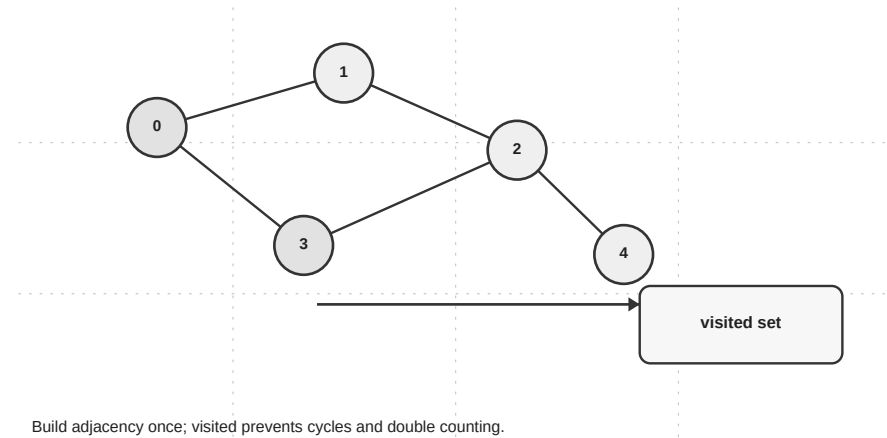
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 417. Pacific Atlantic Water Flow
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? i < n?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def pacific_atlantic_water_flow(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 217 of 229: 269. Alien Dictionary

Pattern: Graph | Sources: NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Given words sorted by an unknown alphabet, return one valid character order, or empty string if impossible.

Example input/output:
Input: words=["wrt","wrf","er","ett","rftt"]

Output: "wertf"

Why: Edges imply $w < e < r < t < f$.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

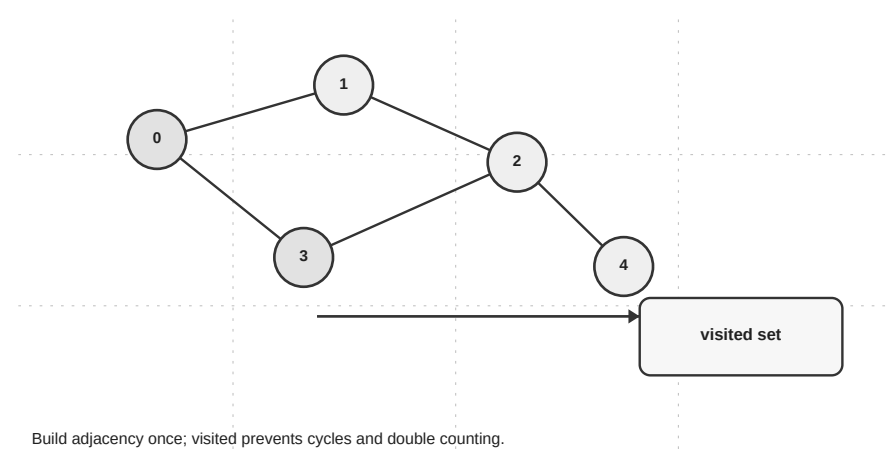
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 269. Alien Dictionary
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def alien_dictionary(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 218 of 229: 261. Graph Valid Tree

Pattern: Graph | Sources: NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Return whether an undirected graph with n nodes is a valid tree: connected and acyclic.

Example input/output:
Input: n=5, edges=[[0,1],[0,2],[0,3],[1,4]]

Output: true

Why: There are n-1 edges and all nodes connect.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

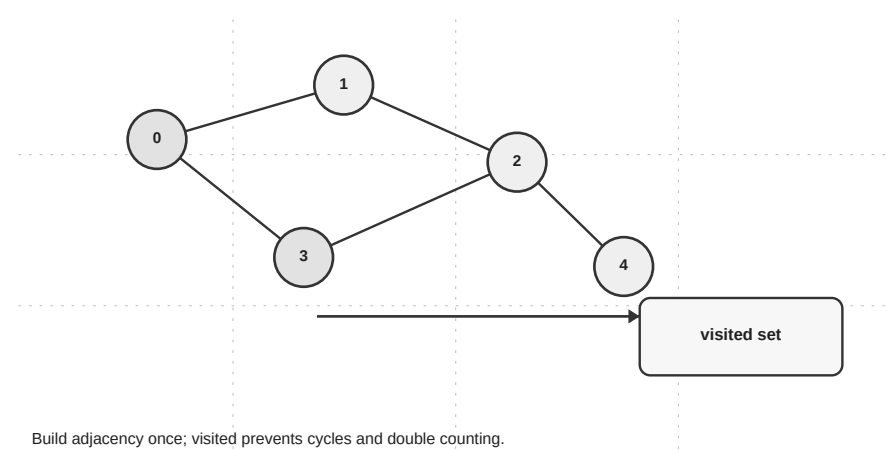
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 261. Graph Valid Tree
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Not marking visited before enqueue; confusing directed and undirected edges.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

- Three drills:
- V1: Detect cycle vs produce traversal order.
 - V2: Handle disconnected components.
 - V3: Convert unweighted BFS to weighted shortest path.

- Follow-ups:
- What happens with disconnected components or cycles?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def graph_valid_tree(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 219 of 229: 323. Number of Connected Components in an Undirected Graph

Pattern: Graph | Sources: NC75 | Companies: Google, Uber, Amazon, Meta (public samples)

Time: $O(V+E)$

Space: $O(V+E)$

Problem statement + sample

Prompt:
Return the number of connected components in an undirected graph.

Example input/output:
Input: $n=5$, $edges=[[0,1],[1,2],[3,4]]$

Output: 2

Why: Components are $\{0,1,2\}$ and $\{3,4\}$.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Try paths without visited/pruning; cycles can explode exponentially.

Optimized approach:
Build adjacency once, then BFS/DFS/topological process with visited/indegree state.

Alternative:
Union-Find is ideal for connectivity; BFS is ideal for shortest unweighted paths.

Implementation checkpoints:

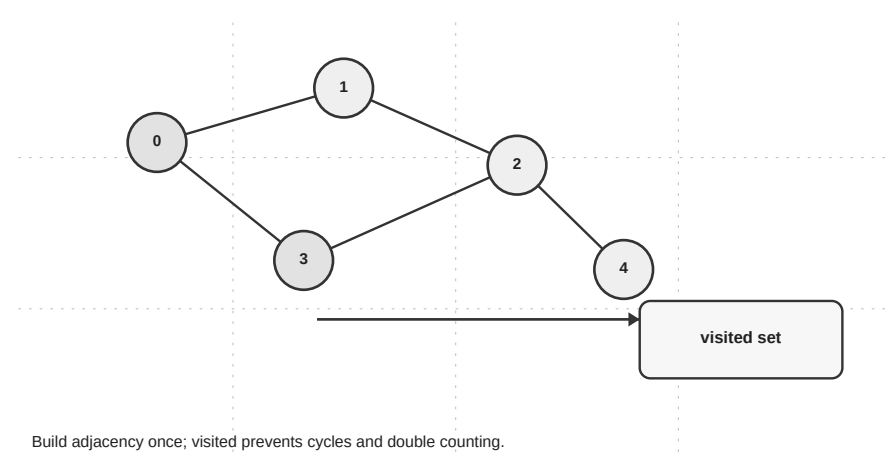
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 323. Number of Connected Components in an Undirected Graph
func solveGraph(n int, edges [][]int) int {
    adj := make([][]int, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        adj[a] = append(adj[a], b)
        adj[b] = append(adj[b], a) // remove for directed graphs
    }
    seen := make([]bool, n)
    answer := 0
    var dfs func(int)
    dfs = func(u int) {
        seen[u] = true
        for _, v := range adj[u] {
            if !seen[v] { dfs(v) }
        }
    }
    for i := 0; i < n; i++ {
        if !seen[i] { answer++; dfs(i) }
    }
    return answer
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
A visited node/component has been fully accounted for and will not be counted again.

Complexity target:
Time $O(V+E)$; Space $O(V+E)$

Why:
Adjacency traversal is $O(V+E)$ time and $O(V+E)$ space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Not marking visited before enqueue; confusing directed and undirected edges.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
disconnected components; directed vs undirected; cycles; self edges; unreachable target.

Variations and follow-ups

Three drills:

- V1: Detect cycle vs produce traversal order.
- V2: Handle disconnected components.
- V3: Convert unweighted BFS to weighted shortest path.

Follow-ups:

- What happens with disconnected components or cycles?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def number_of_connected_components_in_an(n, edges):
    adj = [[] for _ in range(n)]
    for a,b in edges:
        adj[a].append(b); adj[b].append(a)
    seen, ans = set(), 0
    def dfs(u):
        seen.add(u)
        for v in adj[u]:
            if v not in seen: dfs(v)
    for i in range(n):
        if i not in seen: ans += 1; dfs(i)
    return ans

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
A visited node/component has been fully accounted for and will not be counted again.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 220 of 229: 252. Meeting Rooms

Pattern: Intervals | Sources: NC75 | Companies: Google, Meta, Amazon, Bloomberg (public samples)

Time: O(n log n)

Space: O(n)

Problem statement + sample

Prompt:
Given meeting time intervals, return whether one person can attend all meetings.

Example input/output:
Input: intervals=[[0,30],[5,10],[15,20]]

Output: false

Why: The first meeting overlaps the others.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every interval pair repeatedly. O(n^2).

Optimized approach:
Sort once, then maintain the active end or merged interval while scanning.

Alternative:
Sweep line with events is useful for counting overlaps/rooms.

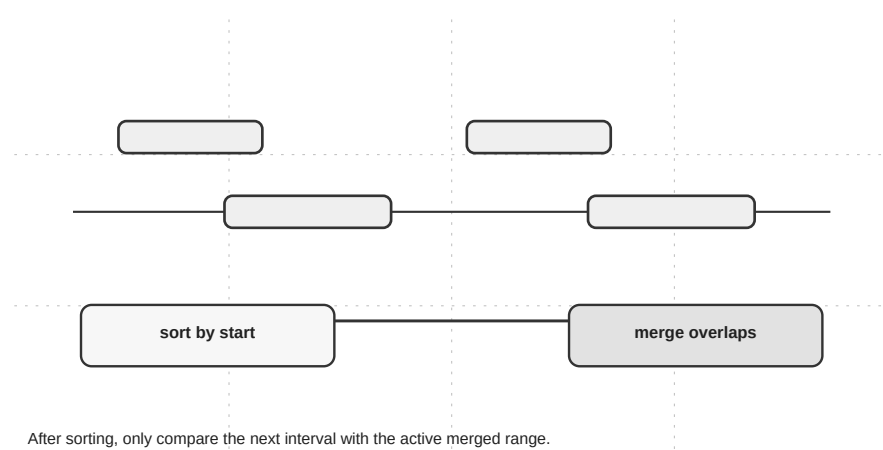
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 252. Meeting Rooms
func solveIntervals(intervals [][]int) [][]int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, in := range intervals {
        if len(merged) == 0 || merged[len(merged)-1][1] < in[0] {
            merged = append(merged, append([]int(nil), in...))
        } else if in[1] > merged[len(merged)-1][1] {
            merged[len(merged)-1][1] = in[1] // extend overlap
        }
    }
    return merged
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Complexity target:
Time O(n log n); Space O(n)

Why:
Sorting dominates at O(n log n); the scan is O(n). Space is O(1) to O(n).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Sorting by wrong key; not merging touching intervals when required.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
touching endpoints; inclusive vs exclusive; empty list; sorting tie-breakers; nested intervals.

Variations and follow-ups

- Three drills:
- V1: Distinguish touching endpoints from overlapping endpoints.
 - V2: Return removed intervals as well as the merged result.
 - V3: Process online insertions into an existing schedule.

- Follow-ups:
- Can you process intervals online?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def meeting_rooms(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All intervals before current have been merged or counted consistently with the chosen end boundary.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 221 of 229: 253. Meeting Rooms II

Pattern: Intervals | Sources: NC75 | Companies: Google, Meta, Amazon, Bloomberg (public samples)

Time: $O(n \log n)$

Space: $O(n)$

Problem statement + sample

Prompt:
Return the minimum number of meeting rooms required for all intervals.

Example input/output:
Input: intervals=[[0,30],[5,10],[15,20]]

Output: 2

Why: At most two meetings overlap at once.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Compare every interval pair repeatedly. $O(n^2)$.

Optimized approach:
Sort once, then maintain the active end or merged interval while scanning.

Alternative:
Sweep line with events is useful for counting overlaps/rooms.

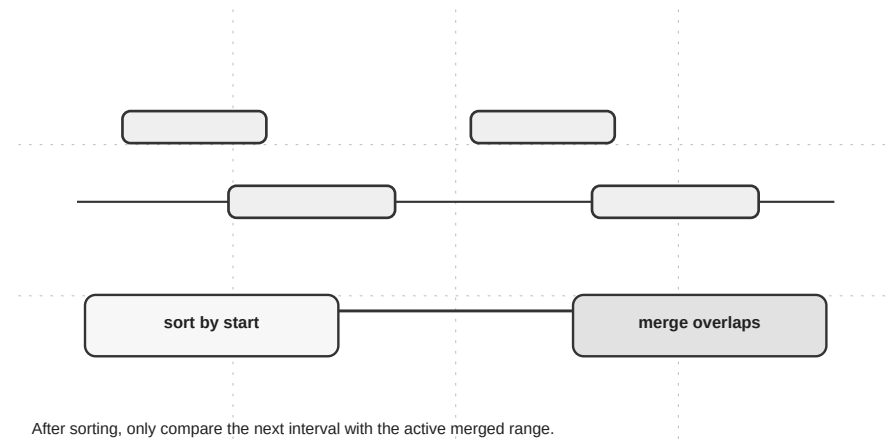
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 253. Meeting Rooms II
func solveIntervals(intervals [][]int) []int {
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][0] < intervals[j][0] })
    merged := [][]int{}
    for _, in := range intervals {
        if len(merged) == 0 || merged[len(merged)-1][1] < in[0] {
            merged = append(merged, append([]int(nil), in...))
        } else if in[1] > merged[len(merged)-1][1] {
            merged[len(merged)-1][1] = in[1] // extend overlap
        }
    }
    return merged
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
All intervals before current have been merged or counted consistently with the chosen end boundary.

Complexity target:
Time $O(n \log n)$; Space $O(n)$

Why:
Sorting dominates at $O(n \log n)$; the scan is $O(n)$. Space is $O(1)$ to $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? $i < n$?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Sorting by wrong key; not merging touching intervals when required.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
touching endpoints; inclusive vs exclusive; empty list; sorting tie-breakers; nested intervals.

Variations and follow-ups

- Three drills:
- V1: Distinguish touching endpoints from overlapping endpoints.
 - V2: Return removed intervals as well as the merged result.
 - V3: Process online insertions into an existing schedule.

- Follow-ups:
- Can you process intervals online?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def meeting_rooms_ii(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
All intervals before current have been merged or counted consistently with the chosen end boundary.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 222 of 229: 143. Reorder List

Pattern: Linked List | Sources: NC75 | Companies: Amazon, Microsoft, Meta, Apple (public samples)

Time: O(n)

Space: O(1) unless recursion/map

Problem statement + sample

Prompt:
Reorder a linked list as L0, Ln, L1, Ln-1, and so on in-place.

Example input/output:
Input: head=[1,2,3,4,5]

Output: [1,5,2,4,3]

Why: Interleave first half with reversed second half.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
Copy values into an array, solve there, then rebuild. Simple but extra O(n) space.

Optimized approach:
Use pointer rewiring, dummy nodes, fast/slow pointers, and careful next preservation.

Alternative:
Recursive list solutions are shorter but can use O(n) call stack.

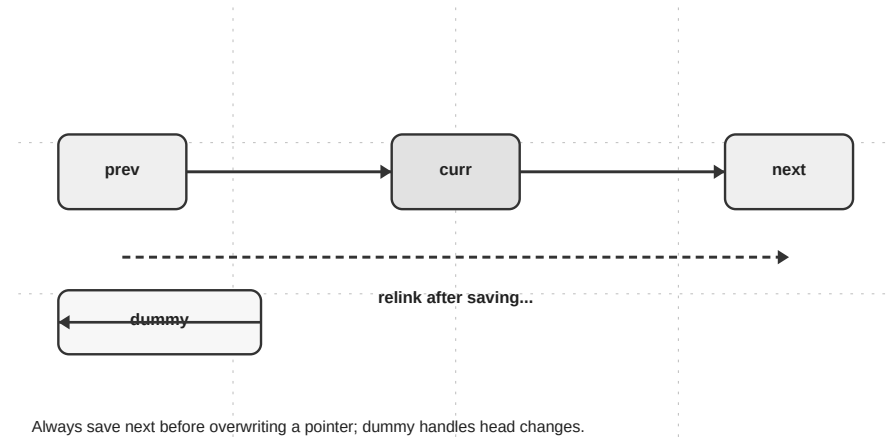
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 143. Reorder List
func solveLinkedList(head *ListNode) *ListNode {
    dummy := &ListNode{Next: head} // protects head changes
    prev, curr := (*ListNode)(nil), head
    for curr != nil {
        next := curr.Next // save before rewiring
        curr.Next = prev // reverse or relink current edge
        prev = curr // processed prefix grows
        curr = next // move to untouched suffix
    }
    _ = dummy // use dummy for deletions/partitions/k-group variants
    return prev
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
The processed prefix is already correctly linked and no unprocessed node is lost.

Complexity target:
Time O(n); Space O(1) unless recursion/map

Why:
Most list pointer solutions are one or two passes: O(n) time, O(1) extra space.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Losing next pointer before rewiring; missing dummy head edge cases.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
empty head; one node; removing head; cycles; preserving next before overwriting pointers.

Variations and follow-ups

- Three drills:
- V1: Do the operation recursively and compare stack use.
 - V2: Handle cycle/random pointers in the same structure.
 - V3: Perform the change for every k-sized group.

- Follow-ups:
- Can you solve it with O(1) extra memory and one pass?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def reorder_list(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The processed prefix is already correctly linked and no unprocessed node is lost.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 223 of 229: 572. Subtree of Another Tree

Pattern: Tree DFS | Sources: NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: O(n)

Space: O(h)

Problem statement + sample

Prompt:
Return whether subRoot is a subtree of root with identical structure and values.

Example input/output:
Input: root=[3,4,5,1,2], subRoot=[4,1,2]

Output: true

Why: The subtree rooted at 4 matches.

- Clarify before coding:
- Empty or singleton input?
 - Mutate input or build output?
 - Duplicate/tie behavior?
 - int vs int64 in Go?
 - Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become O(n^2).

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

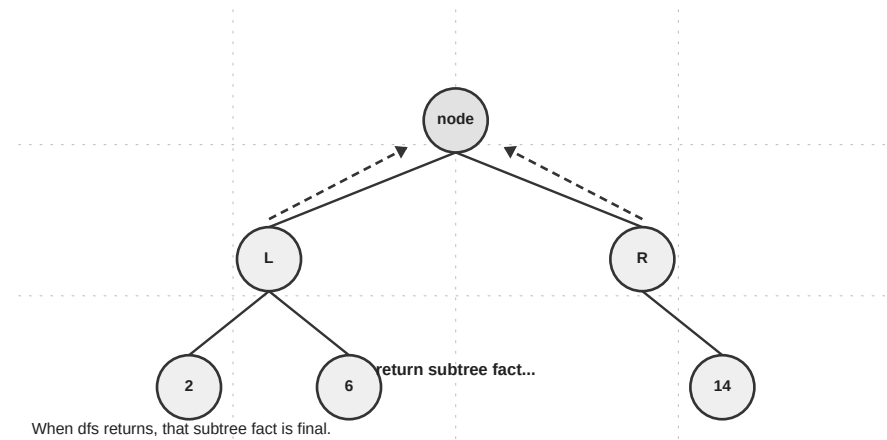
- Implementation checkpoints:
- Define the exact state before coding.
 - Update state in one place.
 - Dry run the sample before typing loops.
 - Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 572. Subtree of Another Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time O(n); Space O(h)

Why:
Each node is visited once: O(n) time. Recursion stack is O(h).

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

- Go line-by-line talk track:
- 1) Inputs and base cases first.
 - 2) State variables match the invariant.
 - 3) The loop moves one pointer/index/event at a time.
 - 4) Record the answer only after the state is valid.
 - 5) Return the exact requested value, not debug state.

- Implementation cautions:
- Bounds: left <= right? i < n?
 - Maps: decrement and delete zero counts when needed.
 - Stack/heap: store indices when value lookup is needed.
 - Trees/graphs: mark visited before recursion/enqueue.
 - DP: base cases and iteration order.

Mistakes + edge checklist

- Common mistakes:
- Returning answer instead of path contribution; wrong null base case.
 - Updating state before saving the next pointer/index.
 - Recording an answer while the window/state is invalid.
 - Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

- Three drills:
- V1: Return the nodes on the path, not just a count/value.
 - V2: Make the tree contain negative values or duplicate values.
 - V3: Convert recursive DFS to iterative stack DFS.

- Follow-ups:
- Can you write iterative DFS? What value does each call return?
 - What changes if constraints are 10x larger?
 - Which line would you unit-test first and why?

Python reference kernel

```
def subtree_of_another_tree(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

- Unit-test matrix:
- Empty/nil input or zero-length string.
 - Single item and all-equal items.
 - Sample case from the prompt.
 - Duplicate/tie/no-solution case.
 - Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 224 of 229: 235. Lowest Common Ancestor of a Binary Search Tree

Pattern: Tree DFS | Sources: NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
In a BST, return the lowest common ancestor of nodes p and q.

Example input/output:
Input: root=[6,2,8,0,4,7,9,null,null,3,5], p=2, q=8

Output: 6

Why: The nodes split on opposite sides of 6.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

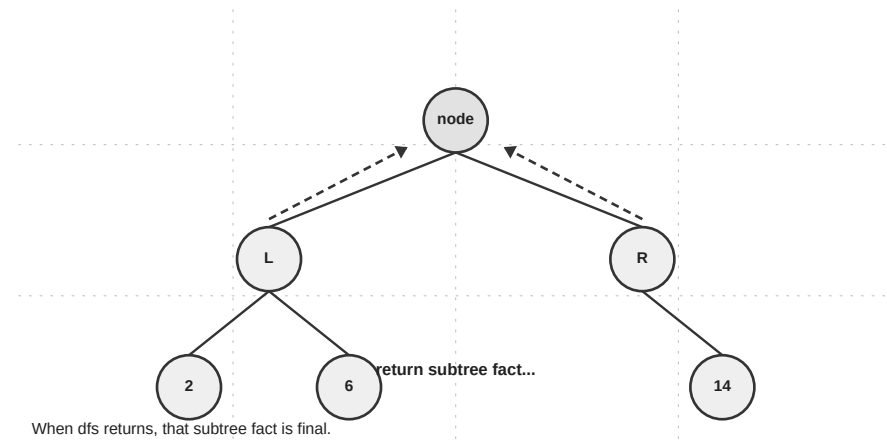
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 235. Lowest Common Ancestor of a Binary Search Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def lowest_common_ancestor_of_a_binary_s(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 225 of 229: 297. Serialize and Deserialize Binary Tree

Pattern: Tree DFS | Sources: NC75 | Companies: Amazon, Google, Meta, Microsoft (public samples)

Time: $O(n)$

Space: $O(h)$

Problem statement + sample

Prompt:
Design serialize and deserialize methods for a binary tree so the original structure and values are recoverable.

Example input/output:
Input: root=[1,2,3,null,null,4,5]

Output: deserialize(serialize(root)) equals the original tree

Why: Null markers preserve structure.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
For each node, recompute subtree facts from scratch; can become $O(n^2)$.

Optimized approach:
Postorder or preorder DFS carries exactly the state needed by children/parent.

Alternative:
Iterative stack avoids recursion depth; BFS works when level order matters.

Implementation checkpoints:

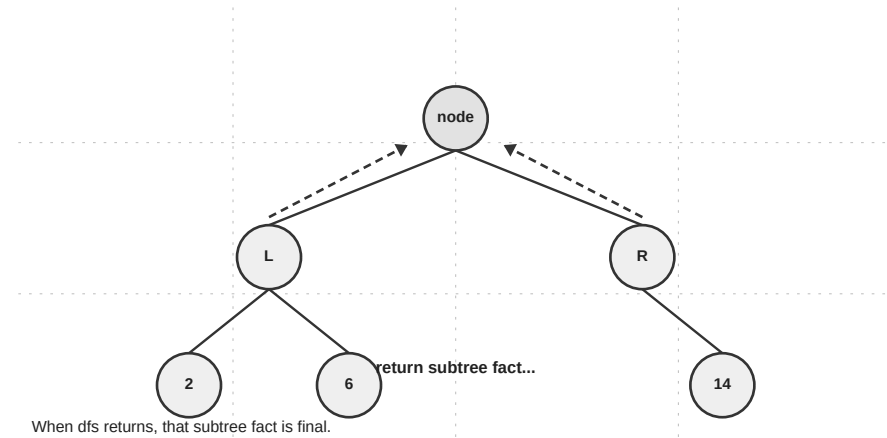
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 297. Serialize and Deserialize Binary Tree
func solveTreeDFS(root *TreeNode) int {
    best := 0
    var dfs func(*TreeNode) int
    dfs = func(node *TreeNode) int {
        if node == nil { return 0 }
        left := dfs(node.Left) // finalized fact from left subtree
        right := dfs(node.Right) // finalized fact from right subtree
        best = max(best, combineThrough(node, left, right))
        return valueForParent(node, left, right)
    }
    dfs(root)
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
When dfs returns from a node, its subtree answer/fact is final.

Complexity target:
Time $O(n)$; Space $O(h)$

Why:
Each node is visited once: $O(n)$ time. Recursion stack is $O(h)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $left \leq right?$ $i < n?$
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Returning answer instead of path contribution; wrong null base case.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
nil root; skewed tree recursion depth; negative values; duplicate values; global vs returned answer.

Variations and follow-ups

Three drills:

- V1: Return the nodes on the path, not just a count/value.
- V2: Make the tree contain negative values or duplicate values.
- V3: Convert recursive DFS to iterative stack DFS.

Follow-ups:

- Can you write iterative DFS? What value does each call return?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def serialize_and_deserialize_binary_tre(root):
    best = 0
    def dfs(node):
        nonlocal best
        if not node: return 0
        left, right = dfs(node.left), dfs(node.right)
        best = max(best, combine(node, left, right))
        return value_for_parent(node, left, right)
    dfs(root)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
When dfs returns from a node, its subtree answer/fact is final.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 226 of 229: 347. Top K Frequent Elements

Pattern: Heap / Priority Queue | Sources: NC75 | Companies: Amazon, Bloomberg, Google, Meta (public samples)

Time: $O(n \log k)$ or $O(n \log n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the k most frequent elements from the array.

Example input/output:
Input: nums=[1,1,1,2,2,3], k=2

Output: [1,2]

Why: 1 and 2 have the highest frequencies.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Repeatedly sort or scan all candidates for each selection: $O(k*n)$ or worse.

Optimized approach:
Use a min/max heap to access the next best candidate in $O(\log n)$.

Alternative:
Quickselect works for one kth statistic; heap is better for streaming or repeated picks.

Implementation checkpoints:

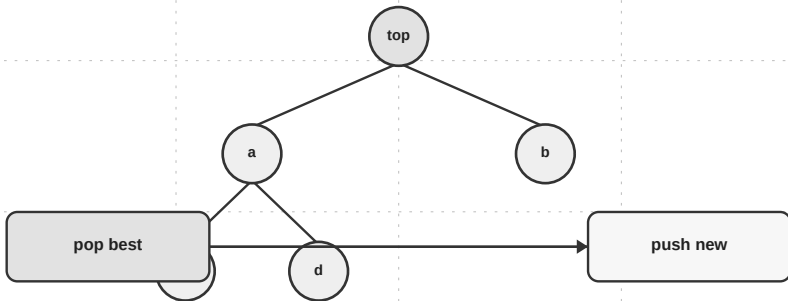
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 347. Top K Frequent Elements
func solveHeap(items []int, k int) int {
    h := &IntHeap{}
    heap.Init(h)
    for _, x := range items {
        heap.Push(h, x) // push eligible candidate
        if h.Len() > k { // keep only useful candidates
            heap.Pop(h)
        }
    }
    // In Go, implement heap.Interface: Len, Less, Swap, Push, Pop.
    return heap.Pop(h).(int)
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Heap top is the next best candidate; discard stale entries after pop.

Invariant + complexity + proof

Invariant:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Complexity target:
Time $O(n \log k)$ or $O(n \log n)$; Space $O(k)$ to $O(n)$

Why:
Heap operations cost $O(\log n)$; total time depends on pushes/pops, often $O(n \log k)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left <= right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Using min-heap/max-heap sign incorrectly; heap grows beyond k.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
stale entries; tie-breakers; min vs max heap in Go; pushing duplicates; empty heap.

Variations and follow-ups

Three drills:

- V1: Return top-k in sorted order after using the heap.
- V2: Handle ties with a secondary key.
- V3: Support streaming insertions and deletions.

Follow-ups:

- Can you reduce memory from $O(n)$ to $O(k)$?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def top_k_frequent_elements(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
The heap contains all eligible candidates not yet chosen, ordered by the needed priority.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 227 of 229: 424. Longest Repeating Character Replacement

Pattern: Sliding Window | Sources: NC75 | Companies: Google, Meta, Amazon, Uber (public samples)

Time: $O(n)$

Space: $O(k)$ to $O(n)$

Problem statement + sample

Prompt:
Return the longest substring length that can be made of one repeated character by replacing at most k characters.

Example input/output:
Input: $s = \text{"AABABBA"} , k = 1$

Output: 4

Why: "AABA" can become all A.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/subarray and test validity from scratch: $O(n^2)$ to $O(n^3)$.

Optimized approach:
Expand right once, update counts, shrink left until the invariant is valid, then record the answer.

Alternative:
Fixed-size windows skip the while loop; variable-size windows need a validity predicate.

Implementation checkpoints:

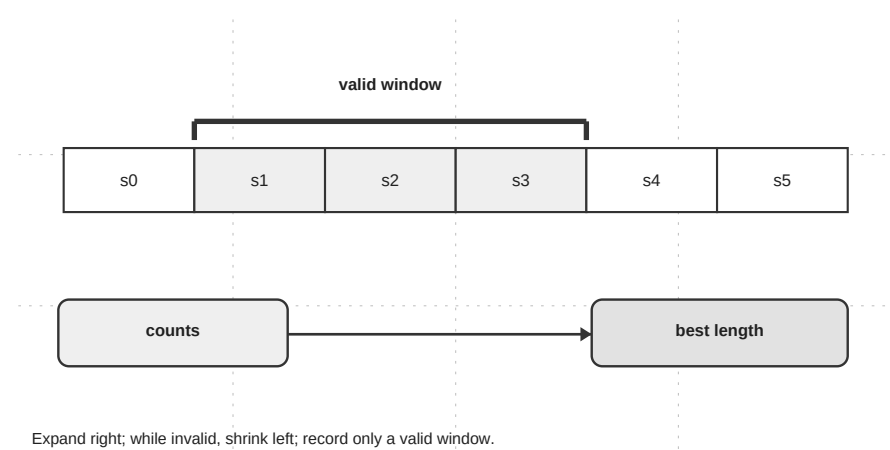
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 424. Longest Repeating Character Replacement
func solveSlidingWindow(s string, k int) int {
    count := map[byte]int{}
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        count[s[right]]++ // include the new right edge
        for !windowValid(count, right-left+1, k) {
            count[s[left]]-- // remove old left edge
            left++
        }
        best = max(best, right-left+1) // record only valid windows
    }
    return best
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Complexity target:
Time $O(n)$; Space $O(k)$ to $O(n)$

Why:
Left and right each move forward at most n times, so $O(n)$. Map space depends on alphabet size.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Shrinking only once when a while loop is needed; stale max/count variables.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
 $k=0$; empty string; repeated chars; removing counts at left; off-by-one length $\text{right-left}+1$.

Variations and follow-ups

Three drills:

- V1: Switch from at-most- k to exactly- k constraints.
- V2: Return the smallest valid window instead of the largest.
- V3: Support multiple character/value constraints at once.

Follow-ups:

- Can you return the actual window, not only its length/count?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def longest_repeating_character_replacem(s, k):
    left = best = 0
    count = {}
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while not window_valid(count, right-left+1, k):
            count[s[left]] -= 1; left += 1
        best = max(best, right-left+1)
    return best

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
At record time, the window satisfies the prompt condition and is minimal/maximal as required.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 228 of 229: 271. Encode and Decode Strings

Pattern: Trie / Encoding | Sources: NC75 | Companies: Google, Amazon, Bloomberg, Microsoft (public samples)

Time: $O(\text{total characters})$

Space: $O(\text{total characters})$

Problem statement + sample

Prompt:
Design encode and decode for a list of strings so any characters, including delimiters, are recoverable.

Example input/output:
Input: strs=["lint","code","love"]

Output: decode(encode(strs))=["lint","code","love"]

Why: Length-prefix encoding is unambiguous.

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Compare every string with every query or split by ambiguous delimiters.

Optimized approach:
Store characters in a trie for prefix search, or encode lengths so decoding is unambiguous.

Alternative:
Hash maps can replace trie when only whole-word lookup is needed.

Implementation checkpoints:

- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

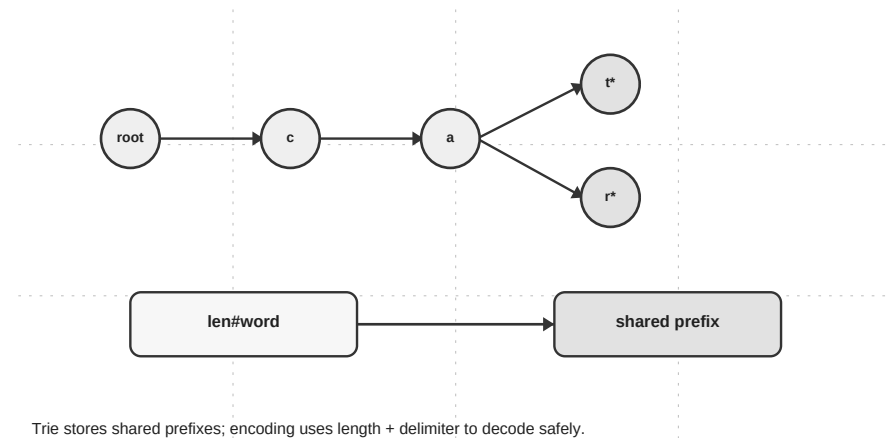
Annotated Go kernel - comments explain the reasoning

```
// 271. Encode and Decode Strings
type TrieNode struct {
    child map[byte]*TrieNode
    end bool
}

func solveTrieEncoding(words []string) *TrieNode {
    root := &TrieNode{child: map[byte]*TrieNode{}}
    for _, w := range words {
        cur := root
        for i := 0; i < len(w); i++ {
            if cur.child[w[i]] == nil { cur.child[w[i]] = &TrieNode{child: map[byte]*TrieNode{}} }
            cur = cur.child[w[i]]
        }
        cur.end = true // complete word marker
    }
    return root
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Every path from root represents exactly one prefix; terminal marks complete words.

Complexity target:
Time $O(\text{total characters})$; Space $O(\text{total characters})$

Why:
 $O(\text{total characters})$ to build; query cost is $O(\text{length of query/prefix})$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: left $\leq$ right? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Terminal marker missing; ambiguous delimiter.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty strings; delimiter appearing in input; duplicate words; memory blowup; lexicographic ordering.

Variations and follow-ups

Three drills:

- V1: Support delete/update operations.
- V2: Return top-k suggestions with lexicographic ties.
- V3: Use length-prefix encoding when delimiter may appear in input.

Follow-ups:

- Can the trie support delete or wildcard search?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def encode_and_decode_strings(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Every path from root represents exactly one prefix; terminal marks complete words.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.

MAIN PROBLEM 229 of 229: 647. Palindromic Substrings

Pattern: DP 2D / Kadane | Sources: NC75 | Companies: Google, Meta, Amazon, Microsoft (public samples)

Time: $O(n*m)$ or $O(n)$

Space: $O(n*m)$ or $O(1)$

Problem statement + sample

Prompt:
Return the number of palindromic substrings in s.

Example input/output:
Input: s="aaa"

Output: 6

Why: Three singles, two "aa", and one "aaa".

Clarify before coding:

- Empty or singleton input?
- Mutate input or build output?
- Duplicate/tie behavior?
- int vs int64 in Go?
- Exact return shape?

Approach ladder

Brute force:
Enumerate every substring/path/subproblem and recompute its score.

Optimized approach:
Pick a state that removes repeated work, then fill in dependency order. For subarrays, keep rolling best.

Alternative:
Space-compress rows/columns when only the previous row/column is needed.

Implementation checkpoints:

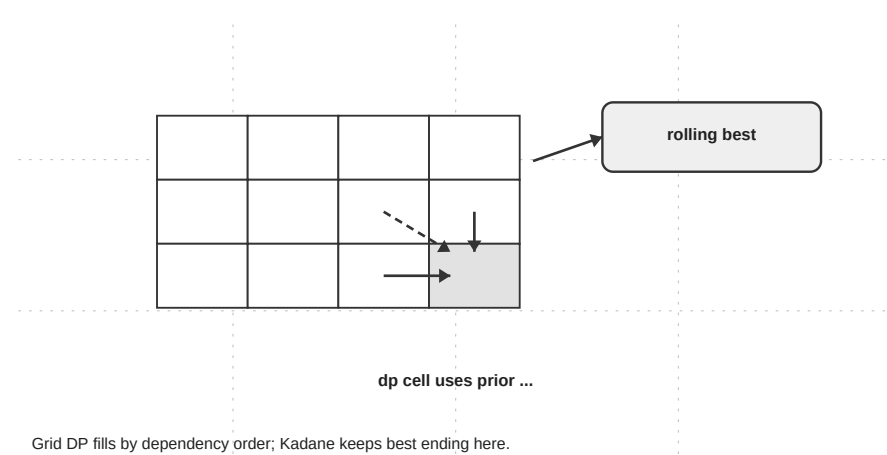
- Define the exact state before coding.
- Update state in one place.
- Dry run the sample before typing loops.
- Say why discarded candidates cannot later win.

Annotated Go kernel - comments explain the reasoning

```
// 647. Palindromic Substrings
func solveDP2DorKadane(grid [][]int) int {
    rows, cols := len(grid), len(grid[0])
    dp := make([][]int, rows)
    for r := range dp { dp[r] = make([]int, cols) }
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            // Combine valid predecessors before writing dp[r][c].
            dp[r][c] = transition2D(grid, dp, r, c)
        }
    }
    return dp[rows-1][cols-1]
}

// Explanation checklist:
// - Initialization creates the invariant.
// - Each loop consumes one new fact or safely discards one candidate.
// - Helper predicates stand for exact prompt logic.
// - Use int64 when sums/products can overflow int.
// - Test: empty, one item, duplicates/ties, no-solution, sample, max-size.
```

Rendered concept diagram



Invariant + complexity + proof

Invariant:
Each filled state stores the optimal answer for exactly its subproblem.

Complexity target:
Time $O(n*m)$ or $O(n)$; Space $O(n*m)$ or $O(1)$

Why:
 $O(\text{number of states} * \text{transition cost})$, commonly $O(n*m)$ or $O(n)$.

Proof sketch:
Initialize state so the invariant is true. Each update preserves it using finalized information only. At the end, every candidate was considered or safely discarded.

Go implementation notes

Go line-by-line talk track:

- 1) Inputs and base cases first.
- 2) State variables match the invariant.
- 3) The loop moves one pointer/index/event at a time.
- 4) Record the answer only after the state is valid.
- 5) Return the exact requested value, not debug state.

Implementation cautions:

- Bounds: $\text{left} \leq \text{right}$? $i < n$?
- Maps: decrement and delete zero counts when needed.
- Stack/heap: store indices when value lookup is needed.
- Trees/graphs: mark visited before recursion/enqueue.
- DP: base cases and iteration order.

Mistakes + edge checklist

Common mistakes:

- Bad initialization; mixing row/column dependencies.
- Updating state before saving the next pointer/index.
- Recording an answer while the window/state is invalid.
- Forgetting nil/empty/base cases.

Edge checklist:
empty dimensions; first row/column; negative-only arrays; diagonal dependencies; memory size.

Variations and follow-ups

Three drills:

- V1: Return the actual subarray/subsequence.
- V2: Reduce 2D memory to rolling rows where safe.
- V3: Handle all-negative values or empty inputs explicitly.

Follow-ups:

- Can the DP be space compressed safely?
- What changes if constraints are 10x larger?
- Which line would you unit-test first and why?

Python reference kernel

```
def palindromic_substrings(*args):
    # State the invariant first, then scan/update once.
    state = init_state(args)
    for item in iterate(args):
        state = update(state, item)
    return answer(state)

# Match exact platform signature and helper logic.
```

State plan + test matrix

State plan for this pattern:
Each filled state stores the optimal answer for exactly its subproblem.

Unit-test matrix:

- Empty/nil input or zero-length string.
- Single item and all-equal items.
- Sample case from the prompt.
- Duplicate/tie/no-solution case.
- Max-size case for time and memory.

Final interview sentence:
Every element/index/node is processed a bounded number of times, so the target complexity follows.